knn

April 5, 2026

[ ]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'CS231n/assignment1'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/MyDrive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/MyDrive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/MyDrive/$FOLDERNAME
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
/content/drive/MyDrive/CS231n/assignment1/cs231n/datasets
/content/drive/MyDrive/CS231n/assignment1
```

1 k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transfering the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

[ ]:
```
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the␣
↪notebook
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Workaround for Python 3.12 where 'imp' is removed
import sys
import importlib
import types
if 'imp' not in sys.modules:
    imp = types.ModuleType('imp')
    imp.reload = importlib.reload
    sys.modules['imp'] = imp

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

[ ]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause␣
↪memory issue)
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)
```

```python
# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Clear previously loaded data.
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

[ ]:
```python
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```

[ ]:
```
# Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

# Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)
```

```
(5000, 3072) (500, 3072)
```

[ ]:
```
from cs231n.classifiers import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are **Ntr** training examples and **Nte** test examples, this stage should result in a **Nte x Ntr** matrix where each element (i,j) is the distance between the i-th test and j-th train example.

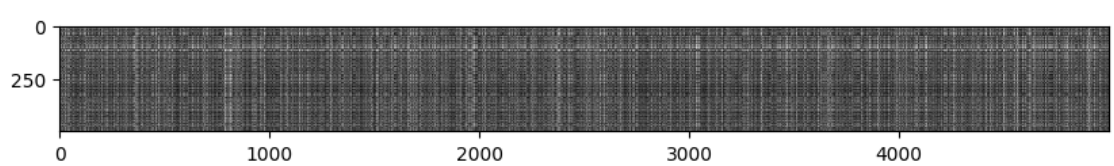
Note: For the three distance computations that we require you to implement in this notebook, you may not use the np.linalg.norm() function that numpy provides.

First, open `cs231n/classifiers/k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

[ ]:
```
# Open cs231n/classifiers/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

```
(500, 5000)
```

[ ]:
```
# We can visualize the distance matrix: each row is a single test example and
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



Inline Question 1

Notice the structured patterns in the distance matrix, where some rows or columns are visibly brighter. (Note that with the default color scheme black indicates low distances while white indicates high distances.)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

YourAnswer : 1. bright rows comes from the i-th test sample sample is far away from all the train sample

2. bright columns comes from that there are no samples distributed around j-th train sample.

[ ]:
```
# Now implement the function predict_labels and run the code below:
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 137 / 500 correct => accuracy: 0.274000
```

You should expect to see approximately 27% accuracy. Now lets try out a larger `k`, say `k = 5`:

[ ]:
```
y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 139 / 500 correct => accuracy: 0.278000
```

You should expect to see a slightly better performance than with `k = 1`.

Inline Question 2

We can also use other distance metrics such as L1 distance. For pixel values $p_{ij}^{(k)}$ at location (i, j) of some image I_k,

the mean μ across all pixels over all images is

$$\mu = \frac{1}{nhw} \sum_{k=1}^{n} \sum_{i=1}^{h} \sum_{j=1}^{w} p_{ij}^{(k)}$$

And the pixel-wise mean μ_{ij} across all images is

$$\mu_{ij} = \frac{1}{n} \sum_{k=1}^{n} p_{ij}^{(k)}.$$

The general standard deviation σ and pixel-wise standard deviation σ_{ij} is defined similarly.

Which of the following preprocessing steps will not change the performance of a Nearest Neighbor classifier that uses L1 distance? Select all that apply. To clarify, both training and test examples are preprocessed in the same way.

1. Subtracting the mean μ ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu$.)
2. Subtracting the per pixel mean μ_{ij} ($\tilde{p}_{ij}^{(k)} = p_{ij}^{(k)} - \mu_{ij}$.)
3. Subtracting the mean μ and dividing by the standard deviation σ.
4. Subtracting the pixel-wise mean μ_{ij} and dividing by the pixel-wise standard deviation σ_{ij}.
5. Rotating the coordinate axes of the data, which means rotating all the images by the same angle. Empty regions in the image caused by rotation are padded with a same pixel value and no interpolation is performed.

$YourAnswer:$ $1, 2, 3$

$YourExplanation:$ 1. The shift is applied to both images and cancels out when taking the difference, so the L1 distance remains unchanged.

2. Similar to the global mean, this per-pixel shift cancels out during the difference calculation, keeping the L1 distance the same.

3. This applies the exact same scaling factor to all distances, preserving their relative order.

4. This applies different scaling factors to different pixels, which distorts the L1 distance and changes the relative ordering of neighbors.

5. The L1 distance is highly sensitive to the orientation of the coordinate axes, so rotation changes the distances computed between images.

[ ]:
```
# Now lets speed up distance matrix computation by using partial vectorization
# with one loop. Implement the function compute_distances_one_loop and run the
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure that it
# agrees with the naive implementation. There are many ways to decide whether
# two matrices are similar; one of the simplest is the Frobenius norm. In case
# you haven't seen it before, the Frobenius norm of two matrices is the square
# root of the squared sum of differences of all elements; in other words,␣
↪reshape
# the matrices into vectors and compute the Euclidean distance between them.
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('One loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
One loop difference was: 0.000000
Good! The distance matrices are the same
```

[ ]:
```
# Now implement the fully vectorized version inside compute_distances_no_loops
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)
```

```
# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('No loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
No loop difference was: 0.000000
Good! The distance matrices are the same
```

[ ]:
```
# Let's compare how fast the implementations are
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it took␣
 ↪to execute.
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_test)
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_test)
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_test)
print('No loop version took %f seconds' % no_loop_time)

# You should see significantly faster performance with the fully vectorized␣
 ↪implementation!

# NOTE: depending on what machine you're using,
# you might not see a speedup when you go from two loops to one loop,
# and might even see a slow-down.
```

```
Two loop version took 37.911475 seconds
One loop version took 37.926412 seconds
No loop version took 0.602099 seconds
```

1.0.1 Cross-validation

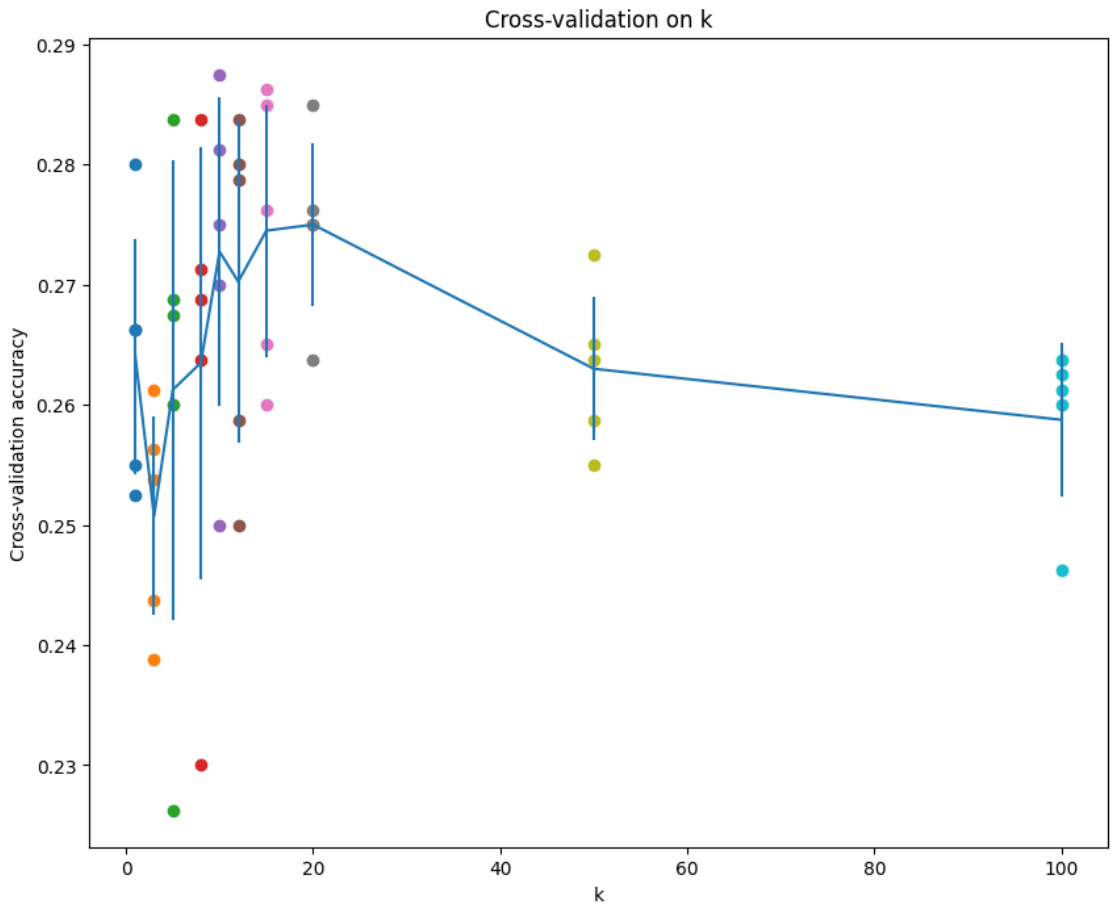
We have implemented the k-Nearest Neighbor classifier but we set the value k = 5 arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

[ ]:
```
num_folds = 5
k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100]

X_train_folds = np.array_split(X_train, num_folds)
y_train_folds = np.array_split(y_train, num_folds)


# A dictionary holding the accuracies for different values of k that we find
# when running cross-validation. After running cross-validation,
# k_to_accuracies[k] should be a list of length num_folds giving the different
# accuracy values that we found when using that value of k.
k_to_accuracies = {}

for k in k_choices:
    k_to_accuracies[k] = []
    KNN = KNearestNeighbor()
    for i in range(num_folds):
      X_train_val = np.concatenate(X_train_folds[:i]+X_train_folds[i+1:])
      y_train_val = np.concatenate(y_train_folds[:i]+y_train_folds[i+1:])
      KNN.train(X_train_val,y_train_val)
      y_pred = KNN.predict(X_train_folds[i],k)
      k_to_accuracies[k].append(np.sum(y_pred==y_train_folds[i])/
 ↪y_train_folds[i].shape[0])
# Print out the computed accuracies
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))
```

```
k = 1, accuracy = 0.248438
k = 1, accuracy = 0.237500
k = 1, accuracy = 0.229687
k = 1, accuracy = 0.285938
k = 1, accuracy = 0.270313
k = 3, accuracy = 0.228125
k = 3, accuracy = 0.232813
k = 3, accuracy = 0.240625
k = 3, accuracy = 0.251563
k = 3, accuracy = 0.248438
k = 5, accuracy = 0.234375
k = 5, accuracy = 0.245312
k = 5, accuracy = 0.257812
k = 5, accuracy = 0.271875
k = 5, accuracy = 0.253125
k = 8, accuracy = 0.231250
k = 8, accuracy = 0.248438
k = 8, accuracy = 0.281250
k = 8, accuracy = 0.271875
```

```
k = 8, accuracy = 0.260937
k = 10, accuracy = 0.231250
k = 10, accuracy = 0.257812
k = 10, accuracy = 0.275000
k = 10, accuracy = 0.276562
k = 10, accuracy = 0.253125
k = 12, accuracy = 0.225000
k = 12, accuracy = 0.259375
k = 12, accuracy = 0.279687
k = 12, accuracy = 0.275000
k = 12, accuracy = 0.242188
k = 15, accuracy = 0.234375
k = 15, accuracy = 0.268750
k = 15, accuracy = 0.284375
k = 15, accuracy = 0.257812
k = 15, accuracy = 0.253125
k = 20, accuracy = 0.245312
k = 20, accuracy = 0.265625
k = 20, accuracy = 0.290625
k = 20, accuracy = 0.284375
k = 20, accuracy = 0.253125
k = 50, accuracy = 0.235937
k = 50, accuracy = 0.243750
k = 50, accuracy = 0.271875
k = 50, accuracy = 0.242188
k = 50, accuracy = 0.254688
k = 100, accuracy = 0.240625
k = 100, accuracy = 0.239063
k = 100, accuracy = 0.254688
k = 100, accuracy = 0.246875
k = 100, accuracy = 0.246875
```

[ ]:
```
# plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)

# plot the trend line with error bars that correspond to standard deviation
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.
 ↪items())])
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.
 ↪items())])
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()
```



[ ]:
```
# Based on the cross-validation results above, choose the best value for k,
# retrain the classifier using all the training data, and test it on the test
# data. You should be able to get above 28% accuracy on the test data.
best_k = 10

classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, accuracy))
```

```
Got 141 / 500 correct => accuracy: 0.282000
```

Inline Question 3

Which of the following statements about k-Nearest Neighbor (k-NN) are true in a classification setting, and for all k? Select all that apply. 1. The decision boundary of the k-NN classifier is

linear. 2. The training error of a 1-NN will always be lower than or equal to that of 5-NN. 3. The test error of a 1-NN will always be lower than that of a 5-NN. 4. The time needed to classify a test example with the k-NN classifier grows with the size of the training set. 5. None of the above.

$YourAnswer:$ $2, 4$

$YourExplanation:$ 1. the boundary is complicated 2. I think this question is a bit subtle, because the nearesst point is itself 3. the test error is displayed above 4. the time complexity of KNN is $O(N)$

softmax

April 5, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'CS231n/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/MyDrive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/MyDrive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/MyDrive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/MyDrive/CS231n/assignment1/cs231n/datasets
/content/drive/MyDrive/CS231n/assignment1
```

1 Softmax Classifier exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

In this exercise you will:

- implement a fully-vectorized **loss function** for the Softmax classifier.
- implement the fully-vectorized expression for its **analytic gradient**
- **check your implementation** using numerical gradient
- use a validation set to **tune the learning rate and regularization** strength
- **optimize** the loss function with **SGD**
- **visualize** the final learned weights

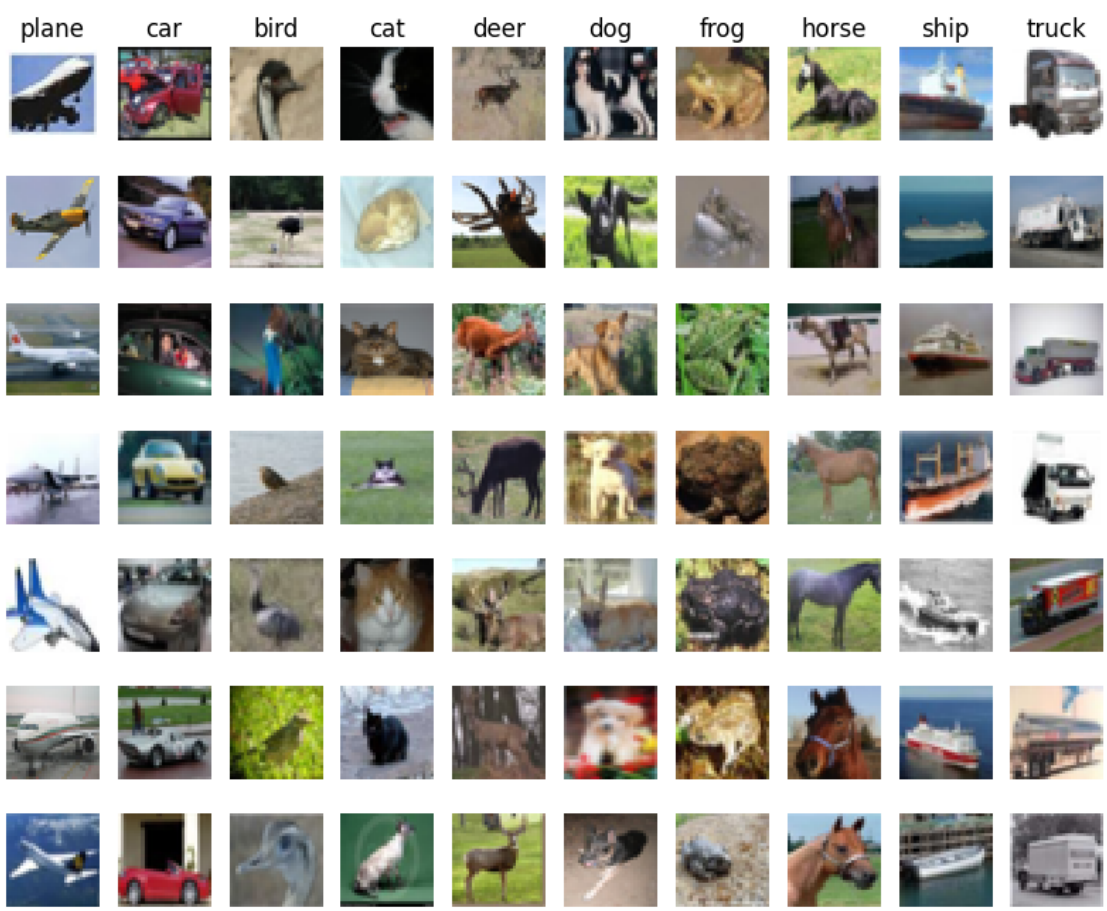
[2]:
```
# Run some setup code for this notebook.
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt
import importlib
sys.modules['imp'] = importlib

# This is a bit of magic to make matplotlib figures appear inline in the
# notebook rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python modules;
# see http://stackoverflow.com/questions/1907993/
 ↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 CIFAR-10 Data Loading and Preprocessing

[3]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

# Cleaning up variables to prevent loading data multiple times (which may cause␣
 ↪memory issue)
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

[4]:
```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
 ↪'ship', 'truck']
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



[5]:
```
# Split the data into train, val, and test sets. In addition we will
# create a small development set as a subset of the training data;
# we can use this for development so our code runs faster.
num_training = 49000
num_validation = 1000
num_test = 1000
num_dev = 500

# Our validation set will be num_validation points from the original
# training set.
mask = range(num_training, num_training + num_validation)
X_val = X_train[mask]
y_val = y_train[mask]

# Our training set will be the first num_train points from the original
# training set.
mask = range(num_training)
X_train = X_train[mask]
y_train = y_train[mask]

# We will also make a development set, which is a small subset of
# the training set.
mask = np.random.choice(num_training, num_dev, replace=False)
X_dev = X_train[mask]
y_dev = y_train[mask]

# We use the first num_test points of the original test set as our
# test set.
mask = range(num_test)
X_test = X_test[mask]
y_test = y_test[mask]

print('Train data shape: ', X_train.shape)
print('Train labels shape: ', y_train.shape)
print('Validation data shape: ', X_val.shape)
print('Validation labels shape: ', y_val.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

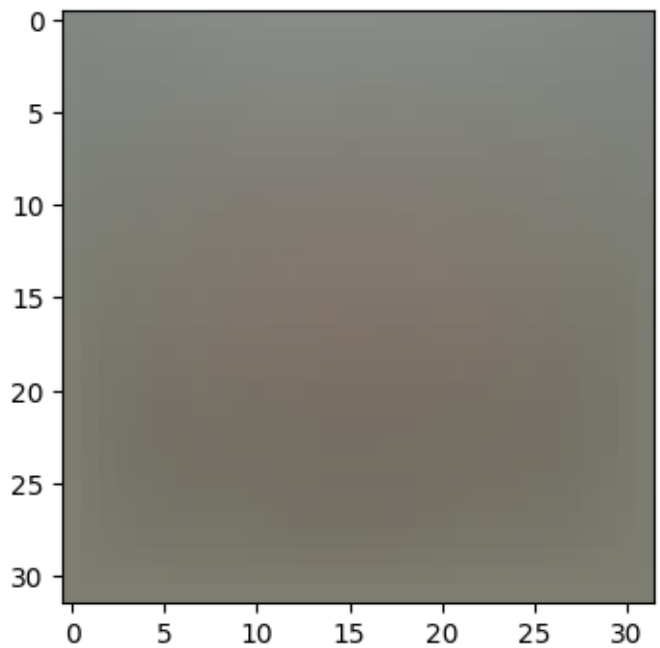
```
Train data shape:  (49000, 32, 32, 3)
Train labels shape:  (49000,)
Validation data shape:  (1000, 32, 32, 3)
Validation labels shape:  (1000,)
Test data shape:  (1000, 32, 32, 3)
Test labels shape:  (1000,)
```

[6]:
```
# Preprocessing: reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_val = np.reshape(X_val, (X_val.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
X_dev = np.reshape(X_dev, (X_dev.shape[0], -1))

# As a sanity check, print out the shapes of the data
print('Training data shape: ', X_train.shape)
print('Validation data shape: ', X_val.shape)
print('Test data shape: ', X_test.shape)
print('dev data shape: ', X_dev.shape)
```

```
Training data shape:  (49000, 3072)
Validation data shape:  (1000, 3072)
Test data shape:  (1000, 3072)
dev data shape:  (500, 3072)
```

[7]:
```
# Preprocessing: subtract the mean image
# first: compute the image mean based on the training data
mean_image = np.mean(X_train, axis=0)
print(mean_image[:10]) # print a few of the elements
plt.figure(figsize=(4,4))
plt.imshow(mean_image.reshape((32,32,3)).astype('uint8')) # visualize the mean␣
↪image
plt.show()

# second: subtract the mean image from train and test data
X_train -= mean_image
X_val -= mean_image
X_test -= mean_image
X_dev -= mean_image

# third: append the bias dimension of ones (i.e. bias trick) so that our␣
↪classifier
# only has to worry about optimizing a single weight matrix W.
X_train = np.hstack([X_train, np.ones((X_train.shape[0], 1))])
X_val = np.hstack([X_val, np.ones((X_val.shape[0], 1))])
X_test = np.hstack([X_test, np.ones((X_test.shape[0], 1))])
X_dev = np.hstack([X_dev, np.ones((X_dev.shape[0], 1))])

print(X_train.shape, X_val.shape, X_test.shape, X_dev.shape)
```

```
[130.64189796 135.98173469 132.47391837 130.05569388 135.34804082
 131.75402041 130.96055102 136.14328571 132.47636735 131.48467347]
```




```
(49000, 3073) (1000, 3073) (1000, 3073) (500, 3073)
```

1.2 Softmax Classifier

Your code for this section will all be written inside `cs231n/classifiers/softmax.py`.

As you can see, we have prefilled the function `softmax_loss_naive` which uses for loops to evaluate the softmax loss function.

[8]:
```
# Evaluate the naive implementation of the loss we provided for you:
from cs231n.classifiers.softmax import softmax_loss_naive
import time

# generate a random Softmax classifier weight matrix of small numbers
W = np.random.randn(3073, 10) * 0.0001

loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
print('loss: %f' % (loss, ))

# As a rough sanity check, our loss should be something close to -log(0.1).
print('loss: %f' % loss)
print('sanity check: %f' % (-np.log(0.1)))
```

```
loss: 2.350497
loss: 2.350497
sanity check: 2.302585
```

Inline Question 1

Why do we expect our loss to be close to -log(0.1)? Explain briefly.**

Your Answer : The initial score for the all 10 classes of each sample is nearly zero, in softmax, one have $\frac{e^s}{\sum_i e^{s_i}}$ is nearly $\frac{1}{10}$

The `grad` returned from the function above is right now all zero. Derive and implement the gradient for the softmax loss function and implement it inline inside the function `softmax_loss_naive`. You will find it helpful to interleave your new code inside the existing function.

To check that you have correctly implemented the gradient, you can numerically estimate the gradient of the loss function and compare the numeric estimate to the gradient that you computed. We have provided code that does this for you:

[9]:
```
# Once you've implemented the gradient, recompute it with the code below
# and gradient check it with the function we provided for you

# Compute the loss and its gradient at W.
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 0.0)

# Numerically compute the gradient along several randomly chosen dimensions, and
# compare them with your analytically computed gradient. The numbers should␣
↪match
# almost exactly along all dimensions.
from cs231n.gradient_check import grad_check_sparse
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 0.0)[0]
grad_numerical = grad_check_sparse(f, W, grad)

# do the gradient check once again with regularization turned on
# you didn't forget the regularization gradient did you?
loss, grad = softmax_loss_naive(W, X_dev, y_dev, 5e1)
f = lambda w: softmax_loss_naive(w, X_dev, y_dev, 5e1)[0]
grad_numerical = grad_check_sparse(f, W, grad)
```

```
numerical: -1.064609 analytic: -1.064609, relative error: 1.062470e-07
numerical: -1.246640 analytic: -1.246641, relative error: 3.624067e-08
numerical: -0.596103 analytic: -0.596103, relative error: 6.081289e-08
numerical: 2.868022 analytic: 2.868022, relative error: 2.425060e-08
numerical: -0.848006 analytic: -0.848006, relative error: 6.825203e-08
numerical: 0.026889 analytic: 0.026889, relative error: 1.603773e-06
numerical: -0.155222 analytic: -0.155222, relative error: 1.665866e-07
numerical: -0.244000 analytic: -0.244000, relative error: 5.716027e-08
numerical: -2.004862 analytic: -2.004862, relative error: 3.217213e-09
numerical: 0.328377 analytic: 0.328377, relative error: 9.137257e-08
numerical: -2.964684 analytic: -2.954952, relative error: 1.644042e-03
numerical: -0.085655 analytic: -0.079601, relative error: 3.663235e-02
numerical: 0.783643 analytic: 0.784310, relative error: 4.253772e-04
numerical: 0.248237 analytic: 0.251689, relative error: 6.904418e-03
numerical: -2.587683 analytic: -2.589673, relative error: 3.843027e-04
```

```
numerical: 0.561000 analytic: 0.561653, relative error: 5.819197e-04
numerical: 2.976733 analytic: 2.977199, relative error: 7.827340e-05
numerical: 4.285372 analytic: 4.284635, relative error: 8.601083e-05
numerical: -1.824495 analytic: -1.825313, relative error: 2.239125e-04
numerical: -0.014342 analytic: -0.020648, relative error: 1.802214e-01
```

Inline Question 2

Although gradcheck is reliable softmax loss, it is possible that for SVM loss, once in a while, a dimension in the gradcheck will not match exactly. What could such a discrepancy be caused by? Is it a reason for concern? What is a simple example in one dimension where a svm loss gradient check could fail? How would change the margin affect of the frequency of this happening?

Note that SVM loss for a sample (x_i, y_i) is defined as:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

where j iterates over all classes except the correct class y_i and s_j denotes the classifier score for j^{th} class. Δ is a scalar margin. For more information, refer to 'Multiclass Support Vector Machine loss' on this page.

Hint: the SVM loss function is not strictly speaking differentiable.

YourAnswer : The discrepancy is caused by the non-differentiable "kinks" in the $\max(0, \cdot)$ function. When the numerical gradient step h crosses such a kink, the finite difference approximation fails to match the exact analytic derivative. This is not a concern as it is an inherent property of the SVM loss.

[10]:
```
# Next implement the function softmax_loss_vectorized; for now only compute the␣
↪loss;
# we will implement the gradient in a moment.
tic = time.time()
loss_naive, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss: %e computed in %fs' % (loss_naive, toc - tic))

from cs231n.classifiers.softmax import softmax_loss_vectorized
tic = time.time()
loss_vectorized, _ = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss: %e computed in %fs' % (loss_vectorized, toc - tic))

# The losses should match but your vectorized implementation should be much␣
↪faster.
print('difference: %f' % (loss_naive - loss_vectorized))
```

```
Naive loss: 2.350497e+00 computed in 0.062845s
Vectorized loss: 2.350497e+00 computed in 0.017202s
difference: 0.000000
```

[11]:
```
# Complete the implementation of softmax_loss_vectorized, and compute the␣
↪gradient
# of the loss function in a vectorized way.

# The naive implementation and the vectorized implementation should match, but
# the vectorized version should still be much faster.
tic = time.time()
_, grad_naive = softmax_loss_naive(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Naive loss and gradient: computed in %fs' % (toc - tic))

tic = time.time()
_, grad_vectorized = softmax_loss_vectorized(W, X_dev, y_dev, 0.000005)
toc = time.time()
print('Vectorized loss and gradient: computed in %fs' % (toc - tic))

# The loss is a single number, so it is easy to compare the values computed
# by the two implementations. The gradient on the other hand is a matrix, so
# we use the Frobenius norm to compare them.
difference = np.linalg.norm(grad_naive - grad_vectorized, ord='fro')
print('difference: %f' % difference)
```

```
Naive loss and gradient: computed in 0.086366s
Vectorized loss and gradient: computed in 0.012908s
difference: 0.000000
```

1.2.1 Stochastic Gradient Descent

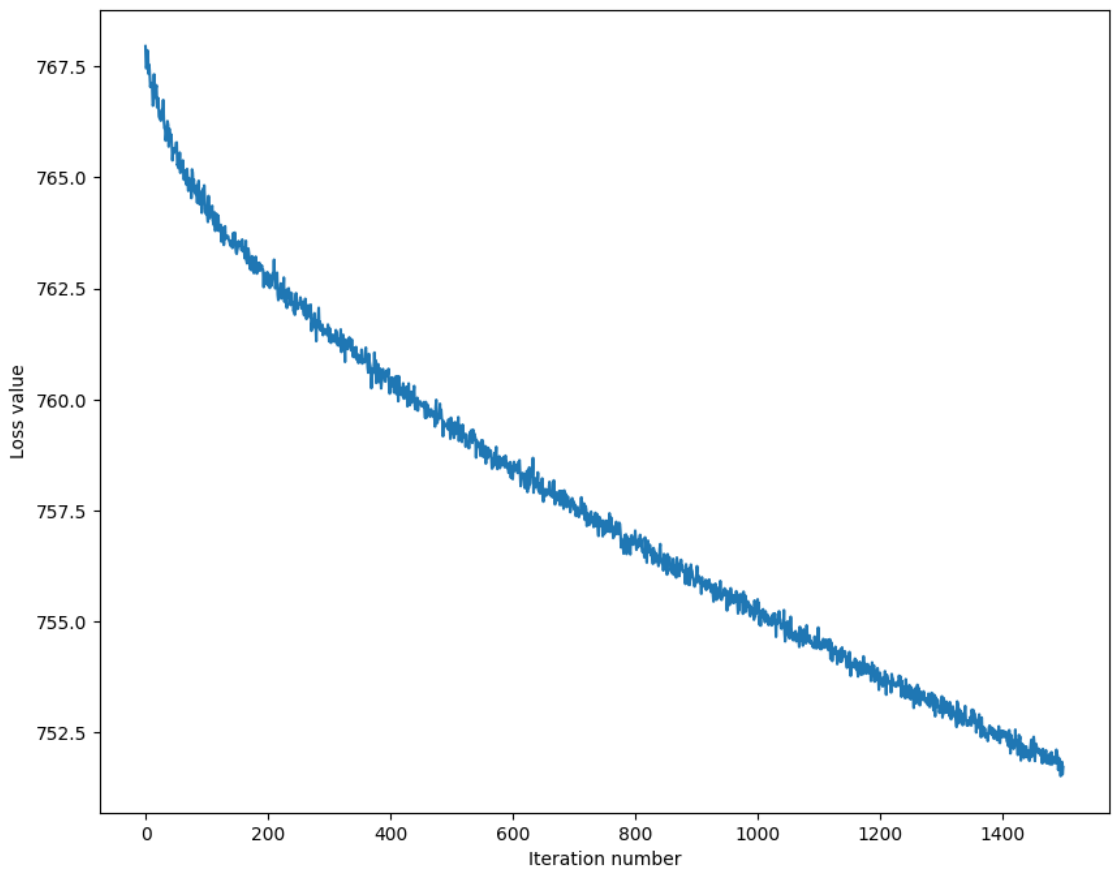
We now have vectorized and efficient expressions for the loss, the gradient and our gradient matches the numerical gradient. We are therefore ready to do SGD to minimize the loss. Your code for this part will be written inside `cs231n/classifiers/linear_classifier.py`.

[12]:
```
# In the file linear_classifier.py, implement SGD in the function
# LinearClassifier.train() and then run it with the code below.
from cs231n.classifiers import Softmax
softmax = Softmax()
tic = time.time()
loss_hist = softmax.train(X_train, y_train, learning_rate=1e-7, reg=2.5e4,
                      num_iters=1500, verbose=True)
toc = time.time()
print('That took %fs' % (toc - tic))
```

```
iteration 0 / 1500: loss 767.951477
iteration 100 / 1500: loss 764.318559
iteration 200 / 1500: loss 762.565177
iteration 300 / 1500: loss 761.307272
iteration 400 / 1500: loss 760.320541
iteration 500 / 1500: loss 759.287527
```

```
iteration 600 / 1500: loss 758.205174
iteration 700 / 1500: loss 757.526313
iteration 800 / 1500: loss 757.046708
iteration 900 / 1500: loss 756.069629
iteration 1000 / 1500: loss 755.500287
iteration 1100 / 1500: loss 754.422790
iteration 1200 / 1500: loss 753.847384
iteration 1300 / 1500: loss 753.048912
iteration 1400 / 1500: loss 752.398976
That took 5.540387s
```

[13]:
```
# A useful debugging strategy is to plot the loss as a function of
# iteration number:
plt.plot(loss_hist)
plt.xlabel('Iteration number')
plt.ylabel('Loss value')
plt.show()
```



[14]:
```
# Write the LinearClassifier.predict function and evaluate the performance on
# both the training and validation set
```

```
# You should get validation accuracy of about 0.34 (> 0.33).
y_train_pred = softmax.predict(X_train)
print('training accuracy: %f' % (np.mean(y_train == y_train_pred), ))
y_val_pred = softmax.predict(X_val)
print('validation accuracy: %f' % (np.mean(y_val == y_val_pred), ))
```

```
training accuracy: 0.246163
validation accuracy: 0.264000
```

[15]:
```
# Save the trained model for autograder.
softmax.save("softmax.npy")
```

```
softmax.npy saved.
```

[36]:
```
# Use the validation set to tune hyperparameters (regularization strength and
# learning rate). You should experiment with different ranges for the learning
# rates and regularization strengths; if you are careful you should be able to
# get a classification accuracy of about 0.365 (> 0.36) on the validation set.

# Note: you may see runtime/overflow warnings during hyper-parameter search.
# This may be caused by extreme values, and is not a bug.

# results is dictionary mapping tuples of the form
# (learning_rate, regularization_strength) to tuples of the form
# (training_accuracy, validation_accuracy). The accuracy is simply the fraction
# of data points that are correctly classified.
results = {}
best_val = -1   # The highest validation accuracy that we have seen so far.
best_softmax = None # The Softmax object that achieved the highest validation␣
 ↪rate.

# Provided as a reference. You may or may not want to change these␣
 ↪hyperparameters
learning_rates = [6.1e-6, 6e-6,6.2e-6,6.3e-6,6.4e-6,6.5e-6]
regularization_strengths = [0.97e4,0.98e4, 1e4,0.99e4,1.01e4,1.02e4,1.03e4,1.
 ↪04e4]
num_itera = 3000
for lr in learning_rates:
  for reg in regularization_strengths:
    softmax = Softmax()
    loss_hist = softmax.train(X_train, y_train, learning_rate=lr, reg=reg,␣
 ↪num_iters=num_itera)
    y_train_pred = softmax.predict(X_train)
    y_val_pred = softmax.predict(X_val)
    train_accuracy = np.mean(y_train == y_train_pred)
    val_accuracy = np.mean(y_val== y_val_pred)
```

```
    print(f"with learning rate {lr} and regularization strength {reg}, train␣
↪accuracy: {train_accuracy}, val accuracy: {val_accuracy}")
    results[(lr, reg)] = (train_accuracy, val_accuracy)
    if val_accuracy > best_val:
      best_val = val_accuracy
      best_softmax = softmax


# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
                lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved during cross-validation: %f' %␣
↪best_val)
```

```
with learning rate 6.1e-06 and regularization strength 9700.0, train accuracy:
0.40053061224489794, val accuracy: 0.343
with learning rate 6.1e-06 and regularization strength 9800.0, train accuracy:
0.3667142857142857, val accuracy: 0.328
with learning rate 6.1e-06 and regularization strength 10000.0, train accuracy:
0.3633469387755102, val accuracy: 0.339
with learning rate 6.1e-06 and regularization strength 9900.0, train accuracy:
0.39979591836734696, val accuracy: 0.383
with learning rate 6.1e-06 and regularization strength 10100.0, train accuracy:
0.3470408163265306, val accuracy: 0.313
with learning rate 6.1e-06 and regularization strength 10200.0, train accuracy:
0.37626530612244896, val accuracy: 0.355
with learning rate 6.1e-06 and regularization strength 10300.0, train accuracy:
0.37512244897959185, val accuracy: 0.357
with learning rate 6.1e-06 and regularization strength 10400.0, train accuracy:
0.39146938775510204, val accuracy: 0.351
with learning rate 6e-06 and regularization strength 9700.0, train accuracy:
0.39491836734693875, val accuracy: 0.354
with learning rate 6e-06 and regularization strength 9800.0, train accuracy:
0.3826530612244898, val accuracy: 0.337
with learning rate 6e-06 and regularization strength 10000.0, train accuracy:
0.37612244897959185, val accuracy: 0.36
with learning rate 6e-06 and regularization strength 9900.0, train accuracy:
0.4166530612244898, val accuracy: 0.381
with learning rate 6e-06 and regularization strength 10100.0, train accuracy:
0.410265306122449, val accuracy: 0.376
with learning rate 6e-06 and regularization strength 10200.0, train accuracy:
0.38651020408163267, val accuracy: 0.354
with learning rate 6e-06 and regularization strength 10300.0, train accuracy:
0.4063265306122449, val accuracy: 0.36
```

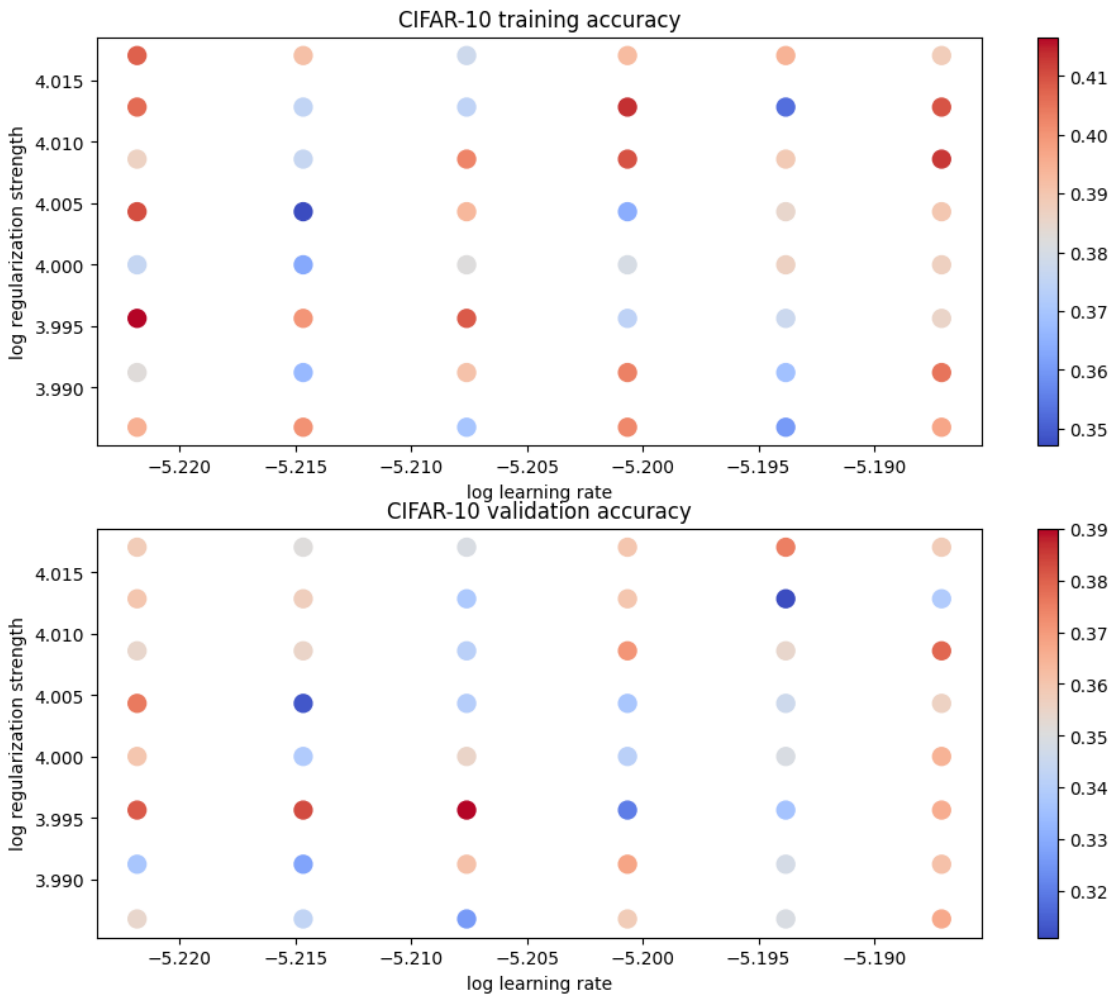
```
with learning rate 6e-06 and regularization strength 10400.0, train accuracy:
0.40791836734693876, val accuracy: 0.358
with learning rate 6.2e-06 and regularization strength 9700.0, train accuracy:
0.36975510204081635, val accuracy: 0.326
with learning rate 6.2e-06 and regularization strength 9800.0, train accuracy:
0.3909183673469388, val accuracy: 0.361
with learning rate 6.2e-06 and regularization strength 10000.0, train accuracy:
0.38195918367346937, val accuracy: 0.355
with learning rate 6.2e-06 and regularization strength 9900.0, train accuracy:
0.40861224489795916, val accuracy: 0.39
with learning rate 6.2e-06 and regularization strength 10100.0, train accuracy:
0.39348979591836736, val accuracy: 0.34
with learning rate 6.2e-06 and regularization strength 10200.0, train accuracy:
0.4027142857142857, val accuracy: 0.341
with learning rate 6.2e-06 and regularization strength 10300.0, train accuracy:
0.37457142857142856, val accuracy: 0.338
with learning rate 6.2e-06 and regularization strength 10400.0, train accuracy:
0.378, val accuracy: 0.349
with learning rate 6.3e-06 and regularization strength 9700.0, train accuracy:
0.40216326530612245, val accuracy: 0.358
with learning rate 6.3e-06 and regularization strength 9800.0, train accuracy:
0.4031224489795918, val accuracy: 0.368
with learning rate 6.3e-06 and regularization strength 10000.0, train accuracy:
0.38018367346938775, val accuracy: 0.341
with learning rate 6.3e-06 and regularization strength 9900.0, train accuracy:
0.37434693877551023, val accuracy: 0.32
with learning rate 6.3e-06 and regularization strength 10100.0, train accuracy:
0.36406122448979594, val accuracy: 0.337
with learning rate 6.3e-06 and regularization strength 10200.0, train accuracy:
0.40969387755102044, val accuracy: 0.371
with learning rate 6.3e-06 and regularization strength 10300.0, train accuracy:
0.41351020408163264, val accuracy: 0.36
with learning rate 6.3e-06 and regularization strength 10400.0, train accuracy:
0.3926938775510204, val accuracy: 0.36
with learning rate 6.4e-06 and regularization strength 9700.0, train accuracy:
0.3603061224489796, val accuracy: 0.349
with learning rate 6.4e-06 and regularization strength 9800.0, train accuracy:
0.3686326530612245, val accuracy: 0.348
with learning rate 6.4e-06 and regularization strength 10000.0, train accuracy:
0.387, val accuracy: 0.349
with learning rate 6.4e-06 and regularization strength 9900.0, train accuracy:
0.3772857142857143, val accuracy: 0.336
with learning rate 6.4e-06 and regularization strength 10100.0, train accuracy:
0.38493877551020406, val accuracy: 0.346
with learning rate 6.4e-06 and regularization strength 10200.0, train accuracy:
0.3890816326530612, val accuracy: 0.354
with learning rate 6.4e-06 and regularization strength 10300.0, train accuracy:
0.3526734693877551, val accuracy: 0.311
```

```
with learning rate 6.4e-06 and regularization strength 10400.0, train accuracy:
0.39455102040816326, val accuracy: 0.375
with learning rate 6.5e-06 and regularization strength 9700.0, train accuracy:
0.3970408163265306, val accuracy: 0.367
with learning rate 6.5e-06 and regularization strength 9800.0, train accuracy:
0.4050204081632653, val accuracy: 0.361
with learning rate 6.5e-06 and regularization strength 10000.0, train accuracy:
0.38718367346938776, val accuracy: 0.365
with learning rate 6.5e-06 and regularization strength 9900.0, train accuracy:
0.3856938775510204, val accuracy: 0.366
with learning rate 6.5e-06 and regularization strength 10100.0, train accuracy:
0.3898979591836735, val accuracy: 0.356
with learning rate 6.5e-06 and regularization strength 10200.0, train accuracy:
0.4127959183673469, val accuracy: 0.379
with learning rate 6.5e-06 and regularization strength 10300.0, train accuracy:
0.40951020408163263, val accuracy: 0.339
with learning rate 6.5e-06 and regularization strength 10400.0, train accuracy:
0.38818367346938776, val accuracy: 0.358
lr 6.000000e-06 reg 9.700000e+03 train accuracy: 0.394918 val accuracy: 0.354000
lr 6.000000e-06 reg 9.800000e+03 train accuracy: 0.382653 val accuracy: 0.337000
lr 6.000000e-06 reg 9.900000e+03 train accuracy: 0.416653 val accuracy: 0.381000
lr 6.000000e-06 reg 1.000000e+04 train accuracy: 0.376122 val accuracy: 0.360000
lr 6.000000e-06 reg 1.010000e+04 train accuracy: 0.410265 val accuracy: 0.376000
lr 6.000000e-06 reg 1.020000e+04 train accuracy: 0.386510 val accuracy: 0.354000
lr 6.000000e-06 reg 1.030000e+04 train accuracy: 0.406327 val accuracy: 0.360000
lr 6.000000e-06 reg 1.040000e+04 train accuracy: 0.407918 val accuracy: 0.358000
lr 6.100000e-06 reg 9.700000e+03 train accuracy: 0.400531 val accuracy: 0.343000
lr 6.100000e-06 reg 9.800000e+03 train accuracy: 0.366714 val accuracy: 0.328000
lr 6.100000e-06 reg 9.900000e+03 train accuracy: 0.399796 val accuracy: 0.383000
lr 6.100000e-06 reg 1.000000e+04 train accuracy: 0.363347 val accuracy: 0.339000
lr 6.100000e-06 reg 1.010000e+04 train accuracy: 0.347041 val accuracy: 0.313000
lr 6.100000e-06 reg 1.020000e+04 train accuracy: 0.376265 val accuracy: 0.355000
lr 6.100000e-06 reg 1.030000e+04 train accuracy: 0.375122 val accuracy: 0.357000
lr 6.100000e-06 reg 1.040000e+04 train accuracy: 0.391469 val accuracy: 0.351000
lr 6.200000e-06 reg 9.700000e+03 train accuracy: 0.369755 val accuracy: 0.326000
lr 6.200000e-06 reg 9.800000e+03 train accuracy: 0.390918 val accuracy: 0.361000
lr 6.200000e-06 reg 9.900000e+03 train accuracy: 0.408612 val accuracy: 0.390000
lr 6.200000e-06 reg 1.000000e+04 train accuracy: 0.381959 val accuracy: 0.355000
lr 6.200000e-06 reg 1.010000e+04 train accuracy: 0.393490 val accuracy: 0.340000
lr 6.200000e-06 reg 1.020000e+04 train accuracy: 0.402714 val accuracy: 0.341000
lr 6.200000e-06 reg 1.030000e+04 train accuracy: 0.374571 val accuracy: 0.338000
lr 6.200000e-06 reg 1.040000e+04 train accuracy: 0.378000 val accuracy: 0.349000
lr 6.300000e-06 reg 9.700000e+03 train accuracy: 0.402163 val accuracy: 0.358000
lr 6.300000e-06 reg 9.800000e+03 train accuracy: 0.403122 val accuracy: 0.368000
lr 6.300000e-06 reg 9.900000e+03 train accuracy: 0.374347 val accuracy: 0.320000
lr 6.300000e-06 reg 1.000000e+04 train accuracy: 0.380184 val accuracy: 0.341000
lr 6.300000e-06 reg 1.010000e+04 train accuracy: 0.364061 val accuracy: 0.337000
lr 6.300000e-06 reg 1.020000e+04 train accuracy: 0.409694 val accuracy: 0.371000
```

```
lr 6.300000e-06 reg 1.030000e+04 train accuracy: 0.413510 val accuracy: 0.360000
lr 6.300000e-06 reg 1.040000e+04 train accuracy: 0.392694 val accuracy: 0.360000
lr 6.400000e-06 reg 9.700000e+03 train accuracy: 0.360306 val accuracy: 0.349000
lr 6.400000e-06 reg 9.800000e+03 train accuracy: 0.368633 val accuracy: 0.348000
lr 6.400000e-06 reg 9.900000e+03 train accuracy: 0.377286 val accuracy: 0.336000
lr 6.400000e-06 reg 1.000000e+04 train accuracy: 0.387000 val accuracy: 0.349000
lr 6.400000e-06 reg 1.010000e+04 train accuracy: 0.384939 val accuracy: 0.346000
lr 6.400000e-06 reg 1.020000e+04 train accuracy: 0.389082 val accuracy: 0.354000
lr 6.400000e-06 reg 1.030000e+04 train accuracy: 0.352673 val accuracy: 0.311000
lr 6.400000e-06 reg 1.040000e+04 train accuracy: 0.394551 val accuracy: 0.375000
lr 6.500000e-06 reg 9.700000e+03 train accuracy: 0.397041 val accuracy: 0.367000
lr 6.500000e-06 reg 9.800000e+03 train accuracy: 0.405020 val accuracy: 0.361000
lr 6.500000e-06 reg 9.900000e+03 train accuracy: 0.385694 val accuracy: 0.366000
lr 6.500000e-06 reg 1.000000e+04 train accuracy: 0.387184 val accuracy: 0.365000
lr 6.500000e-06 reg 1.010000e+04 train accuracy: 0.389898 val accuracy: 0.356000
lr 6.500000e-06 reg 1.020000e+04 train accuracy: 0.412796 val accuracy: 0.379000
lr 6.500000e-06 reg 1.030000e+04 train accuracy: 0.409510 val accuracy: 0.339000
lr 6.500000e-06 reg 1.040000e+04 train accuracy: 0.388184 val accuracy: 0.358000
best validation accuracy achieved during cross-validation: 0.390000
```

[37]:
```python
# Visualize the cross-validation results
import math
import pdb

# pdb.set_trace()

x_scatter = [math.log10(x[0]) for x in results]
y_scatter = [math.log10(x[1]) for x in results]

# plot training accuracy
marker_size = 100
colors = [results[x][0] for x in results]
plt.subplot(2, 1, 1)
plt.tight_layout(pad=3)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
plt.title('CIFAR-10 training accuracy')

# plot validation accuracy
colors = [results[x][1] for x in results] # default size of markers is 20
plt.subplot(2, 1, 2)
plt.scatter(x_scatter, y_scatter, marker_size, c=colors, cmap=plt.cm.coolwarm)
plt.colorbar()
plt.xlabel('log learning rate')
plt.ylabel('log regularization strength')
```

```
plt.title('CIFAR-10 validation accuracy')
plt.show()
```



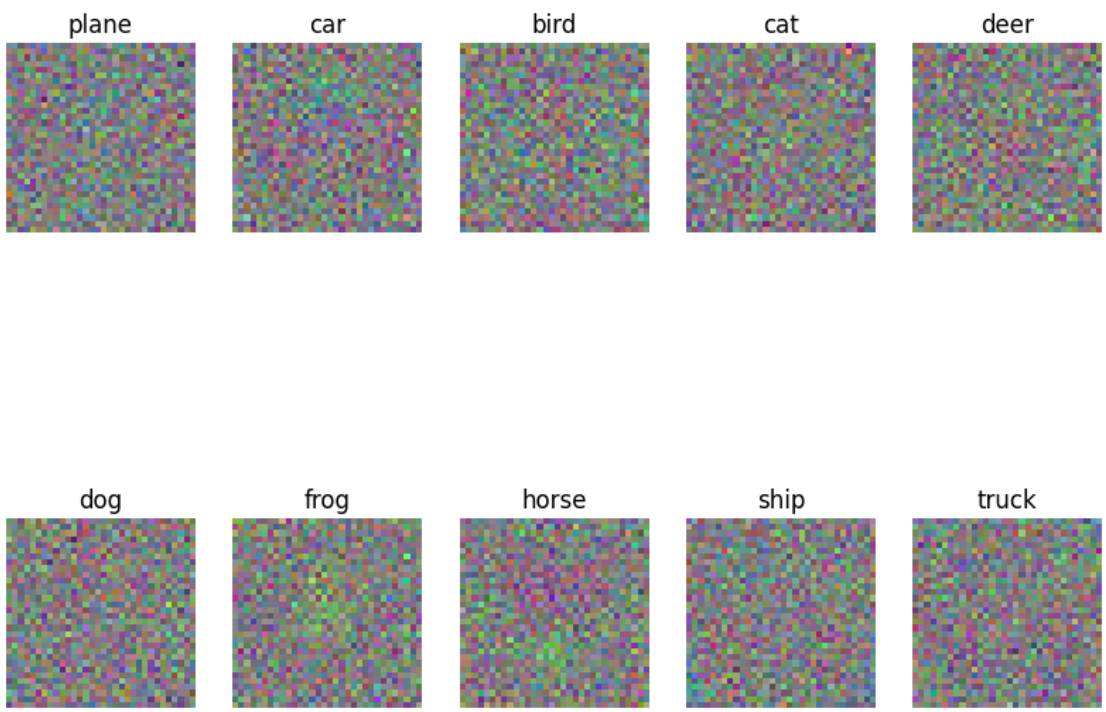
[38]:
```
# Evaluate the best softmax on test set
y_test_pred = best_softmax.predict(X_test)
test_accuracy = np.mean(y_test == y_test_pred)
print('Softmax classifier on raw pixels final test set accuracy: %f' %␣
↪test_accuracy)
```

```
Softmax classifier on raw pixels final test set accuracy: 0.376000
```

[39]:
```
# Save best softmax model
best_softmax.save("best_softmax.npy")
```

```
best_softmax.npy saved.
```

[40]:
```
# Visualize the learned weights for each class.
# Depending on your choice of learning rate and regularization strength, these␣
↪may
# or may not be nice to look at.
w = best_softmax.W[:-1,:] # strip out the bias
w = w.reshape(32, 32, 3, 10)
w_min, w_max = np.min(w), np.max(w)
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
↪'ship', 'truck']
for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Rescale the weights to be between 0 and 255
    wimg = 255.0 * (w[:, :, :, i].squeeze() - w_min) / (w_max - w_min)
    plt.imshow(wimg.astype('uint8'))
    plt.axis('off')
    plt.title(classes[i])
```



Inline question 3

Describe what your visualized Softmax classifier weights look like, and offer a brief explanation for why they look the way they do.

YourAnswer :The picture is difficult to distinguish, i think its beacause the noisy and the linear model is too simple

Inline Question 4 - *True or False*

Suppose the overall training loss is defined as the sum of the per-datapoint loss over all training examples. It is possible to add a new datapoint to a training set that would change the softmax loss, but leave the SVM loss unchanged.

$YourAnswer:$ True

$YourExplanation:$SVM only requires that the separation margin be met; no additional loss will be incurred. But the loss function of softmax needs to analyze the loss of each data point

two_layer_net

April 5, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'CS231n/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/MyDrive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/MyDrive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/MyDrive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/MyDrive/CS231n/assignment1/cs231n/datasets
/content/drive/MyDrive/CS231n/assignment1
```

1 Fully-Connected Neural Nets

In this exercise we will implement fully-connected networks using a modular approach. For each layer we will implement a `forward` and a `backward` function. The `forward` function will receive inputs, weights, and other parameters and will return both an output and a `cache` object storing data needed for the backward pass, like this:

```
def layer_forward(x, w):
  """ Receive inputs x and weights w """
  # Do some computations ...
  z = # ... some intermediate value
  # Do some more computations ...
  out = # the output
```

```
  cache = (x, w, z, out) # Values we need to compute gradients

  return out, cache
```

The backward pass will receive upstream derivatives and the `cache` object, and will return gradients with respect to the inputs and weights, like this:

```
def layer_backward(dout, cache):
  """
  Receive dout (derivative of loss with respect to outputs) and cache,
  and compute derivative with respect to inputs.
  """
  # Unpack cache values
  x, w, z, out = cache

  # Use values in cache to compute derivatives
  dx = # Derivative of loss with respect to x
  dw = # Derivative of loss with respect to w

  return dx, dw
```

After implementing a bunch of layers this way, we will be able to easily combine them to build classifiers with different architectures.

[2]:
```
# As usual, a bit of setup
from __future__ import print_function
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from cs231n.solver import Solver
import sys
import importlib
sys.modules['imp'] = importlib

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/
 ↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

```
def rel_error(x, y):
  """ returns relative error """
  return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

[3]:
```
# Load the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k, v in list(data.items()):
  print(('%s: ' % k, v.shape))
```

```
('X_train: ', (49000, 3, 32, 32))
('y_train: ', (49000,))
('X_val: ', (1000, 3, 32, 32))
('y_val: ', (1000,))
('X_test: ', (1000, 3, 32, 32))
('y_test: ', (1000,))
```

2 Affine layer: forward

Open the file `cs231n/layers.py` and implement the `affine_forward` function.

Once you are done you can test your implementaion by running the following:

[4]:
```
# Test the affine_forward function

num_inputs = 2
input_shape = (4, 5, 6)
output_dim = 3

input_size = num_inputs * np.prod(input_shape)
weight_size = output_dim * np.prod(input_shape)

x = np.linspace(-0.1, 0.5, num=input_size).reshape(num_inputs, *input_shape)
w = np.linspace(-0.2, 0.3, num=weight_size).reshape(np.prod(input_shape),␣
 ↪output_dim)
b = np.linspace(-0.3, 0.1, num=output_dim)

out, _ = affine_forward(x, w, b)
correct_out = np.array([[ 1.49834967,  1.70660132,  1.91485297],
                        [ 3.25553199,  3.5141327,   3.77273342]])

# Compare your output with ours. The error should be around e-9 or less.
print('Testing affine_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing affine_forward function:
difference:  9.769849468192957e-10
```

3 Affine layer: backward

Now implement the `affine_backward` function and test your implementation using numeric gradient checking.

[9]:
```
# Test the affine_backward function
np.random.seed(231)
x = np.random.randn(10, 2, 3)
w = np.random.randn(6, 5)
b = np.random.randn(5)
dout = np.random.randn(10, 5)

dx_num = eval_numerical_gradient_array(lambda x: affine_forward(x, w, b)[0], x,␣
 ↪dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_forward(x, w, b)[0], w,␣
 ↪dout)
db_num = eval_numerical_gradient_array(lambda b: affine_forward(x, w, b)[0], b,␣
 ↪dout)

_, cache = affine_forward(x, w, b)
dx, dw, db = affine_backward(dout, cache)

# The error should be around e-10 or less
print('Testing affine_backward function:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_backward function:
dx error:  5.399100368651805e-11
dw error:  9.904211865398145e-11
db error:  2.4122867568119087e-11
```

4 ReLU activation: forward

Implement the forward pass for the ReLU activation function in the `relu_forward` function and test your implementation using the following:

[10]:
```
# Test the relu_forward function

x = np.linspace(-0.5, 0.5, num=12).reshape(3, 4)

out, _ = relu_forward(x)
correct_out = np.array([[ 0.,          0.,          0.,          0.,        ],
                        [ 0.,          0.,          0.04545455,  0.13636364,],
                        [ 0.22727273,  0.31818182,  0.40909091,  0.5,       ]])
```

```
# Compare your output with ours. The error should be on the order of e-8
print('Testing relu_forward function:')
print('difference: ', rel_error(out, correct_out))
```

```
Testing relu_forward function:
difference:  4.999999798022158e-08
```

5 ReLU activation: backward

Now implement the backward pass for the ReLU activation function in the `relu_backward` function and test your implementation using numeric gradient checking:

[11]:
```
np.random.seed(231)
x = np.random.randn(10, 10)
dout = np.random.randn(*x.shape)

dx_num = eval_numerical_gradient_array(lambda x: relu_forward(x)[0], x, dout)

_, cache = relu_forward(x)
dx = relu_backward(dout, cache)

# The error should be on the order of e-12
print('Testing relu_backward function:')
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing relu_backward function:
dx error:  3.2756349136310288e-12
```

5.1 Inline Question 1:

We've only asked you to implement ReLU, but there are a number of different activation functions that one could use in neural networks, each with its pros and cons. In particular, an issue commonly seen with activation functions is getting zero (or close to zero) gradient flow during backpropagation. Which of the following activation functions have this problem? If you consider these functions in the one dimensional case, what types of input would lead to this behaviour? 1. Sigmoid 2. ReLU 3. Leaky ReLU

YourAnswer : *Sigmoid* and *ReLU*

For *Sigmoid*, it occurred when the absolute value of the input is very large

For *ReLU*, it occurred if the input is smaller that 0

6 "Sandwich" layers

There are some common patterns of layers that are frequently used in neural nets. For example, affine layers are frequently followed by a ReLU nonlinearity. To make these common patterns easy, we define several convenience layers in the file `cs231n/layer_utils.py`.

For now take a look at the `affine_relu_forward` and `affine_relu_backward` functions, and run the following to numerically gradient check the backward pass:

[12]:
```
from cs231n.layer_utils import affine_relu_forward, affine_relu_backward
np.random.seed(231)
x = np.random.randn(2, 3, 4)
w = np.random.randn(12, 10)
b = np.random.randn(10)
dout = np.random.randn(2, 10)

out, cache = affine_relu_forward(x, w, b)
dx, dw, db = affine_relu_backward(dout, cache)

dx_num = eval_numerical_gradient_array(lambda x: affine_relu_forward(x, w,␣
 ↪b)[0], x, dout)
dw_num = eval_numerical_gradient_array(lambda w: affine_relu_forward(x, w,␣
 ↪b)[0], w, dout)
db_num = eval_numerical_gradient_array(lambda b: affine_relu_forward(x, w,␣
 ↪b)[0], b, dout)

# Relative error should be around e-10 or less
print('Testing affine_relu_forward and affine_relu_backward:')
print('dx error: ', rel_error(dx_num, dx))
print('dw error: ', rel_error(dw_num, dw))
print('db error: ', rel_error(db_num, db))
```

```
Testing affine_relu_forward and affine_relu_backward:
dx error:  2.299579177309368e-11
dw error:  8.162011105764925e-11
db error:  7.826724021458994e-12
```

7 Loss layers: Softmax

Now implement the loss and gradient for softmax in the `softmax_loss` function in `cs231n/layers.py`. These should be similar to what you implemented in `cs231n/classifiers/softmax.py`. Other loss functions (e.g. `svm_loss`) can also be implemented in a modular way, however, it is not required for this assignment.

You can make sure that the implementations are correct by running the following:

[13]:
```
np.random.seed(231)
num_classes, num_inputs = 10, 50
x = 0.001 * np.random.randn(num_inputs, num_classes)
y = np.random.randint(num_classes, size=num_inputs)


dx_num = eval_numerical_gradient(lambda x: softmax_loss(x, y)[0], x,␣
 ↪verbose=False)
```

```
loss, dx = softmax_loss(x, y)

# Test softmax_loss function. Loss should be close to 2.3 and dx error should␣
↪be around e-8
print('\nTesting softmax_loss:')
print('loss: ', loss)
print('dx error: ', rel_error(dx_num, dx))
```

```
Testing softmax_loss:
loss:  2.3025458445007376
dx error:  8.234144091578429e-09
```

8 Two-layer network

Open the file `cs231n/classifiers/fc_net.py` and complete the implementation of the `TwoLayerNet` class. Read through it to make sure you understand the API. You can run the cell below to test your implementation.

[40]:
```
np.random.seed(231)
N, D, H, C = 3, 5, 50, 7
X = np.random.randn(N, D)
y = np.random.randint(C, size=N)

std = 1e-3
model = TwoLayerNet(input_dim=D, hidden_dim=H, num_classes=C, weight_scale=std)

print('Testing initialization ... ')
W1_std = abs(model.params['W1'].std() - std)
b1 = model.params['b1']
W2_std = abs(model.params['W2'].std() - std)
b2 = model.params['b2']
assert W1_std < std / 10, 'First layer weights do not seem right'
assert np.all(b1 == 0), 'First layer biases do not seem right'
assert W2_std < std / 10, 'Second layer weights do not seem right'
assert np.all(b2 == 0), 'Second layer biases do not seem right'

print('Testing test-time forward pass ... ')
model.params['W1'] = np.linspace(-0.7, 0.3, num=D*H).reshape(D, H)
model.params['b1'] = np.linspace(-0.1, 0.9, num=H)
model.params['W2'] = np.linspace(-0.3, 0.4, num=H*C).reshape(H, C)
model.params['b2'] = np.linspace(-0.9, 0.1, num=C)
X = np.linspace(-5.5, 4.5, num=N*D).reshape(D, N).T
scores = model.loss(X)
correct_scores = np.asarray(
  [[11.53165108,  12.2917344,   13.05181771,  13.81190102,  14.57198434, 15.
↪33206765,  16.09215096],
```

```
   [12.05769098,  12.74614105,  13.43459113,  14.1230412,   14.81149128, 15.
↪49994135,  16.18839143],
   [12.58373087,  13.20054771,  13.81736455,  14.43418138,  15.05099822, 15.
↪66781506,  16.2846319 ]])
scores_diff = np.abs(scores - correct_scores).sum()
assert scores_diff < 1e-6, 'Problem with test-time forward pass'

print('Testing training loss (no regularization)')
y = np.asarray([0, 5, 1])
loss, grads = model.loss(X, y)
correct_loss = 3.4702243556
assert abs(loss - correct_loss) < 1e-10, 'Problem with training-time loss'

model.reg = 1.0
loss, grads = model.loss(X, y)
correct_loss = 26.5948426952
assert abs(loss - correct_loss) < 1e-10, 'Problem with regularization loss'

# Errors should be around e-7 or less
for reg in [0.0, 0.7]:
  print('Running numeric gradient check with reg = ', reg)
  model.reg = reg
  loss, grads = model.loss(X, y)

  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=False)
    print('%s relative error: %.2e' % (name, rel_error(grad_num, grads[name])))
```

```
Testing initialization …
Testing test-time forward pass …
Testing training loss (no regularization)
Running numeric gradient check with reg =  0.0
W1 relative error: 1.83e-08
W2 relative error: 3.12e-10
b1 relative error: 9.83e-09
b2 relative error: 4.33e-10
Running numeric gradient check with reg =  0.7
W1 relative error: 2.53e-07
W2 relative error: 2.85e-08
b1 relative error: 1.56e-08
b2 relative error: 7.76e-10
```

9 Solver

Open the file `cs231n/solver.py` and read through it to familiarize yourself with the API. After doing so, use a `Solver` instance to train a `TwoLayerNet` that achieves about `36%` accuracy on the

validation set.

[49]:
```
input_size = 32 * 32 * 3
hidden_size = 50
num_classes = 10
model = TwoLayerNet(input_size, hidden_size, num_classes,reg = 0.
 ↪1,weight_scale=1e-3)
solver = Solver(model,data,lr_decay= 0.95,batch_size =␣
 ↪300,optim_config={'learning_rate':1e-3})
solver.train()
accuracy = solver.check_accuracy(data['X_test'],data['y_test'])
print(f"the accuracy is {accuracy}")
```

```
(Iteration 1 / 1630) loss: 2.310984
(Epoch 0 / 10) train acc: 0.119000; val_acc: 0.122000
(Iteration 11 / 1630) loss: 2.274442
(Iteration 21 / 1630) loss: 2.174410
(Iteration 31 / 1630) loss: 2.114370
(Iteration 41 / 1630) loss: 2.025281
(Iteration 51 / 1630) loss: 2.065399
(Iteration 61 / 1630) loss: 1.949118
(Iteration 71 / 1630) loss: 1.910205
(Iteration 81 / 1630) loss: 1.893891
(Iteration 91 / 1630) loss: 1.897379
(Iteration 101 / 1630) loss: 1.789812
(Iteration 111 / 1630) loss: 1.877988
(Iteration 121 / 1630) loss: 1.830869
(Iteration 131 / 1630) loss: 1.704767
(Iteration 141 / 1630) loss: 1.742059
(Iteration 151 / 1630) loss: 1.744377
(Iteration 161 / 1630) loss: 1.757564
(Epoch 1 / 10) train acc: 0.374000; val_acc: 0.394000
(Iteration 171 / 1630) loss: 1.646577
(Iteration 181 / 1630) loss: 1.668712
(Iteration 191 / 1630) loss: 1.832787
(Iteration 201 / 1630) loss: 1.668828
(Iteration 211 / 1630) loss: 1.663967
(Iteration 221 / 1630) loss: 1.577258
(Iteration 231 / 1630) loss: 1.638241
(Iteration 241 / 1630) loss: 1.650992
(Iteration 251 / 1630) loss: 1.552818
(Iteration 261 / 1630) loss: 1.725863
(Iteration 271 / 1630) loss: 1.628345
(Iteration 281 / 1630) loss: 1.586225
(Iteration 291 / 1630) loss: 1.565382
(Iteration 301 / 1630) loss: 1.557223
(Iteration 311 / 1630) loss: 1.529648
(Iteration 321 / 1630) loss: 1.743042
```

```
(Epoch 2 / 10) train acc: 0.438000; val_acc: 0.440000
(Iteration 331 / 1630) loss: 1.589040
(Iteration 341 / 1630) loss: 1.579619
(Iteration 351 / 1630) loss: 1.659527
(Iteration 361 / 1630) loss: 1.595338
(Iteration 371 / 1630) loss: 1.637105
(Iteration 381 / 1630) loss: 1.632566
(Iteration 391 / 1630) loss: 1.664460
(Iteration 401 / 1630) loss: 1.536499
(Iteration 411 / 1630) loss: 1.492763
(Iteration 421 / 1630) loss: 1.536960
(Iteration 431 / 1630) loss: 1.583611
(Iteration 441 / 1630) loss: 1.543556
(Iteration 451 / 1630) loss: 1.646092
(Iteration 461 / 1630) loss: 1.501522
(Iteration 471 / 1630) loss: 1.515100
(Iteration 481 / 1630) loss: 1.578423
(Epoch 3 / 10) train acc: 0.472000; val_acc: 0.435000
(Iteration 491 / 1630) loss: 1.523895
(Iteration 501 / 1630) loss: 1.509781
(Iteration 511 / 1630) loss: 1.493661
(Iteration 521 / 1630) loss: 1.517172
(Iteration 531 / 1630) loss: 1.583157
(Iteration 541 / 1630) loss: 1.573766
(Iteration 551 / 1630) loss: 1.490475
(Iteration 561 / 1630) loss: 1.463080
(Iteration 571 / 1630) loss: 1.575837
(Iteration 581 / 1630) loss: 1.481432
(Iteration 591 / 1630) loss: 1.444784
(Iteration 601 / 1630) loss: 1.559653
(Iteration 611 / 1630) loss: 1.581134
(Iteration 621 / 1630) loss: 1.424404
(Iteration 631 / 1630) loss: 1.511028
(Iteration 641 / 1630) loss: 1.469513
(Iteration 651 / 1630) loss: 1.429297
(Epoch 4 / 10) train acc: 0.512000; val_acc: 0.475000
(Iteration 661 / 1630) loss: 1.473784
(Iteration 671 / 1630) loss: 1.470584
(Iteration 681 / 1630) loss: 1.453400
(Iteration 691 / 1630) loss: 1.451108
(Iteration 701 / 1630) loss: 1.608367
(Iteration 711 / 1630) loss: 1.512974
(Iteration 721 / 1630) loss: 1.502694
(Iteration 731 / 1630) loss: 1.411843
(Iteration 741 / 1630) loss: 1.409715
(Iteration 751 / 1630) loss: 1.451285
(Iteration 761 / 1630) loss: 1.470511
(Iteration 771 / 1630) loss: 1.422931
```

```
(Iteration 781 / 1630) loss: 1.573840
(Iteration 791 / 1630) loss: 1.506768
(Iteration 801 / 1630) loss: 1.493418
(Iteration 811 / 1630) loss: 1.449198
(Epoch 5 / 10) train acc: 0.489000; val_acc: 0.483000
(Iteration 821 / 1630) loss: 1.423950
(Iteration 831 / 1630) loss: 1.448376
(Iteration 841 / 1630) loss: 1.497959
(Iteration 851 / 1630) loss: 1.464961
(Iteration 861 / 1630) loss: 1.412064
(Iteration 871 / 1630) loss: 1.456512
(Iteration 881 / 1630) loss: 1.457572
(Iteration 891 / 1630) loss: 1.422383
(Iteration 901 / 1630) loss: 1.432962
(Iteration 911 / 1630) loss: 1.468222
(Iteration 921 / 1630) loss: 1.432008
(Iteration 931 / 1630) loss: 1.417985
(Iteration 941 / 1630) loss: 1.490449
(Iteration 951 / 1630) loss: 1.539125
(Iteration 961 / 1630) loss: 1.448792
(Iteration 971 / 1630) loss: 1.409672
(Epoch 6 / 10) train acc: 0.491000; val_acc: 0.477000
(Iteration 981 / 1630) loss: 1.382841
(Iteration 991 / 1630) loss: 1.502837
(Iteration 1001 / 1630) loss: 1.395495
(Iteration 1011 / 1630) loss: 1.536271
(Iteration 1021 / 1630) loss: 1.465353
(Iteration 1031 / 1630) loss: 1.413407
(Iteration 1041 / 1630) loss: 1.427314
(Iteration 1051 / 1630) loss: 1.421837
(Iteration 1061 / 1630) loss: 1.436693
(Iteration 1071 / 1630) loss: 1.483116
(Iteration 1081 / 1630) loss: 1.423017
(Iteration 1091 / 1630) loss: 1.324544
(Iteration 1101 / 1630) loss: 1.423852
(Iteration 1111 / 1630) loss: 1.401659
(Iteration 1121 / 1630) loss: 1.400001
(Iteration 1131 / 1630) loss: 1.488262
(Iteration 1141 / 1630) loss: 1.383627
(Epoch 7 / 10) train acc: 0.508000; val_acc: 0.494000
(Iteration 1151 / 1630) loss: 1.432848
(Iteration 1161 / 1630) loss: 1.409322
(Iteration 1171 / 1630) loss: 1.387017
(Iteration 1181 / 1630) loss: 1.478366
(Iteration 1191 / 1630) loss: 1.402271
(Iteration 1201 / 1630) loss: 1.433992
(Iteration 1211 / 1630) loss: 1.290537
(Iteration 1221 / 1630) loss: 1.314160
```

```
(Iteration 1231 / 1630) loss: 1.368667
(Iteration 1241 / 1630) loss: 1.424674
(Iteration 1251 / 1630) loss: 1.338502
(Iteration 1261 / 1630) loss: 1.530388
(Iteration 1271 / 1630) loss: 1.408469
(Iteration 1281 / 1630) loss: 1.367214
(Iteration 1291 / 1630) loss: 1.397374
(Iteration 1301 / 1630) loss: 1.341585
(Epoch 8 / 10) train acc: 0.529000; val_acc: 0.506000
(Iteration 1311 / 1630) loss: 1.392585
(Iteration 1321 / 1630) loss: 1.366614
(Iteration 1331 / 1630) loss: 1.445738
(Iteration 1341 / 1630) loss: 1.429793
(Iteration 1351 / 1630) loss: 1.320774
(Iteration 1361 / 1630) loss: 1.414557
(Iteration 1371 / 1630) loss: 1.351799
(Iteration 1381 / 1630) loss: 1.389376
(Iteration 1391 / 1630) loss: 1.323562
(Iteration 1401 / 1630) loss: 1.322813
(Iteration 1411 / 1630) loss: 1.471339
(Iteration 1421 / 1630) loss: 1.345387
(Iteration 1431 / 1630) loss: 1.415486
(Iteration 1441 / 1630) loss: 1.332088
(Iteration 1451 / 1630) loss: 1.544153
(Iteration 1461 / 1630) loss: 1.343471
(Epoch 9 / 10) train acc: 0.542000; val_acc: 0.502000
(Iteration 1471 / 1630) loss: 1.447285
(Iteration 1481 / 1630) loss: 1.364077
(Iteration 1491 / 1630) loss: 1.469361
(Iteration 1501 / 1630) loss: 1.389206
(Iteration 1511 / 1630) loss: 1.427344
(Iteration 1521 / 1630) loss: 1.333918
(Iteration 1531 / 1630) loss: 1.416328
(Iteration 1541 / 1630) loss: 1.310864
(Iteration 1551 / 1630) loss: 1.334005
(Iteration 1561 / 1630) loss: 1.381396
(Iteration 1571 / 1630) loss: 1.417839
(Iteration 1581 / 1630) loss: 1.286565
(Iteration 1591 / 1630) loss: 1.434776
(Iteration 1601 / 1630) loss: 1.348510
(Iteration 1611 / 1630) loss: 1.396610
(Iteration 1621 / 1630) loss: 1.338344
(Epoch 10 / 10) train acc: 0.543000; val_acc: 0.502000
the accuracy is 0.497
```

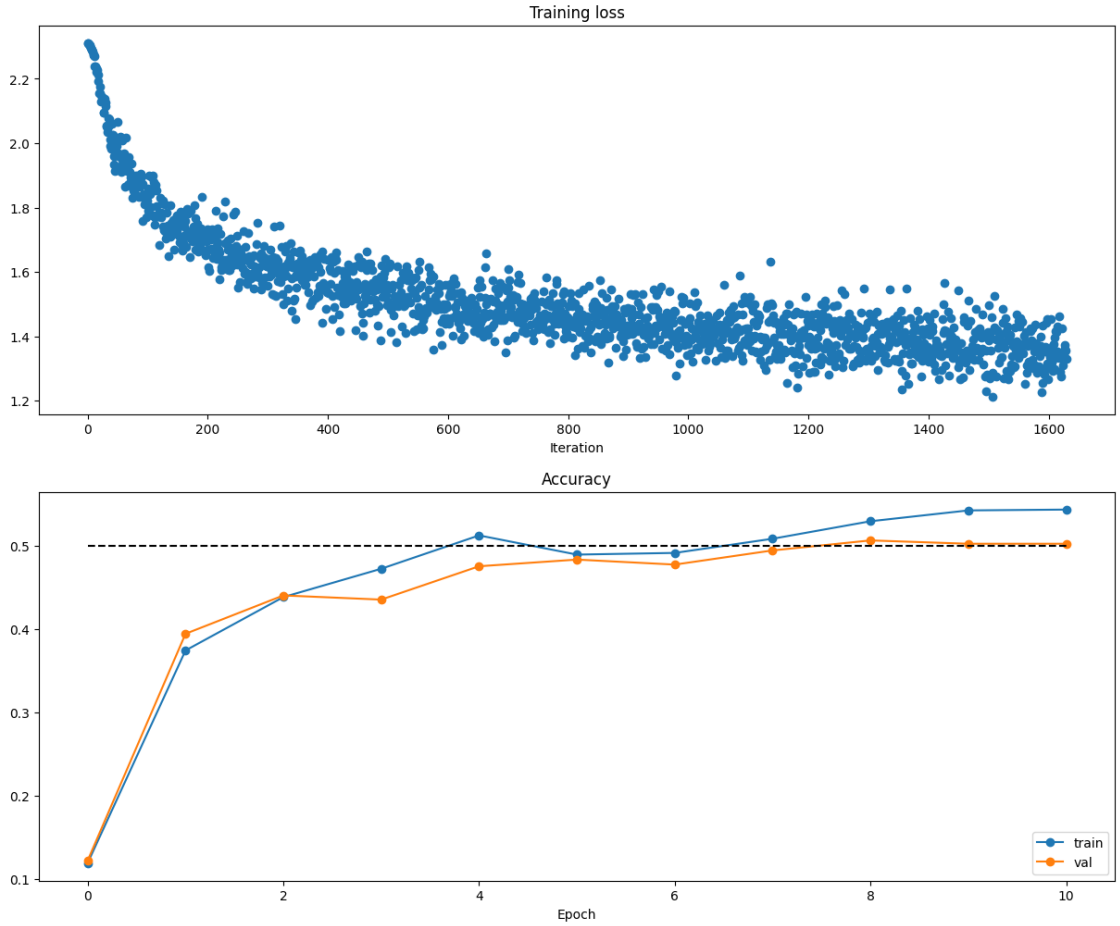
10 Debug the training

With the default parameters we provided above, you should get a validation accuracy of about 0.36 on the validation set. This isn't very good.

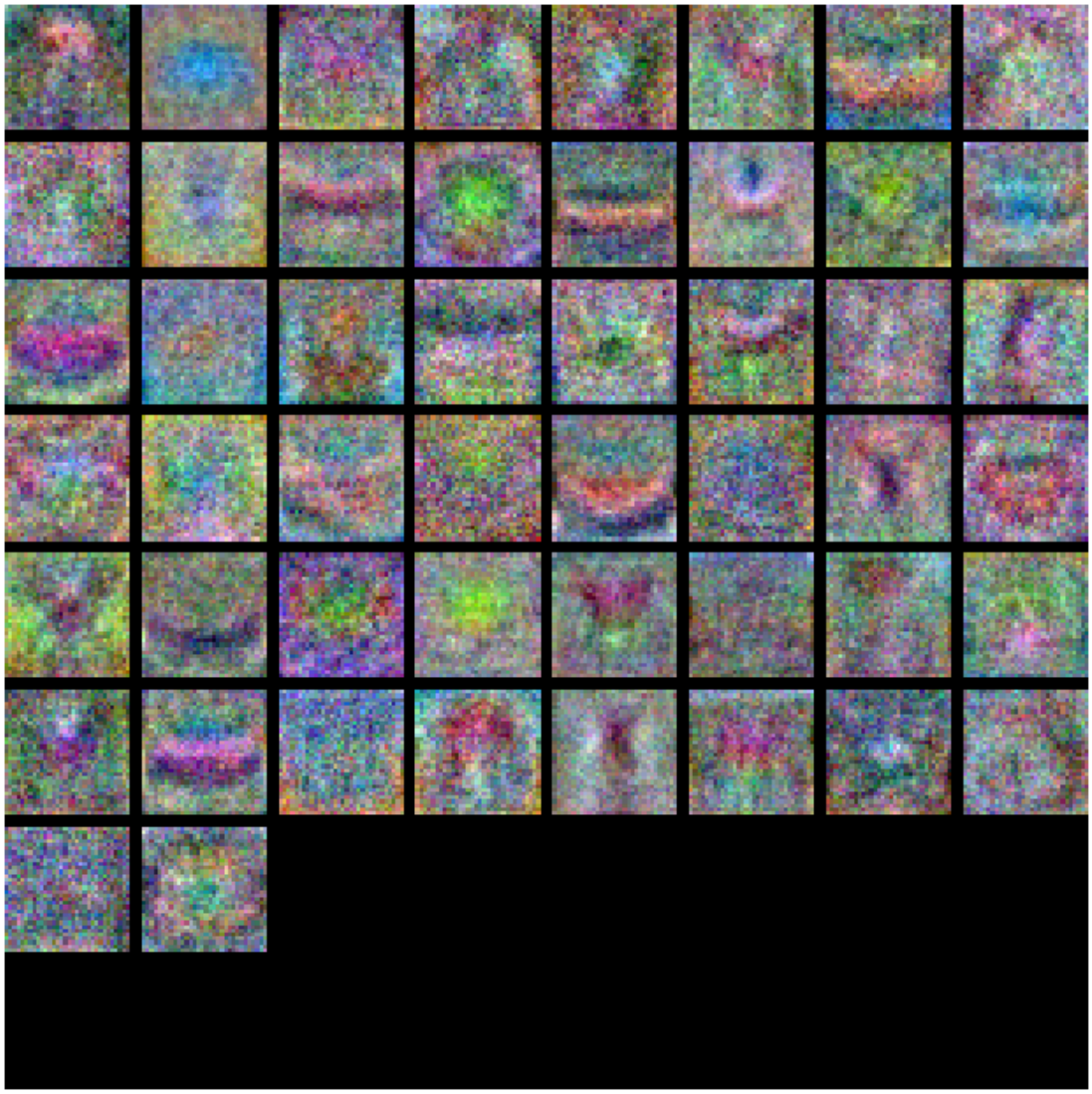
One strategy for getting insight into what's wrong is to plot the loss function and the accuracies on the training and validation sets during optimization.

Another strategy is to visualize the weights that were learned in the first layer of the network. In most neural networks trained on visual data, the first layer weights typically show some visible structure when visualized.

[50]:
```
# Run this cell to visualize training loss and train / val accuracy

plt.subplot(2, 1, 1)
plt.title('Training loss')
plt.plot(solver.loss_history, 'o')
plt.xlabel('Iteration')

plt.subplot(2, 1, 2)
plt.title('Accuracy')
plt.plot(solver.train_acc_history, '-o', label='train')
plt.plot(solver.val_acc_history, '-o', label='val')
plt.plot([0.5] * len(solver.val_acc_history), 'k--')
plt.xlabel('Epoch')
plt.legend(loc='lower right')
plt.gcf().set_size_inches(15, 12)
plt.show()
```



[51]:
```
from cs231n.vis_utils import visualize_grid

# Visualize the weights of the network

def show_net_weights(net):
    W1 = net.params['W1']
    W1 = W1.reshape(3, 32, 32, -1).transpose(3, 1, 2, 0)
    plt.imshow(visualize_grid(W1, padding=3).astype('uint8'))
    plt.gca().axis('off')
    plt.show()

show_net_weights(model)
```



11 Tune your hyperparameters

What's wrong?. Looking at the visualizations above, we see that the loss is decreasing more or less linearly, which seems to suggest that the learning rate may be too low. Moreover, there is no gap between the training and validation accuracy, suggesting that the model we used has low capacity, and that we should increase its size. On the other hand, with a very large model we would expect to see more overfitting, which would manifest itself as a very large gap between the training and validation accuracy.

Tuning. Tuning the hyperparameters and developing intuition for how they affect the final performance is a large part of using Neural Networks, so we want you to get a lot of practice. Below, you should experiment with different values of the various hyperparameters, including hidden layer size, learning rate, numer of training epochs, and regularization strength. You might also consider

tuning the learning rate decay, but you should be able to get good performance using the default value.

Approximate results. You should be aim to achieve a classification accuracy of greater than 48% on the validation set. Our best network gets over 52% on the validation set.

Experiment: You goal in this exercise is to get as good of a result on CIFAR-10 as you can (52% could serve as a reference), with a fully-connected Neural Network. Feel free implement your own techniques (e.g. PCA to reduce dimensionality, or adding dropout, or adding features to the solver, etc.).

[56]:
```
best_model = None
hidden_dim= [100,200]
learing_rate = [1e-3,5e-4]
num_epoch= [10,20]
reg= [0.1,0.25]
best_accuracy = 0
for lr in learing_rate:
  for hidden in hidden_dim:
    for epoch in num_epoch:
      for r in reg:
        model = TwoLayerNet(hidden_dim = hidden,reg =r)
        solver = Solver(model,data,num_epochs = epoch,optim_config =␣
↪{'learning_rate':lr})
        solver.train()
        accuracy = solver.check_accuracy(data['X_val'],data['y_val'])

        print(f"accuracy:{accuracy} with lr:{lr},reg:{r},epoch:
↪{epoch},hidden_dim:{hidden}")
        if accuracy>best_accuracy:
          best_accuracy = accuracy
          best_model = model
```

```
Streaming output truncated to the last 5000 lines.
(Iteration 111 / 9800) loss: 1.859306
(Iteration 121 / 9800) loss: 1.902966
(Iteration 131 / 9800) loss: 1.873426
(Iteration 141 / 9800) loss: 2.050682
(Iteration 151 / 9800) loss: 1.919096
(Iteration 161 / 9800) loss: 1.841012
(Iteration 171 / 9800) loss: 1.835540
(Iteration 181 / 9800) loss: 1.774650
(Iteration 191 / 9800) loss: 1.881383
(Iteration 201 / 9800) loss: 1.825518
(Iteration 211 / 9800) loss: 1.806321
(Iteration 221 / 9800) loss: 1.766442
(Iteration 231 / 9800) loss: 1.919636
(Iteration 241 / 9800) loss: 1.818504
(Iteration 251 / 9800) loss: 1.732610
```

```
(Iteration 261 / 9800) loss: 1.833222
(Iteration 271 / 9800) loss: 1.934611
(Iteration 281 / 9800) loss: 1.713145
(Iteration 291 / 9800) loss: 1.628732
(Iteration 301 / 9800) loss: 1.690760
(Iteration 311 / 9800) loss: 1.849691
(Iteration 321 / 9800) loss: 1.765915
(Iteration 331 / 9800) loss: 1.793818
(Iteration 341 / 9800) loss: 1.591379
(Iteration 351 / 9800) loss: 1.722983
(Iteration 361 / 9800) loss: 1.648344
(Iteration 371 / 9800) loss: 1.740531
(Iteration 381 / 9800) loss: 1.620175
(Iteration 391 / 9800) loss: 1.731573
(Iteration 401 / 9800) loss: 1.766318
(Iteration 411 / 9800) loss: 1.681193
(Iteration 421 / 9800) loss: 1.640505
(Iteration 431 / 9800) loss: 1.680096
(Iteration 441 / 9800) loss: 1.650630
(Iteration 451 / 9800) loss: 1.696586
(Iteration 461 / 9800) loss: 1.834839
(Iteration 471 / 9800) loss: 1.733985
(Iteration 481 / 9800) loss: 1.658363
(Epoch 1 / 20) train acc: 0.435000; val_acc: 0.423000
(Iteration 491 / 9800) loss: 1.551930
(Iteration 501 / 9800) loss: 1.487199
(Iteration 511 / 9800) loss: 1.486123
(Iteration 521 / 9800) loss: 1.483873
(Iteration 531 / 9800) loss: 1.543443
(Iteration 541 / 9800) loss: 1.840254
(Iteration 551 / 9800) loss: 1.681367
(Iteration 561 / 9800) loss: 1.498439
(Iteration 571 / 9800) loss: 1.609452
(Iteration 581 / 9800) loss: 1.391139
(Iteration 591 / 9800) loss: 1.639449
(Iteration 601 / 9800) loss: 1.530070
(Iteration 611 / 9800) loss: 1.610209
(Iteration 621 / 9800) loss: 1.461584
(Iteration 631 / 9800) loss: 1.772098
(Iteration 641 / 9800) loss: 1.601918
(Iteration 651 / 9800) loss: 1.535780
(Iteration 661 / 9800) loss: 1.594366
(Iteration 671 / 9800) loss: 1.620360
(Iteration 681 / 9800) loss: 1.484220
(Iteration 691 / 9800) loss: 1.518095
(Iteration 701 / 9800) loss: 1.381761
(Iteration 711 / 9800) loss: 1.601090
(Iteration 721 / 9800) loss: 1.601197
```

```
(Iteration 731 / 9800) loss: 1.537123
(Iteration 741 / 9800) loss: 1.551184
(Iteration 751 / 9800) loss: 1.670693
(Iteration 761 / 9800) loss: 1.695129
(Iteration 771 / 9800) loss: 1.426745
(Iteration 781 / 9800) loss: 1.491486
(Iteration 791 / 9800) loss: 1.565752
(Iteration 801 / 9800) loss: 1.556517
(Iteration 811 / 9800) loss: 1.703585
(Iteration 821 / 9800) loss: 1.582533
(Iteration 831 / 9800) loss: 1.536261
(Iteration 841 / 9800) loss: 1.501963
(Iteration 851 / 9800) loss: 1.539205
(Iteration 861 / 9800) loss: 1.637370
(Iteration 871 / 9800) loss: 1.589062
(Iteration 881 / 9800) loss: 1.447103
(Iteration 891 / 9800) loss: 1.607870
(Iteration 901 / 9800) loss: 1.606635
(Iteration 911 / 9800) loss: 1.534198
(Iteration 921 / 9800) loss: 1.620942
(Iteration 931 / 9800) loss: 1.660589
(Iteration 941 / 9800) loss: 1.603645
(Iteration 951 / 9800) loss: 1.668274
(Iteration 961 / 9800) loss: 1.485407
(Iteration 971 / 9800) loss: 1.559146
(Epoch 2 / 20) train acc: 0.478000; val_acc: 0.461000
(Iteration 981 / 9800) loss: 1.717009
(Iteration 991 / 9800) loss: 1.468070
(Iteration 1001 / 9800) loss: 1.650650
(Iteration 1011 / 9800) loss: 1.536039
(Iteration 1021 / 9800) loss: 1.460371
(Iteration 1031 / 9800) loss: 1.634521
(Iteration 1041 / 9800) loss: 1.636415
(Iteration 1051 / 9800) loss: 1.379104
(Iteration 1061 / 9800) loss: 1.586156
(Iteration 1071 / 9800) loss: 1.482501
(Iteration 1081 / 9800) loss: 1.516167
(Iteration 1091 / 9800) loss: 1.302179
(Iteration 1101 / 9800) loss: 1.474944
(Iteration 1111 / 9800) loss: 1.330239
(Iteration 1121 / 9800) loss: 1.530353
(Iteration 1131 / 9800) loss: 1.569808
(Iteration 1141 / 9800) loss: 1.425246
(Iteration 1151 / 9800) loss: 1.317548
(Iteration 1161 / 9800) loss: 1.379487
(Iteration 1171 / 9800) loss: 1.385486
(Iteration 1181 / 9800) loss: 1.439790
(Iteration 1191 / 9800) loss: 1.304208
```

```
(Iteration 1201 / 9800) loss: 1.577054
(Iteration 1211 / 9800) loss: 1.524416
(Iteration 1221 / 9800) loss: 1.432658
(Iteration 1231 / 9800) loss: 1.448316
(Iteration 1241 / 9800) loss: 1.323765
(Iteration 1251 / 9800) loss: 1.459412
(Iteration 1261 / 9800) loss: 1.446725
(Iteration 1271 / 9800) loss: 1.436923
(Iteration 1281 / 9800) loss: 1.415162
(Iteration 1291 / 9800) loss: 1.431047
(Iteration 1301 / 9800) loss: 1.496644
(Iteration 1311 / 9800) loss: 1.484421
(Iteration 1321 / 9800) loss: 1.545825
(Iteration 1331 / 9800) loss: 1.484362
(Iteration 1341 / 9800) loss: 1.512854
(Iteration 1351 / 9800) loss: 1.449398
(Iteration 1361 / 9800) loss: 1.390551
(Iteration 1371 / 9800) loss: 1.500854
(Iteration 1381 / 9800) loss: 1.436373
(Iteration 1391 / 9800) loss: 1.303892
(Iteration 1401 / 9800) loss: 1.537160
(Iteration 1411 / 9800) loss: 1.459754
(Iteration 1421 / 9800) loss: 1.431537
(Iteration 1431 / 9800) loss: 1.383021
(Iteration 1441 / 9800) loss: 1.472365
(Iteration 1451 / 9800) loss: 1.425880
(Iteration 1461 / 9800) loss: 1.466827
(Epoch 3 / 20) train acc: 0.500000; val_acc: 0.484000
(Iteration 1471 / 9800) loss: 1.516672
(Iteration 1481 / 9800) loss: 1.513065
(Iteration 1491 / 9800) loss: 1.434597
(Iteration 1501 / 9800) loss: 1.473981
(Iteration 1511 / 9800) loss: 1.452166
(Iteration 1521 / 9800) loss: 1.465766
(Iteration 1531 / 9800) loss: 1.464789
(Iteration 1541 / 9800) loss: 1.591205
(Iteration 1551 / 9800) loss: 1.612078
(Iteration 1561 / 9800) loss: 1.488941
(Iteration 1571 / 9800) loss: 1.332533
(Iteration 1581 / 9800) loss: 1.350934
(Iteration 1591 / 9800) loss: 1.563142
(Iteration 1601 / 9800) loss: 1.369818
(Iteration 1611 / 9800) loss: 1.478330
(Iteration 1621 / 9800) loss: 1.453062
(Iteration 1631 / 9800) loss: 1.455596
(Iteration 1641 / 9800) loss: 1.409500
(Iteration 1651 / 9800) loss: 1.506576
(Iteration 1661 / 9800) loss: 1.657248
```

```
(Iteration 1671 / 9800) loss: 1.474420
(Iteration 1681 / 9800) loss: 1.608975
(Iteration 1691 / 9800) loss: 1.426653
(Iteration 1701 / 9800) loss: 1.383644
(Iteration 1711 / 9800) loss: 1.416427
(Iteration 1721 / 9800) loss: 1.322089
(Iteration 1731 / 9800) loss: 1.629062
(Iteration 1741 / 9800) loss: 1.387924
(Iteration 1751 / 9800) loss: 1.444218
(Iteration 1761 / 9800) loss: 1.514147
(Iteration 1771 / 9800) loss: 1.277978
(Iteration 1781 / 9800) loss: 1.725405
(Iteration 1791 / 9800) loss: 1.365070
(Iteration 1801 / 9800) loss: 1.538013
(Iteration 1811 / 9800) loss: 1.366463
(Iteration 1821 / 9800) loss: 1.335990
(Iteration 1831 / 9800) loss: 1.377306
(Iteration 1841 / 9800) loss: 1.449682
(Iteration 1851 / 9800) loss: 1.285742
(Iteration 1861 / 9800) loss: 1.552686
(Iteration 1871 / 9800) loss: 1.455627
(Iteration 1881 / 9800) loss: 1.493815
(Iteration 1891 / 9800) loss: 1.339172
(Iteration 1901 / 9800) loss: 1.375126
(Iteration 1911 / 9800) loss: 1.440468
(Iteration 1921 / 9800) loss: 1.523631
(Iteration 1931 / 9800) loss: 1.353580
(Iteration 1941 / 9800) loss: 1.385465
(Iteration 1951 / 9800) loss: 1.266459
(Epoch 4 / 20) train acc: 0.527000; val_acc: 0.494000
(Iteration 1961 / 9800) loss: 1.430796
(Iteration 1971 / 9800) loss: 1.328751
(Iteration 1981 / 9800) loss: 1.353617
(Iteration 1991 / 9800) loss: 1.503648
(Iteration 2001 / 9800) loss: 1.553359
(Iteration 2011 / 9800) loss: 1.382199
(Iteration 2021 / 9800) loss: 1.383237
(Iteration 2031 / 9800) loss: 1.444022
(Iteration 2041 / 9800) loss: 1.570445
(Iteration 2051 / 9800) loss: 1.111007
(Iteration 2061 / 9800) loss: 1.510676
(Iteration 2071 / 9800) loss: 1.506419
(Iteration 2081 / 9800) loss: 1.172853
(Iteration 2091 / 9800) loss: 1.551758
(Iteration 2101 / 9800) loss: 1.249072
(Iteration 2111 / 9800) loss: 1.267848
(Iteration 2121 / 9800) loss: 1.395460
(Iteration 2131 / 9800) loss: 1.446181
```

```
(Iteration 2141 / 9800) loss: 1.154070
(Iteration 2151 / 9800) loss: 1.230891
(Iteration 2161 / 9800) loss: 1.538543
(Iteration 2171 / 9800) loss: 1.648601
(Iteration 2181 / 9800) loss: 1.369617
(Iteration 2191 / 9800) loss: 1.338229
(Iteration 2201 / 9800) loss: 1.580975
(Iteration 2211 / 9800) loss: 1.398617
(Iteration 2221 / 9800) loss: 1.407712
(Iteration 2231 / 9800) loss: 1.404096
(Iteration 2241 / 9800) loss: 1.265394
(Iteration 2251 / 9800) loss: 1.521466
(Iteration 2261 / 9800) loss: 1.482832
(Iteration 2271 / 9800) loss: 1.459192
(Iteration 2281 / 9800) loss: 1.224127
(Iteration 2291 / 9800) loss: 1.234952
(Iteration 2301 / 9800) loss: 1.464845
(Iteration 2311 / 9800) loss: 1.241068
(Iteration 2321 / 9800) loss: 1.307686
(Iteration 2331 / 9800) loss: 1.369817
(Iteration 2341 / 9800) loss: 1.447286
(Iteration 2351 / 9800) loss: 1.465731
(Iteration 2361 / 9800) loss: 1.284132
(Iteration 2371 / 9800) loss: 1.082916
(Iteration 2381 / 9800) loss: 1.404063
(Iteration 2391 / 9800) loss: 1.390232
(Iteration 2401 / 9800) loss: 1.467766
(Iteration 2411 / 9800) loss: 1.408084
(Iteration 2421 / 9800) loss: 1.194663
(Iteration 2431 / 9800) loss: 1.349136
(Iteration 2441 / 9800) loss: 1.341204
(Epoch 5 / 20) train acc: 0.534000; val_acc: 0.477000
(Iteration 2451 / 9800) loss: 1.238590
(Iteration 2461 / 9800) loss: 1.487618
(Iteration 2471 / 9800) loss: 1.322762
(Iteration 2481 / 9800) loss: 1.313747
(Iteration 2491 / 9800) loss: 1.315892
(Iteration 2501 / 9800) loss: 1.256060
(Iteration 2511 / 9800) loss: 1.402923
(Iteration 2521 / 9800) loss: 1.378678
(Iteration 2531 / 9800) loss: 1.476212
(Iteration 2541 / 9800) loss: 1.338266
(Iteration 2551 / 9800) loss: 1.242741
(Iteration 2561 / 9800) loss: 1.550895
(Iteration 2571 / 9800) loss: 1.484961
(Iteration 2581 / 9800) loss: 1.261228
(Iteration 2591 / 9800) loss: 1.440332
(Iteration 2601 / 9800) loss: 1.457925
```

```
(Iteration 2611 / 9800) loss: 1.498935
(Iteration 2621 / 9800) loss: 1.308912
(Iteration 2631 / 9800) loss: 1.418008
(Iteration 2641 / 9800) loss: 1.298436
(Iteration 2651 / 9800) loss: 1.449035
(Iteration 2661 / 9800) loss: 1.409313
(Iteration 2671 / 9800) loss: 1.361117
(Iteration 2681 / 9800) loss: 1.390424
(Iteration 2691 / 9800) loss: 1.263274
(Iteration 2701 / 9800) loss: 1.455232
(Iteration 2711 / 9800) loss: 1.501952
(Iteration 2721 / 9800) loss: 1.323810
(Iteration 2731 / 9800) loss: 1.308861
(Iteration 2741 / 9800) loss: 1.175667
(Iteration 2751 / 9800) loss: 1.450206
(Iteration 2761 / 9800) loss: 1.451059
(Iteration 2771 / 9800) loss: 1.299135
(Iteration 2781 / 9800) loss: 1.377232
(Iteration 2791 / 9800) loss: 1.308367
(Iteration 2801 / 9800) loss: 1.353720
(Iteration 2811 / 9800) loss: 1.385486
(Iteration 2821 / 9800) loss: 1.477699
(Iteration 2831 / 9800) loss: 1.431889
(Iteration 2841 / 9800) loss: 1.375767
(Iteration 2851 / 9800) loss: 1.391721
(Iteration 2861 / 9800) loss: 1.330296
(Iteration 2871 / 9800) loss: 1.565131
(Iteration 2881 / 9800) loss: 1.387364
(Iteration 2891 / 9800) loss: 1.424351
(Iteration 2901 / 9800) loss: 1.488912
(Iteration 2911 / 9800) loss: 1.377173
(Iteration 2921 / 9800) loss: 1.221432
(Iteration 2931 / 9800) loss: 1.361010
(Epoch 6 / 20) train acc: 0.568000; val_acc: 0.516000
(Iteration 2941 / 9800) loss: 1.124220
(Iteration 2951 / 9800) loss: 1.210487
(Iteration 2961 / 9800) loss: 1.108295
(Iteration 2971 / 9800) loss: 1.344690
(Iteration 2981 / 9800) loss: 1.466234
(Iteration 2991 / 9800) loss: 1.418568
(Iteration 3001 / 9800) loss: 1.253488
(Iteration 3011 / 9800) loss: 1.455224
(Iteration 3021 / 9800) loss: 1.319939
(Iteration 3031 / 9800) loss: 1.531131
(Iteration 3041 / 9800) loss: 1.251319
(Iteration 3051 / 9800) loss: 1.192016
(Iteration 3061 / 9800) loss: 1.367645
(Iteration 3071 / 9800) loss: 1.239642
```

```
(Iteration 3081 / 9800) loss: 1.322669
(Iteration 3091 / 9800) loss: 1.378711
(Iteration 3101 / 9800) loss: 1.405274
(Iteration 3111 / 9800) loss: 1.477485
(Iteration 3121 / 9800) loss: 1.434640
(Iteration 3131 / 9800) loss: 1.529540
(Iteration 3141 / 9800) loss: 1.133598
(Iteration 3151 / 9800) loss: 1.376755
(Iteration 3161 / 9800) loss: 1.485785
(Iteration 3171 / 9800) loss: 1.367730
(Iteration 3181 / 9800) loss: 1.303890
(Iteration 3191 / 9800) loss: 1.280484
(Iteration 3201 / 9800) loss: 1.436692
(Iteration 3211 / 9800) loss: 1.200331
(Iteration 3221 / 9800) loss: 1.442573
(Iteration 3231 / 9800) loss: 1.308644
(Iteration 3241 / 9800) loss: 1.262738
(Iteration 3251 / 9800) loss: 1.173514
(Iteration 3261 / 9800) loss: 1.239333
(Iteration 3271 / 9800) loss: 1.093483
(Iteration 3281 / 9800) loss: 1.248006
(Iteration 3291 / 9800) loss: 1.472534
(Iteration 3301 / 9800) loss: 1.116851
(Iteration 3311 / 9800) loss: 1.371549
(Iteration 3321 / 9800) loss: 1.169072
(Iteration 3331 / 9800) loss: 1.348320
(Iteration 3341 / 9800) loss: 1.379882
(Iteration 3351 / 9800) loss: 1.214454
(Iteration 3361 / 9800) loss: 1.451259
(Iteration 3371 / 9800) loss: 1.363111
(Iteration 3381 / 9800) loss: 1.070568
(Iteration 3391 / 9800) loss: 1.388270
(Iteration 3401 / 9800) loss: 1.308446
(Iteration 3411 / 9800) loss: 1.204464
(Iteration 3421 / 9800) loss: 1.277663
(Epoch 7 / 20) train acc: 0.554000; val_acc: 0.499000
(Iteration 3431 / 9800) loss: 1.265260
(Iteration 3441 / 9800) loss: 1.105631
(Iteration 3451 / 9800) loss: 1.447103
(Iteration 3461 / 9800) loss: 1.240203
(Iteration 3471 / 9800) loss: 1.348289
(Iteration 3481 / 9800) loss: 1.491118
(Iteration 3491 / 9800) loss: 1.292794
(Iteration 3501 / 9800) loss: 1.225492
(Iteration 3511 / 9800) loss: 1.317333
(Iteration 3521 / 9800) loss: 1.183270
(Iteration 3531 / 9800) loss: 1.400173
(Iteration 3541 / 9800) loss: 1.227089
```

```
(Iteration 3551 / 9800) loss: 1.204329
(Iteration 3561 / 9800) loss: 1.111228
(Iteration 3571 / 9800) loss: 1.291161
(Iteration 3581 / 9800) loss: 1.317167
(Iteration 3591 / 9800) loss: 1.213068
(Iteration 3601 / 9800) loss: 1.336096
(Iteration 3611 / 9800) loss: 1.331881
(Iteration 3621 / 9800) loss: 1.295048
(Iteration 3631 / 9800) loss: 1.290177
(Iteration 3641 / 9800) loss: 1.635210
(Iteration 3651 / 9800) loss: 1.270370
(Iteration 3661 / 9800) loss: 1.232828
(Iteration 3671 / 9800) loss: 1.290740
(Iteration 3681 / 9800) loss: 1.571750
(Iteration 3691 / 9800) loss: 1.413906
(Iteration 3701 / 9800) loss: 1.444019
(Iteration 3711 / 9800) loss: 1.144575
(Iteration 3721 / 9800) loss: 1.319900
(Iteration 3731 / 9800) loss: 1.147808
(Iteration 3741 / 9800) loss: 1.409684
(Iteration 3751 / 9800) loss: 1.321289
(Iteration 3761 / 9800) loss: 1.272193
(Iteration 3771 / 9800) loss: 1.262258
(Iteration 3781 / 9800) loss: 1.250358
(Iteration 3791 / 9800) loss: 1.426892
(Iteration 3801 / 9800) loss: 1.115772
(Iteration 3811 / 9800) loss: 1.259586
(Iteration 3821 / 9800) loss: 1.202898
(Iteration 3831 / 9800) loss: 1.323387
(Iteration 3841 / 9800) loss: 1.287299
(Iteration 3851 / 9800) loss: 1.274589
(Iteration 3861 / 9800) loss: 1.221396
(Iteration 3871 / 9800) loss: 1.103430
(Iteration 3881 / 9800) loss: 1.205929
(Iteration 3891 / 9800) loss: 1.142024
(Iteration 3901 / 9800) loss: 1.208812
(Iteration 3911 / 9800) loss: 1.351023
(Epoch 8 / 20) train acc: 0.570000; val_acc: 0.489000
(Iteration 3921 / 9800) loss: 1.340649
(Iteration 3931 / 9800) loss: 1.482445
(Iteration 3941 / 9800) loss: 1.172239
(Iteration 3951 / 9800) loss: 1.098627
(Iteration 3961 / 9800) loss: 1.316910
(Iteration 3971 / 9800) loss: 1.246308
(Iteration 3981 / 9800) loss: 1.113683
(Iteration 3991 / 9800) loss: 1.346381
(Iteration 4001 / 9800) loss: 1.246246
(Iteration 4011 / 9800) loss: 1.436453
```

```
(Iteration 4021 / 9800) loss: 1.195204
(Iteration 4031 / 9800) loss: 1.223049
(Iteration 4041 / 9800) loss: 1.391871
(Iteration 4051 / 9800) loss: 1.227985
(Iteration 4061 / 9800) loss: 1.251655
(Iteration 4071 / 9800) loss: 1.368620
(Iteration 4081 / 9800) loss: 1.158264
(Iteration 4091 / 9800) loss: 1.250501
(Iteration 4101 / 9800) loss: 1.356396
(Iteration 4111 / 9800) loss: 1.382272
(Iteration 4121 / 9800) loss: 1.344224
(Iteration 4131 / 9800) loss: 1.129795
(Iteration 4141 / 9800) loss: 1.168583
(Iteration 4151 / 9800) loss: 1.463190
(Iteration 4161 / 9800) loss: 1.149612
(Iteration 4171 / 9800) loss: 1.261487
(Iteration 4181 / 9800) loss: 1.373950
(Iteration 4191 / 9800) loss: 1.359325
(Iteration 4201 / 9800) loss: 1.341008
(Iteration 4211 / 9800) loss: 1.181644
(Iteration 4221 / 9800) loss: 1.207493
(Iteration 4231 / 9800) loss: 1.140557
(Iteration 4241 / 9800) loss: 1.193472
(Iteration 4251 / 9800) loss: 1.306498
(Iteration 4261 / 9800) loss: 1.202398
(Iteration 4271 / 9800) loss: 1.389782
(Iteration 4281 / 9800) loss: 1.397883
(Iteration 4291 / 9800) loss: 0.993191
(Iteration 4301 / 9800) loss: 1.122582
(Iteration 4311 / 9800) loss: 1.361873
(Iteration 4321 / 9800) loss: 1.323249
(Iteration 4331 / 9800) loss: 1.234486
(Iteration 4341 / 9800) loss: 1.197518
(Iteration 4351 / 9800) loss: 1.109643
(Iteration 4361 / 9800) loss: 1.337309
(Iteration 4371 / 9800) loss: 1.227759
(Iteration 4381 / 9800) loss: 1.232752
(Iteration 4391 / 9800) loss: 1.180649
(Iteration 4401 / 9800) loss: 1.310640
(Epoch 9 / 20) train acc: 0.588000; val_acc: 0.529000
(Iteration 4411 / 9800) loss: 1.132755
(Iteration 4421 / 9800) loss: 1.334174
(Iteration 4431 / 9800) loss: 1.150635
(Iteration 4441 / 9800) loss: 1.225917
(Iteration 4451 / 9800) loss: 1.330040
(Iteration 4461 / 9800) loss: 1.293681
(Iteration 4471 / 9800) loss: 1.306493
(Iteration 4481 / 9800) loss: 1.234087
```

```
(Iteration 4491 / 9800) loss: 1.470771
(Iteration 4501 / 9800) loss: 1.304698
(Iteration 4511 / 9800) loss: 1.138892
(Iteration 4521 / 9800) loss: 1.171207
(Iteration 4531 / 9800) loss: 1.110881
(Iteration 4541 / 9800) loss: 1.302001
(Iteration 4551 / 9800) loss: 1.178475
(Iteration 4561 / 9800) loss: 1.294712
(Iteration 4571 / 9800) loss: 1.253740
(Iteration 4581 / 9800) loss: 1.205018
(Iteration 4591 / 9800) loss: 1.258289
(Iteration 4601 / 9800) loss: 1.092552
(Iteration 4611 / 9800) loss: 1.218622
(Iteration 4621 / 9800) loss: 1.325522
(Iteration 4631 / 9800) loss: 1.181684
(Iteration 4641 / 9800) loss: 1.206121
(Iteration 4651 / 9800) loss: 1.287179
(Iteration 4661 / 9800) loss: 1.398605
(Iteration 4671 / 9800) loss: 1.200945
(Iteration 4681 / 9800) loss: 1.284110
(Iteration 4691 / 9800) loss: 1.209725
(Iteration 4701 / 9800) loss: 1.299160
(Iteration 4711 / 9800) loss: 1.232947
(Iteration 4721 / 9800) loss: 1.218236
(Iteration 4731 / 9800) loss: 1.369213
(Iteration 4741 / 9800) loss: 1.305322
(Iteration 4751 / 9800) loss: 1.372437
(Iteration 4761 / 9800) loss: 1.309047
(Iteration 4771 / 9800) loss: 1.310153
(Iteration 4781 / 9800) loss: 1.232830
(Iteration 4791 / 9800) loss: 1.038804
(Iteration 4801 / 9800) loss: 1.147334
(Iteration 4811 / 9800) loss: 1.159866
(Iteration 4821 / 9800) loss: 1.342661
(Iteration 4831 / 9800) loss: 1.278787
(Iteration 4841 / 9800) loss: 1.185809
(Iteration 4851 / 9800) loss: 1.282346
(Iteration 4861 / 9800) loss: 1.276942
(Iteration 4871 / 9800) loss: 1.290941
(Iteration 4881 / 9800) loss: 1.277823
(Iteration 4891 / 9800) loss: 1.332933
(Epoch 10 / 20) train acc: 0.585000; val_acc: 0.522000
(Iteration 4901 / 9800) loss: 1.128041
(Iteration 4911 / 9800) loss: 1.234461
(Iteration 4921 / 9800) loss: 1.217363
(Iteration 4931 / 9800) loss: 1.378439
(Iteration 4941 / 9800) loss: 1.280626
(Iteration 4951 / 9800) loss: 1.221554
```

```
(Iteration 4961 / 9800) loss: 1.220751
(Iteration 4971 / 9800) loss: 1.255964
(Iteration 4981 / 9800) loss: 1.215384
(Iteration 4991 / 9800) loss: 1.193846
(Iteration 5001 / 9800) loss: 1.232005
(Iteration 5011 / 9800) loss: 1.094084
(Iteration 5021 / 9800) loss: 1.347038
(Iteration 5031 / 9800) loss: 1.292788
(Iteration 5041 / 9800) loss: 1.310580
(Iteration 5051 / 9800) loss: 1.243118
(Iteration 5061 / 9800) loss: 1.211071
(Iteration 5071 / 9800) loss: 1.253376
(Iteration 5081 / 9800) loss: 1.165668
(Iteration 5091 / 9800) loss: 1.340137
(Iteration 5101 / 9800) loss: 1.253616
(Iteration 5111 / 9800) loss: 1.352731
(Iteration 5121 / 9800) loss: 1.088432
(Iteration 5131 / 9800) loss: 1.144113
(Iteration 5141 / 9800) loss: 1.454227
(Iteration 5151 / 9800) loss: 1.137697
(Iteration 5161 / 9800) loss: 1.212348
(Iteration 5171 / 9800) loss: 1.175050
(Iteration 5181 / 9800) loss: 1.092013
(Iteration 5191 / 9800) loss: 1.260445
(Iteration 5201 / 9800) loss: 1.092102
(Iteration 5211 / 9800) loss: 1.098005
(Iteration 5221 / 9800) loss: 1.448632
(Iteration 5231 / 9800) loss: 1.364030
(Iteration 5241 / 9800) loss: 1.082312
(Iteration 5251 / 9800) loss: 1.243904
(Iteration 5261 / 9800) loss: 1.189926
(Iteration 5271 / 9800) loss: 1.195829
(Iteration 5281 / 9800) loss: 1.245489
(Iteration 5291 / 9800) loss: 1.469221
(Iteration 5301 / 9800) loss: 1.248197
(Iteration 5311 / 9800) loss: 1.079648
(Iteration 5321 / 9800) loss: 1.161706
(Iteration 5331 / 9800) loss: 1.029096
(Iteration 5341 / 9800) loss: 1.202762
(Iteration 5351 / 9800) loss: 1.250783
(Iteration 5361 / 9800) loss: 1.248661
(Iteration 5371 / 9800) loss: 1.271368
(Iteration 5381 / 9800) loss: 1.187898
(Epoch 11 / 20) train acc: 0.594000; val_acc: 0.503000
(Iteration 5391 / 9800) loss: 1.126689
(Iteration 5401 / 9800) loss: 1.302080
(Iteration 5411 / 9800) loss: 1.302556
(Iteration 5421 / 9800) loss: 1.379084
```

```
(Iteration 5431 / 9800) loss: 1.237184
(Iteration 5441 / 9800) loss: 1.318654
(Iteration 5451 / 9800) loss: 1.156245
(Iteration 5461 / 9800) loss: 1.233035
(Iteration 5471 / 9800) loss: 1.130268
(Iteration 5481 / 9800) loss: 1.079014
(Iteration 5491 / 9800) loss: 1.154506
(Iteration 5501 / 9800) loss: 1.203723
(Iteration 5511 / 9800) loss: 1.053416
(Iteration 5521 / 9800) loss: 1.353062
(Iteration 5531 / 9800) loss: 1.329028
(Iteration 5541 / 9800) loss: 1.322376
(Iteration 5551 / 9800) loss: 1.235457
(Iteration 5561 / 9800) loss: 1.202923
(Iteration 5571 / 9800) loss: 1.219826
(Iteration 5581 / 9800) loss: 1.066312
(Iteration 5591 / 9800) loss: 1.290887
(Iteration 5601 / 9800) loss: 1.509687
(Iteration 5611 / 9800) loss: 1.174644
(Iteration 5621 / 9800) loss: 1.198633
(Iteration 5631 / 9800) loss: 1.161798
(Iteration 5641 / 9800) loss: 1.111325
(Iteration 5651 / 9800) loss: 1.208680
(Iteration 5661 / 9800) loss: 1.144597
(Iteration 5671 / 9800) loss: 1.129461
(Iteration 5681 / 9800) loss: 1.215909
(Iteration 5691 / 9800) loss: 1.199573
(Iteration 5701 / 9800) loss: 1.335735
(Iteration 5711 / 9800) loss: 1.130259
(Iteration 5721 / 9800) loss: 1.231789
(Iteration 5731 / 9800) loss: 1.194728
(Iteration 5741 / 9800) loss: 1.149908
(Iteration 5751 / 9800) loss: 1.365665
(Iteration 5761 / 9800) loss: 1.298278
(Iteration 5771 / 9800) loss: 1.220009
(Iteration 5781 / 9800) loss: 1.323466
(Iteration 5791 / 9800) loss: 1.250985
(Iteration 5801 / 9800) loss: 1.162646
(Iteration 5811 / 9800) loss: 1.198116
(Iteration 5821 / 9800) loss: 1.368432
(Iteration 5831 / 9800) loss: 1.321685
(Iteration 5841 / 9800) loss: 1.227707
(Iteration 5851 / 9800) loss: 1.282724
(Iteration 5861 / 9800) loss: 1.087803
(Iteration 5871 / 9800) loss: 1.100880
(Epoch 12 / 20) train acc: 0.556000; val_acc: 0.530000
(Iteration 5881 / 9800) loss: 1.350707
(Iteration 5891 / 9800) loss: 1.185918
```

```
(Iteration 5901 / 9800) loss: 1.088045
(Iteration 5911 / 9800) loss: 1.057007
(Iteration 5921 / 9800) loss: 1.151811
(Iteration 5931 / 9800) loss: 1.171113
(Iteration 5941 / 9800) loss: 1.138338
(Iteration 5951 / 9800) loss: 0.977485
(Iteration 5961 / 9800) loss: 1.167696
(Iteration 5971 / 9800) loss: 1.281086
(Iteration 5981 / 9800) loss: 1.505327
(Iteration 5991 / 9800) loss: 1.322520
(Iteration 6001 / 9800) loss: 1.307399
(Iteration 6011 / 9800) loss: 1.158671
(Iteration 6021 / 9800) loss: 1.319742
(Iteration 6031 / 9800) loss: 1.229231
(Iteration 6041 / 9800) loss: 1.189138
(Iteration 6051 / 9800) loss: 1.118470
(Iteration 6061 / 9800) loss: 1.202317
(Iteration 6071 / 9800) loss: 1.168209
(Iteration 6081 / 9800) loss: 1.346966
(Iteration 6091 / 9800) loss: 1.238955
(Iteration 6101 / 9800) loss: 1.192541
(Iteration 6111 / 9800) loss: 1.316995
(Iteration 6121 / 9800) loss: 0.961350
(Iteration 6131 / 9800) loss: 1.322005
(Iteration 6141 / 9800) loss: 1.304089
(Iteration 6151 / 9800) loss: 1.398993
(Iteration 6161 / 9800) loss: 1.185933
(Iteration 6171 / 9800) loss: 1.121295
(Iteration 6181 / 9800) loss: 1.450525
(Iteration 6191 / 9800) loss: 1.225696
(Iteration 6201 / 9800) loss: 1.278563
(Iteration 6211 / 9800) loss: 1.256292
(Iteration 6221 / 9800) loss: 1.066547
(Iteration 6231 / 9800) loss: 1.273398
(Iteration 6241 / 9800) loss: 1.190287
(Iteration 6251 / 9800) loss: 1.307447
(Iteration 6261 / 9800) loss: 1.106613
(Iteration 6271 / 9800) loss: 1.223677
(Iteration 6281 / 9800) loss: 1.106461
(Iteration 6291 / 9800) loss: 1.227757
(Iteration 6301 / 9800) loss: 1.203883
(Iteration 6311 / 9800) loss: 1.240472
(Iteration 6321 / 9800) loss: 1.457377
(Iteration 6331 / 9800) loss: 1.463366
(Iteration 6341 / 9800) loss: 1.185283
(Iteration 6351 / 9800) loss: 1.275699
(Iteration 6361 / 9800) loss: 1.217473
(Epoch 13 / 20) train acc: 0.611000; val_acc: 0.508000
```

```
(Iteration 6371 / 9800) loss: 1.142292
(Iteration 6381 / 9800) loss: 1.284978
(Iteration 6391 / 9800) loss: 1.199467
(Iteration 6401 / 9800) loss: 1.122270
(Iteration 6411 / 9800) loss: 1.096097
(Iteration 6421 / 9800) loss: 1.288452
(Iteration 6431 / 9800) loss: 1.384591
(Iteration 6441 / 9800) loss: 1.186642
(Iteration 6451 / 9800) loss: 1.379757
(Iteration 6461 / 9800) loss: 1.376469
(Iteration 6471 / 9800) loss: 1.167542
(Iteration 6481 / 9800) loss: 1.146135
(Iteration 6491 / 9800) loss: 1.302456
(Iteration 6501 / 9800) loss: 1.278541
(Iteration 6511 / 9800) loss: 1.157496
(Iteration 6521 / 9800) loss: 1.286895
(Iteration 6531 / 9800) loss: 1.066626
(Iteration 6541 / 9800) loss: 1.004750
(Iteration 6551 / 9800) loss: 1.124553
(Iteration 6561 / 9800) loss: 1.195914
(Iteration 6571 / 9800) loss: 1.371372
(Iteration 6581 / 9800) loss: 1.266750
(Iteration 6591 / 9800) loss: 1.108644
(Iteration 6601 / 9800) loss: 1.276607
(Iteration 6611 / 9800) loss: 1.189821
(Iteration 6621 / 9800) loss: 1.032301
(Iteration 6631 / 9800) loss: 1.138634
(Iteration 6641 / 9800) loss: 1.143103
(Iteration 6651 / 9800) loss: 1.276063
(Iteration 6661 / 9800) loss: 1.185352
(Iteration 6671 / 9800) loss: 1.193462
(Iteration 6681 / 9800) loss: 1.186713
(Iteration 6691 / 9800) loss: 1.111169
(Iteration 6701 / 9800) loss: 1.273690
(Iteration 6711 / 9800) loss: 1.215850
(Iteration 6721 / 9800) loss: 1.279506
(Iteration 6731 / 9800) loss: 1.058323
(Iteration 6741 / 9800) loss: 1.029983
(Iteration 6751 / 9800) loss: 1.091405
(Iteration 6761 / 9800) loss: 1.274988
(Iteration 6771 / 9800) loss: 1.267077
(Iteration 6781 / 9800) loss: 1.051869
(Iteration 6791 / 9800) loss: 1.050066
(Iteration 6801 / 9800) loss: 1.315261
(Iteration 6811 / 9800) loss: 0.985136
(Iteration 6821 / 9800) loss: 1.218884
(Iteration 6831 / 9800) loss: 1.323753
(Iteration 6841 / 9800) loss: 1.093527
```

```
(Iteration 6851 / 9800) loss: 0.974856
(Epoch 14 / 20) train acc: 0.619000; val_acc: 0.524000
(Iteration 6861 / 9800) loss: 1.212476
(Iteration 6871 / 9800) loss: 1.327710
(Iteration 6881 / 9800) loss: 1.171833
(Iteration 6891 / 9800) loss: 1.413646
(Iteration 6901 / 9800) loss: 1.319180
(Iteration 6911 / 9800) loss: 1.178616
(Iteration 6921 / 9800) loss: 1.267671
(Iteration 6931 / 9800) loss: 1.035545
(Iteration 6941 / 9800) loss: 1.297531
(Iteration 6951 / 9800) loss: 1.115835
(Iteration 6961 / 9800) loss: 1.178329
(Iteration 6971 / 9800) loss: 1.226294
(Iteration 6981 / 9800) loss: 1.071441
(Iteration 6991 / 9800) loss: 1.051019
(Iteration 7001 / 9800) loss: 1.127484
(Iteration 7011 / 9800) loss: 1.041824
(Iteration 7021 / 9800) loss: 0.905731
(Iteration 7031 / 9800) loss: 1.273732
(Iteration 7041 / 9800) loss: 1.085610
(Iteration 7051 / 9800) loss: 1.302668
(Iteration 7061 / 9800) loss: 1.213684
(Iteration 7071 / 9800) loss: 1.265858
(Iteration 7081 / 9800) loss: 0.961091
(Iteration 7091 / 9800) loss: 1.052501
(Iteration 7101 / 9800) loss: 1.331817
(Iteration 7111 / 9800) loss: 1.189430
(Iteration 7121 / 9800) loss: 1.131289
(Iteration 7131 / 9800) loss: 1.428018
(Iteration 7141 / 9800) loss: 0.994390
(Iteration 7151 / 9800) loss: 1.229990
(Iteration 7161 / 9800) loss: 1.213501
(Iteration 7171 / 9800) loss: 1.161168
(Iteration 7181 / 9800) loss: 1.277885
(Iteration 7191 / 9800) loss: 1.239397
(Iteration 7201 / 9800) loss: 1.110994
(Iteration 7211 / 9800) loss: 0.987153
(Iteration 7221 / 9800) loss: 1.093390
(Iteration 7231 / 9800) loss: 1.353784
(Iteration 7241 / 9800) loss: 1.058255
(Iteration 7251 / 9800) loss: 1.191608
(Iteration 7261 / 9800) loss: 1.384271
(Iteration 7271 / 9800) loss: 1.276370
(Iteration 7281 / 9800) loss: 1.262721
(Iteration 7291 / 9800) loss: 1.056686
(Iteration 7301 / 9800) loss: 1.157165
(Iteration 7311 / 9800) loss: 1.320441
```

```
(Iteration 7321 / 9800) loss: 1.113126
(Iteration 7331 / 9800) loss: 0.904441
(Iteration 7341 / 9800) loss: 1.082137
(Epoch 15 / 20) train acc: 0.600000; val_acc: 0.498000
(Iteration 7351 / 9800) loss: 1.222092
(Iteration 7361 / 9800) loss: 1.025745
(Iteration 7371 / 9800) loss: 0.942492
(Iteration 7381 / 9800) loss: 1.089163
(Iteration 7391 / 9800) loss: 1.055418
(Iteration 7401 / 9800) loss: 1.356120
(Iteration 7411 / 9800) loss: 1.124456
(Iteration 7421 / 9800) loss: 1.361507
(Iteration 7431 / 9800) loss: 0.968354
(Iteration 7441 / 9800) loss: 1.201711
(Iteration 7451 / 9800) loss: 1.289665
(Iteration 7461 / 9800) loss: 1.204699
(Iteration 7471 / 9800) loss: 1.335328
(Iteration 7481 / 9800) loss: 1.240423
(Iteration 7491 / 9800) loss: 1.038174
(Iteration 7501 / 9800) loss: 1.072208
(Iteration 7511 / 9800) loss: 0.970043
(Iteration 7521 / 9800) loss: 1.150597
(Iteration 7531 / 9800) loss: 1.241509
(Iteration 7541 / 9800) loss: 1.278818
(Iteration 7551 / 9800) loss: 1.193515
(Iteration 7561 / 9800) loss: 1.206597
(Iteration 7571 / 9800) loss: 1.058518
(Iteration 7581 / 9800) loss: 1.274833
(Iteration 7591 / 9800) loss: 1.088936
(Iteration 7601 / 9800) loss: 1.228242
(Iteration 7611 / 9800) loss: 1.201350
(Iteration 7621 / 9800) loss: 1.277063
(Iteration 7631 / 9800) loss: 1.399020
(Iteration 7641 / 9800) loss: 1.312752
(Iteration 7651 / 9800) loss: 1.227741
(Iteration 7661 / 9800) loss: 1.109984
(Iteration 7671 / 9800) loss: 1.088115
(Iteration 7681 / 9800) loss: 1.132990
(Iteration 7691 / 9800) loss: 1.134835
(Iteration 7701 / 9800) loss: 1.248810
(Iteration 7711 / 9800) loss: 1.109564
(Iteration 7721 / 9800) loss: 1.001265
(Iteration 7731 / 9800) loss: 1.146566
(Iteration 7741 / 9800) loss: 1.242963
(Iteration 7751 / 9800) loss: 1.093230
(Iteration 7761 / 9800) loss: 1.184931
(Iteration 7771 / 9800) loss: 1.177849
(Iteration 7781 / 9800) loss: 1.173051
```

```
(Iteration 7791 / 9800) loss: 1.369895
(Iteration 7801 / 9800) loss: 1.274227
(Iteration 7811 / 9800) loss: 1.009425
(Iteration 7821 / 9800) loss: 1.049028
(Iteration 7831 / 9800) loss: 1.194496
(Epoch 16 / 20) train acc: 0.606000; val_acc: 0.507000
(Iteration 7841 / 9800) loss: 1.181047
(Iteration 7851 / 9800) loss: 1.106021
(Iteration 7861 / 9800) loss: 1.280626
(Iteration 7871 / 9800) loss: 1.047592
(Iteration 7881 / 9800) loss: 1.226096
(Iteration 7891 / 9800) loss: 1.365564
(Iteration 7901 / 9800) loss: 1.110627
(Iteration 7911 / 9800) loss: 1.162215
(Iteration 7921 / 9800) loss: 1.466349
(Iteration 7931 / 9800) loss: 1.117565
(Iteration 7941 / 9800) loss: 1.180441
(Iteration 7951 / 9800) loss: 1.286591
(Iteration 7961 / 9800) loss: 1.229871
(Iteration 7971 / 9800) loss: 1.187226
(Iteration 7981 / 9800) loss: 1.098613
(Iteration 7991 / 9800) loss: 1.328731
(Iteration 8001 / 9800) loss: 1.165442
(Iteration 8011 / 9800) loss: 1.306545
(Iteration 8021 / 9800) loss: 1.124092
(Iteration 8031 / 9800) loss: 1.283474
(Iteration 8041 / 9800) loss: 1.206055
(Iteration 8051 / 9800) loss: 1.191586
(Iteration 8061 / 9800) loss: 1.158872
(Iteration 8071 / 9800) loss: 1.287204
(Iteration 8081 / 9800) loss: 1.198818
(Iteration 8091 / 9800) loss: 1.226013
(Iteration 8101 / 9800) loss: 1.323501
(Iteration 8111 / 9800) loss: 1.394547
(Iteration 8121 / 9800) loss: 1.150724
(Iteration 8131 / 9800) loss: 0.920988
(Iteration 8141 / 9800) loss: 1.268992
(Iteration 8151 / 9800) loss: 1.056878
(Iteration 8161 / 9800) loss: 1.179245
(Iteration 8171 / 9800) loss: 1.421538
(Iteration 8181 / 9800) loss: 1.318773
(Iteration 8191 / 9800) loss: 1.080111
(Iteration 8201 / 9800) loss: 1.018866
(Iteration 8211 / 9800) loss: 1.030935
(Iteration 8221 / 9800) loss: 1.248458
(Iteration 8231 / 9800) loss: 1.141021
(Iteration 8241 / 9800) loss: 1.253870
(Iteration 8251 / 9800) loss: 1.243095
```

```
(Iteration 8261 / 9800) loss: 1.145524
(Iteration 8271 / 9800) loss: 0.977043
(Iteration 8281 / 9800) loss: 1.174862
(Iteration 8291 / 9800) loss: 1.097438
(Iteration 8301 / 9800) loss: 1.196449
(Iteration 8311 / 9800) loss: 1.100964
(Iteration 8321 / 9800) loss: 1.241725
(Epoch 17 / 20) train acc: 0.618000; val_acc: 0.512000
(Iteration 8331 / 9800) loss: 1.149558
(Iteration 8341 / 9800) loss: 1.163361
(Iteration 8351 / 9800) loss: 1.100110
(Iteration 8361 / 9800) loss: 1.404053
(Iteration 8371 / 9800) loss: 1.268296
(Iteration 8381 / 9800) loss: 1.078122
(Iteration 8391 / 9800) loss: 1.012863
(Iteration 8401 / 9800) loss: 1.376016
(Iteration 8411 / 9800) loss: 1.189537
(Iteration 8421 / 9800) loss: 1.062920
(Iteration 8431 / 9800) loss: 1.208464
(Iteration 8441 / 9800) loss: 1.269512
(Iteration 8451 / 9800) loss: 1.219427
(Iteration 8461 / 9800) loss: 1.216060
(Iteration 8471 / 9800) loss: 1.169136
(Iteration 8481 / 9800) loss: 1.011225
(Iteration 8491 / 9800) loss: 1.137492
(Iteration 8501 / 9800) loss: 1.091421
(Iteration 8511 / 9800) loss: 1.164044
(Iteration 8521 / 9800) loss: 1.050465
(Iteration 8531 / 9800) loss: 1.065898
(Iteration 8541 / 9800) loss: 1.075230
(Iteration 8551 / 9800) loss: 0.974023
(Iteration 8561 / 9800) loss: 1.173061
(Iteration 8571 / 9800) loss: 1.022131
(Iteration 8581 / 9800) loss: 0.990976
(Iteration 8591 / 9800) loss: 1.160256
(Iteration 8601 / 9800) loss: 1.220820
(Iteration 8611 / 9800) loss: 1.413535
(Iteration 8621 / 9800) loss: 1.125030
(Iteration 8631 / 9800) loss: 0.966872
(Iteration 8641 / 9800) loss: 1.049987
(Iteration 8651 / 9800) loss: 0.959219
(Iteration 8661 / 9800) loss: 1.182827
(Iteration 8671 / 9800) loss: 1.317199
(Iteration 8681 / 9800) loss: 1.136063
(Iteration 8691 / 9800) loss: 1.386701
(Iteration 8701 / 9800) loss: 1.184872
(Iteration 8711 / 9800) loss: 1.077851
(Iteration 8721 / 9800) loss: 1.217903
```

```
(Iteration 8731 / 9800) loss: 1.128086
(Iteration 8741 / 9800) loss: 1.264103
(Iteration 8751 / 9800) loss: 1.217983
(Iteration 8761 / 9800) loss: 1.095648
(Iteration 8771 / 9800) loss: 1.052492
(Iteration 8781 / 9800) loss: 0.930440
(Iteration 8791 / 9800) loss: 1.148640
(Iteration 8801 / 9800) loss: 1.128386
(Iteration 8811 / 9800) loss: 1.142912
(Epoch 18 / 20) train acc: 0.611000; val_acc: 0.518000
(Iteration 8821 / 9800) loss: 1.194415
(Iteration 8831 / 9800) loss: 1.211907
(Iteration 8841 / 9800) loss: 1.104638
(Iteration 8851 / 9800) loss: 1.236388
(Iteration 8861 / 9800) loss: 1.258337
(Iteration 8871 / 9800) loss: 1.233631
(Iteration 8881 / 9800) loss: 1.126845
(Iteration 8891 / 9800) loss: 1.012028
(Iteration 8901 / 9800) loss: 1.224161
(Iteration 8911 / 9800) loss: 1.255326
(Iteration 8921 / 9800) loss: 1.077826
(Iteration 8931 / 9800) loss: 1.088084
(Iteration 8941 / 9800) loss: 1.218937
(Iteration 8951 / 9800) loss: 1.147794
(Iteration 8961 / 9800) loss: 1.180504
(Iteration 8971 / 9800) loss: 1.172117
(Iteration 8981 / 9800) loss: 1.258971
(Iteration 8991 / 9800) loss: 1.211135
(Iteration 9001 / 9800) loss: 1.090531
(Iteration 9011 / 9800) loss: 1.163513
(Iteration 9021 / 9800) loss: 1.341206
(Iteration 9031 / 9800) loss: 1.123767
(Iteration 9041 / 9800) loss: 1.221366
(Iteration 9051 / 9800) loss: 1.110153
(Iteration 9061 / 9800) loss: 1.155458
(Iteration 9071 / 9800) loss: 1.135431
(Iteration 9081 / 9800) loss: 1.046529
(Iteration 9091 / 9800) loss: 1.218778
(Iteration 9101 / 9800) loss: 1.434473
(Iteration 9111 / 9800) loss: 1.327620
(Iteration 9121 / 9800) loss: 1.108884
(Iteration 9131 / 9800) loss: 1.030349
(Iteration 9141 / 9800) loss: 0.991300
(Iteration 9151 / 9800) loss: 1.201793
(Iteration 9161 / 9800) loss: 1.259690
(Iteration 9171 / 9800) loss: 1.077621
(Iteration 9181 / 9800) loss: 1.171699
(Iteration 9191 / 9800) loss: 0.983925
```

```
(Iteration 9201 / 9800) loss: 1.334760
(Iteration 9211 / 9800) loss: 1.021996
(Iteration 9221 / 9800) loss: 1.101197
(Iteration 9231 / 9800) loss: 0.958794
(Iteration 9241 / 9800) loss: 1.170838
(Iteration 9251 / 9800) loss: 1.189998
(Iteration 9261 / 9800) loss: 1.145962
(Iteration 9271 / 9800) loss: 1.082512
(Iteration 9281 / 9800) loss: 1.181312
(Iteration 9291 / 9800) loss: 1.204305
(Iteration 9301 / 9800) loss: 0.926189
(Epoch 19 / 20) train acc: 0.635000; val_acc: 0.504000
(Iteration 9311 / 9800) loss: 1.094775
(Iteration 9321 / 9800) loss: 1.131134
(Iteration 9331 / 9800) loss: 1.080945
(Iteration 9341 / 9800) loss: 1.024751
(Iteration 9351 / 9800) loss: 1.091816
(Iteration 9361 / 9800) loss: 1.320975
(Iteration 9371 / 9800) loss: 1.269185
(Iteration 9381 / 9800) loss: 1.180677
(Iteration 9391 / 9800) loss: 1.176312
(Iteration 9401 / 9800) loss: 1.126258
(Iteration 9411 / 9800) loss: 1.096000
(Iteration 9421 / 9800) loss: 1.000081
(Iteration 9431 / 9800) loss: 1.273139
(Iteration 9441 / 9800) loss: 1.142173
(Iteration 9451 / 9800) loss: 1.122683
(Iteration 9461 / 9800) loss: 1.166326
(Iteration 9471 / 9800) loss: 1.145290
(Iteration 9481 / 9800) loss: 1.243107
(Iteration 9491 / 9800) loss: 1.375399
(Iteration 9501 / 9800) loss: 1.171527
(Iteration 9511 / 9800) loss: 1.192537
(Iteration 9521 / 9800) loss: 1.065235
(Iteration 9531 / 9800) loss: 1.105689
(Iteration 9541 / 9800) loss: 1.296974
(Iteration 9551 / 9800) loss: 1.218481
(Iteration 9561 / 9800) loss: 1.007322
(Iteration 9571 / 9800) loss: 1.338696
(Iteration 9581 / 9800) loss: 1.237601
(Iteration 9591 / 9800) loss: 1.024735
(Iteration 9601 / 9800) loss: 1.081729
(Iteration 9611 / 9800) loss: 0.946917
(Iteration 9621 / 9800) loss: 1.087956
(Iteration 9631 / 9800) loss: 1.347813
(Iteration 9641 / 9800) loss: 1.022045
(Iteration 9651 / 9800) loss: 1.004211
(Iteration 9661 / 9800) loss: 1.239163
```

```
(Iteration 9671 / 9800) loss: 1.151587
(Iteration 9681 / 9800) loss: 1.097635
(Iteration 9691 / 9800) loss: 1.100545
(Iteration 9701 / 9800) loss: 1.216391
(Iteration 9711 / 9800) loss: 1.061428
(Iteration 9721 / 9800) loss: 1.251442
(Iteration 9731 / 9800) loss: 0.988799
(Iteration 9741 / 9800) loss: 1.393854
(Iteration 9751 / 9800) loss: 1.034972
(Iteration 9761 / 9800) loss: 1.070397
(Iteration 9771 / 9800) loss: 1.190859
(Iteration 9781 / 9800) loss: 1.158158
(Iteration 9791 / 9800) loss: 1.051653
(Epoch 20 / 20) train acc: 0.649000; val_acc: 0.517000
accuracy:0.53 with lr:0.0005,reg:0.1,epoch:20,hidden_dim:100
(Iteration 1 / 9800) loss: 2.345576
(Epoch 0 / 20) train acc: 0.131000; val_acc: 0.106000
(Iteration 11 / 9800) loss: 2.312785
(Iteration 21 / 9800) loss: 2.258646
(Iteration 31 / 9800) loss: 2.192371
(Iteration 41 / 9800) loss: 2.127933
(Iteration 51 / 9800) loss: 2.205998
(Iteration 61 / 9800) loss: 2.089507
(Iteration 71 / 9800) loss: 2.005085
(Iteration 81 / 9800) loss: 2.000676
(Iteration 91 / 9800) loss: 1.951737
(Iteration 101 / 9800) loss: 2.014964
(Iteration 111 / 9800) loss: 1.919465
(Iteration 121 / 9800) loss: 1.785906
(Iteration 131 / 9800) loss: 1.980781
(Iteration 141 / 9800) loss: 1.863256
(Iteration 151 / 9800) loss: 2.030773
(Iteration 161 / 9800) loss: 1.787037
(Iteration 171 / 9800) loss: 1.772910
(Iteration 181 / 9800) loss: 1.752354
(Iteration 191 / 9800) loss: 1.832630
(Iteration 201 / 9800) loss: 1.915730
(Iteration 211 / 9800) loss: 1.741926
(Iteration 221 / 9800) loss: 1.791632
(Iteration 231 / 9800) loss: 1.791890
(Iteration 241 / 9800) loss: 1.777020
(Iteration 251 / 9800) loss: 1.829269
(Iteration 261 / 9800) loss: 1.775381
(Iteration 271 / 9800) loss: 1.666240
(Iteration 281 / 9800) loss: 1.746891
(Iteration 291 / 9800) loss: 1.814508
(Iteration 301 / 9800) loss: 1.758714
(Iteration 311 / 9800) loss: 1.740056
```

```
(Iteration 321 / 9800) loss: 1.730679
(Iteration 331 / 9800) loss: 1.602467
(Iteration 341 / 9800) loss: 1.542797
(Iteration 351 / 9800) loss: 1.661050
(Iteration 361 / 9800) loss: 1.783536
(Iteration 371 / 9800) loss: 1.635214
(Iteration 381 / 9800) loss: 1.700718
(Iteration 391 / 9800) loss: 1.697301
(Iteration 401 / 9800) loss: 1.674981
(Iteration 411 / 9800) loss: 1.670634
(Iteration 421 / 9800) loss: 1.606280
(Iteration 431 / 9800) loss: 1.638605
(Iteration 441 / 9800) loss: 1.692622
(Iteration 451 / 9800) loss: 1.506894
(Iteration 461 / 9800) loss: 1.625280
(Iteration 471 / 9800) loss: 1.833259
(Iteration 481 / 9800) loss: 1.594648
(Epoch 1 / 20) train acc: 0.438000; val_acc: 0.439000
(Iteration 491 / 9800) loss: 1.529935
(Iteration 501 / 9800) loss: 1.635780
(Iteration 511 / 9800) loss: 1.704092
(Iteration 521 / 9800) loss: 1.562313
(Iteration 531 / 9800) loss: 1.548626
(Iteration 541 / 9800) loss: 1.699438
(Iteration 551 / 9800) loss: 1.616180
(Iteration 561 / 9800) loss: 1.569252
(Iteration 571 / 9800) loss: 1.518213
(Iteration 581 / 9800) loss: 1.596880
(Iteration 591 / 9800) loss: 1.687339
(Iteration 601 / 9800) loss: 1.536215
(Iteration 611 / 9800) loss: 1.488361
(Iteration 621 / 9800) loss: 1.688502
(Iteration 631 / 9800) loss: 1.481864
(Iteration 641 / 9800) loss: 1.439437
(Iteration 651 / 9800) loss: 1.576281
(Iteration 661 / 9800) loss: 1.505645
(Iteration 671 / 9800) loss: 1.666100
(Iteration 681 / 9800) loss: 1.607381
(Iteration 691 / 9800) loss: 1.540511
(Iteration 701 / 9800) loss: 1.533530
(Iteration 711 / 9800) loss: 1.595124
(Iteration 721 / 9800) loss: 1.394038
(Iteration 731 / 9800) loss: 1.521723
(Iteration 741 / 9800) loss: 1.705705
(Iteration 751 / 9800) loss: 1.630453
(Iteration 761 / 9800) loss: 1.522581
(Iteration 771 / 9800) loss: 1.546555
(Iteration 781 / 9800) loss: 1.490898
```

```
(Iteration 791 / 9800) loss: 1.579460
(Iteration 801 / 9800) loss: 1.397316
(Iteration 811 / 9800) loss: 1.426238
(Iteration 821 / 9800) loss: 1.783172
(Iteration 831 / 9800) loss: 1.555500
(Iteration 841 / 9800) loss: 1.703007
(Iteration 851 / 9800) loss: 1.586280
(Iteration 861 / 9800) loss: 1.636570
(Iteration 871 / 9800) loss: 1.549348
(Iteration 881 / 9800) loss: 1.519049
(Iteration 891 / 9800) loss: 1.359468
(Iteration 901 / 9800) loss: 1.538011
(Iteration 911 / 9800) loss: 1.432054
(Iteration 921 / 9800) loss: 1.579856
(Iteration 931 / 9800) loss: 1.662986
(Iteration 941 / 9800) loss: 1.749198
(Iteration 951 / 9800) loss: 1.402512
(Iteration 961 / 9800) loss: 1.357934
(Iteration 971 / 9800) loss: 1.434067
(Epoch 2 / 20) train acc: 0.451000; val_acc: 0.481000
(Iteration 981 / 9800) loss: 1.624507
(Iteration 991 / 9800) loss: 1.797607
(Iteration 1001 / 9800) loss: 1.627606
(Iteration 1011 / 9800) loss: 1.596631
(Iteration 1021 / 9800) loss: 1.500550
(Iteration 1031 / 9800) loss: 1.701258
(Iteration 1041 / 9800) loss: 1.595856
(Iteration 1051 / 9800) loss: 1.690283
(Iteration 1061 / 9800) loss: 1.441792
(Iteration 1071 / 9800) loss: 1.583256
(Iteration 1081 / 9800) loss: 1.453957
(Iteration 1091 / 9800) loss: 1.753943
(Iteration 1101 / 9800) loss: 1.522871
(Iteration 1111 / 9800) loss: 1.576713
(Iteration 1121 / 9800) loss: 1.519499
(Iteration 1131 / 9800) loss: 1.478397
(Iteration 1141 / 9800) loss: 1.315973
(Iteration 1151 / 9800) loss: 1.591008
(Iteration 1161 / 9800) loss: 1.601289
(Iteration 1171 / 9800) loss: 1.563698
(Iteration 1181 / 9800) loss: 1.498506
(Iteration 1191 / 9800) loss: 1.719289
(Iteration 1201 / 9800) loss: 1.440180
(Iteration 1211 / 9800) loss: 1.487438
(Iteration 1221 / 9800) loss: 1.443427
(Iteration 1231 / 9800) loss: 1.568630
(Iteration 1241 / 9800) loss: 1.388082
(Iteration 1251 / 9800) loss: 1.546839
```

```
(Iteration 1261 / 9800) loss: 1.498639
(Iteration 1271 / 9800) loss: 1.444483
(Iteration 1281 / 9800) loss: 1.525461
(Iteration 1291 / 9800) loss: 1.661084
(Iteration 1301 / 9800) loss: 1.447966
(Iteration 1311 / 9800) loss: 1.512052
(Iteration 1321 / 9800) loss: 1.475007
(Iteration 1331 / 9800) loss: 1.470588
(Iteration 1341 / 9800) loss: 1.500215
(Iteration 1351 / 9800) loss: 1.731349
(Iteration 1361 / 9800) loss: 1.639152
(Iteration 1371 / 9800) loss: 1.517142
(Iteration 1381 / 9800) loss: 1.539408
(Iteration 1391 / 9800) loss: 1.596697
(Iteration 1401 / 9800) loss: 1.381448
(Iteration 1411 / 9800) loss: 1.639921
(Iteration 1421 / 9800) loss: 1.388496
(Iteration 1431 / 9800) loss: 1.605534
(Iteration 1441 / 9800) loss: 1.422668
(Iteration 1451 / 9800) loss: 1.554321
(Iteration 1461 / 9800) loss: 1.512433
(Epoch 3 / 20) train acc: 0.513000; val_acc: 0.467000
(Iteration 1471 / 9800) loss: 1.605557
(Iteration 1481 / 9800) loss: 1.474125
(Iteration 1491 / 9800) loss: 1.666935
(Iteration 1501 / 9800) loss: 1.411447
(Iteration 1511 / 9800) loss: 1.420750
(Iteration 1521 / 9800) loss: 1.333131
(Iteration 1531 / 9800) loss: 1.944306
(Iteration 1541 / 9800) loss: 1.593994
(Iteration 1551 / 9800) loss: 1.360996
(Iteration 1561 / 9800) loss: 1.533381
(Iteration 1571 / 9800) loss: 1.471702
(Iteration 1581 / 9800) loss: 1.443150
(Iteration 1591 / 9800) loss: 1.570318
(Iteration 1601 / 9800) loss: 1.333999
(Iteration 1611 / 9800) loss: 1.539815
(Iteration 1621 / 9800) loss: 1.588281
(Iteration 1631 / 9800) loss: 1.470689
(Iteration 1641 / 9800) loss: 1.317521
(Iteration 1651 / 9800) loss: 1.426353
(Iteration 1661 / 9800) loss: 1.590646
(Iteration 1671 / 9800) loss: 1.582659
(Iteration 1681 / 9800) loss: 1.533283
(Iteration 1691 / 9800) loss: 1.449632
(Iteration 1701 / 9800) loss: 1.527144
(Iteration 1711 / 9800) loss: 1.480555
(Iteration 1721 / 9800) loss: 1.669941
```

```
(Iteration 1731 / 9800) loss: 1.509172
(Iteration 1741 / 9800) loss: 1.496508
(Iteration 1751 / 9800) loss: 1.441849
(Iteration 1761 / 9800) loss: 1.616752
(Iteration 1771 / 9800) loss: 1.304021
(Iteration 1781 / 9800) loss: 1.411613
(Iteration 1791 / 9800) loss: 1.694083
(Iteration 1801 / 9800) loss: 1.390429
(Iteration 1811 / 9800) loss: 1.426970
(Iteration 1821 / 9800) loss: 1.404268
(Iteration 1831 / 9800) loss: 1.317510
(Iteration 1841 / 9800) loss: 1.424094
(Iteration 1851 / 9800) loss: 1.535307
(Iteration 1861 / 9800) loss: 1.423473
(Iteration 1871 / 9800) loss: 1.493152
(Iteration 1881 / 9800) loss: 1.462597
(Iteration 1891 / 9800) loss: 1.392557
(Iteration 1901 / 9800) loss: 1.330434
(Iteration 1911 / 9800) loss: 1.465854
(Iteration 1921 / 9800) loss: 1.464565
(Iteration 1931 / 9800) loss: 1.532171
(Iteration 1941 / 9800) loss: 1.405248
(Iteration 1951 / 9800) loss: 1.380983
(Epoch 4 / 20) train acc: 0.520000; val_acc: 0.489000
(Iteration 1961 / 9800) loss: 1.493438
(Iteration 1971 / 9800) loss: 1.642668
(Iteration 1981 / 9800) loss: 1.578406
(Iteration 1991 / 9800) loss: 1.495650
(Iteration 2001 / 9800) loss: 1.433696
(Iteration 2011 / 9800) loss: 1.529419
(Iteration 2021 / 9800) loss: 1.385397
(Iteration 2031 / 9800) loss: 1.462565
(Iteration 2041 / 9800) loss: 1.310970
(Iteration 2051 / 9800) loss: 1.444991
(Iteration 2061 / 9800) loss: 1.403410
(Iteration 2071 / 9800) loss: 1.393791
(Iteration 2081 / 9800) loss: 1.268228
(Iteration 2091 / 9800) loss: 1.434289
(Iteration 2101 / 9800) loss: 1.495163
(Iteration 2111 / 9800) loss: 1.603488
(Iteration 2121 / 9800) loss: 1.224203
(Iteration 2131 / 9800) loss: 1.487030
(Iteration 2141 / 9800) loss: 1.627875
(Iteration 2151 / 9800) loss: 1.368549
(Iteration 2161 / 9800) loss: 1.518011
(Iteration 2171 / 9800) loss: 1.228147
(Iteration 2181 / 9800) loss: 1.444773
(Iteration 2191 / 9800) loss: 1.530775
```

```
(Iteration 2201 / 9800) loss: 1.355749
(Iteration 2211 / 9800) loss: 1.408280
(Iteration 2221 / 9800) loss: 1.482026
(Iteration 2231 / 9800) loss: 1.210364
(Iteration 2241 / 9800) loss: 1.359044
(Iteration 2251 / 9800) loss: 1.278482
(Iteration 2261 / 9800) loss: 1.446461
(Iteration 2271 / 9800) loss: 1.443501
(Iteration 2281 / 9800) loss: 1.298311
(Iteration 2291 / 9800) loss: 1.681958
(Iteration 2301 / 9800) loss: 1.323730
(Iteration 2311 / 9800) loss: 1.485998
(Iteration 2321 / 9800) loss: 1.355887
(Iteration 2331 / 9800) loss: 1.400542
(Iteration 2341 / 9800) loss: 1.404174
(Iteration 2351 / 9800) loss: 1.366838
(Iteration 2361 / 9800) loss: 1.400751
(Iteration 2371 / 9800) loss: 1.428037
(Iteration 2381 / 9800) loss: 1.288769
(Iteration 2391 / 9800) loss: 1.283179
(Iteration 2401 / 9800) loss: 1.362704
(Iteration 2411 / 9800) loss: 1.372014
(Iteration 2421 / 9800) loss: 1.444089
(Iteration 2431 / 9800) loss: 1.438607
(Iteration 2441 / 9800) loss: 1.388359
(Epoch 5 / 20) train acc: 0.518000; val_acc: 0.504000
(Iteration 2451 / 9800) loss: 1.265128
(Iteration 2461 / 9800) loss: 1.254612
(Iteration 2471 / 9800) loss: 1.381394
(Iteration 2481 / 9800) loss: 1.527583
(Iteration 2491 / 9800) loss: 1.391487
(Iteration 2501 / 9800) loss: 1.313341
(Iteration 2511 / 9800) loss: 1.197920
(Iteration 2521 / 9800) loss: 1.403007
(Iteration 2531 / 9800) loss: 1.538361
(Iteration 2541 / 9800) loss: 1.433998
(Iteration 2551 / 9800) loss: 1.463134
(Iteration 2561 / 9800) loss: 1.295714
(Iteration 2571 / 9800) loss: 1.454400
(Iteration 2581 / 9800) loss: 1.365274
(Iteration 2591 / 9800) loss: 1.236493
(Iteration 2601 / 9800) loss: 1.544646
(Iteration 2611 / 9800) loss: 1.292574
(Iteration 2621 / 9800) loss: 1.403754
(Iteration 2631 / 9800) loss: 1.409692
(Iteration 2641 / 9800) loss: 1.314409
(Iteration 2651 / 9800) loss: 1.481087
(Iteration 2661 / 9800) loss: 1.440496
```

```
(Iteration 2671 / 9800) loss: 1.260081
(Iteration 2681 / 9800) loss: 1.192969
(Iteration 2691 / 9800) loss: 1.216578
(Iteration 2701 / 9800) loss: 1.331974
(Iteration 2711 / 9800) loss: 1.277617
(Iteration 2721 / 9800) loss: 1.411234
(Iteration 2731 / 9800) loss: 1.489397
(Iteration 2741 / 9800) loss: 1.409937
(Iteration 2751 / 9800) loss: 1.240187
(Iteration 2761 / 9800) loss: 1.350771
(Iteration 2771 / 9800) loss: 1.459722
(Iteration 2781 / 9800) loss: 1.315481
(Iteration 2791 / 9800) loss: 1.542188
(Iteration 2801 / 9800) loss: 1.368906
(Iteration 2811 / 9800) loss: 1.322309
(Iteration 2821 / 9800) loss: 1.355186
(Iteration 2831 / 9800) loss: 1.403510
(Iteration 2841 / 9800) loss: 1.369478
(Iteration 2851 / 9800) loss: 1.293965
(Iteration 2861 / 9800) loss: 1.456332
(Iteration 2871 / 9800) loss: 1.429297
(Iteration 2881 / 9800) loss: 1.416493
(Iteration 2891 / 9800) loss: 1.333409
(Iteration 2901 / 9800) loss: 1.400003
(Iteration 2911 / 9800) loss: 1.330074
(Iteration 2921 / 9800) loss: 1.399331
(Iteration 2931 / 9800) loss: 1.621871
(Epoch 6 / 20) train acc: 0.551000; val_acc: 0.503000
(Iteration 2941 / 9800) loss: 1.353456
(Iteration 2951 / 9800) loss: 1.368771
(Iteration 2961 / 9800) loss: 1.435849
(Iteration 2971 / 9800) loss: 1.244476
(Iteration 2981 / 9800) loss: 1.615110
(Iteration 2991 / 9800) loss: 1.640555
(Iteration 3001 / 9800) loss: 1.302832
(Iteration 3011 / 9800) loss: 1.463822
(Iteration 3021 / 9800) loss: 1.470232
(Iteration 3031 / 9800) loss: 1.363603
(Iteration 3041 / 9800) loss: 1.391089
(Iteration 3051 / 9800) loss: 1.251933
(Iteration 3061 / 9800) loss: 1.437612
(Iteration 3071 / 9800) loss: 1.557547
(Iteration 3081 / 9800) loss: 1.309778
(Iteration 3091 / 9800) loss: 1.445644
(Iteration 3101 / 9800) loss: 1.191300
(Iteration 3111 / 9800) loss: 1.131752
(Iteration 3121 / 9800) loss: 1.277322
(Iteration 3131 / 9800) loss: 1.318256
```

```
(Iteration 3141 / 9800) loss: 1.374886
(Iteration 3151 / 9800) loss: 1.289685
(Iteration 3161 / 9800) loss: 1.433909
(Iteration 3171 / 9800) loss: 1.237690
(Iteration 3181 / 9800) loss: 1.322097
(Iteration 3191 / 9800) loss: 1.276104
(Iteration 3201 / 9800) loss: 1.303850
(Iteration 3211 / 9800) loss: 1.378115
(Iteration 3221 / 9800) loss: 1.462015
(Iteration 3231 / 9800) loss: 1.225363
(Iteration 3241 / 9800) loss: 1.238378
(Iteration 3251 / 9800) loss: 1.362202
(Iteration 3261 / 9800) loss: 1.457042
(Iteration 3271 / 9800) loss: 1.228499
(Iteration 3281 / 9800) loss: 1.402740
(Iteration 3291 / 9800) loss: 1.342065
(Iteration 3301 / 9800) loss: 1.429201
(Iteration 3311 / 9800) loss: 1.385677
(Iteration 3321 / 9800) loss: 1.327783
(Iteration 3331 / 9800) loss: 1.461627
(Iteration 3341 / 9800) loss: 1.504350
(Iteration 3351 / 9800) loss: 1.477219
(Iteration 3361 / 9800) loss: 1.309435
(Iteration 3371 / 9800) loss: 1.271090
(Iteration 3381 / 9800) loss: 1.317493
(Iteration 3391 / 9800) loss: 1.536623
(Iteration 3401 / 9800) loss: 1.271597
(Iteration 3411 / 9800) loss: 1.509919
(Iteration 3421 / 9800) loss: 1.452959
(Epoch 7 / 20) train acc: 0.538000; val_acc: 0.483000
(Iteration 3431 / 9800) loss: 1.490480
(Iteration 3441 / 9800) loss: 1.246253
(Iteration 3451 / 9800) loss: 1.529649
(Iteration 3461 / 9800) loss: 1.552931
(Iteration 3471 / 9800) loss: 1.085692
(Iteration 3481 / 9800) loss: 1.378800
(Iteration 3491 / 9800) loss: 1.320394
(Iteration 3501 / 9800) loss: 1.457226
(Iteration 3511 / 9800) loss: 1.564146
(Iteration 3521 / 9800) loss: 1.353130
(Iteration 3531 / 9800) loss: 1.299819
(Iteration 3541 / 9800) loss: 1.492080
(Iteration 3551 / 9800) loss: 1.324874
(Iteration 3561 / 9800) loss: 1.373307
(Iteration 3571 / 9800) loss: 1.313048
(Iteration 3581 / 9800) loss: 1.343589
(Iteration 3591 / 9800) loss: 1.422473
(Iteration 3601 / 9800) loss: 1.420519
```

```
(Iteration 3611 / 9800) loss: 1.292337
(Iteration 3621 / 9800) loss: 1.441757
(Iteration 3631 / 9800) loss: 1.215322
(Iteration 3641 / 9800) loss: 1.375224
(Iteration 3651 / 9800) loss: 1.419336
(Iteration 3661 / 9800) loss: 1.337373
(Iteration 3671 / 9800) loss: 1.356890
(Iteration 3681 / 9800) loss: 1.506790
(Iteration 3691 / 9800) loss: 1.426286
(Iteration 3701 / 9800) loss: 1.345967
(Iteration 3711 / 9800) loss: 1.219270
(Iteration 3721 / 9800) loss: 1.588881
(Iteration 3731 / 9800) loss: 1.548854
(Iteration 3741 / 9800) loss: 1.218555
(Iteration 3751 / 9800) loss: 1.407967
(Iteration 3761 / 9800) loss: 1.129708
(Iteration 3771 / 9800) loss: 1.289760
(Iteration 3781 / 9800) loss: 1.487221
(Iteration 3791 / 9800) loss: 1.355386
(Iteration 3801 / 9800) loss: 1.310503
(Iteration 3811 / 9800) loss: 1.142658
(Iteration 3821 / 9800) loss: 1.394030
(Iteration 3831 / 9800) loss: 1.239268
(Iteration 3841 / 9800) loss: 1.425822
(Iteration 3851 / 9800) loss: 1.273368
(Iteration 3861 / 9800) loss: 1.337346
(Iteration 3871 / 9800) loss: 1.245081
(Iteration 3881 / 9800) loss: 1.348860
(Iteration 3891 / 9800) loss: 1.584635
(Iteration 3901 / 9800) loss: 1.278140
(Iteration 3911 / 9800) loss: 1.575322
(Epoch 8 / 20) train acc: 0.593000; val_acc: 0.509000
(Iteration 3921 / 9800) loss: 1.333700
(Iteration 3931 / 9800) loss: 1.325948
(Iteration 3941 / 9800) loss: 1.285139
(Iteration 3951 / 9800) loss: 1.519850
(Iteration 3961 / 9800) loss: 1.244768
(Iteration 3971 / 9800) loss: 1.347987
(Iteration 3981 / 9800) loss: 1.295644
(Iteration 3991 / 9800) loss: 1.642448
(Iteration 4001 / 9800) loss: 1.361305
(Iteration 4011 / 9800) loss: 1.470351
(Iteration 4021 / 9800) loss: 1.453378
(Iteration 4031 / 9800) loss: 1.248016
(Iteration 4041 / 9800) loss: 1.274996
(Iteration 4051 / 9800) loss: 1.324678
(Iteration 4061 / 9800) loss: 1.350824
(Iteration 4071 / 9800) loss: 1.395853
```

```
(Iteration 4081 / 9800) loss: 1.368761
(Iteration 4091 / 9800) loss: 1.448163
(Iteration 4101 / 9800) loss: 1.445475
(Iteration 4111 / 9800) loss: 1.461415
(Iteration 4121 / 9800) loss: 1.212505
(Iteration 4131 / 9800) loss: 1.459738
(Iteration 4141 / 9800) loss: 1.290450
(Iteration 4151 / 9800) loss: 1.341539
(Iteration 4161 / 9800) loss: 1.227226
(Iteration 4171 / 9800) loss: 1.165629
(Iteration 4181 / 9800) loss: 1.245591
(Iteration 4191 / 9800) loss: 1.263547
(Iteration 4201 / 9800) loss: 1.256963
(Iteration 4211 / 9800) loss: 1.523418
(Iteration 4221 / 9800) loss: 1.497344
(Iteration 4231 / 9800) loss: 1.372163
(Iteration 4241 / 9800) loss: 1.289503
(Iteration 4251 / 9800) loss: 1.133865
(Iteration 4261 / 9800) loss: 1.362429
(Iteration 4271 / 9800) loss: 1.411993
(Iteration 4281 / 9800) loss: 1.297588
(Iteration 4291 / 9800) loss: 1.144528
(Iteration 4301 / 9800) loss: 1.494865
(Iteration 4311 / 9800) loss: 1.266834
(Iteration 4321 / 9800) loss: 1.347545
(Iteration 4331 / 9800) loss: 1.261516
(Iteration 4341 / 9800) loss: 1.327956
(Iteration 4351 / 9800) loss: 1.369606
(Iteration 4361 / 9800) loss: 1.316318
(Iteration 4371 / 9800) loss: 1.190993
(Iteration 4381 / 9800) loss: 1.521490
(Iteration 4391 / 9800) loss: 1.223796
(Iteration 4401 / 9800) loss: 1.364615
(Epoch 9 / 20) train acc: 0.607000; val_acc: 0.514000
(Iteration 4411 / 9800) loss: 1.357954
(Iteration 4421 / 9800) loss: 1.215586
(Iteration 4431 / 9800) loss: 1.353002
(Iteration 4441 / 9800) loss: 1.187472
(Iteration 4451 / 9800) loss: 1.356613
(Iteration 4461 / 9800) loss: 1.319744
(Iteration 4471 / 9800) loss: 1.194931
(Iteration 4481 / 9800) loss: 1.105864
(Iteration 4491 / 9800) loss: 1.291870
(Iteration 4501 / 9800) loss: 1.587424
(Iteration 4511 / 9800) loss: 1.501511
(Iteration 4521 / 9800) loss: 1.141070
(Iteration 4531 / 9800) loss: 1.190275
(Iteration 4541 / 9800) loss: 1.273233
```

```
(Iteration 4551 / 9800) loss: 1.233068
(Iteration 4561 / 9800) loss: 1.566450
(Iteration 4571 / 9800) loss: 1.246277
(Iteration 4581 / 9800) loss: 1.281973
(Iteration 4591 / 9800) loss: 1.198144
(Iteration 4601 / 9800) loss: 1.394804
(Iteration 4611 / 9800) loss: 1.457227
(Iteration 4621 / 9800) loss: 1.424718
(Iteration 4631 / 9800) loss: 1.294477
(Iteration 4641 / 9800) loss: 1.455222
(Iteration 4651 / 9800) loss: 1.295435
(Iteration 4661 / 9800) loss: 1.348814
(Iteration 4671 / 9800) loss: 1.315544
(Iteration 4681 / 9800) loss: 1.377837
(Iteration 4691 / 9800) loss: 1.550587
(Iteration 4701 / 9800) loss: 1.528621
(Iteration 4711 / 9800) loss: 1.422050
(Iteration 4721 / 9800) loss: 1.237672
(Iteration 4731 / 9800) loss: 1.441945
(Iteration 4741 / 9800) loss: 1.152223
(Iteration 4751 / 9800) loss: 1.386856
(Iteration 4761 / 9800) loss: 1.424870
(Iteration 4771 / 9800) loss: 1.178195
(Iteration 4781 / 9800) loss: 1.562323
(Iteration 4791 / 9800) loss: 1.383139
(Iteration 4801 / 9800) loss: 1.246034
(Iteration 4811 / 9800) loss: 1.350874
(Iteration 4821 / 9800) loss: 1.178852
(Iteration 4831 / 9800) loss: 1.317904
(Iteration 4841 / 9800) loss: 1.290507
(Iteration 4851 / 9800) loss: 1.265440
(Iteration 4861 / 9800) loss: 1.299206
(Iteration 4871 / 9800) loss: 1.352892
(Iteration 4881 / 9800) loss: 1.433128
(Iteration 4891 / 9800) loss: 1.419354
(Epoch 10 / 20) train acc: 0.575000; val_acc: 0.524000
(Iteration 4901 / 9800) loss: 1.268974
(Iteration 4911 / 9800) loss: 1.284767
(Iteration 4921 / 9800) loss: 1.261023
(Iteration 4931 / 9800) loss: 1.334021
(Iteration 4941 / 9800) loss: 1.403823
(Iteration 4951 / 9800) loss: 1.369007
(Iteration 4961 / 9800) loss: 1.438373
(Iteration 4971 / 9800) loss: 1.327002
(Iteration 4981 / 9800) loss: 1.283365
(Iteration 4991 / 9800) loss: 1.235862
(Iteration 5001 / 9800) loss: 1.332806
(Iteration 5011 / 9800) loss: 1.471866
```

```
(Iteration 5021 / 9800) loss: 1.368258
(Iteration 5031 / 9800) loss: 1.334503
(Iteration 5041 / 9800) loss: 1.202106
(Iteration 5051 / 9800) loss: 1.341952
(Iteration 5061 / 9800) loss: 1.303268
(Iteration 5071 / 9800) loss: 1.401255
(Iteration 5081 / 9800) loss: 1.282475
(Iteration 5091 / 9800) loss: 1.369313
(Iteration 5101 / 9800) loss: 1.224662
(Iteration 5111 / 9800) loss: 1.284272
(Iteration 5121 / 9800) loss: 1.387965
(Iteration 5131 / 9800) loss: 1.409106
(Iteration 5141 / 9800) loss: 1.438223
(Iteration 5151 / 9800) loss: 1.379539
(Iteration 5161 / 9800) loss: 1.212131
(Iteration 5171 / 9800) loss: 1.149190
(Iteration 5181 / 9800) loss: 1.319196
(Iteration 5191 / 9800) loss: 1.203638
(Iteration 5201 / 9800) loss: 1.285416
(Iteration 5211 / 9800) loss: 1.285193
(Iteration 5221 / 9800) loss: 1.387522
(Iteration 5231 / 9800) loss: 1.309072
(Iteration 5241 / 9800) loss: 1.267653
(Iteration 5251 / 9800) loss: 1.242305
(Iteration 5261 / 9800) loss: 1.327665
(Iteration 5271 / 9800) loss: 1.350733
(Iteration 5281 / 9800) loss: 1.205487
(Iteration 5291 / 9800) loss: 1.405944
(Iteration 5301 / 9800) loss: 1.302256
(Iteration 5311 / 9800) loss: 1.233420
(Iteration 5321 / 9800) loss: 1.323609
(Iteration 5331 / 9800) loss: 1.246027
(Iteration 5341 / 9800) loss: 1.338127
(Iteration 5351 / 9800) loss: 1.392113
(Iteration 5361 / 9800) loss: 1.309441
(Iteration 5371 / 9800) loss: 1.351532
(Iteration 5381 / 9800) loss: 1.284578
(Epoch 11 / 20) train acc: 0.594000; val_acc: 0.514000
(Iteration 5391 / 9800) loss: 1.258247
(Iteration 5401 / 9800) loss: 1.478381
(Iteration 5411 / 9800) loss: 1.380232
(Iteration 5421 / 9800) loss: 1.182535
(Iteration 5431 / 9800) loss: 1.251595
(Iteration 5441 / 9800) loss: 1.418585
(Iteration 5451 / 9800) loss: 1.320680
(Iteration 5461 / 9800) loss: 1.091902
(Iteration 5471 / 9800) loss: 1.175135
(Iteration 5481 / 9800) loss: 1.250530
```

```
(Iteration 5491 / 9800) loss: 1.335056
(Iteration 5501 / 9800) loss: 1.491343
(Iteration 5511 / 9800) loss: 1.347700
(Iteration 5521 / 9800) loss: 1.452023
(Iteration 5531 / 9800) loss: 1.391700
(Iteration 5541 / 9800) loss: 1.228898
(Iteration 5551 / 9800) loss: 1.339747
(Iteration 5561 / 9800) loss: 1.430299
(Iteration 5571 / 9800) loss: 1.058647
(Iteration 5581 / 9800) loss: 1.458070
(Iteration 5591 / 9800) loss: 1.245982
(Iteration 5601 / 9800) loss: 1.598095
(Iteration 5611 / 9800) loss: 1.291221
(Iteration 5621 / 9800) loss: 1.351075
(Iteration 5631 / 9800) loss: 1.371899
(Iteration 5641 / 9800) loss: 1.363347
(Iteration 5651 / 9800) loss: 1.186063
(Iteration 5661 / 9800) loss: 1.457852
(Iteration 5671 / 9800) loss: 1.290543
(Iteration 5681 / 9800) loss: 1.417027
(Iteration 5691 / 9800) loss: 1.325513
(Iteration 5701 / 9800) loss: 1.413383
(Iteration 5711 / 9800) loss: 1.230465
(Iteration 5721 / 9800) loss: 1.044903
(Iteration 5731 / 9800) loss: 1.233736
(Iteration 5741 / 9800) loss: 1.321862
(Iteration 5751 / 9800) loss: 1.374906
(Iteration 5761 / 9800) loss: 1.235019
(Iteration 5771 / 9800) loss: 1.290353
(Iteration 5781 / 9800) loss: 1.367308
(Iteration 5791 / 9800) loss: 1.279963
(Iteration 5801 / 9800) loss: 1.190984
(Iteration 5811 / 9800) loss: 1.412246
(Iteration 5821 / 9800) loss: 1.299030
(Iteration 5831 / 9800) loss: 1.147950
(Iteration 5841 / 9800) loss: 1.188732
(Iteration 5851 / 9800) loss: 1.283527
(Iteration 5861 / 9800) loss: 1.218851
(Iteration 5871 / 9800) loss: 1.527431
(Epoch 12 / 20) train acc: 0.570000; val_acc: 0.503000
(Iteration 5881 / 9800) loss: 1.237678
(Iteration 5891 / 9800) loss: 1.400381
(Iteration 5901 / 9800) loss: 1.345771
(Iteration 5911 / 9800) loss: 1.200238
(Iteration 5921 / 9800) loss: 1.478993
(Iteration 5931 / 9800) loss: 1.268570
(Iteration 5941 / 9800) loss: 1.262192
(Iteration 5951 / 9800) loss: 1.331752
```

```
(Iteration 5961 / 9800) loss: 1.263169
(Iteration 5971 / 9800) loss: 1.304017
(Iteration 5981 / 9800) loss: 1.438258
(Iteration 5991 / 9800) loss: 1.405927
(Iteration 6001 / 9800) loss: 1.342367
(Iteration 6011 / 9800) loss: 1.287966
(Iteration 6021 / 9800) loss: 1.128630
(Iteration 6031 / 9800) loss: 1.439659
(Iteration 6041 / 9800) loss: 1.253986
(Iteration 6051 / 9800) loss: 1.267901
(Iteration 6061 / 9800) loss: 1.274758
(Iteration 6071 / 9800) loss: 1.429164
(Iteration 6081 / 9800) loss: 1.336461
(Iteration 6091 / 9800) loss: 1.204007
(Iteration 6101 / 9800) loss: 1.283564
(Iteration 6111 / 9800) loss: 1.292629
(Iteration 6121 / 9800) loss: 1.352609
(Iteration 6131 / 9800) loss: 1.240729
(Iteration 6141 / 9800) loss: 1.253540
(Iteration 6151 / 9800) loss: 1.247580
(Iteration 6161 / 9800) loss: 1.309112
(Iteration 6171 / 9800) loss: 1.313120
(Iteration 6181 / 9800) loss: 1.160954
(Iteration 6191 / 9800) loss: 1.244301
(Iteration 6201 / 9800) loss: 1.208231
(Iteration 6211 / 9800) loss: 1.296016
(Iteration 6221 / 9800) loss: 1.213152
(Iteration 6231 / 9800) loss: 1.144964
(Iteration 6241 / 9800) loss: 1.076759
(Iteration 6251 / 9800) loss: 1.242260
(Iteration 6261 / 9800) loss: 1.253263
(Iteration 6271 / 9800) loss: 1.363553
(Iteration 6281 / 9800) loss: 1.382404
(Iteration 6291 / 9800) loss: 1.175115
(Iteration 6301 / 9800) loss: 1.311361
(Iteration 6311 / 9800) loss: 1.442185
(Iteration 6321 / 9800) loss: 1.340377
(Iteration 6331 / 9800) loss: 1.313368
(Iteration 6341 / 9800) loss: 1.072718
(Iteration 6351 / 9800) loss: 1.158603
(Iteration 6361 / 9800) loss: 1.582293
(Epoch 13 / 20) train acc: 0.567000; val_acc: 0.516000
(Iteration 6371 / 9800) loss: 1.218261
(Iteration 6381 / 9800) loss: 1.532375
(Iteration 6391 / 9800) loss: 1.295925
(Iteration 6401 / 9800) loss: 1.657022
(Iteration 6411 / 9800) loss: 1.418564
(Iteration 6421 / 9800) loss: 1.393268
```

```
(Iteration 6431 / 9800) loss: 1.203421
(Iteration 6441 / 9800) loss: 1.187751
(Iteration 6451 / 9800) loss: 1.373996
(Iteration 6461 / 9800) loss: 1.210041
(Iteration 6471 / 9800) loss: 1.238010
(Iteration 6481 / 9800) loss: 1.398772
(Iteration 6491 / 9800) loss: 1.201284
(Iteration 6501 / 9800) loss: 1.233716
(Iteration 6511 / 9800) loss: 1.380067
(Iteration 6521 / 9800) loss: 1.386122
(Iteration 6531 / 9800) loss: 1.256564
(Iteration 6541 / 9800) loss: 1.269312
(Iteration 6551 / 9800) loss: 1.361276
(Iteration 6561 / 9800) loss: 1.192085
(Iteration 6571 / 9800) loss: 1.275303
(Iteration 6581 / 9800) loss: 1.362479
(Iteration 6591 / 9800) loss: 1.235418
(Iteration 6601 / 9800) loss: 1.168102
(Iteration 6611 / 9800) loss: 1.167064
(Iteration 6621 / 9800) loss: 1.224120
(Iteration 6631 / 9800) loss: 1.067454
(Iteration 6641 / 9800) loss: 1.217513
(Iteration 6651 / 9800) loss: 1.226245
(Iteration 6661 / 9800) loss: 1.252268
(Iteration 6671 / 9800) loss: 1.202014
(Iteration 6681 / 9800) loss: 1.200263
(Iteration 6691 / 9800) loss: 1.189838
(Iteration 6701 / 9800) loss: 1.318750
(Iteration 6711 / 9800) loss: 1.187180
(Iteration 6721 / 9800) loss: 1.204389
(Iteration 6731 / 9800) loss: 1.162424
(Iteration 6741 / 9800) loss: 1.169145
(Iteration 6751 / 9800) loss: 1.446160
(Iteration 6761 / 9800) loss: 1.355710
(Iteration 6771 / 9800) loss: 1.357090
(Iteration 6781 / 9800) loss: 1.411304
(Iteration 6791 / 9800) loss: 1.329316
(Iteration 6801 / 9800) loss: 1.399684
(Iteration 6811 / 9800) loss: 1.396036
(Iteration 6821 / 9800) loss: 1.312581
(Iteration 6831 / 9800) loss: 1.243595
(Iteration 6841 / 9800) loss: 1.213037
(Iteration 6851 / 9800) loss: 1.480055
(Epoch 14 / 20) train acc: 0.609000; val_acc: 0.518000
(Iteration 6861 / 9800) loss: 1.266044
(Iteration 6871 / 9800) loss: 1.354011
(Iteration 6881 / 9800) loss: 1.306378
(Iteration 6891 / 9800) loss: 1.140520
```

```
(Iteration 6901 / 9800) loss: 1.276348
(Iteration 6911 / 9800) loss: 1.210514
(Iteration 6921 / 9800) loss: 1.233117
(Iteration 6931 / 9800) loss: 1.207676
(Iteration 6941 / 9800) loss: 1.333612
(Iteration 6951 / 9800) loss: 1.261400
(Iteration 6961 / 9800) loss: 1.211241
(Iteration 6971 / 9800) loss: 1.252184
(Iteration 6981 / 9800) loss: 1.561200
(Iteration 6991 / 9800) loss: 1.379465
(Iteration 7001 / 9800) loss: 1.191069
(Iteration 7011 / 9800) loss: 1.341831
(Iteration 7021 / 9800) loss: 1.389475
(Iteration 7031 / 9800) loss: 1.488020
(Iteration 7041 / 9800) loss: 1.151523
(Iteration 7051 / 9800) loss: 1.365238
(Iteration 7061 / 9800) loss: 1.297241
(Iteration 7071 / 9800) loss: 1.329921
(Iteration 7081 / 9800) loss: 1.274406
(Iteration 7091 / 9800) loss: 1.111380
(Iteration 7101 / 9800) loss: 1.506175
(Iteration 7111 / 9800) loss: 1.369610
(Iteration 7121 / 9800) loss: 1.173902
(Iteration 7131 / 9800) loss: 1.119927
(Iteration 7141 / 9800) loss: 1.470361
(Iteration 7151 / 9800) loss: 1.366379
(Iteration 7161 / 9800) loss: 1.266451
(Iteration 7171 / 9800) loss: 1.184952
(Iteration 7181 / 9800) loss: 1.285609
(Iteration 7191 / 9800) loss: 1.194863
(Iteration 7201 / 9800) loss: 1.397702
(Iteration 7211 / 9800) loss: 1.419660
(Iteration 7221 / 9800) loss: 1.287202
(Iteration 7231 / 9800) loss: 1.372728
(Iteration 7241 / 9800) loss: 1.138694
(Iteration 7251 / 9800) loss: 1.309894
(Iteration 7261 / 9800) loss: 1.149097
(Iteration 7271 / 9800) loss: 1.085548
(Iteration 7281 / 9800) loss: 1.202478
(Iteration 7291 / 9800) loss: 1.143957
(Iteration 7301 / 9800) loss: 1.437627
(Iteration 7311 / 9800) loss: 1.174041
(Iteration 7321 / 9800) loss: 1.388440
(Iteration 7331 / 9800) loss: 1.329205
(Iteration 7341 / 9800) loss: 1.196120
(Epoch 15 / 20) train acc: 0.603000; val_acc: 0.528000
(Iteration 7351 / 9800) loss: 1.460395
(Iteration 7361 / 9800) loss: 1.248018
```

```
(Iteration 7371 / 9800) loss: 1.257212
(Iteration 7381 / 9800) loss: 1.283590
(Iteration 7391 / 9800) loss: 1.203351
(Iteration 7401 / 9800) loss: 1.103133
(Iteration 7411 / 9800) loss: 1.287308
(Iteration 7421 / 9800) loss: 1.430225
(Iteration 7431 / 9800) loss: 1.442278
(Iteration 7441 / 9800) loss: 1.222754
(Iteration 7451 / 9800) loss: 1.217968
(Iteration 7461 / 9800) loss: 1.308908
(Iteration 7471 / 9800) loss: 1.253782
(Iteration 7481 / 9800) loss: 1.293157
(Iteration 7491 / 9800) loss: 1.215279
(Iteration 7501 / 9800) loss: 1.144821
(Iteration 7511 / 9800) loss: 1.050847
(Iteration 7521 / 9800) loss: 1.284891
(Iteration 7531 / 9800) loss: 1.148203
(Iteration 7541 / 9800) loss: 0.998273
(Iteration 7551 / 9800) loss: 1.364778
(Iteration 7561 / 9800) loss: 1.390828
(Iteration 7571 / 9800) loss: 1.310729
(Iteration 7581 / 9800) loss: 1.315335
(Iteration 7591 / 9800) loss: 1.267540
(Iteration 7601 / 9800) loss: 1.199101
(Iteration 7611 / 9800) loss: 1.400759
(Iteration 7621 / 9800) loss: 1.168889
(Iteration 7631 / 9800) loss: 1.200922
(Iteration 7641 / 9800) loss: 1.073017
(Iteration 7651 / 9800) loss: 1.189224
(Iteration 7661 / 9800) loss: 1.357085
(Iteration 7671 / 9800) loss: 1.094949
(Iteration 7681 / 9800) loss: 1.102563
(Iteration 7691 / 9800) loss: 1.279954
(Iteration 7701 / 9800) loss: 1.083870
(Iteration 7711 / 9800) loss: 1.312679
(Iteration 7721 / 9800) loss: 1.395313
(Iteration 7731 / 9800) loss: 1.133644
(Iteration 7741 / 9800) loss: 1.235159
(Iteration 7751 / 9800) loss: 1.226822
(Iteration 7761 / 9800) loss: 1.286629
(Iteration 7771 / 9800) loss: 1.410690
(Iteration 7781 / 9800) loss: 1.332337
(Iteration 7791 / 9800) loss: 1.212847
(Iteration 7801 / 9800) loss: 1.194667
(Iteration 7811 / 9800) loss: 1.421199
(Iteration 7821 / 9800) loss: 1.383660
(Iteration 7831 / 9800) loss: 1.442720
(Epoch 16 / 20) train acc: 0.585000; val_acc: 0.523000
```

```
(Iteration 7841 / 9800) loss: 1.296083
(Iteration 7851 / 9800) loss: 1.468313
(Iteration 7861 / 9800) loss: 1.178579
(Iteration 7871 / 9800) loss: 1.228564
(Iteration 7881 / 9800) loss: 1.180042
(Iteration 7891 / 9800) loss: 1.166701
(Iteration 7901 / 9800) loss: 1.242305
(Iteration 7911 / 9800) loss: 1.267140
(Iteration 7921 / 9800) loss: 1.409211
(Iteration 7931 / 9800) loss: 1.495287
(Iteration 7941 / 9800) loss: 1.204852
(Iteration 7951 / 9800) loss: 1.247204
(Iteration 7961 / 9800) loss: 1.333547
(Iteration 7971 / 9800) loss: 1.273678
(Iteration 7981 / 9800) loss: 1.462316
(Iteration 7991 / 9800) loss: 1.397434
(Iteration 8001 / 9800) loss: 1.222033
(Iteration 8011 / 9800) loss: 1.202341
(Iteration 8021 / 9800) loss: 1.281271
(Iteration 8031 / 9800) loss: 1.341428
(Iteration 8041 / 9800) loss: 1.369757
(Iteration 8051 / 9800) loss: 1.239256
(Iteration 8061 / 9800) loss: 1.163795
(Iteration 8071 / 9800) loss: 1.127236
(Iteration 8081 / 9800) loss: 1.219583
(Iteration 8091 / 9800) loss: 1.103029
(Iteration 8101 / 9800) loss: 1.210864
(Iteration 8111 / 9800) loss: 1.205897
(Iteration 8121 / 9800) loss: 1.222558
(Iteration 8131 / 9800) loss: 1.156753
(Iteration 8141 / 9800) loss: 1.247133
(Iteration 8151 / 9800) loss: 1.332216
(Iteration 8161 / 9800) loss: 1.486525
(Iteration 8171 / 9800) loss: 1.475694
(Iteration 8181 / 9800) loss: 1.290096
(Iteration 8191 / 9800) loss: 1.224993
(Iteration 8201 / 9800) loss: 1.297934
(Iteration 8211 / 9800) loss: 1.134460
(Iteration 8221 / 9800) loss: 1.438978
(Iteration 8231 / 9800) loss: 1.229573
(Iteration 8241 / 9800) loss: 1.254230
(Iteration 8251 / 9800) loss: 1.458418
(Iteration 8261 / 9800) loss: 1.301374
(Iteration 8271 / 9800) loss: 1.099748
(Iteration 8281 / 9800) loss: 1.254481
(Iteration 8291 / 9800) loss: 1.283844
(Iteration 8301 / 9800) loss: 1.027833
(Iteration 8311 / 9800) loss: 1.409621
```

```
(Iteration 8321 / 9800) loss: 1.168277
(Epoch 17 / 20) train acc: 0.577000; val_acc: 0.496000
(Iteration 8331 / 9800) loss: 1.358661
(Iteration 8341 / 9800) loss: 1.451694
(Iteration 8351 / 9800) loss: 1.192413
(Iteration 8361 / 9800) loss: 0.970514
(Iteration 8371 / 9800) loss: 1.263911
(Iteration 8381 / 9800) loss: 1.486369
(Iteration 8391 / 9800) loss: 1.206785
(Iteration 8401 / 9800) loss: 1.249844
(Iteration 8411 / 9800) loss: 1.266518
(Iteration 8421 / 9800) loss: 1.213493
(Iteration 8431 / 9800) loss: 1.507175
(Iteration 8441 / 9800) loss: 1.096459
(Iteration 8451 / 9800) loss: 1.162788
(Iteration 8461 / 9800) loss: 1.279452
(Iteration 8471 / 9800) loss: 1.107722
(Iteration 8481 / 9800) loss: 1.212965
(Iteration 8491 / 9800) loss: 1.224820
(Iteration 8501 / 9800) loss: 1.215957
(Iteration 8511 / 9800) loss: 1.244741
(Iteration 8521 / 9800) loss: 1.128823
(Iteration 8531 / 9800) loss: 1.152951
(Iteration 8541 / 9800) loss: 1.253665
(Iteration 8551 / 9800) loss: 1.555345
(Iteration 8561 / 9800) loss: 1.310360
(Iteration 8571 / 9800) loss: 1.405679
(Iteration 8581 / 9800) loss: 1.253795
(Iteration 8591 / 9800) loss: 1.384572
(Iteration 8601 / 9800) loss: 1.222643
(Iteration 8611 / 9800) loss: 1.198954
(Iteration 8621 / 9800) loss: 1.172121
(Iteration 8631 / 9800) loss: 1.126314
(Iteration 8641 / 9800) loss: 1.173999
(Iteration 8651 / 9800) loss: 1.153252
(Iteration 8661 / 9800) loss: 1.216130
(Iteration 8671 / 9800) loss: 1.278123
(Iteration 8681 / 9800) loss: 1.392868
(Iteration 8691 / 9800) loss: 1.069154
(Iteration 8701 / 9800) loss: 1.330202
(Iteration 8711 / 9800) loss: 1.493254
(Iteration 8721 / 9800) loss: 1.286734
(Iteration 8731 / 9800) loss: 1.407367
(Iteration 8741 / 9800) loss: 1.144580
(Iteration 8751 / 9800) loss: 1.162973
(Iteration 8761 / 9800) loss: 1.162179
(Iteration 8771 / 9800) loss: 1.365447
(Iteration 8781 / 9800) loss: 1.562149
```

```
(Iteration 8791 / 9800) loss: 1.264397
(Iteration 8801 / 9800) loss: 1.267924
(Iteration 8811 / 9800) loss: 1.228729
(Epoch 18 / 20) train acc: 0.591000; val_acc: 0.526000
(Iteration 8821 / 9800) loss: 1.261723
(Iteration 8831 / 9800) loss: 1.162297
(Iteration 8841 / 9800) loss: 1.241367
(Iteration 8851 / 9800) loss: 1.311610
(Iteration 8861 / 9800) loss: 1.310379
(Iteration 8871 / 9800) loss: 1.154147
(Iteration 8881 / 9800) loss: 1.083228
(Iteration 8891 / 9800) loss: 1.259011
(Iteration 8901 / 9800) loss: 1.133570
(Iteration 8911 / 9800) loss: 1.265915
(Iteration 8921 / 9800) loss: 1.067702
(Iteration 8931 / 9800) loss: 1.183272
(Iteration 8941 / 9800) loss: 1.300488
(Iteration 8951 / 9800) loss: 1.143421
(Iteration 8961 / 9800) loss: 1.185577
(Iteration 8971 / 9800) loss: 1.441512
(Iteration 8981 / 9800) loss: 1.285900
(Iteration 8991 / 9800) loss: 1.224242
(Iteration 9001 / 9800) loss: 1.286565
(Iteration 9011 / 9800) loss: 1.174901
(Iteration 9021 / 9800) loss: 1.297543
(Iteration 9031 / 9800) loss: 1.264150
(Iteration 9041 / 9800) loss: 1.273310
(Iteration 9051 / 9800) loss: 1.130911
(Iteration 9061 / 9800) loss: 1.509814
(Iteration 9071 / 9800) loss: 1.301578
(Iteration 9081 / 9800) loss: 1.212083
(Iteration 9091 / 9800) loss: 1.149881
(Iteration 9101 / 9800) loss: 1.329548
(Iteration 9111 / 9800) loss: 1.323112
(Iteration 9121 / 9800) loss: 1.205652
(Iteration 9131 / 9800) loss: 1.360328
(Iteration 9141 / 9800) loss: 1.482968
(Iteration 9151 / 9800) loss: 1.129178
(Iteration 9161 / 9800) loss: 1.233466
(Iteration 9171 / 9800) loss: 1.101068
(Iteration 9181 / 9800) loss: 1.155922
(Iteration 9191 / 9800) loss: 0.984023
(Iteration 9201 / 9800) loss: 1.288293
(Iteration 9211 / 9800) loss: 1.262230
(Iteration 9221 / 9800) loss: 1.154903
(Iteration 9231 / 9800) loss: 1.339150
(Iteration 9241 / 9800) loss: 1.390212
(Iteration 9251 / 9800) loss: 1.161360
```

```
(Iteration 9261 / 9800) loss: 1.384152
(Iteration 9271 / 9800) loss: 1.276100
(Iteration 9281 / 9800) loss: 1.401167
(Iteration 9291 / 9800) loss: 1.244488
(Iteration 9301 / 9800) loss: 1.158727
(Epoch 19 / 20) train acc: 0.604000; val_acc: 0.515000
(Iteration 9311 / 9800) loss: 1.184453
(Iteration 9321 / 9800) loss: 1.179264
(Iteration 9331 / 9800) loss: 1.159542
(Iteration 9341 / 9800) loss: 1.188765
(Iteration 9351 / 9800) loss: 1.265228
(Iteration 9361 / 9800) loss: 1.388650
(Iteration 9371 / 9800) loss: 1.125984
(Iteration 9381 / 9800) loss: 1.284171
(Iteration 9391 / 9800) loss: 1.109045
(Iteration 9401 / 9800) loss: 1.362102
(Iteration 9411 / 9800) loss: 1.386897
(Iteration 9421 / 9800) loss: 1.099160
(Iteration 9431 / 9800) loss: 1.228140
(Iteration 9441 / 9800) loss: 1.382766
(Iteration 9451 / 9800) loss: 1.359814
(Iteration 9461 / 9800) loss: 1.132735
(Iteration 9471 / 9800) loss: 1.473135
(Iteration 9481 / 9800) loss: 1.291919
(Iteration 9491 / 9800) loss: 1.433466
(Iteration 9501 / 9800) loss: 1.092671
(Iteration 9511 / 9800) loss: 1.187341
(Iteration 9521 / 9800) loss: 1.205030
(Iteration 9531 / 9800) loss: 1.216217
(Iteration 9541 / 9800) loss: 1.394838
(Iteration 9551 / 9800) loss: 1.329576
(Iteration 9561 / 9800) loss: 1.162072
(Iteration 9571 / 9800) loss: 1.262059
(Iteration 9581 / 9800) loss: 1.466078
(Iteration 9591 / 9800) loss: 1.051385
(Iteration 9601 / 9800) loss: 1.155945
(Iteration 9611 / 9800) loss: 1.069860
(Iteration 9621 / 9800) loss: 1.294980
(Iteration 9631 / 9800) loss: 1.290461
(Iteration 9641 / 9800) loss: 1.235274
(Iteration 9651 / 9800) loss: 1.152226
(Iteration 9661 / 9800) loss: 1.398562
(Iteration 9671 / 9800) loss: 1.236520
(Iteration 9681 / 9800) loss: 1.159139
(Iteration 9691 / 9800) loss: 1.084311
(Iteration 9701 / 9800) loss: 1.469333
(Iteration 9711 / 9800) loss: 1.176784
(Iteration 9721 / 9800) loss: 1.116448
```

```
(Iteration 9731 / 9800) loss: 1.146925
(Iteration 9741 / 9800) loss: 1.436041
(Iteration 9751 / 9800) loss: 1.115619
(Iteration 9761 / 9800) loss: 1.235001
(Iteration 9771 / 9800) loss: 1.212260
(Iteration 9781 / 9800) loss: 1.053125
(Iteration 9791 / 9800) loss: 1.208765
(Epoch 20 / 20) train acc: 0.614000; val_acc: 0.544000
accuracy:0.544 with lr:0.0005,reg:0.25,epoch:20,hidden_dim:100
(Iteration 1 / 4900) loss: 2.336272
(Epoch 0 / 10) train acc: 0.136000; val_acc: 0.114000
(Iteration 11 / 4900) loss: 2.301169
(Iteration 21 / 4900) loss: 2.270486
(Iteration 31 / 4900) loss: 2.132866
(Iteration 41 / 4900) loss: 2.206335
(Iteration 51 / 4900) loss: 2.113370
(Iteration 61 / 4900) loss: 1.993013
(Iteration 71 / 4900) loss: 2.022531
(Iteration 81 / 4900) loss: 1.905526
(Iteration 91 / 4900) loss: 1.924174
(Iteration 101 / 4900) loss: 1.899368
(Iteration 111 / 4900) loss: 1.916746
(Iteration 121 / 4900) loss: 1.973509
(Iteration 131 / 4900) loss: 1.919512
(Iteration 141 / 4900) loss: 1.884298
(Iteration 151 / 4900) loss: 1.920083
(Iteration 161 / 4900) loss: 1.828133
(Iteration 171 / 4900) loss: 1.904653
(Iteration 181 / 4900) loss: 1.885518
(Iteration 191 / 4900) loss: 1.703484
(Iteration 201 / 4900) loss: 1.734553
(Iteration 211 / 4900) loss: 1.821337
(Iteration 221 / 4900) loss: 1.703481
(Iteration 231 / 4900) loss: 1.881798
(Iteration 241 / 4900) loss: 1.757636
(Iteration 251 / 4900) loss: 1.895498
(Iteration 261 / 4900) loss: 1.651260
(Iteration 271 / 4900) loss: 2.055926
(Iteration 281 / 4900) loss: 1.789300
(Iteration 291 / 4900) loss: 1.698013
(Iteration 301 / 4900) loss: 1.729696
(Iteration 311 / 4900) loss: 1.802997
(Iteration 321 / 4900) loss: 1.668224
(Iteration 331 / 4900) loss: 1.946934
(Iteration 341 / 4900) loss: 1.669382
(Iteration 351 / 4900) loss: 1.770482
(Iteration 361 / 4900) loss: 1.745667
(Iteration 371 / 4900) loss: 1.656960
```

```
(Iteration 381 / 4900) loss: 1.709072
(Iteration 391 / 4900) loss: 1.544716
(Iteration 401 / 4900) loss: 1.746746
(Iteration 411 / 4900) loss: 1.671257
(Iteration 421 / 4900) loss: 1.766551
(Iteration 431 / 4900) loss: 1.611978
(Iteration 441 / 4900) loss: 1.588953
(Iteration 451 / 4900) loss: 1.486347
(Iteration 461 / 4900) loss: 1.830013
(Iteration 471 / 4900) loss: 1.753290
(Iteration 481 / 4900) loss: 1.766296
(Epoch 1 / 10) train acc: 0.434000; val_acc: 0.451000
(Iteration 491 / 4900) loss: 1.695699
(Iteration 501 / 4900) loss: 1.476121
(Iteration 511 / 4900) loss: 1.733065
(Iteration 521 / 4900) loss: 1.733320
(Iteration 531 / 4900) loss: 1.571242
(Iteration 541 / 4900) loss: 1.731221
(Iteration 551 / 4900) loss: 1.496505
(Iteration 561 / 4900) loss: 1.636413
(Iteration 571 / 4900) loss: 1.633746
(Iteration 581 / 4900) loss: 1.444034
(Iteration 591 / 4900) loss: 1.563067
(Iteration 601 / 4900) loss: 1.677805
(Iteration 611 / 4900) loss: 1.548955
(Iteration 621 / 4900) loss: 1.894374
(Iteration 631 / 4900) loss: 1.575190
(Iteration 641 / 4900) loss: 1.824126
(Iteration 651 / 4900) loss: 1.621979
(Iteration 661 / 4900) loss: 1.655588
(Iteration 671 / 4900) loss: 1.595704
(Iteration 681 / 4900) loss: 1.613800
(Iteration 691 / 4900) loss: 1.357651
(Iteration 701 / 4900) loss: 1.528895
(Iteration 711 / 4900) loss: 1.714966
(Iteration 721 / 4900) loss: 1.632448
(Iteration 731 / 4900) loss: 1.360613
(Iteration 741 / 4900) loss: 1.516389
(Iteration 751 / 4900) loss: 1.346558
(Iteration 761 / 4900) loss: 1.410149
(Iteration 771 / 4900) loss: 1.465327
(Iteration 781 / 4900) loss: 1.558060
(Iteration 791 / 4900) loss: 1.607393
(Iteration 801 / 4900) loss: 1.509902
(Iteration 811 / 4900) loss: 1.726309
(Iteration 821 / 4900) loss: 1.361147
(Iteration 831 / 4900) loss: 1.669437
(Iteration 841 / 4900) loss: 1.516301
```

```
(Iteration 851 / 4900) loss: 1.533920
(Iteration 861 / 4900) loss: 1.415106
(Iteration 871 / 4900) loss: 1.409745
(Iteration 881 / 4900) loss: 1.517307
(Iteration 891 / 4900) loss: 1.535101
(Iteration 901 / 4900) loss: 1.394923
(Iteration 911 / 4900) loss: 1.491839
(Iteration 921 / 4900) loss: 1.515737
(Iteration 931 / 4900) loss: 1.699351
(Iteration 941 / 4900) loss: 1.512845
(Iteration 951 / 4900) loss: 1.605881
(Iteration 961 / 4900) loss: 1.511261
(Iteration 971 / 4900) loss: 1.634046
(Epoch 2 / 10) train acc: 0.472000; val_acc: 0.475000
(Iteration 981 / 4900) loss: 1.612673
(Iteration 991 / 4900) loss: 1.511104
(Iteration 1001 / 4900) loss: 1.448445
(Iteration 1011 / 4900) loss: 1.599823
(Iteration 1021 / 4900) loss: 1.546873
(Iteration 1031 / 4900) loss: 1.399658
(Iteration 1041 / 4900) loss: 1.429500
(Iteration 1051 / 4900) loss: 1.472042
(Iteration 1061 / 4900) loss: 1.553187
(Iteration 1071 / 4900) loss: 1.394033
(Iteration 1081 / 4900) loss: 1.426647
(Iteration 1091 / 4900) loss: 1.522943
(Iteration 1101 / 4900) loss: 1.292624
(Iteration 1111 / 4900) loss: 1.498828
(Iteration 1121 / 4900) loss: 1.492555
(Iteration 1131 / 4900) loss: 1.598366
(Iteration 1141 / 4900) loss: 1.436101
(Iteration 1151 / 4900) loss: 1.476401
(Iteration 1161 / 4900) loss: 1.477158
(Iteration 1171 / 4900) loss: 1.402768
(Iteration 1181 / 4900) loss: 1.365372
(Iteration 1191 / 4900) loss: 1.643556
(Iteration 1201 / 4900) loss: 1.289520
(Iteration 1211 / 4900) loss: 1.537452
(Iteration 1221 / 4900) loss: 1.503126
(Iteration 1231 / 4900) loss: 1.493036
(Iteration 1241 / 4900) loss: 1.296548
(Iteration 1251 / 4900) loss: 1.452821
(Iteration 1261 / 4900) loss: 1.384777
(Iteration 1271 / 4900) loss: 1.432757
(Iteration 1281 / 4900) loss: 1.629342
(Iteration 1291 / 4900) loss: 1.509635
(Iteration 1301 / 4900) loss: 1.360832
(Iteration 1311 / 4900) loss: 1.518655
```

```
(Iteration 1321 / 4900) loss: 1.263660
(Iteration 1331 / 4900) loss: 1.357569
(Iteration 1341 / 4900) loss: 1.404614
(Iteration 1351 / 4900) loss: 1.430200
(Iteration 1361 / 4900) loss: 1.473129
(Iteration 1371 / 4900) loss: 1.568611
(Iteration 1381 / 4900) loss: 1.403500
(Iteration 1391 / 4900) loss: 1.476094
(Iteration 1401 / 4900) loss: 1.469521
(Iteration 1411 / 4900) loss: 1.451222
(Iteration 1421 / 4900) loss: 1.567573
(Iteration 1431 / 4900) loss: 1.304699
(Iteration 1441 / 4900) loss: 1.253665
(Iteration 1451 / 4900) loss: 1.447875
(Iteration 1461 / 4900) loss: 1.347241
(Epoch 3 / 10) train acc: 0.517000; val_acc: 0.491000
(Iteration 1471 / 4900) loss: 1.470711
(Iteration 1481 / 4900) loss: 1.581620
(Iteration 1491 / 4900) loss: 1.486862
(Iteration 1501 / 4900) loss: 1.317789
(Iteration 1511 / 4900) loss: 1.480473
(Iteration 1521 / 4900) loss: 1.445681
(Iteration 1531 / 4900) loss: 1.347939
(Iteration 1541 / 4900) loss: 1.547991
(Iteration 1551 / 4900) loss: 1.447793
(Iteration 1561 / 4900) loss: 1.402031
(Iteration 1571 / 4900) loss: 1.414782
(Iteration 1581 / 4900) loss: 1.389867
(Iteration 1591 / 4900) loss: 1.376865
(Iteration 1601 / 4900) loss: 1.288426
(Iteration 1611 / 4900) loss: 1.255564
(Iteration 1621 / 4900) loss: 1.387798
(Iteration 1631 / 4900) loss: 1.409148
(Iteration 1641 / 4900) loss: 1.277256
(Iteration 1651 / 4900) loss: 1.502869
(Iteration 1661 / 4900) loss: 1.438398
(Iteration 1671 / 4900) loss: 1.508628
(Iteration 1681 / 4900) loss: 1.508793
(Iteration 1691 / 4900) loss: 1.474418
(Iteration 1701 / 4900) loss: 1.295026
(Iteration 1711 / 4900) loss: 1.362692
(Iteration 1721 / 4900) loss: 1.354698
(Iteration 1731 / 4900) loss: 1.290478
(Iteration 1741 / 4900) loss: 1.622768
(Iteration 1751 / 4900) loss: 1.344392
(Iteration 1761 / 4900) loss: 1.395793
(Iteration 1771 / 4900) loss: 1.542947
(Iteration 1781 / 4900) loss: 1.153699
```

```
(Iteration 1791 / 4900) loss: 1.318845
(Iteration 1801 / 4900) loss: 1.351454
(Iteration 1811 / 4900) loss: 1.313705
(Iteration 1821 / 4900) loss: 1.194726
(Iteration 1831 / 4900) loss: 1.461792
(Iteration 1841 / 4900) loss: 1.514703
(Iteration 1851 / 4900) loss: 1.339335
(Iteration 1861 / 4900) loss: 1.452617
(Iteration 1871 / 4900) loss: 1.323451
(Iteration 1881 / 4900) loss: 1.423197
(Iteration 1891 / 4900) loss: 1.356468
(Iteration 1901 / 4900) loss: 1.388331
(Iteration 1911 / 4900) loss: 1.524601
(Iteration 1921 / 4900) loss: 1.494961
(Iteration 1931 / 4900) loss: 1.387907
(Iteration 1941 / 4900) loss: 1.346120
(Iteration 1951 / 4900) loss: 1.324741
(Epoch 4 / 10) train acc: 0.537000; val_acc: 0.485000
(Iteration 1961 / 4900) loss: 1.321604
(Iteration 1971 / 4900) loss: 1.412021
(Iteration 1981 / 4900) loss: 1.449929
(Iteration 1991 / 4900) loss: 1.377398
(Iteration 2001 / 4900) loss: 1.475870
(Iteration 2011 / 4900) loss: 1.387066
(Iteration 2021 / 4900) loss: 1.326535
(Iteration 2031 / 4900) loss: 1.411193
(Iteration 2041 / 4900) loss: 1.275776
(Iteration 2051 / 4900) loss: 1.409213
(Iteration 2061 / 4900) loss: 1.371904
(Iteration 2071 / 4900) loss: 1.213862
(Iteration 2081 / 4900) loss: 1.301064
(Iteration 2091 / 4900) loss: 1.287216
(Iteration 2101 / 4900) loss: 1.407770
(Iteration 2111 / 4900) loss: 1.135867
(Iteration 2121 / 4900) loss: 1.456814
(Iteration 2131 / 4900) loss: 1.324789
(Iteration 2141 / 4900) loss: 1.403494
(Iteration 2151 / 4900) loss: 1.509846
(Iteration 2161 / 4900) loss: 1.460175
(Iteration 2171 / 4900) loss: 1.539653
(Iteration 2181 / 4900) loss: 1.431942
(Iteration 2191 / 4900) loss: 1.388967
(Iteration 2201 / 4900) loss: 1.391127
(Iteration 2211 / 4900) loss: 1.492167
(Iteration 2221 / 4900) loss: 1.160486
(Iteration 2231 / 4900) loss: 1.328333
(Iteration 2241 / 4900) loss: 1.480752
(Iteration 2251 / 4900) loss: 1.331064
```

```
(Iteration 2261 / 4900) loss: 1.213591
(Iteration 2271 / 4900) loss: 1.306084
(Iteration 2281 / 4900) loss: 1.188008
(Iteration 2291 / 4900) loss: 1.399217
(Iteration 2301 / 4900) loss: 1.439293
(Iteration 2311 / 4900) loss: 1.310297
(Iteration 2321 / 4900) loss: 1.250711
(Iteration 2331 / 4900) loss: 1.577746
(Iteration 2341 / 4900) loss: 1.412878
(Iteration 2351 / 4900) loss: 1.389816
(Iteration 2361 / 4900) loss: 1.377744
(Iteration 2371 / 4900) loss: 1.389631
(Iteration 2381 / 4900) loss: 1.341845
(Iteration 2391 / 4900) loss: 1.411610
(Iteration 2401 / 4900) loss: 1.531072
(Iteration 2411 / 4900) loss: 1.551743
(Iteration 2421 / 4900) loss: 1.132416
(Iteration 2431 / 4900) loss: 1.400939
(Iteration 2441 / 4900) loss: 1.291057
(Epoch 5 / 10) train acc: 0.562000; val_acc: 0.495000
(Iteration 2451 / 4900) loss: 1.346066
(Iteration 2461 / 4900) loss: 1.343200
(Iteration 2471 / 4900) loss: 1.402170
(Iteration 2481 / 4900) loss: 1.347290
(Iteration 2491 / 4900) loss: 1.415568
(Iteration 2501 / 4900) loss: 1.320435
(Iteration 2511 / 4900) loss: 1.376345
(Iteration 2521 / 4900) loss: 1.344777
(Iteration 2531 / 4900) loss: 1.408909
(Iteration 2541 / 4900) loss: 1.155646
(Iteration 2551 / 4900) loss: 1.278763
(Iteration 2561 / 4900) loss: 1.144586
(Iteration 2571 / 4900) loss: 1.324934
(Iteration 2581 / 4900) loss: 1.144635
(Iteration 2591 / 4900) loss: 1.377339
(Iteration 2601 / 4900) loss: 1.247090
(Iteration 2611 / 4900) loss: 1.314755
(Iteration 2621 / 4900) loss: 1.305847
(Iteration 2631 / 4900) loss: 1.384267
(Iteration 2641 / 4900) loss: 1.335362
(Iteration 2651 / 4900) loss: 1.477021
(Iteration 2661 / 4900) loss: 1.427370
(Iteration 2671 / 4900) loss: 1.421334
(Iteration 2681 / 4900) loss: 1.322918
(Iteration 2691 / 4900) loss: 1.133688
(Iteration 2701 / 4900) loss: 1.265391
(Iteration 2711 / 4900) loss: 1.329431
(Iteration 2721 / 4900) loss: 1.337108
```

```
(Iteration 2731 / 4900) loss: 1.088884
(Iteration 2741 / 4900) loss: 1.499491
(Iteration 2751 / 4900) loss: 1.312316
(Iteration 2761 / 4900) loss: 1.673378
(Iteration 2771 / 4900) loss: 1.475133
(Iteration 2781 / 4900) loss: 1.250021
(Iteration 2791 / 4900) loss: 1.298365
(Iteration 2801 / 4900) loss: 1.323803
(Iteration 2811 / 4900) loss: 1.345936
(Iteration 2821 / 4900) loss: 1.435137
(Iteration 2831 / 4900) loss: 1.450739
(Iteration 2841 / 4900) loss: 1.270586
(Iteration 2851 / 4900) loss: 1.153063
(Iteration 2861 / 4900) loss: 1.174004
(Iteration 2871 / 4900) loss: 1.315954
(Iteration 2881 / 4900) loss: 1.317200
(Iteration 2891 / 4900) loss: 1.486072
(Iteration 2901 / 4900) loss: 1.406075
(Iteration 2911 / 4900) loss: 1.270534
(Iteration 2921 / 4900) loss: 1.435162
(Iteration 2931 / 4900) loss: 1.295551
(Epoch 6 / 10) train acc: 0.564000; val_acc: 0.494000
(Iteration 2941 / 4900) loss: 1.187820
(Iteration 2951 / 4900) loss: 1.288400
(Iteration 2961 / 4900) loss: 1.067575
(Iteration 2971 / 4900) loss: 1.268518
(Iteration 2981 / 4900) loss: 1.296958
(Iteration 2991 / 4900) loss: 1.623798
(Iteration 3001 / 4900) loss: 1.218546
(Iteration 3011 / 4900) loss: 1.116417
(Iteration 3021 / 4900) loss: 1.156661
(Iteration 3031 / 4900) loss: 1.049811
(Iteration 3041 / 4900) loss: 1.266623
(Iteration 3051 / 4900) loss: 1.322261
(Iteration 3061 / 4900) loss: 1.301092
(Iteration 3071 / 4900) loss: 1.245158
(Iteration 3081 / 4900) loss: 1.393797
(Iteration 3091 / 4900) loss: 1.202868
(Iteration 3101 / 4900) loss: 1.375162
(Iteration 3111 / 4900) loss: 1.298339
(Iteration 3121 / 4900) loss: 1.251215
(Iteration 3131 / 4900) loss: 1.307128
(Iteration 3141 / 4900) loss: 1.354268
(Iteration 3151 / 4900) loss: 1.339304
(Iteration 3161 / 4900) loss: 1.325616
(Iteration 3171 / 4900) loss: 1.207587
(Iteration 3181 / 4900) loss: 1.302146
(Iteration 3191 / 4900) loss: 1.237826
```

```
(Iteration 3201 / 4900) loss: 1.275373
(Iteration 3211 / 4900) loss: 1.211164
(Iteration 3221 / 4900) loss: 1.240343
(Iteration 3231 / 4900) loss: 1.303024
(Iteration 3241 / 4900) loss: 1.194440
(Iteration 3251 / 4900) loss: 1.209748
(Iteration 3261 / 4900) loss: 1.096432
(Iteration 3271 / 4900) loss: 1.357203
(Iteration 3281 / 4900) loss: 1.160047
(Iteration 3291 / 4900) loss: 1.311183
(Iteration 3301 / 4900) loss: 1.358931
(Iteration 3311 / 4900) loss: 1.474287
(Iteration 3321 / 4900) loss: 1.255714
(Iteration 3331 / 4900) loss: 1.360880
(Iteration 3341 / 4900) loss: 1.263006
(Iteration 3351 / 4900) loss: 1.304013
(Iteration 3361 / 4900) loss: 1.169823
(Iteration 3371 / 4900) loss: 1.161061
(Iteration 3381 / 4900) loss: 1.087861
(Iteration 3391 / 4900) loss: 1.123565
(Iteration 3401 / 4900) loss: 1.339959
(Iteration 3411 / 4900) loss: 1.256773
(Iteration 3421 / 4900) loss: 1.420948
(Epoch 7 / 10) train acc: 0.582000; val_acc: 0.508000
(Iteration 3431 / 4900) loss: 1.206367
(Iteration 3441 / 4900) loss: 1.466845
(Iteration 3451 / 4900) loss: 1.138899
(Iteration 3461 / 4900) loss: 1.199246
(Iteration 3471 / 4900) loss: 1.220039
(Iteration 3481 / 4900) loss: 1.115849
(Iteration 3491 / 4900) loss: 1.337434
(Iteration 3501 / 4900) loss: 1.085495
(Iteration 3511 / 4900) loss: 1.192982
(Iteration 3521 / 4900) loss: 1.344439
(Iteration 3531 / 4900) loss: 1.308662
(Iteration 3541 / 4900) loss: 1.118741
(Iteration 3551 / 4900) loss: 1.413192
(Iteration 3561 / 4900) loss: 1.299366
(Iteration 3571 / 4900) loss: 1.219742
(Iteration 3581 / 4900) loss: 1.272889
(Iteration 3591 / 4900) loss: 1.314435
(Iteration 3601 / 4900) loss: 1.119988
(Iteration 3611 / 4900) loss: 1.209820
(Iteration 3621 / 4900) loss: 1.137616
(Iteration 3631 / 4900) loss: 1.121989
(Iteration 3641 / 4900) loss: 1.221893
(Iteration 3651 / 4900) loss: 1.479311
(Iteration 3661 / 4900) loss: 1.232441
```

```
(Iteration 3671 / 4900) loss: 1.170512
(Iteration 3681 / 4900) loss: 1.376888
(Iteration 3691 / 4900) loss: 1.266264
(Iteration 3701 / 4900) loss: 1.463501
(Iteration 3711 / 4900) loss: 1.078551
(Iteration 3721 / 4900) loss: 1.536470
(Iteration 3731 / 4900) loss: 1.319108
(Iteration 3741 / 4900) loss: 1.366768
(Iteration 3751 / 4900) loss: 1.062963
(Iteration 3761 / 4900) loss: 1.146887
(Iteration 3771 / 4900) loss: 1.382608
(Iteration 3781 / 4900) loss: 1.297277
(Iteration 3791 / 4900) loss: 0.868229
(Iteration 3801 / 4900) loss: 1.384589
(Iteration 3811 / 4900) loss: 1.236792
(Iteration 3821 / 4900) loss: 1.178038
(Iteration 3831 / 4900) loss: 1.214358
(Iteration 3841 / 4900) loss: 1.193149
(Iteration 3851 / 4900) loss: 1.280730
(Iteration 3861 / 4900) loss: 1.264692
(Iteration 3871 / 4900) loss: 1.392718
(Iteration 3881 / 4900) loss: 1.152689
(Iteration 3891 / 4900) loss: 1.129105
(Iteration 3901 / 4900) loss: 1.447270
(Iteration 3911 / 4900) loss: 1.395115
(Epoch 8 / 10) train acc: 0.585000; val_acc: 0.507000
(Iteration 3921 / 4900) loss: 1.266728
(Iteration 3931 / 4900) loss: 1.315651
(Iteration 3941 / 4900) loss: 1.197119
(Iteration 3951 / 4900) loss: 1.131125
(Iteration 3961 / 4900) loss: 1.262152
(Iteration 3971 / 4900) loss: 1.500152
(Iteration 3981 / 4900) loss: 1.197744
(Iteration 3991 / 4900) loss: 1.298616
(Iteration 4001 / 4900) loss: 1.144823
(Iteration 4011 / 4900) loss: 1.092349
(Iteration 4021 / 4900) loss: 1.216239
(Iteration 4031 / 4900) loss: 1.294668
(Iteration 4041 / 4900) loss: 1.102512
(Iteration 4051 / 4900) loss: 1.284162
(Iteration 4061 / 4900) loss: 1.308285
(Iteration 4071 / 4900) loss: 1.213938
(Iteration 4081 / 4900) loss: 1.236953
(Iteration 4091 / 4900) loss: 1.285295
(Iteration 4101 / 4900) loss: 1.377314
(Iteration 4111 / 4900) loss: 1.266796
(Iteration 4121 / 4900) loss: 1.465403
(Iteration 4131 / 4900) loss: 1.320915
```

```
(Iteration 4141 / 4900) loss: 1.331391
(Iteration 4151 / 4900) loss: 1.146616
(Iteration 4161 / 4900) loss: 1.179783
(Iteration 4171 / 4900) loss: 1.289194
(Iteration 4181 / 4900) loss: 1.279271
(Iteration 4191 / 4900) loss: 0.966836
(Iteration 4201 / 4900) loss: 1.324307
(Iteration 4211 / 4900) loss: 1.166755
(Iteration 4221 / 4900) loss: 1.208396
(Iteration 4231 / 4900) loss: 1.061965
(Iteration 4241 / 4900) loss: 1.369439
(Iteration 4251 / 4900) loss: 1.234951
(Iteration 4261 / 4900) loss: 1.109912
(Iteration 4271 / 4900) loss: 1.144019
(Iteration 4281 / 4900) loss: 1.260717
(Iteration 4291 / 4900) loss: 1.224669
(Iteration 4301 / 4900) loss: 1.121178
(Iteration 4311 / 4900) loss: 1.220598
(Iteration 4321 / 4900) loss: 1.279772
(Iteration 4331 / 4900) loss: 1.435485
(Iteration 4341 / 4900) loss: 1.314785
(Iteration 4351 / 4900) loss: 1.318872
(Iteration 4361 / 4900) loss: 1.189929
(Iteration 4371 / 4900) loss: 1.120682
(Iteration 4381 / 4900) loss: 1.191530
(Iteration 4391 / 4900) loss: 1.194193
(Iteration 4401 / 4900) loss: 1.383352
(Epoch 9 / 10) train acc: 0.621000; val_acc: 0.481000
(Iteration 4411 / 4900) loss: 1.481846
(Iteration 4421 / 4900) loss: 1.086262
(Iteration 4431 / 4900) loss: 1.047933
(Iteration 4441 / 4900) loss: 1.173588
(Iteration 4451 / 4900) loss: 1.158386
(Iteration 4461 / 4900) loss: 1.087892
(Iteration 4471 / 4900) loss: 1.596524
(Iteration 4481 / 4900) loss: 1.464623
(Iteration 4491 / 4900) loss: 1.239102
(Iteration 4501 / 4900) loss: 1.134133
(Iteration 4511 / 4900) loss: 1.350772
(Iteration 4521 / 4900) loss: 1.391768
(Iteration 4531 / 4900) loss: 1.183058
(Iteration 4541 / 4900) loss: 1.190547
(Iteration 4551 / 4900) loss: 1.313046
(Iteration 4561 / 4900) loss: 1.328695
(Iteration 4571 / 4900) loss: 1.265186
(Iteration 4581 / 4900) loss: 1.136808
(Iteration 4591 / 4900) loss: 1.190596
(Iteration 4601 / 4900) loss: 1.047988
```

```
(Iteration 4611 / 4900) loss: 1.332955
(Iteration 4621 / 4900) loss: 1.512399
(Iteration 4631 / 4900) loss: 1.183968
(Iteration 4641 / 4900) loss: 1.256040
(Iteration 4651 / 4900) loss: 1.489293
(Iteration 4661 / 4900) loss: 1.257823
(Iteration 4671 / 4900) loss: 1.427401
(Iteration 4681 / 4900) loss: 1.180373
(Iteration 4691 / 4900) loss: 1.136050
(Iteration 4701 / 4900) loss: 1.207360
(Iteration 4711 / 4900) loss: 1.194323
(Iteration 4721 / 4900) loss: 1.183141
(Iteration 4731 / 4900) loss: 1.049293
(Iteration 4741 / 4900) loss: 1.263010
(Iteration 4751 / 4900) loss: 1.174196
(Iteration 4761 / 4900) loss: 1.321043
(Iteration 4771 / 4900) loss: 1.290343
(Iteration 4781 / 4900) loss: 1.267899
(Iteration 4791 / 4900) loss: 1.102327
(Iteration 4801 / 4900) loss: 1.258675
(Iteration 4811 / 4900) loss: 1.063385
(Iteration 4821 / 4900) loss: 1.186038
(Iteration 4831 / 4900) loss: 0.984147
(Iteration 4841 / 4900) loss: 1.194646
(Iteration 4851 / 4900) loss: 1.058270
(Iteration 4861 / 4900) loss: 1.199403
(Iteration 4871 / 4900) loss: 1.242919
(Iteration 4881 / 4900) loss: 1.310722
(Iteration 4891 / 4900) loss: 1.344011
(Epoch 10 / 10) train acc: 0.654000; val_acc: 0.509000
accuracy:0.509 with lr:0.0005,reg:0.1,epoch:10,hidden_dim:200
(Iteration 1 / 4900) loss: 2.381791
(Epoch 0 / 10) train acc: 0.122000; val_acc: 0.116000
(Iteration 11 / 4900) loss: 2.346755
(Iteration 21 / 4900) loss: 2.279245
(Iteration 31 / 4900) loss: 2.210375
(Iteration 41 / 4900) loss: 2.132019
(Iteration 51 / 4900) loss: 2.155636
(Iteration 61 / 4900) loss: 1.992246
(Iteration 71 / 4900) loss: 2.057374
(Iteration 81 / 4900) loss: 2.084588
(Iteration 91 / 4900) loss: 1.976405
(Iteration 101 / 4900) loss: 1.940796
(Iteration 111 / 4900) loss: 1.939807
(Iteration 121 / 4900) loss: 1.964921
(Iteration 131 / 4900) loss: 1.966064
(Iteration 141 / 4900) loss: 1.879193
(Iteration 151 / 4900) loss: 1.891202
```

```
(Iteration 161 / 4900) loss: 1.748210
(Iteration 171 / 4900) loss: 1.910443
(Iteration 181 / 4900) loss: 1.988188
(Iteration 191 / 4900) loss: 1.747177
(Iteration 201 / 4900) loss: 1.707992
(Iteration 211 / 4900) loss: 1.902131
(Iteration 221 / 4900) loss: 1.772584
(Iteration 231 / 4900) loss: 1.769441
(Iteration 241 / 4900) loss: 1.830521
(Iteration 251 / 4900) loss: 1.776463
(Iteration 261 / 4900) loss: 2.027946
(Iteration 271 / 4900) loss: 1.679390
(Iteration 281 / 4900) loss: 1.792686
(Iteration 291 / 4900) loss: 1.800751
(Iteration 301 / 4900) loss: 1.760238
(Iteration 311 / 4900) loss: 1.714756
(Iteration 321 / 4900) loss: 1.867977
(Iteration 331 / 4900) loss: 1.799584
(Iteration 341 / 4900) loss: 1.710987
(Iteration 351 / 4900) loss: 1.721534
(Iteration 361 / 4900) loss: 1.691947
(Iteration 371 / 4900) loss: 1.782388
(Iteration 381 / 4900) loss: 1.703237
(Iteration 391 / 4900) loss: 1.855648
(Iteration 401 / 4900) loss: 1.747980
(Iteration 411 / 4900) loss: 1.685876
(Iteration 421 / 4900) loss: 1.552015
(Iteration 431 / 4900) loss: 1.676273
(Iteration 441 / 4900) loss: 1.766370
(Iteration 451 / 4900) loss: 1.714443
(Iteration 461 / 4900) loss: 1.585628
(Iteration 471 / 4900) loss: 1.797010
(Iteration 481 / 4900) loss: 1.598347
(Epoch 1 / 10) train acc: 0.426000; val_acc: 0.427000
(Iteration 491 / 4900) loss: 1.627248
(Iteration 501 / 4900) loss: 1.636011
(Iteration 511 / 4900) loss: 1.732673
(Iteration 521 / 4900) loss: 1.612663
(Iteration 531 / 4900) loss: 1.864038
(Iteration 541 / 4900) loss: 1.600553
(Iteration 551 / 4900) loss: 1.404311
(Iteration 561 / 4900) loss: 1.553852
(Iteration 571 / 4900) loss: 1.625544
(Iteration 581 / 4900) loss: 1.608677
(Iteration 591 / 4900) loss: 1.573409
(Iteration 601 / 4900) loss: 1.476360
(Iteration 611 / 4900) loss: 1.760128
(Iteration 621 / 4900) loss: 1.642446
```

```
(Iteration 631 / 4900) loss: 1.589602
(Iteration 641 / 4900) loss: 1.760819
(Iteration 651 / 4900) loss: 1.901604
(Iteration 661 / 4900) loss: 1.711627
(Iteration 671 / 4900) loss: 1.646379
(Iteration 681 / 4900) loss: 1.681084
(Iteration 691 / 4900) loss: 1.537286
(Iteration 701 / 4900) loss: 1.634060
(Iteration 711 / 4900) loss: 1.787462
(Iteration 721 / 4900) loss: 1.496824
(Iteration 731 / 4900) loss: 1.748802
(Iteration 741 / 4900) loss: 1.546595
(Iteration 751 / 4900) loss: 1.465575
(Iteration 761 / 4900) loss: 1.712697
(Iteration 771 / 4900) loss: 1.536924
(Iteration 781 / 4900) loss: 1.672417
(Iteration 791 / 4900) loss: 1.547666
(Iteration 801 / 4900) loss: 1.762188
(Iteration 811 / 4900) loss: 1.530811
(Iteration 821 / 4900) loss: 1.453778
(Iteration 831 / 4900) loss: 1.391400
(Iteration 841 / 4900) loss: 1.448011
(Iteration 851 / 4900) loss: 1.424604
(Iteration 861 / 4900) loss: 1.580420
(Iteration 871 / 4900) loss: 1.541611
(Iteration 881 / 4900) loss: 1.373310
(Iteration 891 / 4900) loss: 1.646427
(Iteration 901 / 4900) loss: 1.671341
(Iteration 911 / 4900) loss: 1.641855
(Iteration 921 / 4900) loss: 1.434448
(Iteration 931 / 4900) loss: 1.488981
(Iteration 941 / 4900) loss: 1.541414
(Iteration 951 / 4900) loss: 1.490621
(Iteration 961 / 4900) loss: 1.616482
(Iteration 971 / 4900) loss: 1.578580
(Epoch 2 / 10) train acc: 0.487000; val_acc: 0.466000
(Iteration 981 / 4900) loss: 1.417420
(Iteration 991 / 4900) loss: 1.544010
(Iteration 1001 / 4900) loss: 1.627923
(Iteration 1011 / 4900) loss: 1.506455
(Iteration 1021 / 4900) loss: 1.556659
(Iteration 1031 / 4900) loss: 1.477954
(Iteration 1041 / 4900) loss: 1.617622
(Iteration 1051 / 4900) loss: 1.605861
(Iteration 1061 / 4900) loss: 1.538479
(Iteration 1071 / 4900) loss: 1.636170
(Iteration 1081 / 4900) loss: 1.704552
(Iteration 1091 / 4900) loss: 1.488139
```

```
(Iteration 1101 / 4900) loss: 1.417982
(Iteration 1111 / 4900) loss: 1.577658
(Iteration 1121 / 4900) loss: 1.387293
(Iteration 1131 / 4900) loss: 1.602261
(Iteration 1141 / 4900) loss: 1.360815
(Iteration 1151 / 4900) loss: 1.577788
(Iteration 1161 / 4900) loss: 1.661454
(Iteration 1171 / 4900) loss: 1.501056
(Iteration 1181 / 4900) loss: 1.499395
(Iteration 1191 / 4900) loss: 1.580519
(Iteration 1201 / 4900) loss: 1.499487
(Iteration 1211 / 4900) loss: 1.686402
(Iteration 1221 / 4900) loss: 1.627484
(Iteration 1231 / 4900) loss: 1.675455
(Iteration 1241 / 4900) loss: 1.413737
(Iteration 1251 / 4900) loss: 1.490561
(Iteration 1261 / 4900) loss: 1.407304
(Iteration 1271 / 4900) loss: 1.464136
(Iteration 1281 / 4900) loss: 1.290018
(Iteration 1291 / 4900) loss: 1.574030
(Iteration 1301 / 4900) loss: 1.588625
(Iteration 1311 / 4900) loss: 1.566181
(Iteration 1321 / 4900) loss: 1.614433
(Iteration 1331 / 4900) loss: 1.496763
(Iteration 1341 / 4900) loss: 1.578887
(Iteration 1351 / 4900) loss: 1.492053
(Iteration 1361 / 4900) loss: 1.409793
(Iteration 1371 / 4900) loss: 1.294145
(Iteration 1381 / 4900) loss: 1.367656
(Iteration 1391 / 4900) loss: 1.386201
(Iteration 1401 / 4900) loss: 1.413321
(Iteration 1411 / 4900) loss: 1.507263
(Iteration 1421 / 4900) loss: 1.543700
(Iteration 1431 / 4900) loss: 1.571968
(Iteration 1441 / 4900) loss: 1.497270
(Iteration 1451 / 4900) loss: 1.449663
(Iteration 1461 / 4900) loss: 1.585407
(Epoch 3 / 10) train acc: 0.526000; val_acc: 0.502000
(Iteration 1471 / 4900) loss: 1.570417
(Iteration 1481 / 4900) loss: 1.508557
(Iteration 1491 / 4900) loss: 1.551169
(Iteration 1501 / 4900) loss: 1.496628
(Iteration 1511 / 4900) loss: 1.626560
(Iteration 1521 / 4900) loss: 1.670237
(Iteration 1531 / 4900) loss: 1.395409
(Iteration 1541 / 4900) loss: 1.603583
(Iteration 1551 / 4900) loss: 1.642588
(Iteration 1561 / 4900) loss: 1.470407
```

```
(Iteration 1571 / 4900) loss: 1.671357
(Iteration 1581 / 4900) loss: 1.338604
(Iteration 1591 / 4900) loss: 1.419170
(Iteration 1601 / 4900) loss: 1.453107
(Iteration 1611 / 4900) loss: 1.659581
(Iteration 1621 / 4900) loss: 1.282454
(Iteration 1631 / 4900) loss: 1.384513
(Iteration 1641 / 4900) loss: 1.381196
(Iteration 1651 / 4900) loss: 1.429565
(Iteration 1661 / 4900) loss: 1.504646
(Iteration 1671 / 4900) loss: 1.392080
(Iteration 1681 / 4900) loss: 1.592353
(Iteration 1691 / 4900) loss: 1.538674
(Iteration 1701 / 4900) loss: 1.553625
(Iteration 1711 / 4900) loss: 1.480436
(Iteration 1721 / 4900) loss: 1.256852
(Iteration 1731 / 4900) loss: 1.510613
(Iteration 1741 / 4900) loss: 1.506578
(Iteration 1751 / 4900) loss: 1.323424
(Iteration 1761 / 4900) loss: 1.484155
(Iteration 1771 / 4900) loss: 1.499344
(Iteration 1781 / 4900) loss: 1.389517
(Iteration 1791 / 4900) loss: 1.516094
(Iteration 1801 / 4900) loss: 1.536055
(Iteration 1811 / 4900) loss: 1.406455
(Iteration 1821 / 4900) loss: 1.512569
(Iteration 1831 / 4900) loss: 1.513133
(Iteration 1841 / 4900) loss: 1.315619
(Iteration 1851 / 4900) loss: 1.384699
(Iteration 1861 / 4900) loss: 1.535631
(Iteration 1871 / 4900) loss: 1.497906
(Iteration 1881 / 4900) loss: 1.446054
(Iteration 1891 / 4900) loss: 1.418731
(Iteration 1901 / 4900) loss: 1.506246
(Iteration 1911 / 4900) loss: 1.324927
(Iteration 1921 / 4900) loss: 1.405496
(Iteration 1931 / 4900) loss: 1.458051
(Iteration 1941 / 4900) loss: 1.485055
(Iteration 1951 / 4900) loss: 1.500331
(Epoch 4 / 10) train acc: 0.545000; val_acc: 0.481000
(Iteration 1961 / 4900) loss: 1.511394
(Iteration 1971 / 4900) loss: 1.482585
(Iteration 1981 / 4900) loss: 1.336994
(Iteration 1991 / 4900) loss: 1.473120
(Iteration 2001 / 4900) loss: 1.360828
(Iteration 2011 / 4900) loss: 1.487871
(Iteration 2021 / 4900) loss: 1.410942
(Iteration 2031 / 4900) loss: 1.310640
```

```
(Iteration 2041 / 4900) loss: 1.448962
(Iteration 2051 / 4900) loss: 1.362771
(Iteration 2061 / 4900) loss: 1.501907
(Iteration 2071 / 4900) loss: 1.418221
(Iteration 2081 / 4900) loss: 1.182219
(Iteration 2091 / 4900) loss: 1.387459
(Iteration 2101 / 4900) loss: 1.329769
(Iteration 2111 / 4900) loss: 1.225590
(Iteration 2121 / 4900) loss: 1.295390
(Iteration 2131 / 4900) loss: 1.516685
(Iteration 2141 / 4900) loss: 1.657054
(Iteration 2151 / 4900) loss: 1.380048
(Iteration 2161 / 4900) loss: 1.455682
(Iteration 2171 / 4900) loss: 1.192553
(Iteration 2181 / 4900) loss: 1.477451
(Iteration 2191 / 4900) loss: 1.506306
(Iteration 2201 / 4900) loss: 1.137549
(Iteration 2211 / 4900) loss: 1.411274
(Iteration 2221 / 4900) loss: 1.342678
(Iteration 2231 / 4900) loss: 1.460114
(Iteration 2241 / 4900) loss: 1.455214
(Iteration 2251 / 4900) loss: 1.307313
(Iteration 2261 / 4900) loss: 1.433313
(Iteration 2271 / 4900) loss: 1.404855
(Iteration 2281 / 4900) loss: 1.398769
(Iteration 2291 / 4900) loss: 1.290963
(Iteration 2301 / 4900) loss: 1.329597
(Iteration 2311 / 4900) loss: 1.138611
(Iteration 2321 / 4900) loss: 1.260201
(Iteration 2331 / 4900) loss: 1.571135
(Iteration 2341 / 4900) loss: 1.637240
(Iteration 2351 / 4900) loss: 1.447405
(Iteration 2361 / 4900) loss: 1.276304
(Iteration 2371 / 4900) loss: 1.524688
(Iteration 2381 / 4900) loss: 1.299621
(Iteration 2391 / 4900) loss: 1.312603
(Iteration 2401 / 4900) loss: 1.260305
(Iteration 2411 / 4900) loss: 1.597179
(Iteration 2421 / 4900) loss: 1.437685
(Iteration 2431 / 4900) loss: 1.329419
(Iteration 2441 / 4900) loss: 1.540395
(Epoch 5 / 10) train acc: 0.540000; val_acc: 0.496000
(Iteration 2451 / 4900) loss: 1.548253
(Iteration 2461 / 4900) loss: 1.556414
(Iteration 2471 / 4900) loss: 1.473894
(Iteration 2481 / 4900) loss: 1.425121
(Iteration 2491 / 4900) loss: 1.408096
(Iteration 2501 / 4900) loss: 1.231998
```

```
(Iteration 2511 / 4900) loss: 1.359930
(Iteration 2521 / 4900) loss: 1.254738
(Iteration 2531 / 4900) loss: 1.847418
(Iteration 2541 / 4900) loss: 1.309813
(Iteration 2551 / 4900) loss: 1.460978
(Iteration 2561 / 4900) loss: 1.396393
(Iteration 2571 / 4900) loss: 1.342398
(Iteration 2581 / 4900) loss: 1.495983
(Iteration 2591 / 4900) loss: 1.343549
(Iteration 2601 / 4900) loss: 1.354891
(Iteration 2611 / 4900) loss: 1.243158
(Iteration 2621 / 4900) loss: 1.420383
(Iteration 2631 / 4900) loss: 1.444264
(Iteration 2641 / 4900) loss: 1.407313
(Iteration 2651 / 4900) loss: 1.233706
(Iteration 2661 / 4900) loss: 1.406197
(Iteration 2671 / 4900) loss: 1.519977
(Iteration 2681 / 4900) loss: 1.293011
(Iteration 2691 / 4900) loss: 1.274132
(Iteration 2701 / 4900) loss: 1.455364
(Iteration 2711 / 4900) loss: 1.470887
(Iteration 2721 / 4900) loss: 1.294411
(Iteration 2731 / 4900) loss: 1.414875
(Iteration 2741 / 4900) loss: 1.372672
(Iteration 2751 / 4900) loss: 1.336139
(Iteration 2761 / 4900) loss: 1.537328
(Iteration 2771 / 4900) loss: 1.276401
(Iteration 2781 / 4900) loss: 1.493497
(Iteration 2791 / 4900) loss: 1.541149
(Iteration 2801 / 4900) loss: 1.512956
(Iteration 2811 / 4900) loss: 1.358697
(Iteration 2821 / 4900) loss: 1.491839
(Iteration 2831 / 4900) loss: 1.451203
(Iteration 2841 / 4900) loss: 1.447139
(Iteration 2851 / 4900) loss: 1.309468
(Iteration 2861 / 4900) loss: 1.419924
(Iteration 2871 / 4900) loss: 1.553058
(Iteration 2881 / 4900) loss: 1.460827
(Iteration 2891 / 4900) loss: 1.298387
(Iteration 2901 / 4900) loss: 1.346068
(Iteration 2911 / 4900) loss: 1.529275
(Iteration 2921 / 4900) loss: 1.214695
(Iteration 2931 / 4900) loss: 1.331987
(Epoch 6 / 10) train acc: 0.559000; val_acc: 0.491000
(Iteration 2941 / 4900) loss: 1.256253
(Iteration 2951 / 4900) loss: 1.459719
(Iteration 2961 / 4900) loss: 1.282639
(Iteration 2971 / 4900) loss: 1.500972
```

```
(Iteration 2981 / 4900) loss: 1.394317
(Iteration 2991 / 4900) loss: 1.367910
(Iteration 3001 / 4900) loss: 1.432154
(Iteration 3011 / 4900) loss: 1.431026
(Iteration 3021 / 4900) loss: 1.264560
(Iteration 3031 / 4900) loss: 1.380126
(Iteration 3041 / 4900) loss: 1.391278
(Iteration 3051 / 4900) loss: 1.473872
(Iteration 3061 / 4900) loss: 1.337005
(Iteration 3071 / 4900) loss: 1.360767
(Iteration 3081 / 4900) loss: 1.507385
(Iteration 3091 / 4900) loss: 1.308135
(Iteration 3101 / 4900) loss: 1.420247
(Iteration 3111 / 4900) loss: 1.683056
(Iteration 3121 / 4900) loss: 1.306156
(Iteration 3131 / 4900) loss: 1.397618
(Iteration 3141 / 4900) loss: 1.441845
(Iteration 3151 / 4900) loss: 1.411685
(Iteration 3161 / 4900) loss: 1.343486
(Iteration 3171 / 4900) loss: 1.385348
(Iteration 3181 / 4900) loss: 1.301335
(Iteration 3191 / 4900) loss: 1.155797
(Iteration 3201 / 4900) loss: 1.289734
(Iteration 3211 / 4900) loss: 1.098102
(Iteration 3221 / 4900) loss: 1.342214
(Iteration 3231 / 4900) loss: 1.236977
(Iteration 3241 / 4900) loss: 1.412538
(Iteration 3251 / 4900) loss: 1.370631
(Iteration 3261 / 4900) loss: 1.303792
(Iteration 3271 / 4900) loss: 1.580381
(Iteration 3281 / 4900) loss: 1.496170
(Iteration 3291 / 4900) loss: 1.457803
(Iteration 3301 / 4900) loss: 1.508771
(Iteration 3311 / 4900) loss: 1.384400
(Iteration 3321 / 4900) loss: 1.554162
(Iteration 3331 / 4900) loss: 1.350055
(Iteration 3341 / 4900) loss: 1.458863
(Iteration 3351 / 4900) loss: 1.398127
(Iteration 3361 / 4900) loss: 1.291247
(Iteration 3371 / 4900) loss: 1.350855
(Iteration 3381 / 4900) loss: 1.371879
(Iteration 3391 / 4900) loss: 1.467678
(Iteration 3401 / 4900) loss: 1.352456
(Iteration 3411 / 4900) loss: 1.470475
(Iteration 3421 / 4900) loss: 1.420853
(Epoch 7 / 10) train acc: 0.557000; val_acc: 0.496000
(Iteration 3431 / 4900) loss: 1.231230
(Iteration 3441 / 4900) loss: 1.334879
```

```
(Iteration 3451 / 4900) loss: 1.454046
(Iteration 3461 / 4900) loss: 1.409755
(Iteration 3471 / 4900) loss: 1.268985
(Iteration 3481 / 4900) loss: 1.202117
(Iteration 3491 / 4900) loss: 1.364011
(Iteration 3501 / 4900) loss: 1.444318
(Iteration 3511 / 4900) loss: 1.309518
(Iteration 3521 / 4900) loss: 1.316009
(Iteration 3531 / 4900) loss: 1.479109
(Iteration 3541 / 4900) loss: 1.153756
(Iteration 3551 / 4900) loss: 1.208381
(Iteration 3561 / 4900) loss: 1.427924
(Iteration 3571 / 4900) loss: 1.200397
(Iteration 3581 / 4900) loss: 1.499126
(Iteration 3591 / 4900) loss: 1.182323
(Iteration 3601 / 4900) loss: 1.443097
(Iteration 3611 / 4900) loss: 1.396640
(Iteration 3621 / 4900) loss: 1.351526
(Iteration 3631 / 4900) loss: 1.182683
(Iteration 3641 / 4900) loss: 1.452273
(Iteration 3651 / 4900) loss: 1.606753
(Iteration 3661 / 4900) loss: 1.192464
(Iteration 3671 / 4900) loss: 1.333201
(Iteration 3681 / 4900) loss: 1.361816
(Iteration 3691 / 4900) loss: 1.378885
(Iteration 3701 / 4900) loss: 1.258756
(Iteration 3711 / 4900) loss: 1.203404
(Iteration 3721 / 4900) loss: 1.381876
(Iteration 3731 / 4900) loss: 1.277632
(Iteration 3741 / 4900) loss: 1.512661
(Iteration 3751 / 4900) loss: 1.252316
(Iteration 3761 / 4900) loss: 1.334275
(Iteration 3771 / 4900) loss: 1.268915
(Iteration 3781 / 4900) loss: 1.257608
(Iteration 3791 / 4900) loss: 1.261224
(Iteration 3801 / 4900) loss: 1.238238
(Iteration 3811 / 4900) loss: 1.278782
(Iteration 3821 / 4900) loss: 1.406620
(Iteration 3831 / 4900) loss: 1.234164
(Iteration 3841 / 4900) loss: 1.346953
(Iteration 3851 / 4900) loss: 1.291451
(Iteration 3861 / 4900) loss: 1.275581
(Iteration 3871 / 4900) loss: 1.254924
(Iteration 3881 / 4900) loss: 1.448290
(Iteration 3891 / 4900) loss: 1.315074
(Iteration 3901 / 4900) loss: 1.444689
(Iteration 3911 / 4900) loss: 1.339737
(Epoch 8 / 10) train acc: 0.596000; val_acc: 0.516000
```

```
(Iteration 3921 / 4900) loss: 1.511091
(Iteration 3931 / 4900) loss: 1.335759
(Iteration 3941 / 4900) loss: 1.351024
(Iteration 3951 / 4900) loss: 1.201490
(Iteration 3961 / 4900) loss: 1.438147
(Iteration 3971 / 4900) loss: 1.299443
(Iteration 3981 / 4900) loss: 1.251656
(Iteration 3991 / 4900) loss: 1.384759
(Iteration 4001 / 4900) loss: 1.478659
(Iteration 4011 / 4900) loss: 1.442711
(Iteration 4021 / 4900) loss: 1.299196
(Iteration 4031 / 4900) loss: 1.444781
(Iteration 4041 / 4900) loss: 1.176564
(Iteration 4051 / 4900) loss: 1.343958
(Iteration 4061 / 4900) loss: 1.335054
(Iteration 4071 / 4900) loss: 1.335756
(Iteration 4081 / 4900) loss: 1.336193
(Iteration 4091 / 4900) loss: 1.212384
(Iteration 4101 / 4900) loss: 1.441513
(Iteration 4111 / 4900) loss: 1.286551
(Iteration 4121 / 4900) loss: 1.462644
(Iteration 4131 / 4900) loss: 1.166523
(Iteration 4141 / 4900) loss: 1.332162
(Iteration 4151 / 4900) loss: 1.376105
(Iteration 4161 / 4900) loss: 1.265367
(Iteration 4171 / 4900) loss: 1.193985
(Iteration 4181 / 4900) loss: 1.514559
(Iteration 4191 / 4900) loss: 1.142432
(Iteration 4201 / 4900) loss: 1.161008
(Iteration 4211 / 4900) loss: 1.269967
(Iteration 4221 / 4900) loss: 1.158492
(Iteration 4231 / 4900) loss: 1.273425
(Iteration 4241 / 4900) loss: 1.342158
(Iteration 4251 / 4900) loss: 1.323374
(Iteration 4261 / 4900) loss: 1.229169
(Iteration 4271 / 4900) loss: 1.229752
(Iteration 4281 / 4900) loss: 1.401446
(Iteration 4291 / 4900) loss: 1.371341
(Iteration 4301 / 4900) loss: 1.397193
(Iteration 4311 / 4900) loss: 1.153004
(Iteration 4321 / 4900) loss: 1.282844
(Iteration 4331 / 4900) loss: 1.265378
(Iteration 4341 / 4900) loss: 1.513881
(Iteration 4351 / 4900) loss: 1.343137
(Iteration 4361 / 4900) loss: 1.330608
(Iteration 4371 / 4900) loss: 1.426543
(Iteration 4381 / 4900) loss: 1.471723
(Iteration 4391 / 4900) loss: 1.320880
```

```
(Iteration 4401 / 4900) loss: 1.266510
(Epoch 9 / 10) train acc: 0.607000; val_acc: 0.531000
(Iteration 4411 / 4900) loss: 1.384949
(Iteration 4421 / 4900) loss: 1.138456
(Iteration 4431 / 4900) loss: 1.175600
(Iteration 4441 / 4900) loss: 1.320654
(Iteration 4451 / 4900) loss: 1.209898
(Iteration 4461 / 4900) loss: 1.241051
(Iteration 4471 / 4900) loss: 1.277990
(Iteration 4481 / 4900) loss: 1.280927
(Iteration 4491 / 4900) loss: 1.088957
(Iteration 4501 / 4900) loss: 1.207734
(Iteration 4511 / 4900) loss: 1.367237
(Iteration 4521 / 4900) loss: 1.419657
(Iteration 4531 / 4900) loss: 1.175284
(Iteration 4541 / 4900) loss: 1.269553
(Iteration 4551 / 4900) loss: 1.431247
(Iteration 4561 / 4900) loss: 1.164116
(Iteration 4571 / 4900) loss: 1.245311
(Iteration 4581 / 4900) loss: 1.348673
(Iteration 4591 / 4900) loss: 1.457160
(Iteration 4601 / 4900) loss: 1.359520
(Iteration 4611 / 4900) loss: 1.461595
(Iteration 4621 / 4900) loss: 1.259642
(Iteration 4631 / 4900) loss: 1.197097
(Iteration 4641 / 4900) loss: 1.202133
(Iteration 4651 / 4900) loss: 1.126366
(Iteration 4661 / 4900) loss: 1.265288
(Iteration 4671 / 4900) loss: 1.272321
(Iteration 4681 / 4900) loss: 1.280073
(Iteration 4691 / 4900) loss: 1.128231
(Iteration 4701 / 4900) loss: 1.695190
(Iteration 4711 / 4900) loss: 1.401494
(Iteration 4721 / 4900) loss: 1.249558
(Iteration 4731 / 4900) loss: 1.182755
(Iteration 4741 / 4900) loss: 1.237395
(Iteration 4751 / 4900) loss: 1.216658
(Iteration 4761 / 4900) loss: 1.461640
(Iteration 4771 / 4900) loss: 1.283407
(Iteration 4781 / 4900) loss: 1.225974
(Iteration 4791 / 4900) loss: 1.362438
(Iteration 4801 / 4900) loss: 1.160537
(Iteration 4811 / 4900) loss: 1.343423
(Iteration 4821 / 4900) loss: 1.250442
(Iteration 4831 / 4900) loss: 1.393924
(Iteration 4841 / 4900) loss: 1.469265
(Iteration 4851 / 4900) loss: 1.093642
(Iteration 4861 / 4900) loss: 1.278704
```

```
(Iteration 4871 / 4900) loss: 1.201710
(Iteration 4881 / 4900) loss: 1.511036
(Iteration 4891 / 4900) loss: 1.307714
(Epoch 10 / 10) train acc: 0.617000; val_acc: 0.533000
accuracy:0.533 with lr:0.0005,reg:0.25,epoch:10,hidden_dim:200
(Iteration 1 / 9800) loss: 2.329979
(Epoch 0 / 20) train acc: 0.142000; val_acc: 0.166000
(Iteration 11 / 9800) loss: 2.283443
(Iteration 21 / 9800) loss: 2.207766
(Iteration 31 / 9800) loss: 2.176248
(Iteration 41 / 9800) loss: 2.141788
(Iteration 51 / 9800) loss: 2.029651
(Iteration 61 / 9800) loss: 2.063163
(Iteration 71 / 9800) loss: 1.965802
(Iteration 81 / 9800) loss: 2.061966
(Iteration 91 / 9800) loss: 1.932757
(Iteration 101 / 9800) loss: 1.919753
(Iteration 111 / 9800) loss: 1.913110
(Iteration 121 / 9800) loss: 1.803273
(Iteration 131 / 9800) loss: 1.880968
(Iteration 141 / 9800) loss: 1.824441
(Iteration 151 / 9800) loss: 1.844214
(Iteration 161 / 9800) loss: 1.896843
(Iteration 171 / 9800) loss: 1.915407
(Iteration 181 / 9800) loss: 1.697700
(Iteration 191 / 9800) loss: 1.791309
(Iteration 201 / 9800) loss: 1.793219
(Iteration 211 / 9800) loss: 1.828062
(Iteration 221 / 9800) loss: 1.694713
(Iteration 231 / 9800) loss: 1.789553
(Iteration 241 / 9800) loss: 1.792083
(Iteration 251 / 9800) loss: 1.871470
(Iteration 261 / 9800) loss: 1.827321
(Iteration 271 / 9800) loss: 1.729937
(Iteration 281 / 9800) loss: 1.682150
(Iteration 291 / 9800) loss: 1.744318
(Iteration 301 / 9800) loss: 1.642351
(Iteration 311 / 9800) loss: 1.821347
(Iteration 321 / 9800) loss: 1.803269
(Iteration 331 / 9800) loss: 1.713637
(Iteration 341 / 9800) loss: 1.747410
(Iteration 351 / 9800) loss: 1.609383
(Iteration 361 / 9800) loss: 1.788802
(Iteration 371 / 9800) loss: 1.564810
(Iteration 381 / 9800) loss: 1.672068
(Iteration 391 / 9800) loss: 1.629053
(Iteration 401 / 9800) loss: 1.812101
(Iteration 411 / 9800) loss: 1.544386
```

```
(Iteration 421 / 9800) loss: 1.722364
(Iteration 431 / 9800) loss: 1.520429
(Iteration 441 / 9800) loss: 1.597103
(Iteration 451 / 9800) loss: 1.598075
(Iteration 461 / 9800) loss: 1.729377
(Iteration 471 / 9800) loss: 1.579863
(Iteration 481 / 9800) loss: 1.676057
(Epoch 1 / 20) train acc: 0.435000; val_acc: 0.447000
(Iteration 491 / 9800) loss: 1.555332
(Iteration 501 / 9800) loss: 1.881347
(Iteration 511 / 9800) loss: 1.546862
(Iteration 521 / 9800) loss: 1.497568
(Iteration 531 / 9800) loss: 1.655273
(Iteration 541 / 9800) loss: 1.657410
(Iteration 551 / 9800) loss: 1.648441
(Iteration 561 / 9800) loss: 1.693257
(Iteration 571 / 9800) loss: 1.541352
(Iteration 581 / 9800) loss: 1.626227
(Iteration 591 / 9800) loss: 1.519190
(Iteration 601 / 9800) loss: 1.611942
(Iteration 611 / 9800) loss: 1.657954
(Iteration 621 / 9800) loss: 1.536038
(Iteration 631 / 9800) loss: 1.610214
(Iteration 641 / 9800) loss: 1.524669
(Iteration 651 / 9800) loss: 1.566624
(Iteration 661 / 9800) loss: 1.541733
(Iteration 671 / 9800) loss: 1.632730
(Iteration 681 / 9800) loss: 1.520612
(Iteration 691 / 9800) loss: 1.428985
(Iteration 701 / 9800) loss: 1.578734
(Iteration 711 / 9800) loss: 1.601470
(Iteration 721 / 9800) loss: 1.448874
(Iteration 731 / 9800) loss: 1.571286
(Iteration 741 / 9800) loss: 1.509111
(Iteration 751 / 9800) loss: 1.586850
(Iteration 761 / 9800) loss: 1.523225
(Iteration 771 / 9800) loss: 1.611810
(Iteration 781 / 9800) loss: 1.521706
(Iteration 791 / 9800) loss: 1.630888
(Iteration 801 / 9800) loss: 1.609055
(Iteration 811 / 9800) loss: 1.580281
(Iteration 821 / 9800) loss: 1.649852
(Iteration 831 / 9800) loss: 1.373615
(Iteration 841 / 9800) loss: 1.626247
(Iteration 851 / 9800) loss: 1.471303
(Iteration 861 / 9800) loss: 1.571884
(Iteration 871 / 9800) loss: 1.515401
(Iteration 881 / 9800) loss: 1.520260
```

```
(Iteration 891 / 9800) loss: 1.445514
(Iteration 901 / 9800) loss: 1.433290
(Iteration 911 / 9800) loss: 1.370967
(Iteration 921 / 9800) loss: 1.527343
(Iteration 931 / 9800) loss: 1.524177
(Iteration 941 / 9800) loss: 1.412921
(Iteration 951 / 9800) loss: 1.593090
(Iteration 961 / 9800) loss: 1.479245
(Iteration 971 / 9800) loss: 1.486494
(Epoch 2 / 20) train acc: 0.513000; val_acc: 0.466000
(Iteration 981 / 9800) loss: 1.467789
(Iteration 991 / 9800) loss: 1.393983
(Iteration 1001 / 9800) loss: 1.410430
(Iteration 1011 / 9800) loss: 1.528937
(Iteration 1021 / 9800) loss: 1.307875
(Iteration 1031 / 9800) loss: 1.436289
(Iteration 1041 / 9800) loss: 1.556560
(Iteration 1051 / 9800) loss: 1.732151
(Iteration 1061 / 9800) loss: 1.440605
(Iteration 1071 / 9800) loss: 1.676134
(Iteration 1081 / 9800) loss: 1.578985
(Iteration 1091 / 9800) loss: 1.475889
(Iteration 1101 / 9800) loss: 1.536260
(Iteration 1111 / 9800) loss: 1.533728
(Iteration 1121 / 9800) loss: 1.465615
(Iteration 1131 / 9800) loss: 1.425549
(Iteration 1141 / 9800) loss: 1.312700
(Iteration 1151 / 9800) loss: 1.595149
(Iteration 1161 / 9800) loss: 1.805590
(Iteration 1171 / 9800) loss: 1.498481
(Iteration 1181 / 9800) loss: 1.490988
(Iteration 1191 / 9800) loss: 1.567032
(Iteration 1201 / 9800) loss: 1.536212
(Iteration 1211 / 9800) loss: 1.528007
(Iteration 1221 / 9800) loss: 1.358952
(Iteration 1231 / 9800) loss: 1.502587
(Iteration 1241 / 9800) loss: 1.422493
(Iteration 1251 / 9800) loss: 1.558964
(Iteration 1261 / 9800) loss: 1.342239
(Iteration 1271 / 9800) loss: 1.596322
(Iteration 1281 / 9800) loss: 1.518611
(Iteration 1291 / 9800) loss: 1.400062
(Iteration 1301 / 9800) loss: 1.404657
(Iteration 1311 / 9800) loss: 1.388774
(Iteration 1321 / 9800) loss: 1.463464
(Iteration 1331 / 9800) loss: 1.510193
(Iteration 1341 / 9800) loss: 1.396342
(Iteration 1351 / 9800) loss: 1.400423
```

```
(Iteration 1361 / 9800) loss: 1.439978
(Iteration 1371 / 9800) loss: 1.311756
(Iteration 1381 / 9800) loss: 1.604747
(Iteration 1391 / 9800) loss: 1.302290
(Iteration 1401 / 9800) loss: 1.354045
(Iteration 1411 / 9800) loss: 1.400297
(Iteration 1421 / 9800) loss: 1.561230
(Iteration 1431 / 9800) loss: 1.414178
(Iteration 1441 / 9800) loss: 1.712914
(Iteration 1451 / 9800) loss: 1.380712
(Iteration 1461 / 9800) loss: 1.602325
(Epoch 3 / 20) train acc: 0.516000; val_acc: 0.486000
(Iteration 1471 / 9800) loss: 1.418236
(Iteration 1481 / 9800) loss: 1.341148
(Iteration 1491 / 9800) loss: 1.322834
(Iteration 1501 / 9800) loss: 1.436649
(Iteration 1511 / 9800) loss: 1.563981
(Iteration 1521 / 9800) loss: 1.412858
(Iteration 1531 / 9800) loss: 1.429067
(Iteration 1541 / 9800) loss: 1.578635
(Iteration 1551 / 9800) loss: 1.425976
(Iteration 1561 / 9800) loss: 1.288944
(Iteration 1571 / 9800) loss: 1.256652
(Iteration 1581 / 9800) loss: 1.233263
(Iteration 1591 / 9800) loss: 1.404100
(Iteration 1601 / 9800) loss: 1.324602
(Iteration 1611 / 9800) loss: 1.435800
(Iteration 1621 / 9800) loss: 1.535487
(Iteration 1631 / 9800) loss: 1.423398
(Iteration 1641 / 9800) loss: 1.336844
(Iteration 1651 / 9800) loss: 1.306412
(Iteration 1661 / 9800) loss: 1.454769
(Iteration 1671 / 9800) loss: 1.384303
(Iteration 1681 / 9800) loss: 1.598816
(Iteration 1691 / 9800) loss: 1.614433
(Iteration 1701 / 9800) loss: 1.306904
(Iteration 1711 / 9800) loss: 1.686452
(Iteration 1721 / 9800) loss: 1.559088
(Iteration 1731 / 9800) loss: 1.446578
(Iteration 1741 / 9800) loss: 1.222127
(Iteration 1751 / 9800) loss: 1.351882
(Iteration 1761 / 9800) loss: 1.336734
(Iteration 1771 / 9800) loss: 1.471409
(Iteration 1781 / 9800) loss: 1.378140
(Iteration 1791 / 9800) loss: 1.572221
(Iteration 1801 / 9800) loss: 1.405774
(Iteration 1811 / 9800) loss: 1.344555
(Iteration 1821 / 9800) loss: 1.442266
```

```
(Iteration 1831 / 9800) loss: 1.338320
(Iteration 1841 / 9800) loss: 1.279327
(Iteration 1851 / 9800) loss: 1.472598
(Iteration 1861 / 9800) loss: 1.494483
(Iteration 1871 / 9800) loss: 1.431456
(Iteration 1881 / 9800) loss: 1.258351
(Iteration 1891 / 9800) loss: 1.373076
(Iteration 1901 / 9800) loss: 1.259389
(Iteration 1911 / 9800) loss: 1.350306
(Iteration 1921 / 9800) loss: 1.417392
(Iteration 1931 / 9800) loss: 1.284548
(Iteration 1941 / 9800) loss: 1.295023
(Iteration 1951 / 9800) loss: 1.338141
(Epoch 4 / 20) train acc: 0.565000; val_acc: 0.494000
(Iteration 1961 / 9800) loss: 1.275636
(Iteration 1971 / 9800) loss: 1.137253
(Iteration 1981 / 9800) loss: 1.458636
(Iteration 1991 / 9800) loss: 1.299912
(Iteration 2001 / 9800) loss: 1.608545
(Iteration 2011 / 9800) loss: 1.432440
(Iteration 2021 / 9800) loss: 1.256040
(Iteration 2031 / 9800) loss: 1.724890
(Iteration 2041 / 9800) loss: 1.583108
(Iteration 2051 / 9800) loss: 1.552788
(Iteration 2061 / 9800) loss: 1.348811
(Iteration 2071 / 9800) loss: 1.211899
(Iteration 2081 / 9800) loss: 1.443509
(Iteration 2091 / 9800) loss: 1.321167
(Iteration 2101 / 9800) loss: 1.320915
(Iteration 2111 / 9800) loss: 1.299685
(Iteration 2121 / 9800) loss: 1.351371
(Iteration 2131 / 9800) loss: 1.383618
(Iteration 2141 / 9800) loss: 1.331790
(Iteration 2151 / 9800) loss: 1.299256
(Iteration 2161 / 9800) loss: 1.154441
(Iteration 2171 / 9800) loss: 1.358824
(Iteration 2181 / 9800) loss: 1.428777
(Iteration 2191 / 9800) loss: 1.269584
(Iteration 2201 / 9800) loss: 1.287232
(Iteration 2211 / 9800) loss: 1.263270
(Iteration 2221 / 9800) loss: 1.257872
(Iteration 2231 / 9800) loss: 1.346510
(Iteration 2241 / 9800) loss: 1.214386
(Iteration 2251 / 9800) loss: 1.319820
(Iteration 2261 / 9800) loss: 1.521685
(Iteration 2271 / 9800) loss: 1.342840
(Iteration 2281 / 9800) loss: 1.298233
(Iteration 2291 / 9800) loss: 1.410165
```

```
(Iteration 2301 / 9800) loss: 1.310282
(Iteration 2311 / 9800) loss: 1.412075
(Iteration 2321 / 9800) loss: 1.506270
(Iteration 2331 / 9800) loss: 1.541460
(Iteration 2341 / 9800) loss: 1.177156
(Iteration 2351 / 9800) loss: 1.024040
(Iteration 2361 / 9800) loss: 1.196004
(Iteration 2371 / 9800) loss: 1.467188
(Iteration 2381 / 9800) loss: 1.469683
(Iteration 2391 / 9800) loss: 1.267859
(Iteration 2401 / 9800) loss: 1.255060
(Iteration 2411 / 9800) loss: 1.417968
(Iteration 2421 / 9800) loss: 1.243101
(Iteration 2431 / 9800) loss: 1.246729
(Iteration 2441 / 9800) loss: 1.237327
(Epoch 5 / 20) train acc: 0.525000; val_acc: 0.514000
(Iteration 2451 / 9800) loss: 1.414432
(Iteration 2461 / 9800) loss: 1.314480
(Iteration 2471 / 9800) loss: 1.489937
(Iteration 2481 / 9800) loss: 1.313000
(Iteration 2491 / 9800) loss: 1.370153
(Iteration 2501 / 9800) loss: 1.296674
(Iteration 2511 / 9800) loss: 1.287173
(Iteration 2521 / 9800) loss: 1.335908
(Iteration 2531 / 9800) loss: 1.194963
(Iteration 2541 / 9800) loss: 1.246192
(Iteration 2551 / 9800) loss: 1.295071
(Iteration 2561 / 9800) loss: 1.337390
(Iteration 2571 / 9800) loss: 1.179099
(Iteration 2581 / 9800) loss: 1.332042
(Iteration 2591 / 9800) loss: 1.221543
(Iteration 2601 / 9800) loss: 1.341994
(Iteration 2611 / 9800) loss: 1.176064
(Iteration 2621 / 9800) loss: 1.357853
(Iteration 2631 / 9800) loss: 1.234300
(Iteration 2641 / 9800) loss: 1.320660
(Iteration 2651 / 9800) loss: 1.324822
(Iteration 2661 / 9800) loss: 1.617654
(Iteration 2671 / 9800) loss: 1.225251
(Iteration 2681 / 9800) loss: 1.372404
(Iteration 2691 / 9800) loss: 1.201496
(Iteration 2701 / 9800) loss: 1.274037
(Iteration 2711 / 9800) loss: 1.480330
(Iteration 2721 / 9800) loss: 1.394491
(Iteration 2731 / 9800) loss: 1.321053
(Iteration 2741 / 9800) loss: 1.295605
(Iteration 2751 / 9800) loss: 1.406372
(Iteration 2761 / 9800) loss: 1.218536
```

```
(Iteration 2771 / 9800) loss: 1.440764
(Iteration 2781 / 9800) loss: 1.236494
(Iteration 2791 / 9800) loss: 1.237881
(Iteration 2801 / 9800) loss: 1.171217
(Iteration 2811 / 9800) loss: 1.170763
(Iteration 2821 / 9800) loss: 1.303003
(Iteration 2831 / 9800) loss: 1.229353
(Iteration 2841 / 9800) loss: 1.341817
(Iteration 2851 / 9800) loss: 1.250826
(Iteration 2861 / 9800) loss: 1.397300
(Iteration 2871 / 9800) loss: 1.358423
(Iteration 2881 / 9800) loss: 1.444836
(Iteration 2891 / 9800) loss: 1.477291
(Iteration 2901 / 9800) loss: 1.281189
(Iteration 2911 / 9800) loss: 1.337408
(Iteration 2921 / 9800) loss: 1.184415
(Iteration 2931 / 9800) loss: 1.236042
(Epoch 6 / 20) train acc: 0.551000; val_acc: 0.507000
(Iteration 2941 / 9800) loss: 1.272549
(Iteration 2951 / 9800) loss: 1.310808
(Iteration 2961 / 9800) loss: 1.160783
(Iteration 2971 / 9800) loss: 1.350938
(Iteration 2981 / 9800) loss: 1.252909
(Iteration 2991 / 9800) loss: 1.346509
(Iteration 3001 / 9800) loss: 1.214445
(Iteration 3011 / 9800) loss: 1.281584
(Iteration 3021 / 9800) loss: 1.338202
(Iteration 3031 / 9800) loss: 1.199175
(Iteration 3041 / 9800) loss: 1.316301
(Iteration 3051 / 9800) loss: 1.143871
(Iteration 3061 / 9800) loss: 1.448081
(Iteration 3071 / 9800) loss: 1.362603
(Iteration 3081 / 9800) loss: 1.254789
(Iteration 3091 / 9800) loss: 1.113841
(Iteration 3101 / 9800) loss: 1.220785
(Iteration 3111 / 9800) loss: 1.271983
(Iteration 3121 / 9800) loss: 1.372088
(Iteration 3131 / 9800) loss: 1.426541
(Iteration 3141 / 9800) loss: 1.402614
(Iteration 3151 / 9800) loss: 1.217237
(Iteration 3161 / 9800) loss: 1.391851
(Iteration 3171 / 9800) loss: 1.200681
(Iteration 3181 / 9800) loss: 1.163518
(Iteration 3191 / 9800) loss: 1.391024
(Iteration 3201 / 9800) loss: 1.145540
(Iteration 3211 / 9800) loss: 1.377774
(Iteration 3221 / 9800) loss: 1.381482
(Iteration 3231 / 9800) loss: 1.171512
```

```
(Iteration 3241 / 9800) loss: 1.266164
(Iteration 3251 / 9800) loss: 1.293684
(Iteration 3261 / 9800) loss: 1.365282
(Iteration 3271 / 9800) loss: 1.156500
(Iteration 3281 / 9800) loss: 1.293086
(Iteration 3291 / 9800) loss: 1.156330
(Iteration 3301 / 9800) loss: 1.190309
(Iteration 3311 / 9800) loss: 1.472985
(Iteration 3321 / 9800) loss: 1.471507
(Iteration 3331 / 9800) loss: 1.282056
(Iteration 3341 / 9800) loss: 1.268641
(Iteration 3351 / 9800) loss: 1.243083
(Iteration 3361 / 9800) loss: 1.113220
(Iteration 3371 / 9800) loss: 1.350233
(Iteration 3381 / 9800) loss: 1.333273
(Iteration 3391 / 9800) loss: 1.246903
(Iteration 3401 / 9800) loss: 1.185756
(Iteration 3411 / 9800) loss: 1.301193
(Iteration 3421 / 9800) loss: 1.363076
(Epoch 7 / 20) train acc: 0.585000; val_acc: 0.510000
(Iteration 3431 / 9800) loss: 1.209002
(Iteration 3441 / 9800) loss: 1.161767
(Iteration 3451 / 9800) loss: 1.081350
(Iteration 3461 / 9800) loss: 1.184182
(Iteration 3471 / 9800) loss: 1.408012
(Iteration 3481 / 9800) loss: 1.190004
(Iteration 3491 / 9800) loss: 1.241879
(Iteration 3501 / 9800) loss: 1.189189
(Iteration 3511 / 9800) loss: 1.083304
(Iteration 3521 / 9800) loss: 1.281669
(Iteration 3531 / 9800) loss: 1.188382
(Iteration 3541 / 9800) loss: 1.345062
(Iteration 3551 / 9800) loss: 1.378942
(Iteration 3561 / 9800) loss: 1.446233
(Iteration 3571 / 9800) loss: 1.262383
(Iteration 3581 / 9800) loss: 1.110969
(Iteration 3591 / 9800) loss: 1.144440
(Iteration 3601 / 9800) loss: 1.333056
(Iteration 3611 / 9800) loss: 1.417791
(Iteration 3621 / 9800) loss: 1.274531
(Iteration 3631 / 9800) loss: 1.375622
(Iteration 3641 / 9800) loss: 1.234477
(Iteration 3651 / 9800) loss: 1.274001
(Iteration 3661 / 9800) loss: 1.152829
(Iteration 3671 / 9800) loss: 1.032562
(Iteration 3681 / 9800) loss: 1.231913
(Iteration 3691 / 9800) loss: 1.177681
(Iteration 3701 / 9800) loss: 1.321202
```

```
(Iteration 3711 / 9800) loss: 1.412534
(Iteration 3721 / 9800) loss: 1.179474
(Iteration 3731 / 9800) loss: 1.409904
(Iteration 3741 / 9800) loss: 1.177319
(Iteration 3751 / 9800) loss: 1.017777
(Iteration 3761 / 9800) loss: 1.301115
(Iteration 3771 / 9800) loss: 1.275191
(Iteration 3781 / 9800) loss: 1.100958
(Iteration 3791 / 9800) loss: 1.293782
(Iteration 3801 / 9800) loss: 1.264731
(Iteration 3811 / 9800) loss: 1.330004
(Iteration 3821 / 9800) loss: 1.200629
(Iteration 3831 / 9800) loss: 1.326434
(Iteration 3841 / 9800) loss: 1.173101
(Iteration 3851 / 9800) loss: 1.424872
(Iteration 3861 / 9800) loss: 1.249337
(Iteration 3871 / 9800) loss: 1.172034
(Iteration 3881 / 9800) loss: 1.342974
(Iteration 3891 / 9800) loss: 1.301817
(Iteration 3901 / 9800) loss: 1.270471
(Iteration 3911 / 9800) loss: 1.339189
(Epoch 8 / 20) train acc: 0.602000; val_acc: 0.510000
(Iteration 3921 / 9800) loss: 1.166227
(Iteration 3931 / 9800) loss: 1.089608
(Iteration 3941 / 9800) loss: 1.238489
(Iteration 3951 / 9800) loss: 1.115946
(Iteration 3961 / 9800) loss: 1.186545
(Iteration 3971 / 9800) loss: 1.219484
(Iteration 3981 / 9800) loss: 1.072520
(Iteration 3991 / 9800) loss: 1.252926
(Iteration 4001 / 9800) loss: 1.166300
(Iteration 4011 / 9800) loss: 1.309957
(Iteration 4021 / 9800) loss: 1.277472
(Iteration 4031 / 9800) loss: 1.171622
(Iteration 4041 / 9800) loss: 1.338563
(Iteration 4051 / 9800) loss: 1.191969
(Iteration 4061 / 9800) loss: 1.119896
(Iteration 4071 / 9800) loss: 1.299956
(Iteration 4081 / 9800) loss: 1.221608
(Iteration 4091 / 9800) loss: 1.251258
(Iteration 4101 / 9800) loss: 1.394385
(Iteration 4111 / 9800) loss: 1.393438
(Iteration 4121 / 9800) loss: 1.379502
(Iteration 4131 / 9800) loss: 1.189105
(Iteration 4141 / 9800) loss: 1.114624
(Iteration 4151 / 9800) loss: 1.248349
(Iteration 4161 / 9800) loss: 1.184640
(Iteration 4171 / 9800) loss: 1.383718
```

```
(Iteration 4181 / 9800) loss: 1.063640
(Iteration 4191 / 9800) loss: 1.358246
(Iteration 4201 / 9800) loss: 1.130936
(Iteration 4211 / 9800) loss: 1.282646
(Iteration 4221 / 9800) loss: 1.142180
(Iteration 4231 / 9800) loss: 1.365414
(Iteration 4241 / 9800) loss: 1.049219
(Iteration 4251 / 9800) loss: 1.141818
(Iteration 4261 / 9800) loss: 1.249677
(Iteration 4271 / 9800) loss: 1.008368
(Iteration 4281 / 9800) loss: 1.186901
(Iteration 4291 / 9800) loss: 1.070666
(Iteration 4301 / 9800) loss: 1.281967
(Iteration 4311 / 9800) loss: 1.247162
(Iteration 4321 / 9800) loss: 1.398071
(Iteration 4331 / 9800) loss: 1.078045
(Iteration 4341 / 9800) loss: 1.249164
(Iteration 4351 / 9800) loss: 1.227578
(Iteration 4361 / 9800) loss: 0.992882
(Iteration 4371 / 9800) loss: 1.102346
(Iteration 4381 / 9800) loss: 1.155997
(Iteration 4391 / 9800) loss: 1.307169
(Iteration 4401 / 9800) loss: 1.325154
(Epoch 9 / 20) train acc: 0.616000; val_acc: 0.517000
(Iteration 4411 / 9800) loss: 1.277432
(Iteration 4421 / 9800) loss: 1.321761
(Iteration 4431 / 9800) loss: 1.218983
(Iteration 4441 / 9800) loss: 1.325236
(Iteration 4451 / 9800) loss: 1.177171
(Iteration 4461 / 9800) loss: 1.317166
(Iteration 4471 / 9800) loss: 1.020657
(Iteration 4481 / 9800) loss: 1.222454
(Iteration 4491 / 9800) loss: 1.224938
(Iteration 4501 / 9800) loss: 1.116534
(Iteration 4511 / 9800) loss: 1.104685
(Iteration 4521 / 9800) loss: 1.099834
(Iteration 4531 / 9800) loss: 1.138993
(Iteration 4541 / 9800) loss: 1.247728
(Iteration 4551 / 9800) loss: 1.249268
(Iteration 4561 / 9800) loss: 1.239872
(Iteration 4571 / 9800) loss: 1.058131
(Iteration 4581 / 9800) loss: 1.288981
(Iteration 4591 / 9800) loss: 1.185206
(Iteration 4601 / 9800) loss: 1.187936
(Iteration 4611 / 9800) loss: 1.574117
(Iteration 4621 / 9800) loss: 1.259087
(Iteration 4631 / 9800) loss: 1.241061
(Iteration 4641 / 9800) loss: 1.208826
```

```
(Iteration 4651 / 9800) loss: 1.172704
(Iteration 4661 / 9800) loss: 1.199911
(Iteration 4671 / 9800) loss: 1.139596
(Iteration 4681 / 9800) loss: 1.346764
(Iteration 4691 / 9800) loss: 1.209447
(Iteration 4701 / 9800) loss: 1.252989
(Iteration 4711 / 9800) loss: 1.059944
(Iteration 4721 / 9800) loss: 1.111214
(Iteration 4731 / 9800) loss: 1.087378
(Iteration 4741 / 9800) loss: 1.215526
(Iteration 4751 / 9800) loss: 1.259201
(Iteration 4761 / 9800) loss: 1.098469
(Iteration 4771 / 9800) loss: 1.241050
(Iteration 4781 / 9800) loss: 0.992929
(Iteration 4791 / 9800) loss: 1.229780
(Iteration 4801 / 9800) loss: 1.096459
(Iteration 4811 / 9800) loss: 1.206250
(Iteration 4821 / 9800) loss: 1.248963
(Iteration 4831 / 9800) loss: 1.134467
(Iteration 4841 / 9800) loss: 1.126240
(Iteration 4851 / 9800) loss: 1.231932
(Iteration 4861 / 9800) loss: 1.142309
(Iteration 4871 / 9800) loss: 1.135483
(Iteration 4881 / 9800) loss: 1.125551
(Iteration 4891 / 9800) loss: 1.067538
(Epoch 10 / 20) train acc: 0.651000; val_acc: 0.543000
(Iteration 4901 / 9800) loss: 0.961115
(Iteration 4911 / 9800) loss: 0.986491
(Iteration 4921 / 9800) loss: 1.012764
(Iteration 4931 / 9800) loss: 1.137336
(Iteration 4941 / 9800) loss: 1.531745
(Iteration 4951 / 9800) loss: 1.142373
(Iteration 4961 / 9800) loss: 1.352131
(Iteration 4971 / 9800) loss: 1.291407
(Iteration 4981 / 9800) loss: 1.240376
(Iteration 4991 / 9800) loss: 1.237543
(Iteration 5001 / 9800) loss: 0.961463
(Iteration 5011 / 9800) loss: 1.414477
(Iteration 5021 / 9800) loss: 1.113294
(Iteration 5031 / 9800) loss: 1.146237
(Iteration 5041 / 9800) loss: 1.166592
(Iteration 5051 / 9800) loss: 1.007359
(Iteration 5061 / 9800) loss: 1.313219
(Iteration 5071 / 9800) loss: 1.067518
(Iteration 5081 / 9800) loss: 1.073886
(Iteration 5091 / 9800) loss: 1.067092
(Iteration 5101 / 9800) loss: 1.190584
(Iteration 5111 / 9800) loss: 1.024485
```

```
(Iteration 5121 / 9800) loss: 1.120217
(Iteration 5131 / 9800) loss: 1.229999
(Iteration 5141 / 9800) loss: 1.160862
(Iteration 5151 / 9800) loss: 1.329462
(Iteration 5161 / 9800) loss: 1.111139
(Iteration 5171 / 9800) loss: 1.277412
(Iteration 5181 / 9800) loss: 1.193077
(Iteration 5191 / 9800) loss: 1.307094
(Iteration 5201 / 9800) loss: 1.264090
(Iteration 5211 / 9800) loss: 1.071812
(Iteration 5221 / 9800) loss: 1.186761
(Iteration 5231 / 9800) loss: 1.446820
(Iteration 5241 / 9800) loss: 1.239090
(Iteration 5251 / 9800) loss: 1.167312
(Iteration 5261 / 9800) loss: 1.259500
(Iteration 5271 / 9800) loss: 1.045090
(Iteration 5281 / 9800) loss: 0.940462
(Iteration 5291 / 9800) loss: 1.383351
(Iteration 5301 / 9800) loss: 1.199712
(Iteration 5311 / 9800) loss: 1.231503
(Iteration 5321 / 9800) loss: 1.110553
(Iteration 5331 / 9800) loss: 1.268673
(Iteration 5341 / 9800) loss: 1.275714
(Iteration 5351 / 9800) loss: 1.078135
(Iteration 5361 / 9800) loss: 1.048049
(Iteration 5371 / 9800) loss: 1.219884
(Iteration 5381 / 9800) loss: 1.174242
(Epoch 11 / 20) train acc: 0.611000; val_acc: 0.509000
(Iteration 5391 / 9800) loss: 1.337777
(Iteration 5401 / 9800) loss: 1.080839
(Iteration 5411 / 9800) loss: 1.111761
(Iteration 5421 / 9800) loss: 1.391246
(Iteration 5431 / 9800) loss: 1.209044
(Iteration 5441 / 9800) loss: 1.222030
(Iteration 5451 / 9800) loss: 1.098194
(Iteration 5461 / 9800) loss: 1.097807
(Iteration 5471 / 9800) loss: 1.050674
(Iteration 5481 / 9800) loss: 1.332001
(Iteration 5491 / 9800) loss: 1.345823
(Iteration 5501 / 9800) loss: 1.189717
(Iteration 5511 / 9800) loss: 1.244856
(Iteration 5521 / 9800) loss: 1.352873
(Iteration 5531 / 9800) loss: 1.097053
(Iteration 5541 / 9800) loss: 1.275544
(Iteration 5551 / 9800) loss: 1.021958
(Iteration 5561 / 9800) loss: 1.057446
(Iteration 5571 / 9800) loss: 1.153711
(Iteration 5581 / 9800) loss: 1.150956
```

```
(Iteration 5591 / 9800) loss: 1.158976
(Iteration 5601 / 9800) loss: 1.268306
(Iteration 5611 / 9800) loss: 1.330313
(Iteration 5621 / 9800) loss: 1.027918
(Iteration 5631 / 9800) loss: 1.373772
(Iteration 5641 / 9800) loss: 1.301236
(Iteration 5651 / 9800) loss: 1.128404
(Iteration 5661 / 9800) loss: 1.310874
(Iteration 5671 / 9800) loss: 1.070530
(Iteration 5681 / 9800) loss: 0.968226
(Iteration 5691 / 9800) loss: 1.054921
(Iteration 5701 / 9800) loss: 1.203736
(Iteration 5711 / 9800) loss: 1.367652
(Iteration 5721 / 9800) loss: 1.220476
(Iteration 5731 / 9800) loss: 1.111621
(Iteration 5741 / 9800) loss: 1.061065
(Iteration 5751 / 9800) loss: 1.093569
(Iteration 5761 / 9800) loss: 1.230148
(Iteration 5771 / 9800) loss: 1.178357
(Iteration 5781 / 9800) loss: 1.201159
(Iteration 5791 / 9800) loss: 1.165212
(Iteration 5801 / 9800) loss: 1.067923
(Iteration 5811 / 9800) loss: 1.306083
(Iteration 5821 / 9800) loss: 1.109932
(Iteration 5831 / 9800) loss: 1.111030
(Iteration 5841 / 9800) loss: 1.102027
(Iteration 5851 / 9800) loss: 1.307560
(Iteration 5861 / 9800) loss: 1.246265
(Iteration 5871 / 9800) loss: 1.027980
(Epoch 12 / 20) train acc: 0.628000; val_acc: 0.525000
(Iteration 5881 / 9800) loss: 1.229268
(Iteration 5891 / 9800) loss: 1.193636
(Iteration 5901 / 9800) loss: 1.140893
(Iteration 5911 / 9800) loss: 1.080677
(Iteration 5921 / 9800) loss: 1.246139
(Iteration 5931 / 9800) loss: 0.986091
(Iteration 5941 / 9800) loss: 1.061448
(Iteration 5951 / 9800) loss: 1.115149
(Iteration 5961 / 9800) loss: 1.079524
(Iteration 5971 / 9800) loss: 1.166059
(Iteration 5981 / 9800) loss: 1.024615
(Iteration 5991 / 9800) loss: 1.289047
(Iteration 6001 / 9800) loss: 1.210847
(Iteration 6011 / 9800) loss: 1.110809
(Iteration 6021 / 9800) loss: 1.213990
(Iteration 6031 / 9800) loss: 1.095054
(Iteration 6041 / 9800) loss: 1.364698
(Iteration 6051 / 9800) loss: 1.329035
```

```
(Iteration 6061 / 9800) loss: 1.233828
(Iteration 6071 / 9800) loss: 1.099518
(Iteration 6081 / 9800) loss: 1.161503
(Iteration 6091 / 9800) loss: 1.053864
(Iteration 6101 / 9800) loss: 0.953990
(Iteration 6111 / 9800) loss: 1.434079
(Iteration 6121 / 9800) loss: 1.106493
(Iteration 6131 / 9800) loss: 1.229247
(Iteration 6141 / 9800) loss: 1.299329
(Iteration 6151 / 9800) loss: 1.273109
(Iteration 6161 / 9800) loss: 1.272520
(Iteration 6171 / 9800) loss: 1.244209
(Iteration 6181 / 9800) loss: 0.974135
(Iteration 6191 / 9800) loss: 1.096558
(Iteration 6201 / 9800) loss: 1.191278
(Iteration 6211 / 9800) loss: 1.111938
(Iteration 6221 / 9800) loss: 1.142923
(Iteration 6231 / 9800) loss: 0.932948
(Iteration 6241 / 9800) loss: 1.335512
(Iteration 6251 / 9800) loss: 1.179605
(Iteration 6261 / 9800) loss: 1.118926
(Iteration 6271 / 9800) loss: 1.171615
(Iteration 6281 / 9800) loss: 1.192793
(Iteration 6291 / 9800) loss: 1.132583
(Iteration 6301 / 9800) loss: 1.275952
(Iteration 6311 / 9800) loss: 1.099699
(Iteration 6321 / 9800) loss: 1.159624
(Iteration 6331 / 9800) loss: 1.157289
(Iteration 6341 / 9800) loss: 1.169285
(Iteration 6351 / 9800) loss: 1.127803
(Iteration 6361 / 9800) loss: 1.159332
(Epoch 13 / 20) train acc: 0.635000; val_acc: 0.510000
(Iteration 6371 / 9800) loss: 1.228189
(Iteration 6381 / 9800) loss: 1.273958
(Iteration 6391 / 9800) loss: 1.170508
(Iteration 6401 / 9800) loss: 1.214013
(Iteration 6411 / 9800) loss: 1.181990
(Iteration 6421 / 9800) loss: 1.028996
(Iteration 6431 / 9800) loss: 1.250551
(Iteration 6441 / 9800) loss: 1.151581
(Iteration 6451 / 9800) loss: 1.109060
(Iteration 6461 / 9800) loss: 1.247191
(Iteration 6471 / 9800) loss: 1.089058
(Iteration 6481 / 9800) loss: 1.065417
(Iteration 6491 / 9800) loss: 1.022940
(Iteration 6501 / 9800) loss: 1.147336
(Iteration 6511 / 9800) loss: 1.306753
(Iteration 6521 / 9800) loss: 1.042737
```

```
(Iteration 6531 / 9800) loss: 1.158160
(Iteration 6541 / 9800) loss: 1.099286
(Iteration 6551 / 9800) loss: 0.991594
(Iteration 6561 / 9800) loss: 1.211508
(Iteration 6571 / 9800) loss: 1.038308
(Iteration 6581 / 9800) loss: 1.116537
(Iteration 6591 / 9800) loss: 1.203629
(Iteration 6601 / 9800) loss: 0.946438
(Iteration 6611 / 9800) loss: 1.175253
(Iteration 6621 / 9800) loss: 1.131695
(Iteration 6631 / 9800) loss: 1.059237
(Iteration 6641 / 9800) loss: 1.007720
(Iteration 6651 / 9800) loss: 1.018807
(Iteration 6661 / 9800) loss: 0.933400
(Iteration 6671 / 9800) loss: 1.089268
(Iteration 6681 / 9800) loss: 1.099720
(Iteration 6691 / 9800) loss: 1.024811
(Iteration 6701 / 9800) loss: 1.193737
(Iteration 6711 / 9800) loss: 1.336146
(Iteration 6721 / 9800) loss: 0.827025
(Iteration 6731 / 9800) loss: 1.208970
(Iteration 6741 / 9800) loss: 1.285498
(Iteration 6751 / 9800) loss: 1.173043
(Iteration 6761 / 9800) loss: 1.103103
(Iteration 6771 / 9800) loss: 1.338433
(Iteration 6781 / 9800) loss: 1.032693
(Iteration 6791 / 9800) loss: 1.188205
(Iteration 6801 / 9800) loss: 1.224924
(Iteration 6811 / 9800) loss: 1.094447
(Iteration 6821 / 9800) loss: 1.042502
(Iteration 6831 / 9800) loss: 1.293753
(Iteration 6841 / 9800) loss: 1.073939
(Iteration 6851 / 9800) loss: 1.208021
(Epoch 14 / 20) train acc: 0.655000; val_acc: 0.515000
(Iteration 6861 / 9800) loss: 1.101865
(Iteration 6871 / 9800) loss: 1.037589
(Iteration 6881 / 9800) loss: 1.024748
(Iteration 6891 / 9800) loss: 1.097113
(Iteration 6901 / 9800) loss: 1.187467
(Iteration 6911 / 9800) loss: 1.134951
(Iteration 6921 / 9800) loss: 1.175560
(Iteration 6931 / 9800) loss: 0.994963
(Iteration 6941 / 9800) loss: 1.108261
(Iteration 6951 / 9800) loss: 1.270276
(Iteration 6961 / 9800) loss: 1.077368
(Iteration 6971 / 9800) loss: 1.091440
(Iteration 6981 / 9800) loss: 0.925041
(Iteration 6991 / 9800) loss: 1.007995
```

```
(Iteration 7001 / 9800) loss: 1.137205
(Iteration 7011 / 9800) loss: 1.090217
(Iteration 7021 / 9800) loss: 1.188174
(Iteration 7031 / 9800) loss: 1.188936
(Iteration 7041 / 9800) loss: 1.108131
(Iteration 7051 / 9800) loss: 1.071259
(Iteration 7061 / 9800) loss: 1.152544
(Iteration 7071 / 9800) loss: 1.330588
(Iteration 7081 / 9800) loss: 1.077823
(Iteration 7091 / 9800) loss: 1.234527
(Iteration 7101 / 9800) loss: 1.066372
(Iteration 7111 / 9800) loss: 1.292032
(Iteration 7121 / 9800) loss: 1.306403
(Iteration 7131 / 9800) loss: 1.051640
(Iteration 7141 / 9800) loss: 1.223294
(Iteration 7151 / 9800) loss: 1.167007
(Iteration 7161 / 9800) loss: 0.949088
(Iteration 7171 / 9800) loss: 1.142530
(Iteration 7181 / 9800) loss: 1.028404
(Iteration 7191 / 9800) loss: 1.111086
(Iteration 7201 / 9800) loss: 0.977683
(Iteration 7211 / 9800) loss: 1.031633
(Iteration 7221 / 9800) loss: 1.067996
(Iteration 7231 / 9800) loss: 1.152332
(Iteration 7241 / 9800) loss: 1.004715
(Iteration 7251 / 9800) loss: 1.036092
(Iteration 7261 / 9800) loss: 1.051548
(Iteration 7271 / 9800) loss: 0.972533
(Iteration 7281 / 9800) loss: 1.130758
(Iteration 7291 / 9800) loss: 0.990227
(Iteration 7301 / 9800) loss: 1.221541
(Iteration 7311 / 9800) loss: 1.191900
(Iteration 7321 / 9800) loss: 1.137014
(Iteration 7331 / 9800) loss: 1.036083
(Iteration 7341 / 9800) loss: 0.943927
(Epoch 15 / 20) train acc: 0.666000; val_acc: 0.515000
(Iteration 7351 / 9800) loss: 1.049612
(Iteration 7361 / 9800) loss: 1.030261
(Iteration 7371 / 9800) loss: 1.011529
(Iteration 7381 / 9800) loss: 1.179346
(Iteration 7391 / 9800) loss: 1.004706
(Iteration 7401 / 9800) loss: 1.021788
(Iteration 7411 / 9800) loss: 0.923104
(Iteration 7421 / 9800) loss: 1.026477
(Iteration 7431 / 9800) loss: 1.136335
(Iteration 7441 / 9800) loss: 0.869789
(Iteration 7451 / 9800) loss: 0.967273
(Iteration 7461 / 9800) loss: 1.277311
```

```
(Iteration 7471 / 9800) loss: 0.998339
(Iteration 7481 / 9800) loss: 1.172426
(Iteration 7491 / 9800) loss: 1.169474
(Iteration 7501 / 9800) loss: 0.989528
(Iteration 7511 / 9800) loss: 1.014156
(Iteration 7521 / 9800) loss: 1.209149
(Iteration 7531 / 9800) loss: 1.045853
(Iteration 7541 / 9800) loss: 1.149116
(Iteration 7551 / 9800) loss: 1.089233
(Iteration 7561 / 9800) loss: 1.072786
(Iteration 7571 / 9800) loss: 1.115404
(Iteration 7581 / 9800) loss: 1.312156
(Iteration 7591 / 9800) loss: 1.100165
(Iteration 7601 / 9800) loss: 0.838182
(Iteration 7611 / 9800) loss: 1.233361
(Iteration 7621 / 9800) loss: 1.084712
(Iteration 7631 / 9800) loss: 1.031081
(Iteration 7641 / 9800) loss: 0.865773
(Iteration 7651 / 9800) loss: 1.019016
(Iteration 7661 / 9800) loss: 1.047257
(Iteration 7671 / 9800) loss: 0.975172
(Iteration 7681 / 9800) loss: 1.039816
(Iteration 7691 / 9800) loss: 1.209504
(Iteration 7701 / 9800) loss: 1.063674
(Iteration 7711 / 9800) loss: 0.978152
(Iteration 7721 / 9800) loss: 1.060715
(Iteration 7731 / 9800) loss: 1.202422
(Iteration 7741 / 9800) loss: 0.969700
(Iteration 7751 / 9800) loss: 1.080400
(Iteration 7761 / 9800) loss: 1.111139
(Iteration 7771 / 9800) loss: 1.054113
(Iteration 7781 / 9800) loss: 0.982350
(Iteration 7791 / 9800) loss: 0.955049
(Iteration 7801 / 9800) loss: 0.939857
(Iteration 7811 / 9800) loss: 1.171169
(Iteration 7821 / 9800) loss: 1.087069
(Iteration 7831 / 9800) loss: 0.934624
(Epoch 16 / 20) train acc: 0.637000; val_acc: 0.525000
(Iteration 7841 / 9800) loss: 1.007471
(Iteration 7851 / 9800) loss: 0.936938
(Iteration 7861 / 9800) loss: 0.987656
(Iteration 7871 / 9800) loss: 1.231979
(Iteration 7881 / 9800) loss: 1.123377
(Iteration 7891 / 9800) loss: 1.011667
(Iteration 7901 / 9800) loss: 0.961167
(Iteration 7911 / 9800) loss: 1.241199
(Iteration 7921 / 9800) loss: 1.102130
(Iteration 7931 / 9800) loss: 1.112973
```

```
(Iteration 7941 / 9800) loss: 1.180151
(Iteration 7951 / 9800) loss: 1.287680
(Iteration 7961 / 9800) loss: 1.094844
(Iteration 7971 / 9800) loss: 1.243182
(Iteration 7981 / 9800) loss: 0.883948
(Iteration 7991 / 9800) loss: 1.220379
(Iteration 8001 / 9800) loss: 1.032775
(Iteration 8011 / 9800) loss: 1.139401
(Iteration 8021 / 9800) loss: 1.045552
(Iteration 8031 / 9800) loss: 0.995086
(Iteration 8041 / 9800) loss: 1.086033
(Iteration 8051 / 9800) loss: 1.161934
(Iteration 8061 / 9800) loss: 1.143605
(Iteration 8071 / 9800) loss: 1.134240
(Iteration 8081 / 9800) loss: 1.147995
(Iteration 8091 / 9800) loss: 1.023259
(Iteration 8101 / 9800) loss: 1.181699
(Iteration 8111 / 9800) loss: 1.373975
(Iteration 8121 / 9800) loss: 1.084528
(Iteration 8131 / 9800) loss: 1.160855
(Iteration 8141 / 9800) loss: 1.111692
(Iteration 8151 / 9800) loss: 1.121117
(Iteration 8161 / 9800) loss: 1.084458
(Iteration 8171 / 9800) loss: 1.262126
(Iteration 8181 / 9800) loss: 1.142169
(Iteration 8191 / 9800) loss: 0.960673
(Iteration 8201 / 9800) loss: 0.989727
(Iteration 8211 / 9800) loss: 1.254569
(Iteration 8221 / 9800) loss: 1.169747
(Iteration 8231 / 9800) loss: 0.885493
(Iteration 8241 / 9800) loss: 1.096277
(Iteration 8251 / 9800) loss: 1.040404
(Iteration 8261 / 9800) loss: 1.103169
(Iteration 8271 / 9800) loss: 1.102422
(Iteration 8281 / 9800) loss: 0.992528
(Iteration 8291 / 9800) loss: 1.269324
(Iteration 8301 / 9800) loss: 1.075205
(Iteration 8311 / 9800) loss: 1.039257
(Iteration 8321 / 9800) loss: 1.027761
(Epoch 17 / 20) train acc: 0.691000; val_acc: 0.543000
(Iteration 8331 / 9800) loss: 1.494238
(Iteration 8341 / 9800) loss: 1.098580
(Iteration 8351 / 9800) loss: 1.119176
(Iteration 8361 / 9800) loss: 1.140926
(Iteration 8371 / 9800) loss: 1.149797
(Iteration 8381 / 9800) loss: 1.246298
(Iteration 8391 / 9800) loss: 1.093896
(Iteration 8401 / 9800) loss: 1.044911
```

```
(Iteration 8411 / 9800) loss: 1.071006
(Iteration 8421 / 9800) loss: 0.927280
(Iteration 8431 / 9800) loss: 1.184353
(Iteration 8441 / 9800) loss: 1.058967
(Iteration 8451 / 9800) loss: 1.422620
(Iteration 8461 / 9800) loss: 1.093139
(Iteration 8471 / 9800) loss: 1.081386
(Iteration 8481 / 9800) loss: 1.019128
(Iteration 8491 / 9800) loss: 1.302242
(Iteration 8501 / 9800) loss: 1.199787
(Iteration 8511 / 9800) loss: 1.067100
(Iteration 8521 / 9800) loss: 0.993127
(Iteration 8531 / 9800) loss: 1.089262
(Iteration 8541 / 9800) loss: 1.018691
(Iteration 8551 / 9800) loss: 0.969948
(Iteration 8561 / 9800) loss: 0.927672
(Iteration 8571 / 9800) loss: 0.959244
(Iteration 8581 / 9800) loss: 0.951504
(Iteration 8591 / 9800) loss: 1.008738
(Iteration 8601 / 9800) loss: 1.061585
(Iteration 8611 / 9800) loss: 1.007721
(Iteration 8621 / 9800) loss: 1.045398
(Iteration 8631 / 9800) loss: 0.816929
(Iteration 8641 / 9800) loss: 0.961766
(Iteration 8651 / 9800) loss: 1.045650
(Iteration 8661 / 9800) loss: 1.052445
(Iteration 8671 / 9800) loss: 1.222736
(Iteration 8681 / 9800) loss: 1.233010
(Iteration 8691 / 9800) loss: 1.105267
(Iteration 8701 / 9800) loss: 1.064255
(Iteration 8711 / 9800) loss: 1.132519
(Iteration 8721 / 9800) loss: 1.119258
(Iteration 8731 / 9800) loss: 0.983290
(Iteration 8741 / 9800) loss: 1.104748
(Iteration 8751 / 9800) loss: 0.930216
(Iteration 8761 / 9800) loss: 1.191898
(Iteration 8771 / 9800) loss: 1.112245
(Iteration 8781 / 9800) loss: 1.049062
(Iteration 8791 / 9800) loss: 0.999654
(Iteration 8801 / 9800) loss: 0.843064
(Iteration 8811 / 9800) loss: 1.064716
(Epoch 18 / 20) train acc: 0.666000; val_acc: 0.521000
(Iteration 8821 / 9800) loss: 1.105867
(Iteration 8831 / 9800) loss: 1.096241
(Iteration 8841 / 9800) loss: 1.092866
(Iteration 8851 / 9800) loss: 0.928566
(Iteration 8861 / 9800) loss: 1.148820
(Iteration 8871 / 9800) loss: 1.035897
```

```
(Iteration 8881 / 9800) loss: 1.177601
(Iteration 8891 / 9800) loss: 0.924206
(Iteration 8901 / 9800) loss: 1.034254
(Iteration 8911 / 9800) loss: 1.018664
(Iteration 8921 / 9800) loss: 1.025005
(Iteration 8931 / 9800) loss: 1.062014
(Iteration 8941 / 9800) loss: 0.926946
(Iteration 8951 / 9800) loss: 1.036484
(Iteration 8961 / 9800) loss: 1.064523
(Iteration 8971 / 9800) loss: 0.944256
(Iteration 8981 / 9800) loss: 1.184308
(Iteration 8991 / 9800) loss: 1.072583
(Iteration 9001 / 9800) loss: 1.038786
(Iteration 9011 / 9800) loss: 1.093233
(Iteration 9021 / 9800) loss: 0.873463
(Iteration 9031 / 9800) loss: 0.905079
(Iteration 9041 / 9800) loss: 1.096940
(Iteration 9051 / 9800) loss: 1.020996
(Iteration 9061 / 9800) loss: 1.019088
(Iteration 9071 / 9800) loss: 1.112123
(Iteration 9081 / 9800) loss: 1.094359
(Iteration 9091 / 9800) loss: 1.234435
(Iteration 9101 / 9800) loss: 1.054574
(Iteration 9111 / 9800) loss: 0.999379
(Iteration 9121 / 9800) loss: 1.143528
(Iteration 9131 / 9800) loss: 0.955655
(Iteration 9141 / 9800) loss: 1.083068
(Iteration 9151 / 9800) loss: 1.015170
(Iteration 9161 / 9800) loss: 1.165545
(Iteration 9171 / 9800) loss: 1.051957
(Iteration 9181 / 9800) loss: 1.195180
(Iteration 9191 / 9800) loss: 0.930207
(Iteration 9201 / 9800) loss: 1.168000
(Iteration 9211 / 9800) loss: 1.005892
(Iteration 9221 / 9800) loss: 0.924386
(Iteration 9231 / 9800) loss: 1.009745
(Iteration 9241 / 9800) loss: 1.301386
(Iteration 9251 / 9800) loss: 0.882459
(Iteration 9261 / 9800) loss: 1.065841
(Iteration 9271 / 9800) loss: 0.958170
(Iteration 9281 / 9800) loss: 1.111218
(Iteration 9291 / 9800) loss: 1.131964
(Iteration 9301 / 9800) loss: 1.114873
(Epoch 19 / 20) train acc: 0.671000; val_acc: 0.514000
(Iteration 9311 / 9800) loss: 0.828249
(Iteration 9321 / 9800) loss: 1.073451
(Iteration 9331 / 9800) loss: 1.264521
(Iteration 9341 / 9800) loss: 0.858981
```

```
(Iteration 9351 / 9800) loss: 0.976301
(Iteration 9361 / 9800) loss: 1.172609
(Iteration 9371 / 9800) loss: 1.104080
(Iteration 9381 / 9800) loss: 1.170523
(Iteration 9391 / 9800) loss: 0.932775
(Iteration 9401 / 9800) loss: 1.034237
(Iteration 9411 / 9800) loss: 0.954058
(Iteration 9421 / 9800) loss: 0.984734
(Iteration 9431 / 9800) loss: 1.211318
(Iteration 9441 / 9800) loss: 1.157250
(Iteration 9451 / 9800) loss: 0.855733
(Iteration 9461 / 9800) loss: 0.980386
(Iteration 9471 / 9800) loss: 1.182613
(Iteration 9481 / 9800) loss: 0.851755
(Iteration 9491 / 9800) loss: 1.006444
(Iteration 9501 / 9800) loss: 1.184517
(Iteration 9511 / 9800) loss: 1.249047
(Iteration 9521 / 9800) loss: 1.001267
(Iteration 9531 / 9800) loss: 1.346382
(Iteration 9541 / 9800) loss: 1.018070
(Iteration 9551 / 9800) loss: 1.006526
(Iteration 9561 / 9800) loss: 1.119997
(Iteration 9571 / 9800) loss: 1.105955
(Iteration 9581 / 9800) loss: 1.116797
(Iteration 9591 / 9800) loss: 1.097641
(Iteration 9601 / 9800) loss: 0.924388
(Iteration 9611 / 9800) loss: 1.093765
(Iteration 9621 / 9800) loss: 1.275140
(Iteration 9631 / 9800) loss: 1.056993
(Iteration 9641 / 9800) loss: 1.028042
(Iteration 9651 / 9800) loss: 0.997823
(Iteration 9661 / 9800) loss: 1.059211
(Iteration 9671 / 9800) loss: 1.058384
(Iteration 9681 / 9800) loss: 0.852402
(Iteration 9691 / 9800) loss: 1.190700
(Iteration 9701 / 9800) loss: 0.970414
(Iteration 9711 / 9800) loss: 0.916068
(Iteration 9721 / 9800) loss: 1.133978
(Iteration 9731 / 9800) loss: 1.075402
(Iteration 9741 / 9800) loss: 1.054591
(Iteration 9751 / 9800) loss: 0.865407
(Iteration 9761 / 9800) loss: 1.062643
(Iteration 9771 / 9800) loss: 0.976997
(Iteration 9781 / 9800) loss: 1.120026
(Iteration 9791 / 9800) loss: 1.034989
(Epoch 20 / 20) train acc: 0.715000; val_acc: 0.539000
accuracy:0.543 with lr:0.0005,reg:0.1,epoch:20,hidden_dim:200
(Iteration 1 / 9800) loss: 2.376819
```

```
(Epoch 0 / 20) train acc: 0.142000; val_acc: 0.139000
(Iteration 11 / 9800) loss: 2.310984
(Iteration 21 / 9800) loss: 2.281809
(Iteration 31 / 9800) loss: 2.177850
(Iteration 41 / 9800) loss: 2.089085
(Iteration 51 / 9800) loss: 2.200912
(Iteration 61 / 9800) loss: 2.141459
(Iteration 71 / 9800) loss: 1.951717
(Iteration 81 / 9800) loss: 1.921045
(Iteration 91 / 9800) loss: 2.072389
(Iteration 101 / 9800) loss: 2.019898
(Iteration 111 / 9800) loss: 2.019726
(Iteration 121 / 9800) loss: 1.999647
(Iteration 131 / 9800) loss: 1.942526
(Iteration 141 / 9800) loss: 1.819173
(Iteration 151 / 9800) loss: 1.866991
(Iteration 161 / 9800) loss: 1.998578
(Iteration 171 / 9800) loss: 1.780871
(Iteration 181 / 9800) loss: 1.974465
(Iteration 191 / 9800) loss: 1.913369
(Iteration 201 / 9800) loss: 1.782992
(Iteration 211 / 9800) loss: 1.794418
(Iteration 221 / 9800) loss: 1.827950
(Iteration 231 / 9800) loss: 1.914119
(Iteration 241 / 9800) loss: 1.900234
(Iteration 251 / 9800) loss: 1.881519
(Iteration 261 / 9800) loss: 1.826530
(Iteration 271 / 9800) loss: 1.898343
(Iteration 281 / 9800) loss: 1.780369
(Iteration 291 / 9800) loss: 1.869895
(Iteration 301 / 9800) loss: 1.777707
(Iteration 311 / 9800) loss: 1.793714
(Iteration 321 / 9800) loss: 1.604018
(Iteration 331 / 9800) loss: 1.739224
(Iteration 341 / 9800) loss: 1.748157
(Iteration 351 / 9800) loss: 1.777043
(Iteration 361 / 9800) loss: 1.768223
(Iteration 371 / 9800) loss: 1.696772
(Iteration 381 / 9800) loss: 1.716271
(Iteration 391 / 9800) loss: 1.751652
(Iteration 401 / 9800) loss: 1.693369
(Iteration 411 / 9800) loss: 1.780886
(Iteration 421 / 9800) loss: 1.610305
(Iteration 431 / 9800) loss: 1.637056
(Iteration 441 / 9800) loss: 1.684133
(Iteration 451 / 9800) loss: 1.780854
(Iteration 461 / 9800) loss: 1.735888
(Iteration 471 / 9800) loss: 1.539125
```

```
(Iteration 481 / 9800) loss: 1.705266
(Epoch 1 / 20) train acc: 0.439000; val_acc: 0.451000
(Iteration 491 / 9800) loss: 1.788549
(Iteration 501 / 9800) loss: 1.672754
(Iteration 511 / 9800) loss: 1.766759
(Iteration 521 / 9800) loss: 1.708312
(Iteration 531 / 9800) loss: 1.787060
(Iteration 541 / 9800) loss: 1.564828
(Iteration 551 / 9800) loss: 1.567356
(Iteration 561 / 9800) loss: 1.761735
(Iteration 571 / 9800) loss: 1.753149
(Iteration 581 / 9800) loss: 1.522242
(Iteration 591 / 9800) loss: 1.710672
(Iteration 601 / 9800) loss: 1.707683
(Iteration 611 / 9800) loss: 1.613594
(Iteration 621 / 9800) loss: 1.839240
(Iteration 631 / 9800) loss: 1.414729
(Iteration 641 / 9800) loss: 1.658812
(Iteration 651 / 9800) loss: 1.598965
(Iteration 661 / 9800) loss: 1.814654
(Iteration 671 / 9800) loss: 1.507728
(Iteration 681 / 9800) loss: 1.559319
(Iteration 691 / 9800) loss: 1.503079
(Iteration 701 / 9800) loss: 1.581556
(Iteration 711 / 9800) loss: 1.489947
(Iteration 721 / 9800) loss: 1.632051
(Iteration 731 / 9800) loss: 1.671880
(Iteration 741 / 9800) loss: 1.619137
(Iteration 751 / 9800) loss: 1.791767
(Iteration 761 / 9800) loss: 1.611881
(Iteration 771 / 9800) loss: 1.544285
(Iteration 781 / 9800) loss: 1.428496
(Iteration 791 / 9800) loss: 1.644901
(Iteration 801 / 9800) loss: 1.609305
(Iteration 811 / 9800) loss: 1.444278
(Iteration 821 / 9800) loss: 1.588703
(Iteration 831 / 9800) loss: 1.748826
(Iteration 841 / 9800) loss: 1.713267
(Iteration 851 / 9800) loss: 1.594737
(Iteration 861 / 9800) loss: 1.756835
(Iteration 871 / 9800) loss: 1.609650
(Iteration 881 / 9800) loss: 1.648279
(Iteration 891 / 9800) loss: 1.615146
(Iteration 901 / 9800) loss: 1.701784
(Iteration 911 / 9800) loss: 1.499408
(Iteration 921 / 9800) loss: 1.388910
(Iteration 931 / 9800) loss: 1.635094
(Iteration 941 / 9800) loss: 1.516038
```

```
(Iteration 951 / 9800) loss: 1.572910
(Iteration 961 / 9800) loss: 1.461119
(Iteration 971 / 9800) loss: 1.621387
(Epoch 2 / 20) train acc: 0.516000; val_acc: 0.470000
(Iteration 981 / 9800) loss: 1.440053
(Iteration 991 / 9800) loss: 1.627443
(Iteration 1001 / 9800) loss: 1.453202
(Iteration 1011 / 9800) loss: 1.546498
(Iteration 1021 / 9800) loss: 1.530984
(Iteration 1031 / 9800) loss: 1.450439
(Iteration 1041 / 9800) loss: 1.557160
(Iteration 1051 / 9800) loss: 1.439600
(Iteration 1061 / 9800) loss: 1.525012
(Iteration 1071 / 9800) loss: 1.481833
(Iteration 1081 / 9800) loss: 1.435178
(Iteration 1091 / 9800) loss: 1.677440
(Iteration 1101 / 9800) loss: 1.572109
(Iteration 1111 / 9800) loss: 1.665599
(Iteration 1121 / 9800) loss: 1.534174
(Iteration 1131 / 9800) loss: 1.704065
(Iteration 1141 / 9800) loss: 1.589184
(Iteration 1151 / 9800) loss: 1.430618
(Iteration 1161 / 9800) loss: 1.555589
(Iteration 1171 / 9800) loss: 1.647464
(Iteration 1181 / 9800) loss: 1.463370
(Iteration 1191 / 9800) loss: 1.734476
(Iteration 1201 / 9800) loss: 1.473417
(Iteration 1211 / 9800) loss: 1.388117
(Iteration 1221 / 9800) loss: 1.482502
(Iteration 1231 / 9800) loss: 1.700696
(Iteration 1241 / 9800) loss: 1.543725
(Iteration 1251 / 9800) loss: 1.484088
(Iteration 1261 / 9800) loss: 1.440089
(Iteration 1271 / 9800) loss: 1.603478
(Iteration 1281 / 9800) loss: 1.519276
(Iteration 1291 / 9800) loss: 1.660623
(Iteration 1301 / 9800) loss: 1.577615
(Iteration 1311 / 9800) loss: 1.575148
(Iteration 1321 / 9800) loss: 1.701015
(Iteration 1331 / 9800) loss: 1.363723
(Iteration 1341 / 9800) loss: 1.420637
(Iteration 1351 / 9800) loss: 1.400961
(Iteration 1361 / 9800) loss: 1.404815
(Iteration 1371 / 9800) loss: 1.404611
(Iteration 1381 / 9800) loss: 1.301515
(Iteration 1391 / 9800) loss: 1.606256
(Iteration 1401 / 9800) loss: 1.496667
(Iteration 1411 / 9800) loss: 1.424478
```

```
(Iteration 1421 / 9800) loss: 1.627378
(Iteration 1431 / 9800) loss: 1.363121
(Iteration 1441 / 9800) loss: 1.453104
(Iteration 1451 / 9800) loss: 1.577955
(Iteration 1461 / 9800) loss: 1.513062
(Epoch 3 / 20) train acc: 0.523000; val_acc: 0.485000
(Iteration 1471 / 9800) loss: 1.428064
(Iteration 1481 / 9800) loss: 1.303320
(Iteration 1491 / 9800) loss: 1.377109
(Iteration 1501 / 9800) loss: 1.454084
(Iteration 1511 / 9800) loss: 1.553933
(Iteration 1521 / 9800) loss: 1.530294
(Iteration 1531 / 9800) loss: 1.570209
(Iteration 1541 / 9800) loss: 1.318230
(Iteration 1551 / 9800) loss: 1.680429
(Iteration 1561 / 9800) loss: 1.454873
(Iteration 1571 / 9800) loss: 1.452048
(Iteration 1581 / 9800) loss: 1.444456
(Iteration 1591 / 9800) loss: 1.592862
(Iteration 1601 / 9800) loss: 1.631770
(Iteration 1611 / 9800) loss: 1.379280
(Iteration 1621 / 9800) loss: 1.311157
(Iteration 1631 / 9800) loss: 1.506810
(Iteration 1641 / 9800) loss: 1.400283
(Iteration 1651 / 9800) loss: 1.545883
(Iteration 1661 / 9800) loss: 1.394674
(Iteration 1671 / 9800) loss: 1.328967
(Iteration 1681 / 9800) loss: 1.354115
(Iteration 1691 / 9800) loss: 1.503988
(Iteration 1701 / 9800) loss: 1.473484
(Iteration 1711 / 9800) loss: 1.294647
(Iteration 1721 / 9800) loss: 1.429982
(Iteration 1731 / 9800) loss: 1.568164
(Iteration 1741 / 9800) loss: 1.465731
(Iteration 1751 / 9800) loss: 1.575245
(Iteration 1761 / 9800) loss: 1.459131
(Iteration 1771 / 9800) loss: 1.333267
(Iteration 1781 / 9800) loss: 1.411572
(Iteration 1791 / 9800) loss: 1.414482
(Iteration 1801 / 9800) loss: 1.633507
(Iteration 1811 / 9800) loss: 1.485815
(Iteration 1821 / 9800) loss: 1.388748
(Iteration 1831 / 9800) loss: 1.673972
(Iteration 1841 / 9800) loss: 1.377239
(Iteration 1851 / 9800) loss: 1.281006
(Iteration 1861 / 9800) loss: 1.420176
(Iteration 1871 / 9800) loss: 1.437727
(Iteration 1881 / 9800) loss: 1.433184
```

```
(Iteration 1891 / 9800) loss: 1.583774
(Iteration 1901 / 9800) loss: 1.410844
(Iteration 1911 / 9800) loss: 1.551592
(Iteration 1921 / 9800) loss: 1.416319
(Iteration 1931 / 9800) loss: 1.536470
(Iteration 1941 / 9800) loss: 1.429325
(Iteration 1951 / 9800) loss: 1.330766
(Epoch 4 / 20) train acc: 0.507000; val_acc: 0.498000
(Iteration 1961 / 9800) loss: 1.503149
(Iteration 1971 / 9800) loss: 1.259342
(Iteration 1981 / 9800) loss: 1.562462
(Iteration 1991 / 9800) loss: 1.492844
(Iteration 2001 / 9800) loss: 1.460913
(Iteration 2011 / 9800) loss: 1.469269
(Iteration 2021 / 9800) loss: 1.300600
(Iteration 2031 / 9800) loss: 1.281202
(Iteration 2041 / 9800) loss: 1.424001
(Iteration 2051 / 9800) loss: 1.527232
(Iteration 2061 / 9800) loss: 1.486019
(Iteration 2071 / 9800) loss: 1.310346
(Iteration 2081 / 9800) loss: 1.277007
(Iteration 2091 / 9800) loss: 1.279429
(Iteration 2101 / 9800) loss: 1.468747
(Iteration 2111 / 9800) loss: 1.513064
(Iteration 2121 / 9800) loss: 1.585476
(Iteration 2131 / 9800) loss: 1.525221
(Iteration 2141 / 9800) loss: 1.525908
(Iteration 2151 / 9800) loss: 1.417231
(Iteration 2161 / 9800) loss: 1.463211
(Iteration 2171 / 9800) loss: 1.299343
(Iteration 2181 / 9800) loss: 1.456655
(Iteration 2191 / 9800) loss: 1.716555
(Iteration 2201 / 9800) loss: 1.443054
(Iteration 2211 / 9800) loss: 1.619861
(Iteration 2221 / 9800) loss: 1.368013
(Iteration 2231 / 9800) loss: 1.326448
(Iteration 2241 / 9800) loss: 1.268665
(Iteration 2251 / 9800) loss: 1.747678
(Iteration 2261 / 9800) loss: 1.382125
(Iteration 2271 / 9800) loss: 1.506796
(Iteration 2281 / 9800) loss: 1.416926
(Iteration 2291 / 9800) loss: 1.319243
(Iteration 2301 / 9800) loss: 1.645025
(Iteration 2311 / 9800) loss: 1.551650
(Iteration 2321 / 9800) loss: 1.489245
(Iteration 2331 / 9800) loss: 1.642953
(Iteration 2341 / 9800) loss: 1.442399
(Iteration 2351 / 9800) loss: 1.333602
```

```
(Iteration 2361 / 9800) loss: 1.399100
(Iteration 2371 / 9800) loss: 1.174379
(Iteration 2381 / 9800) loss: 1.399222
(Iteration 2391 / 9800) loss: 1.289118
(Iteration 2401 / 9800) loss: 1.308398
(Iteration 2411 / 9800) loss: 1.484090
(Iteration 2421 / 9800) loss: 1.386564
(Iteration 2431 / 9800) loss: 1.250068
(Iteration 2441 / 9800) loss: 1.502901
(Epoch 5 / 20) train acc: 0.562000; val_acc: 0.528000
(Iteration 2451 / 9800) loss: 1.301410
(Iteration 2461 / 9800) loss: 1.692146
(Iteration 2471 / 9800) loss: 1.207155
(Iteration 2481 / 9800) loss: 1.355626
(Iteration 2491 / 9800) loss: 1.411398
(Iteration 2501 / 9800) loss: 1.439224
(Iteration 2511 / 9800) loss: 1.411360
(Iteration 2521 / 9800) loss: 1.502402
(Iteration 2531 / 9800) loss: 1.426052
(Iteration 2541 / 9800) loss: 1.432543
(Iteration 2551 / 9800) loss: 1.369556
(Iteration 2561 / 9800) loss: 1.437582
(Iteration 2571 / 9800) loss: 1.426483
(Iteration 2581 / 9800) loss: 1.358352
(Iteration 2591 / 9800) loss: 1.569054
(Iteration 2601 / 9800) loss: 1.504758
(Iteration 2611 / 9800) loss: 1.494876
(Iteration 2621 / 9800) loss: 1.167946
(Iteration 2631 / 9800) loss: 1.558125
(Iteration 2641 / 9800) loss: 1.376545
(Iteration 2651 / 9800) loss: 1.348897
(Iteration 2661 / 9800) loss: 1.455483
(Iteration 2671 / 9800) loss: 1.514574
(Iteration 2681 / 9800) loss: 1.245829
(Iteration 2691 / 9800) loss: 1.331229
(Iteration 2701 / 9800) loss: 1.390793
(Iteration 2711 / 9800) loss: 1.257064
(Iteration 2721 / 9800) loss: 1.403236
(Iteration 2731 / 9800) loss: 1.375577
(Iteration 2741 / 9800) loss: 1.495735
(Iteration 2751 / 9800) loss: 1.297767
(Iteration 2761 / 9800) loss: 1.209932
(Iteration 2771 / 9800) loss: 1.241129
(Iteration 2781 / 9800) loss: 1.457316
(Iteration 2791 / 9800) loss: 1.238764
(Iteration 2801 / 9800) loss: 1.318493
(Iteration 2811 / 9800) loss: 1.303967
(Iteration 2821 / 9800) loss: 1.374259
```

```
(Iteration 2831 / 9800) loss: 1.469227
(Iteration 2841 / 9800) loss: 1.390872
(Iteration 2851 / 9800) loss: 1.298660
(Iteration 2861 / 9800) loss: 1.222349
(Iteration 2871 / 9800) loss: 1.367556
(Iteration 2881 / 9800) loss: 1.384674
(Iteration 2891 / 9800) loss: 1.206550
(Iteration 2901 / 9800) loss: 1.277757
(Iteration 2911 / 9800) loss: 1.345925
(Iteration 2921 / 9800) loss: 1.433864
(Iteration 2931 / 9800) loss: 1.182994
(Epoch 6 / 20) train acc: 0.557000; val_acc: 0.511000
(Iteration 2941 / 9800) loss: 1.365841
(Iteration 2951 / 9800) loss: 1.304377
(Iteration 2961 / 9800) loss: 1.474848
(Iteration 2971 / 9800) loss: 1.687419
(Iteration 2981 / 9800) loss: 1.347122
(Iteration 2991 / 9800) loss: 1.517643
(Iteration 3001 / 9800) loss: 1.293378
(Iteration 3011 / 9800) loss: 1.336643
(Iteration 3021 / 9800) loss: 1.160252
(Iteration 3031 / 9800) loss: 1.297200
(Iteration 3041 / 9800) loss: 1.320944
(Iteration 3051 / 9800) loss: 1.349706
(Iteration 3061 / 9800) loss: 1.256265
(Iteration 3071 / 9800) loss: 1.438569
(Iteration 3081 / 9800) loss: 1.222834
(Iteration 3091 / 9800) loss: 1.353613
(Iteration 3101 / 9800) loss: 1.223594
(Iteration 3111 / 9800) loss: 1.456579
(Iteration 3121 / 9800) loss: 1.388252
(Iteration 3131 / 9800) loss: 1.176734
(Iteration 3141 / 9800) loss: 1.223698
(Iteration 3151 / 9800) loss: 1.172479
(Iteration 3161 / 9800) loss: 1.414230
(Iteration 3171 / 9800) loss: 1.389265
(Iteration 3181 / 9800) loss: 1.507062
(Iteration 3191 / 9800) loss: 1.430593
(Iteration 3201 / 9800) loss: 1.253397
(Iteration 3211 / 9800) loss: 1.204758
(Iteration 3221 / 9800) loss: 1.274258
(Iteration 3231 / 9800) loss: 1.195993
(Iteration 3241 / 9800) loss: 1.416429
(Iteration 3251 / 9800) loss: 1.242283
(Iteration 3261 / 9800) loss: 1.250077
(Iteration 3271 / 9800) loss: 1.528891
(Iteration 3281 / 9800) loss: 1.445830
(Iteration 3291 / 9800) loss: 1.295075
```

```
(Iteration 3301 / 9800) loss: 1.539178
(Iteration 3311 / 9800) loss: 1.374182
(Iteration 3321 / 9800) loss: 1.279364
(Iteration 3331 / 9800) loss: 1.376821
(Iteration 3341 / 9800) loss: 1.238862
(Iteration 3351 / 9800) loss: 1.528530
(Iteration 3361 / 9800) loss: 1.297499
(Iteration 3371 / 9800) loss: 1.253035
(Iteration 3381 / 9800) loss: 1.414407
(Iteration 3391 / 9800) loss: 1.555889
(Iteration 3401 / 9800) loss: 1.313077
(Iteration 3411 / 9800) loss: 1.301912
(Iteration 3421 / 9800) loss: 1.508568
(Epoch 7 / 20) train acc: 0.580000; val_acc: 0.520000
(Iteration 3431 / 9800) loss: 1.258798
(Iteration 3441 / 9800) loss: 1.459909
(Iteration 3451 / 9800) loss: 1.258706
(Iteration 3461 / 9800) loss: 1.251932
(Iteration 3471 / 9800) loss: 1.582967
(Iteration 3481 / 9800) loss: 1.544615
(Iteration 3491 / 9800) loss: 1.344656
(Iteration 3501 / 9800) loss: 1.463495
(Iteration 3511 / 9800) loss: 1.388551
(Iteration 3521 / 9800) loss: 1.120668
(Iteration 3531 / 9800) loss: 1.134796
(Iteration 3541 / 9800) loss: 1.223054
(Iteration 3551 / 9800) loss: 1.192590
(Iteration 3561 / 9800) loss: 1.102979
(Iteration 3571 / 9800) loss: 1.365972
(Iteration 3581 / 9800) loss: 1.403456
(Iteration 3591 / 9800) loss: 1.320904
(Iteration 3601 / 9800) loss: 1.374100
(Iteration 3611 / 9800) loss: 1.223217
(Iteration 3621 / 9800) loss: 1.291492
(Iteration 3631 / 9800) loss: 1.211227
(Iteration 3641 / 9800) loss: 1.208334
(Iteration 3651 / 9800) loss: 1.446636
(Iteration 3661 / 9800) loss: 1.392414
(Iteration 3671 / 9800) loss: 1.515472
(Iteration 3681 / 9800) loss: 1.387363
(Iteration 3691 / 9800) loss: 1.222121
(Iteration 3701 / 9800) loss: 1.254646
(Iteration 3711 / 9800) loss: 1.736311
(Iteration 3721 / 9800) loss: 1.261120
(Iteration 3731 / 9800) loss: 1.461295
(Iteration 3741 / 9800) loss: 1.367199
(Iteration 3751 / 9800) loss: 1.108566
(Iteration 3761 / 9800) loss: 1.240168
```

```
(Iteration 3771 / 9800) loss: 1.338669
(Iteration 3781 / 9800) loss: 1.363473
(Iteration 3791 / 9800) loss: 1.247347
(Iteration 3801 / 9800) loss: 1.318170
(Iteration 3811 / 9800) loss: 1.224477
(Iteration 3821 / 9800) loss: 1.223916
(Iteration 3831 / 9800) loss: 1.294747
(Iteration 3841 / 9800) loss: 1.378359
(Iteration 3851 / 9800) loss: 1.188452
(Iteration 3861 / 9800) loss: 1.405447
(Iteration 3871 / 9800) loss: 1.184832
(Iteration 3881 / 9800) loss: 1.441762
(Iteration 3891 / 9800) loss: 1.314802
(Iteration 3901 / 9800) loss: 1.260528
(Iteration 3911 / 9800) loss: 1.260390
(Epoch 8 / 20) train acc: 0.592000; val_acc: 0.497000
(Iteration 3921 / 9800) loss: 1.230300
(Iteration 3931 / 9800) loss: 1.129231
(Iteration 3941 / 9800) loss: 1.141837
(Iteration 3951 / 9800) loss: 1.325371
(Iteration 3961 / 9800) loss: 1.180868
(Iteration 3971 / 9800) loss: 1.226544
(Iteration 3981 / 9800) loss: 1.216644
(Iteration 3991 / 9800) loss: 1.102473
(Iteration 4001 / 9800) loss: 1.034904
(Iteration 4011 / 9800) loss: 1.381897
(Iteration 4021 / 9800) loss: 1.206099
(Iteration 4031 / 9800) loss: 1.345799
(Iteration 4041 / 9800) loss: 1.163720
(Iteration 4051 / 9800) loss: 1.422203
(Iteration 4061 / 9800) loss: 1.356379
(Iteration 4071 / 9800) loss: 1.102323
(Iteration 4081 / 9800) loss: 1.265898
(Iteration 4091 / 9800) loss: 1.258728
(Iteration 4101 / 9800) loss: 1.390054
(Iteration 4111 / 9800) loss: 1.375897
(Iteration 4121 / 9800) loss: 1.397197
(Iteration 4131 / 9800) loss: 1.279508
(Iteration 4141 / 9800) loss: 1.265588
(Iteration 4151 / 9800) loss: 1.384785
(Iteration 4161 / 9800) loss: 1.576728
(Iteration 4171 / 9800) loss: 1.204694
(Iteration 4181 / 9800) loss: 1.384555
(Iteration 4191 / 9800) loss: 1.443199
(Iteration 4201 / 9800) loss: 1.399029
(Iteration 4211 / 9800) loss: 1.161911
(Iteration 4221 / 9800) loss: 1.302234
(Iteration 4231 / 9800) loss: 1.285393
```

```
(Iteration 4241 / 9800) loss: 1.274063
(Iteration 4251 / 9800) loss: 1.199188
(Iteration 4261 / 9800) loss: 1.300292
(Iteration 4271 / 9800) loss: 1.503780
(Iteration 4281 / 9800) loss: 1.468962
(Iteration 4291 / 9800) loss: 1.549138
(Iteration 4301 / 9800) loss: 1.484911
(Iteration 4311 / 9800) loss: 1.403796
(Iteration 4321 / 9800) loss: 1.323313
(Iteration 4331 / 9800) loss: 1.407516
(Iteration 4341 / 9800) loss: 1.337037
(Iteration 4351 / 9800) loss: 1.244685
(Iteration 4361 / 9800) loss: 1.274263
(Iteration 4371 / 9800) loss: 1.553304
(Iteration 4381 / 9800) loss: 1.443278
(Iteration 4391 / 9800) loss: 1.194985
(Iteration 4401 / 9800) loss: 1.335232
(Epoch 9 / 20) train acc: 0.610000; val_acc: 0.515000
(Iteration 4411 / 9800) loss: 1.378326
(Iteration 4421 / 9800) loss: 1.265391
(Iteration 4431 / 9800) loss: 1.408555
(Iteration 4441 / 9800) loss: 1.193972
(Iteration 4451 / 9800) loss: 1.246045
(Iteration 4461 / 9800) loss: 1.291980
(Iteration 4471 / 9800) loss: 1.283779
(Iteration 4481 / 9800) loss: 1.270805
(Iteration 4491 / 9800) loss: 1.240090
(Iteration 4501 / 9800) loss: 1.213376
(Iteration 4511 / 9800) loss: 1.185238
(Iteration 4521 / 9800) loss: 1.401146
(Iteration 4531 / 9800) loss: 1.434774
(Iteration 4541 / 9800) loss: 1.249036
(Iteration 4551 / 9800) loss: 1.287938
(Iteration 4561 / 9800) loss: 1.185439
(Iteration 4571 / 9800) loss: 1.288142
(Iteration 4581 / 9800) loss: 1.103039
(Iteration 4591 / 9800) loss: 1.234937
(Iteration 4601 / 9800) loss: 1.115666
(Iteration 4611 / 9800) loss: 1.218666
(Iteration 4621 / 9800) loss: 1.349821
(Iteration 4631 / 9800) loss: 1.208490
(Iteration 4641 / 9800) loss: 1.438303
(Iteration 4651 / 9800) loss: 1.175584
(Iteration 4661 / 9800) loss: 1.176928
(Iteration 4671 / 9800) loss: 1.171600
(Iteration 4681 / 9800) loss: 1.207728
(Iteration 4691 / 9800) loss: 1.144225
(Iteration 4701 / 9800) loss: 1.131599
```

```
(Iteration 4711 / 9800) loss: 1.285385
(Iteration 4721 / 9800) loss: 1.145964
(Iteration 4731 / 9800) loss: 1.360044
(Iteration 4741 / 9800) loss: 1.376669
(Iteration 4751 / 9800) loss: 1.345755
(Iteration 4761 / 9800) loss: 1.326740
(Iteration 4771 / 9800) loss: 1.396667
(Iteration 4781 / 9800) loss: 1.242840
(Iteration 4791 / 9800) loss: 1.177658
(Iteration 4801 / 9800) loss: 1.340837
(Iteration 4811 / 9800) loss: 1.194459
(Iteration 4821 / 9800) loss: 1.237242
(Iteration 4831 / 9800) loss: 1.281130
(Iteration 4841 / 9800) loss: 1.485118
(Iteration 4851 / 9800) loss: 1.347148
(Iteration 4861 / 9800) loss: 1.226851
(Iteration 4871 / 9800) loss: 1.166049
(Iteration 4881 / 9800) loss: 1.273035
(Iteration 4891 / 9800) loss: 1.236115
(Epoch 10 / 20) train acc: 0.562000; val_acc: 0.509000
(Iteration 4901 / 9800) loss: 1.407576
(Iteration 4911 / 9800) loss: 1.407698
(Iteration 4921 / 9800) loss: 1.176365
(Iteration 4931 / 9800) loss: 1.176675
(Iteration 4941 / 9800) loss: 1.285817
(Iteration 4951 / 9800) loss: 1.356195
(Iteration 4961 / 9800) loss: 1.192102
(Iteration 4971 / 9800) loss: 1.116043
(Iteration 4981 / 9800) loss: 1.124977
(Iteration 4991 / 9800) loss: 1.294937
(Iteration 5001 / 9800) loss: 1.539668
(Iteration 5011 / 9800) loss: 1.285348
(Iteration 5021 / 9800) loss: 1.177239
(Iteration 5031 / 9800) loss: 1.128726
(Iteration 5041 / 9800) loss: 1.176789
(Iteration 5051 / 9800) loss: 1.524356
(Iteration 5061 / 9800) loss: 1.403818
(Iteration 5071 / 9800) loss: 1.421440
(Iteration 5081 / 9800) loss: 1.198682
(Iteration 5091 / 9800) loss: 1.194080
(Iteration 5101 / 9800) loss: 1.318425
(Iteration 5111 / 9800) loss: 1.268803
(Iteration 5121 / 9800) loss: 1.229151
(Iteration 5131 / 9800) loss: 1.366567
(Iteration 5141 / 9800) loss: 1.204591
(Iteration 5151 / 9800) loss: 1.110843
(Iteration 5161 / 9800) loss: 1.216653
(Iteration 5171 / 9800) loss: 1.303751
```

```
(Iteration 5181 / 9800) loss: 1.289128
(Iteration 5191 / 9800) loss: 1.427886
(Iteration 5201 / 9800) loss: 1.497762
(Iteration 5211 / 9800) loss: 1.201370
(Iteration 5221 / 9800) loss: 1.378180
(Iteration 5231 / 9800) loss: 1.190975
(Iteration 5241 / 9800) loss: 1.318422
(Iteration 5251 / 9800) loss: 1.277822
(Iteration 5261 / 9800) loss: 1.463612
(Iteration 5271 / 9800) loss: 1.406401
(Iteration 5281 / 9800) loss: 1.120225
(Iteration 5291 / 9800) loss: 1.363194
(Iteration 5301 / 9800) loss: 1.197549
(Iteration 5311 / 9800) loss: 1.211421
(Iteration 5321 / 9800) loss: 1.186769
(Iteration 5331 / 9800) loss: 1.187415
(Iteration 5341 / 9800) loss: 1.199579
(Iteration 5351 / 9800) loss: 1.460901
(Iteration 5361 / 9800) loss: 1.115202
(Iteration 5371 / 9800) loss: 1.274977
(Iteration 5381 / 9800) loss: 1.060320
(Epoch 11 / 20) train acc: 0.633000; val_acc: 0.538000
(Iteration 5391 / 9800) loss: 1.222672
(Iteration 5401 / 9800) loss: 1.545770
(Iteration 5411 / 9800) loss: 1.184328
(Iteration 5421 / 9800) loss: 1.429567
(Iteration 5431 / 9800) loss: 1.141682
(Iteration 5441 / 9800) loss: 1.273308
(Iteration 5451 / 9800) loss: 1.318672
(Iteration 5461 / 9800) loss: 1.411025
(Iteration 5471 / 9800) loss: 1.333437
(Iteration 5481 / 9800) loss: 1.185950
(Iteration 5491 / 9800) loss: 1.355353
(Iteration 5501 / 9800) loss: 1.260125
(Iteration 5511 / 9800) loss: 1.297774
(Iteration 5521 / 9800) loss: 1.343165
(Iteration 5531 / 9800) loss: 1.163584
(Iteration 5541 / 9800) loss: 1.250547
(Iteration 5551 / 9800) loss: 1.254990
(Iteration 5561 / 9800) loss: 1.257447
(Iteration 5571 / 9800) loss: 1.246290
(Iteration 5581 / 9800) loss: 1.344104
(Iteration 5591 / 9800) loss: 1.056394
(Iteration 5601 / 9800) loss: 1.241318
(Iteration 5611 / 9800) loss: 1.411567
(Iteration 5621 / 9800) loss: 1.225440
(Iteration 5631 / 9800) loss: 1.144607
(Iteration 5641 / 9800) loss: 1.494638
```

```
(Iteration 5651 / 9800) loss: 1.287791
(Iteration 5661 / 9800) loss: 1.188887
(Iteration 5671 / 9800) loss: 1.307539
(Iteration 5681 / 9800) loss: 1.166322
(Iteration 5691 / 9800) loss: 1.319469
(Iteration 5701 / 9800) loss: 1.189842
(Iteration 5711 / 9800) loss: 1.207013
(Iteration 5721 / 9800) loss: 1.192019
(Iteration 5731 / 9800) loss: 1.224946
(Iteration 5741 / 9800) loss: 1.432430
(Iteration 5751 / 9800) loss: 1.424123
(Iteration 5761 / 9800) loss: 1.327317
(Iteration 5771 / 9800) loss: 1.206796
(Iteration 5781 / 9800) loss: 1.139765
(Iteration 5791 / 9800) loss: 1.063866
(Iteration 5801 / 9800) loss: 1.244379
(Iteration 5811 / 9800) loss: 1.146665
(Iteration 5821 / 9800) loss: 1.312619
(Iteration 5831 / 9800) loss: 1.123106
(Iteration 5841 / 9800) loss: 1.306000
(Iteration 5851 / 9800) loss: 1.211413
(Iteration 5861 / 9800) loss: 1.438921
(Iteration 5871 / 9800) loss: 1.311159
(Epoch 12 / 20) train acc: 0.585000; val_acc: 0.500000
(Iteration 5881 / 9800) loss: 1.265194
(Iteration 5891 / 9800) loss: 1.275013
(Iteration 5901 / 9800) loss: 1.265881
(Iteration 5911 / 9800) loss: 1.171567
(Iteration 5921 / 9800) loss: 1.133880
(Iteration 5931 / 9800) loss: 1.144572
(Iteration 5941 / 9800) loss: 1.293962
(Iteration 5951 / 9800) loss: 1.225642
(Iteration 5961 / 9800) loss: 1.183302
(Iteration 5971 / 9800) loss: 1.394319
(Iteration 5981 / 9800) loss: 1.272669
(Iteration 5991 / 9800) loss: 1.254003
(Iteration 6001 / 9800) loss: 1.242664
(Iteration 6011 / 9800) loss: 1.260992
(Iteration 6021 / 9800) loss: 1.313357
(Iteration 6031 / 9800) loss: 1.369458
(Iteration 6041 / 9800) loss: 1.181795
(Iteration 6051 / 9800) loss: 1.351834
(Iteration 6061 / 9800) loss: 1.427290
(Iteration 6071 / 9800) loss: 1.068899
(Iteration 6081 / 9800) loss: 1.152099
(Iteration 6091 / 9800) loss: 1.210448
(Iteration 6101 / 9800) loss: 1.329346
(Iteration 6111 / 9800) loss: 1.166604
```

```
(Iteration 6121 / 9800) loss: 1.266873
(Iteration 6131 / 9800) loss: 1.262746
(Iteration 6141 / 9800) loss: 1.423713
(Iteration 6151 / 9800) loss: 1.154822
(Iteration 6161 / 9800) loss: 1.261708
(Iteration 6171 / 9800) loss: 1.297196
(Iteration 6181 / 9800) loss: 1.398910
(Iteration 6191 / 9800) loss: 1.194779
(Iteration 6201 / 9800) loss: 1.241180
(Iteration 6211 / 9800) loss: 1.125755
(Iteration 6221 / 9800) loss: 1.158061
(Iteration 6231 / 9800) loss: 1.337126
(Iteration 6241 / 9800) loss: 1.200312
(Iteration 6251 / 9800) loss: 1.251770
(Iteration 6261 / 9800) loss: 1.278435
(Iteration 6271 / 9800) loss: 1.148029
(Iteration 6281 / 9800) loss: 1.413345
(Iteration 6291 / 9800) loss: 1.465445
(Iteration 6301 / 9800) loss: 1.153789
(Iteration 6311 / 9800) loss: 1.275153
(Iteration 6321 / 9800) loss: 1.085435
(Iteration 6331 / 9800) loss: 1.114114
(Iteration 6341 / 9800) loss: 1.263227
(Iteration 6351 / 9800) loss: 1.024115
(Iteration 6361 / 9800) loss: 1.138434
(Epoch 13 / 20) train acc: 0.640000; val_acc: 0.539000
(Iteration 6371 / 9800) loss: 1.195087
(Iteration 6381 / 9800) loss: 1.418665
(Iteration 6391 / 9800) loss: 1.188656
(Iteration 6401 / 9800) loss: 1.207138
(Iteration 6411 / 9800) loss: 1.251701
(Iteration 6421 / 9800) loss: 1.148769
(Iteration 6431 / 9800) loss: 1.368463
(Iteration 6441 / 9800) loss: 1.226262
(Iteration 6451 / 9800) loss: 1.272143
(Iteration 6461 / 9800) loss: 1.184271
(Iteration 6471 / 9800) loss: 1.222969
(Iteration 6481 / 9800) loss: 1.422734
(Iteration 6491 / 9800) loss: 1.239674
(Iteration 6501 / 9800) loss: 1.115930
(Iteration 6511 / 9800) loss: 1.128530
(Iteration 6521 / 9800) loss: 1.122608
(Iteration 6531 / 9800) loss: 1.189609
(Iteration 6541 / 9800) loss: 1.554706
(Iteration 6551 / 9800) loss: 1.134382
(Iteration 6561 / 9800) loss: 1.042391
(Iteration 6571 / 9800) loss: 1.245612
(Iteration 6581 / 9800) loss: 1.077687
```

```
(Iteration 6591 / 9800) loss: 1.188439
(Iteration 6601 / 9800) loss: 1.079197
(Iteration 6611 / 9800) loss: 1.444483
(Iteration 6621 / 9800) loss: 1.104979
(Iteration 6631 / 9800) loss: 1.389176
(Iteration 6641 / 9800) loss: 1.234071
(Iteration 6651 / 9800) loss: 1.199464
(Iteration 6661 / 9800) loss: 1.277398
(Iteration 6671 / 9800) loss: 1.296373
(Iteration 6681 / 9800) loss: 1.127519
(Iteration 6691 / 9800) loss: 1.213548
(Iteration 6701 / 9800) loss: 1.390491
(Iteration 6711 / 9800) loss: 1.255831
(Iteration 6721 / 9800) loss: 1.332575
(Iteration 6731 / 9800) loss: 1.159131
(Iteration 6741 / 9800) loss: 1.290622
(Iteration 6751 / 9800) loss: 1.112251
(Iteration 6761 / 9800) loss: 1.300143
(Iteration 6771 / 9800) loss: 0.952854
(Iteration 6781 / 9800) loss: 1.188395
(Iteration 6791 / 9800) loss: 1.229393
(Iteration 6801 / 9800) loss: 1.162394
(Iteration 6811 / 9800) loss: 1.145859
(Iteration 6821 / 9800) loss: 1.257312
(Iteration 6831 / 9800) loss: 1.156524
(Iteration 6841 / 9800) loss: 1.256304
(Iteration 6851 / 9800) loss: 1.392301
(Epoch 14 / 20) train acc: 0.653000; val_acc: 0.514000
(Iteration 6861 / 9800) loss: 1.222273
(Iteration 6871 / 9800) loss: 1.351212
(Iteration 6881 / 9800) loss: 1.126152
(Iteration 6891 / 9800) loss: 1.200792
(Iteration 6901 / 9800) loss: 1.316109
(Iteration 6911 / 9800) loss: 1.177804
(Iteration 6921 / 9800) loss: 1.272075
(Iteration 6931 / 9800) loss: 1.262562
(Iteration 6941 / 9800) loss: 1.430995
(Iteration 6951 / 9800) loss: 1.161631
(Iteration 6961 / 9800) loss: 1.385471
(Iteration 6971 / 9800) loss: 1.138991
(Iteration 6981 / 9800) loss: 1.246087
(Iteration 6991 / 9800) loss: 1.436773
(Iteration 7001 / 9800) loss: 1.273329
(Iteration 7011 / 9800) loss: 1.245464
(Iteration 7021 / 9800) loss: 1.227117
(Iteration 7031 / 9800) loss: 1.159664
(Iteration 7041 / 9800) loss: 1.254646
(Iteration 7051 / 9800) loss: 1.137693
```

```
(Iteration 7061 / 9800) loss: 1.177331
(Iteration 7071 / 9800) loss: 1.145725
(Iteration 7081 / 9800) loss: 1.230228
(Iteration 7091 / 9800) loss: 1.126865
(Iteration 7101 / 9800) loss: 1.276002
(Iteration 7111 / 9800) loss: 1.319403
(Iteration 7121 / 9800) loss: 1.273220
(Iteration 7131 / 9800) loss: 1.157426
(Iteration 7141 / 9800) loss: 1.309697
(Iteration 7151 / 9800) loss: 1.125387
(Iteration 7161 / 9800) loss: 1.153469
(Iteration 7171 / 9800) loss: 1.055907
(Iteration 7181 / 9800) loss: 1.247964
(Iteration 7191 / 9800) loss: 1.282492
(Iteration 7201 / 9800) loss: 1.088692
(Iteration 7211 / 9800) loss: 1.226127
(Iteration 7221 / 9800) loss: 1.129239
(Iteration 7231 / 9800) loss: 1.257379
(Iteration 7241 / 9800) loss: 1.354428
(Iteration 7251 / 9800) loss: 1.152808
(Iteration 7261 / 9800) loss: 1.289572
(Iteration 7271 / 9800) loss: 1.316132
(Iteration 7281 / 9800) loss: 1.473022
(Iteration 7291 / 9800) loss: 1.206527
(Iteration 7301 / 9800) loss: 1.216889
(Iteration 7311 / 9800) loss: 1.205672
(Iteration 7321 / 9800) loss: 1.318447
(Iteration 7331 / 9800) loss: 1.413096
(Iteration 7341 / 9800) loss: 1.076017
(Epoch 15 / 20) train acc: 0.665000; val_acc: 0.503000
(Iteration 7351 / 9800) loss: 1.272356
(Iteration 7361 / 9800) loss: 1.223145
(Iteration 7371 / 9800) loss: 1.469479
(Iteration 7381 / 9800) loss: 1.233555
(Iteration 7391 / 9800) loss: 1.344279
(Iteration 7401 / 9800) loss: 1.141000
(Iteration 7411 / 9800) loss: 1.120681
(Iteration 7421 / 9800) loss: 1.128023
(Iteration 7431 / 9800) loss: 1.238095
(Iteration 7441 / 9800) loss: 1.031211
(Iteration 7451 / 9800) loss: 1.201477
(Iteration 7461 / 9800) loss: 1.417961
(Iteration 7471 / 9800) loss: 1.212324
(Iteration 7481 / 9800) loss: 1.242116
(Iteration 7491 / 9800) loss: 1.146325
(Iteration 7501 / 9800) loss: 1.359556
(Iteration 7511 / 9800) loss: 1.049157
(Iteration 7521 / 9800) loss: 1.170401
```

```
(Iteration 7531 / 9800) loss: 1.093452
(Iteration 7541 / 9800) loss: 1.195099
(Iteration 7551 / 9800) loss: 1.187026
(Iteration 7561 / 9800) loss: 1.141979
(Iteration 7571 / 9800) loss: 1.253666
(Iteration 7581 / 9800) loss: 1.222613
(Iteration 7591 / 9800) loss: 1.213884
(Iteration 7601 / 9800) loss: 1.382617
(Iteration 7611 / 9800) loss: 1.148625
(Iteration 7621 / 9800) loss: 1.209990
(Iteration 7631 / 9800) loss: 1.211379
(Iteration 7641 / 9800) loss: 1.234188
(Iteration 7651 / 9800) loss: 1.047561
(Iteration 7661 / 9800) loss: 1.206367
(Iteration 7671 / 9800) loss: 1.378305
(Iteration 7681 / 9800) loss: 1.094704
(Iteration 7691 / 9800) loss: 1.004561
(Iteration 7701 / 9800) loss: 1.259957
(Iteration 7711 / 9800) loss: 1.360608
(Iteration 7721 / 9800) loss: 1.187527
(Iteration 7731 / 9800) loss: 1.091185
(Iteration 7741 / 9800) loss: 1.181170
(Iteration 7751 / 9800) loss: 1.256770
(Iteration 7761 / 9800) loss: 1.108869
(Iteration 7771 / 9800) loss: 1.095670
(Iteration 7781 / 9800) loss: 1.265667
(Iteration 7791 / 9800) loss: 1.134471
(Iteration 7801 / 9800) loss: 1.206623
(Iteration 7811 / 9800) loss: 1.120189
(Iteration 7821 / 9800) loss: 1.350594
(Iteration 7831 / 9800) loss: 1.101580
(Epoch 16 / 20) train acc: 0.690000; val_acc: 0.546000
(Iteration 7841 / 9800) loss: 1.135915
(Iteration 7851 / 9800) loss: 1.211402
(Iteration 7861 / 9800) loss: 1.117332
(Iteration 7871 / 9800) loss: 1.327648
(Iteration 7881 / 9800) loss: 1.170860
(Iteration 7891 / 9800) loss: 1.183962
(Iteration 7901 / 9800) loss: 1.432588
(Iteration 7911 / 9800) loss: 1.424398
(Iteration 7921 / 9800) loss: 1.293478
(Iteration 7931 / 9800) loss: 1.455146
(Iteration 7941 / 9800) loss: 1.189758
(Iteration 7951 / 9800) loss: 1.199818
(Iteration 7961 / 9800) loss: 1.114537
(Iteration 7971 / 9800) loss: 1.296528
(Iteration 7981 / 9800) loss: 1.127563
(Iteration 7991 / 9800) loss: 1.408052
```

```
(Iteration 8001 / 9800) loss: 1.308278
(Iteration 8011 / 9800) loss: 1.292434
(Iteration 8021 / 9800) loss: 1.039092
(Iteration 8031 / 9800) loss: 1.247329
(Iteration 8041 / 9800) loss: 1.127224
(Iteration 8051 / 9800) loss: 1.211990
(Iteration 8061 / 9800) loss: 1.055531
(Iteration 8071 / 9800) loss: 1.253240
(Iteration 8081 / 9800) loss: 1.038737
(Iteration 8091 / 9800) loss: 1.365326
(Iteration 8101 / 9800) loss: 1.240879
(Iteration 8111 / 9800) loss: 1.202831
(Iteration 8121 / 9800) loss: 1.228335
(Iteration 8131 / 9800) loss: 1.196505
(Iteration 8141 / 9800) loss: 1.227734
(Iteration 8151 / 9800) loss: 1.122600
(Iteration 8161 / 9800) loss: 1.110062
(Iteration 8171 / 9800) loss: 1.211381
(Iteration 8181 / 9800) loss: 0.975720
(Iteration 8191 / 9800) loss: 1.113583
(Iteration 8201 / 9800) loss: 1.002887
(Iteration 8211 / 9800) loss: 1.044141
(Iteration 8221 / 9800) loss: 1.169417
(Iteration 8231 / 9800) loss: 1.334395
(Iteration 8241 / 9800) loss: 1.195073
(Iteration 8251 / 9800) loss: 1.412497
(Iteration 8261 / 9800) loss: 1.328913
(Iteration 8271 / 9800) loss: 1.141798
(Iteration 8281 / 9800) loss: 1.140082
(Iteration 8291 / 9800) loss: 1.132996
(Iteration 8301 / 9800) loss: 1.299977
(Iteration 8311 / 9800) loss: 1.208352
(Iteration 8321 / 9800) loss: 1.283188
(Epoch 17 / 20) train acc: 0.636000; val_acc: 0.513000
(Iteration 8331 / 9800) loss: 0.920611
(Iteration 8341 / 9800) loss: 1.220976
(Iteration 8351 / 9800) loss: 1.204258
(Iteration 8361 / 9800) loss: 1.226082
(Iteration 8371 / 9800) loss: 1.356979
(Iteration 8381 / 9800) loss: 1.371787
(Iteration 8391 / 9800) loss: 1.165556
(Iteration 8401 / 9800) loss: 1.221296
(Iteration 8411 / 9800) loss: 1.449446
(Iteration 8421 / 9800) loss: 1.341414
(Iteration 8431 / 9800) loss: 1.124748
(Iteration 8441 / 9800) loss: 1.249058
(Iteration 8451 / 9800) loss: 1.203571
(Iteration 8461 / 9800) loss: 1.235257
```

```
(Iteration 8471 / 9800) loss: 1.347212
(Iteration 8481 / 9800) loss: 1.089507
(Iteration 8491 / 9800) loss: 1.298929
(Iteration 8501 / 9800) loss: 1.488343
(Iteration 8511 / 9800) loss: 1.221892
(Iteration 8521 / 9800) loss: 1.298834
(Iteration 8531 / 9800) loss: 1.066716
(Iteration 8541 / 9800) loss: 1.124937
(Iteration 8551 / 9800) loss: 1.138074
(Iteration 8561 / 9800) loss: 1.271548
(Iteration 8571 / 9800) loss: 1.429738
(Iteration 8581 / 9800) loss: 1.259808
(Iteration 8591 / 9800) loss: 1.183837
(Iteration 8601 / 9800) loss: 1.409798
(Iteration 8611 / 9800) loss: 1.160607
(Iteration 8621 / 9800) loss: 0.989859
(Iteration 8631 / 9800) loss: 1.047364
(Iteration 8641 / 9800) loss: 1.282423
(Iteration 8651 / 9800) loss: 1.319182
(Iteration 8661 / 9800) loss: 1.167713
(Iteration 8671 / 9800) loss: 1.187288
(Iteration 8681 / 9800) loss: 1.154145
(Iteration 8691 / 9800) loss: 1.008658
(Iteration 8701 / 9800) loss: 1.181503
(Iteration 8711 / 9800) loss: 1.171266
(Iteration 8721 / 9800) loss: 1.223244
(Iteration 8731 / 9800) loss: 1.113182
(Iteration 8741 / 9800) loss: 1.373320
(Iteration 8751 / 9800) loss: 1.209679
(Iteration 8761 / 9800) loss: 1.165420
(Iteration 8771 / 9800) loss: 1.136345
(Iteration 8781 / 9800) loss: 1.348163
(Iteration 8791 / 9800) loss: 1.066720
(Iteration 8801 / 9800) loss: 1.232856
(Iteration 8811 / 9800) loss: 1.247493
(Epoch 18 / 20) train acc: 0.635000; val_acc: 0.528000
(Iteration 8821 / 9800) loss: 1.186001
(Iteration 8831 / 9800) loss: 1.299744
(Iteration 8841 / 9800) loss: 1.159068
(Iteration 8851 / 9800) loss: 1.411378
(Iteration 8861 / 9800) loss: 1.365558
(Iteration 8871 / 9800) loss: 1.249594
(Iteration 8881 / 9800) loss: 1.067948
(Iteration 8891 / 9800) loss: 1.298397
(Iteration 8901 / 9800) loss: 1.100387
(Iteration 8911 / 9800) loss: 1.262060
(Iteration 8921 / 9800) loss: 1.243726
(Iteration 8931 / 9800) loss: 1.271023
```

```
(Iteration 8941 / 9800) loss: 1.089998
(Iteration 8951 / 9800) loss: 1.210775
(Iteration 8961 / 9800) loss: 1.216554
(Iteration 8971 / 9800) loss: 0.987086
(Iteration 8981 / 9800) loss: 1.342420
(Iteration 8991 / 9800) loss: 1.248789
(Iteration 9001 / 9800) loss: 1.119142
(Iteration 9011 / 9800) loss: 1.352741
(Iteration 9021 / 9800) loss: 1.232669
(Iteration 9031 / 9800) loss: 1.156438
(Iteration 9041 / 9800) loss: 1.159139
(Iteration 9051 / 9800) loss: 1.378195
(Iteration 9061 / 9800) loss: 1.264127
(Iteration 9071 / 9800) loss: 1.069758
(Iteration 9081 / 9800) loss: 1.290970
(Iteration 9091 / 9800) loss: 1.254766
(Iteration 9101 / 9800) loss: 1.223777
(Iteration 9111 / 9800) loss: 0.966419
(Iteration 9121 / 9800) loss: 1.219408
(Iteration 9131 / 9800) loss: 1.052498
(Iteration 9141 / 9800) loss: 1.235054
(Iteration 9151 / 9800) loss: 1.151434
(Iteration 9161 / 9800) loss: 1.173165
(Iteration 9171 / 9800) loss: 1.171080
(Iteration 9181 / 9800) loss: 1.296391
(Iteration 9191 / 9800) loss: 1.257547
(Iteration 9201 / 9800) loss: 1.329993
(Iteration 9211 / 9800) loss: 1.189401
(Iteration 9221 / 9800) loss: 1.086897
(Iteration 9231 / 9800) loss: 1.037103
(Iteration 9241 / 9800) loss: 1.194306
(Iteration 9251 / 9800) loss: 1.208471
(Iteration 9261 / 9800) loss: 1.102040
(Iteration 9271 / 9800) loss: 1.248478
(Iteration 9281 / 9800) loss: 1.342505
(Iteration 9291 / 9800) loss: 1.084636
(Iteration 9301 / 9800) loss: 1.166743
(Epoch 19 / 20) train acc: 0.629000; val_acc: 0.517000
(Iteration 9311 / 9800) loss: 1.164881
(Iteration 9321 / 9800) loss: 1.141232
(Iteration 9331 / 9800) loss: 1.045256
(Iteration 9341 / 9800) loss: 1.301431
(Iteration 9351 / 9800) loss: 0.896541
(Iteration 9361 / 9800) loss: 1.139687
(Iteration 9371 / 9800) loss: 1.358971
(Iteration 9381 / 9800) loss: 1.111421
(Iteration 9391 / 9800) loss: 1.152047
(Iteration 9401 / 9800) loss: 1.097160
```

```
(Iteration 9411 / 9800) loss: 1.466027
(Iteration 9421 / 9800) loss: 1.097201
(Iteration 9431 / 9800) loss: 1.402436
(Iteration 9441 / 9800) loss: 1.267901
(Iteration 9451 / 9800) loss: 1.178481
(Iteration 9461 / 9800) loss: 0.983023
(Iteration 9471 / 9800) loss: 1.313538
(Iteration 9481 / 9800) loss: 1.092788
(Iteration 9491 / 9800) loss: 1.198055
(Iteration 9501 / 9800) loss: 1.304072
(Iteration 9511 / 9800) loss: 1.216356
(Iteration 9521 / 9800) loss: 1.406329
(Iteration 9531 / 9800) loss: 1.287373
(Iteration 9541 / 9800) loss: 1.095945
(Iteration 9551 / 9800) loss: 1.112464
(Iteration 9561 / 9800) loss: 1.327322
(Iteration 9571 / 9800) loss: 1.259631
(Iteration 9581 / 9800) loss: 1.210582
(Iteration 9591 / 9800) loss: 1.306815
(Iteration 9601 / 9800) loss: 1.099885
(Iteration 9611 / 9800) loss: 0.988755
(Iteration 9621 / 9800) loss: 1.425241
(Iteration 9631 / 9800) loss: 1.170380
(Iteration 9641 / 9800) loss: 0.998875
(Iteration 9651 / 9800) loss: 1.180091
(Iteration 9661 / 9800) loss: 1.394850
(Iteration 9671 / 9800) loss: 1.263577
(Iteration 9681 / 9800) loss: 1.164673
(Iteration 9691 / 9800) loss: 1.096495
(Iteration 9701 / 9800) loss: 1.019520
(Iteration 9711 / 9800) loss: 1.320902
(Iteration 9721 / 9800) loss: 1.123054
(Iteration 9731 / 9800) loss: 1.192688
(Iteration 9741 / 9800) loss: 1.318354
(Iteration 9751 / 9800) loss: 1.169932
(Iteration 9761 / 9800) loss: 1.267529
(Iteration 9771 / 9800) loss: 1.272300
(Iteration 9781 / 9800) loss: 1.081860
(Iteration 9791 / 9800) loss: 1.270252
(Epoch 20 / 20) train acc: 0.673000; val_acc: 0.517000
accuracy:0.546 with lr:0.0005,reg:0.25,epoch:20,hidden_dim:200
```

12 Test your model!

Run your best model on the validation and test sets. You should achieve above 48% accuracy on the validation set and the test set.

[57]:
```
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
```

```
Validation set accuracy:  0.546
```

[58]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Test set accuracy:  0.531
```

[59]:
```
# Save best model
best_model.save("best_two_layer_net.npy")
```

```
best_two_layer_net.npy saved.
```

12.1 Inline Question 2:

Now that you have trained a Neural Network classifier, you may find that your testing accuracy is much lower than the training accuracy. In what ways can we decrease this gap? Select all that apply.

1. Train on a larger dataset.
2. Add more hidden units.
3. Increase the regularization strength.
4. None of the above.

$Your Answer:$ 1 and 3

$Your Explanation:$ The mistake is overfitting. 1. a larger dataset forces the model to learn the essential features of data. 2. add hidden units will make the model more complicated and in some cases, the model would highly depend on some units, the other are dead 3. regularization would reduce overfitting, which is a common sense

features

April 5, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'CS231n/assignment1'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/MyDrive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/MyDrive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/MyDrive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/MyDrive/CS231n/assignment1/cs231n/datasets
/content/drive/MyDrive/CS231n/assignment1
```

1 Image features exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission. For more details see the assignments page on the course website.

We have seen that we can achieve reasonable performance on an image classification task by training a linear classifier on the pixels of the input image. In this exercise we will show that we can improve our classification performance by training linear classifiers not on raw pixels but on features that are computed from the raw pixels.

All of your work for this exercise will be done in this notebook.

[2]:
```
import random
import numpy as np
from cs231n.data_utils import load_CIFAR10
import matplotlib.pyplot as plt
import sys
import importlib
sys.modules['imp'] = importlib

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading extenrnal modules
# see http://stackoverflow.com/questions/1907993/
 ↪autoreload-of-modules-in-ipython
%load_ext autoreload
%autoreload 2
```

1.1 Load data

Similar to previous exercises, we will load CIFAR-10 data from disk.

[25]:
```
from cs231n.features import color_histogram_hsv, hog_feature

def get_CIFAR10_data(num_training=49000, num_validation=1000, num_test=1000):
    # Load the raw CIFAR-10 data
    cifar10_dir = 'cs231n/datasets/cifar-10-batches-py'

    # Cleaning up variables to prevent loading data multiple times (which may␣
 ↪cause memory issue)
    try:
       del X_train, y_train
       del X_test, y_test
       print('Clear previously loaded data.')
    except:
       pass

    X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

    # Subsample the data
    mask = list(range(num_training, num_training + num_validation))
    X_val = X_train[mask]
    y_val = y_train[mask]
    mask = list(range(num_training))
    X_train = X_train[mask]
    y_train = y_train[mask]
```

```
    mask = list(range(num_test))
    X_test = X_test[mask]
    y_test = y_test[mask]

    return X_train, y_train, X_val, y_val, X_test, y_test

X_train, y_train, X_val, y_val, X_test, y_test = get_CIFAR10_data()
```

1.2 Extract Features

For each image we will compute a Histogram of Oriented Gradients (HOG) as well as a color histogram using the hue channel in HSV color space. We form our final feature vector for each image by concatenating the HOG and color histogram feature vectors.

Roughly speaking, HOG should capture the texture of the image while ignoring color information, and the color histogram represents the color of the input image while ignoring texture. As a result, we expect that using both together ought to work better than using either alone. Verifying this assumption would be a good thing to try for your own interest.

The `hog_feature` and `color_histogram_hsv` functions both operate on a single image and return a feature vector for that image. The extract_features function takes a set of images and a list of feature functions and evaluates each feature function on each image, storing the results in a matrix where each column is the concatenation of all feature vectors for a single image.

[26]:
```
from cs231n.features import *

# num_color_bins = 10 # Number of bins in the color histogram
num_color_bins = 25 # Number of bins in the color histogram
feature_fns = [hog_feature, lambda img: color_histogram_hsv(img,␣
 ↪nbin=num_color_bins)]
X_train_feats = extract_features(X_train, feature_fns, verbose=True)
X_val_feats = extract_features(X_val, feature_fns)
X_test_feats = extract_features(X_test, feature_fns)

# Preprocessing: Subtract the mean feature
mean_feat = np.mean(X_train_feats, axis=0, keepdims=True)
X_train_feats -= mean_feat
X_val_feats -= mean_feat
X_test_feats -= mean_feat

# Preprocessing: Divide by standard deviation. This ensures that each feature
# has roughly the same scale.
std_feat = np.std(X_train_feats, axis=0, keepdims=True)
X_train_feats /= std_feat
X_val_feats /= std_feat
X_test_feats /= std_feat

# Preprocessing: Add a bias dimension
```

```
X_train_feats = np.hstack([X_train_feats, np.ones((X_train_feats.shape[0], 1))])
X_val_feats = np.hstack([X_val_feats, np.ones((X_val_feats.shape[0], 1))])
X_test_feats = np.hstack([X_test_feats, np.ones((X_test_feats.shape[0], 1))])
```

```
Done extracting features for 1000 / 49000 images
Done extracting features for 2000 / 49000 images
Done extracting features for 3000 / 49000 images
Done extracting features for 4000 / 49000 images
Done extracting features for 5000 / 49000 images
Done extracting features for 6000 / 49000 images
Done extracting features for 7000 / 49000 images
Done extracting features for 8000 / 49000 images
Done extracting features for 9000 / 49000 images
Done extracting features for 10000 / 49000 images
Done extracting features for 11000 / 49000 images
Done extracting features for 12000 / 49000 images
Done extracting features for 13000 / 49000 images
Done extracting features for 14000 / 49000 images
Done extracting features for 15000 / 49000 images
Done extracting features for 16000 / 49000 images
Done extracting features for 17000 / 49000 images
Done extracting features for 18000 / 49000 images
Done extracting features for 19000 / 49000 images
Done extracting features for 20000 / 49000 images
Done extracting features for 21000 / 49000 images
Done extracting features for 22000 / 49000 images
Done extracting features for 23000 / 49000 images
Done extracting features for 24000 / 49000 images
Done extracting features for 25000 / 49000 images
Done extracting features for 26000 / 49000 images
Done extracting features for 27000 / 49000 images
Done extracting features for 28000 / 49000 images
Done extracting features for 29000 / 49000 images
Done extracting features for 30000 / 49000 images
Done extracting features for 31000 / 49000 images
Done extracting features for 32000 / 49000 images
Done extracting features for 33000 / 49000 images
Done extracting features for 34000 / 49000 images
Done extracting features for 35000 / 49000 images
Done extracting features for 36000 / 49000 images
Done extracting features for 37000 / 49000 images
Done extracting features for 38000 / 49000 images
Done extracting features for 39000 / 49000 images
Done extracting features for 40000 / 49000 images
Done extracting features for 41000 / 49000 images
Done extracting features for 42000 / 49000 images
Done extracting features for 43000 / 49000 images
```

```
Done extracting features for 44000 / 49000 images
Done extracting features for 45000 / 49000 images
Done extracting features for 46000 / 49000 images
Done extracting features for 47000 / 49000 images
Done extracting features for 48000 / 49000 images
Done extracting features for 49000 / 49000 images
```

1.3 Train Softmax classifier on features

Using the Softmax code developed earlier in the assignment, train Softmax classifiers on top of the features extracted above; this should achieve better results than training them directly on top of raw pixels.

[5]:
```
# Use the validation set to tune the learning rate and regularization strength

from cs231n.classifiers.linear_classifier import Softmax

learning_rates = [1e-3, 1e-2]
regularization_strengths = [5, 6]

results = {}
best_val = -1
best_softmax = None

for lr in learning_rates:
  for reg in regularization_strengths:
    softmax = Softmax()
    loss_hist = softmax.train(X_train_feats, y_train, learning_rate=lr, reg=reg)
    y_train_pred = softmax.predict(X_train_feats)
    y_val_pred = softmax.predict(X_val_feats)
    train_accuracy = np.mean(y_train == y_train_pred)
    val_accuracy = np.mean(y_val== y_val_pred)
    print(f"with learning rate {lr} and regularization strength {reg}, train␣
 ↪accuracy: {train_accuracy}, val accuracy: {val_accuracy}")
    results[(lr, reg)] = (train_accuracy, val_accuracy)
    if val_accuracy > best_val:
      best_val = val_accuracy
      best_softmax = softmax

# Print out results.
for lr, reg in sorted(results):
    train_accuracy, val_accuracy = results[(lr, reg)]
    print('lr %e reg %e train accuracy: %f val accuracy: %f' % (
                lr, reg, train_accuracy, val_accuracy))

print('best validation accuracy achieved: %f' % best_val)
```

```
with learning rate 0.001 and regularization strength 5, train accuracy:
0.4034081632653061, val accuracy: 0.401
with learning rate 0.001 and regularization strength 6, train accuracy:
0.4149795918367347, val accuracy: 0.41
with learning rate 0.01 and regularization strength 5, train accuracy:
0.4436938775510204, val accuracy: 0.442
with learning rate 0.01 and regularization strength 6, train accuracy:
0.44281632653061226, val accuracy: 0.457
lr 1.000000e-03 reg 5.000000e+00 train accuracy: 0.403408 val accuracy: 0.401000
lr 1.000000e-03 reg 6.000000e+00 train accuracy: 0.414980 val accuracy: 0.410000
lr 1.000000e-02 reg 5.000000e+00 train accuracy: 0.443694 val accuracy: 0.442000
lr 1.000000e-02 reg 6.000000e+00 train accuracy: 0.442816 val accuracy: 0.457000
best validation accuracy achieved: 0.457000
```

[6]:
```
# Evaluate your trained Softmax on the test set: you should be able to get at␣
↪least 0.42
y_test_pred = best_softmax.predict(X_test_feats)
test_accuracy = np.mean(y_test == y_test_pred)
print(test_accuracy)
```

```
0.442
```

[7]:
```
# Save best softmax model
best_softmax.save("best_softmax_features.npy")
```

```
best_softmax_features.npy saved.
```

[8]:
```
# An important way to gain intuition about how an algorithm works is to
# visualize the mistakes that it makes. In this visualization, we show examples
# of images that are misclassified by our current system. The first column
# shows images that our system labeled as "plane" but whose true label is
# something other than "plane".

examples_per_class = 8
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',␣
↪'ship', 'truck']
for cls, cls_name in enumerate(classes):
    idxs = np.where((y_test != cls) & (y_test_pred == cls))[0]
    idxs = np.random.choice(idxs, examples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt.subplot(examples_per_class, len(classes), i * len(classes) + cls +␣
↪1)
        plt.imshow(X_test[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls_name)
plt.show()
```

1.3.1 Inline question 1:

Describe the misclassification results that you see. Do they make sense?

Your Answer : Similar things are grouped together, e.g., ships and planes. Yes, they make sense

1.4 Neural Network on image features

Earlier in this assigment we saw that training a two-layer neural network on raw pixels achieved better classification performance than linear classifiers on raw pixels. In this notebook we have seen that linear classifiers on image features outperform linear classifiers on raw pixels.

For completeness, we should also try training a neural network on image features. This approach should outperform all previous approaches: you should easily be able to achieve over 55% classification accuracy on the test set; our best model achieves about 60% classification accuracy.

[27]:
```
# Preprocessing: Remove the bias dimension
# Make sure to run this cell only ONCE
print(X_train_feats.shape)
X_train_feats = X_train_feats[:, :-1]
```

```
X_val_feats = X_val_feats[:, :-1]
X_test_feats = X_test_feats[:, :-1]

print(X_train_feats.shape)
```

```
(49000, 170)
(49000, 169)
```

[69]:
```
from cs231n.classifiers.fc_net import TwoLayerNet
from cs231n.solver import Solver

input_dim = X_train_feats.shape[1]
hidden_dim = 500
num_classes = 10

data = {
    'X_train': X_train_feats,
    'y_train': y_train,
    'X_val': X_val_feats,
    'y_val': y_val,
    'X_test': X_test_feats,
    'y_test': y_test,
}

learning_rate = [0.1,1,0.5]
regularization_strength = [0,6.2e-6,5.8e-6]
epochs = 30
weight_scale = 1e-2

best_net = None
best_acc = 0
for lr in learning_rate:
  for reg in regularization_strength:
    model = TwoLayerNet(input_dim,hidden_dim,num_classes,reg = reg,weight_scale␣
↪= weight_scale)
    solver = Solver(model,data,optim_config = {'learning_rate':lr},lr_decay = 0.
↪95,batch_size = 100,num_epochs=epochs,verbose=True)
    solver.train()
    acc = solver.check_accuracy(data['X_val'],data['y_val'])
    print(f"with learning rate {lr}, regularization strength {reg}, accuracy:␣
↪{acc}")
    if acc > best_acc:
      best_acc = acc
      best_net = model
print(f"the best accuracy on validation set is {best_acc}")
```

```
Streaming output truncated to the last 5000 lines.
```

```
(Iteration 9881 / 14700) loss: 0.091740
(Iteration 9891 / 14700) loss: 0.076832
(Iteration 9901 / 14700) loss: 0.074200
(Iteration 9911 / 14700) loss: 0.105743
(Iteration 9921 / 14700) loss: 0.112253
(Iteration 9931 / 14700) loss: 0.082554
(Iteration 9941 / 14700) loss: 0.127490
(Iteration 9951 / 14700) loss: 0.087635
(Iteration 9961 / 14700) loss: 0.116057
(Iteration 9971 / 14700) loss: 0.077254
(Iteration 9981 / 14700) loss: 0.074009
(Iteration 9991 / 14700) loss: 0.162231
(Iteration 10001 / 14700) loss: 0.111064
(Iteration 10011 / 14700) loss: 0.069329
(Iteration 10021 / 14700) loss: 0.137538
(Iteration 10031 / 14700) loss: 0.092633
(Iteration 10041 / 14700) loss: 0.111816
(Iteration 10051 / 14700) loss: 0.096663
(Iteration 10061 / 14700) loss: 0.108107
(Iteration 10071 / 14700) loss: 0.116963
(Iteration 10081 / 14700) loss: 0.135194
(Iteration 10091 / 14700) loss: 0.066423
(Iteration 10101 / 14700) loss: 0.070911
(Iteration 10111 / 14700) loss: 0.075466
(Iteration 10121 / 14700) loss: 0.153181
(Iteration 10131 / 14700) loss: 0.055024
(Iteration 10141 / 14700) loss: 0.130276
(Iteration 10151 / 14700) loss: 0.058680
(Iteration 10161 / 14700) loss: 0.075605
(Iteration 10171 / 14700) loss: 0.067290
(Iteration 10181 / 14700) loss: 0.074143
(Iteration 10191 / 14700) loss: 0.159291
(Iteration 10201 / 14700) loss: 0.119277
(Iteration 10211 / 14700) loss: 0.100117
(Iteration 10221 / 14700) loss: 0.090203
(Iteration 10231 / 14700) loss: 0.095899
(Iteration 10241 / 14700) loss: 0.082597
(Iteration 10251 / 14700) loss: 0.114595
(Iteration 10261 / 14700) loss: 0.133035
(Iteration 10271 / 14700) loss: 0.121810
(Iteration 10281 / 14700) loss: 0.125225
(Epoch 21 / 30) train acc: 0.982000; val_acc: 0.577000
(Iteration 10291 / 14700) loss: 0.124060
(Iteration 10301 / 14700) loss: 0.068758
(Iteration 10311 / 14700) loss: 0.135865
(Iteration 10321 / 14700) loss: 0.093624
(Iteration 10331 / 14700) loss: 0.042890
(Iteration 10341 / 14700) loss: 0.132959
```

```
(Iteration 10351 / 14700) loss: 0.138536
(Iteration 10361 / 14700) loss: 0.082626
(Iteration 10371 / 14700) loss: 0.092702
(Iteration 10381 / 14700) loss: 0.064841
(Iteration 10391 / 14700) loss: 0.106889
(Iteration 10401 / 14700) loss: 0.072318
(Iteration 10411 / 14700) loss: 0.099167
(Iteration 10421 / 14700) loss: 0.074746
(Iteration 10431 / 14700) loss: 0.083702
(Iteration 10441 / 14700) loss: 0.081164
(Iteration 10451 / 14700) loss: 0.047867
(Iteration 10461 / 14700) loss: 0.075445
(Iteration 10471 / 14700) loss: 0.040502
(Iteration 10481 / 14700) loss: 0.062583
(Iteration 10491 / 14700) loss: 0.118183
(Iteration 10501 / 14700) loss: 0.130877
(Iteration 10511 / 14700) loss: 0.126745
(Iteration 10521 / 14700) loss: 0.064438
(Iteration 10531 / 14700) loss: 0.103537
(Iteration 10541 / 14700) loss: 0.041502
(Iteration 10551 / 14700) loss: 0.046002
(Iteration 10561 / 14700) loss: 0.088604
(Iteration 10571 / 14700) loss: 0.094867
(Iteration 10581 / 14700) loss: 0.083767
(Iteration 10591 / 14700) loss: 0.069630
(Iteration 10601 / 14700) loss: 0.062721
(Iteration 10611 / 14700) loss: 0.057579
(Iteration 10621 / 14700) loss: 0.089941
(Iteration 10631 / 14700) loss: 0.053274
(Iteration 10641 / 14700) loss: 0.103060
(Iteration 10651 / 14700) loss: 0.119145
(Iteration 10661 / 14700) loss: 0.054181
(Iteration 10671 / 14700) loss: 0.106955
(Iteration 10681 / 14700) loss: 0.064842
(Iteration 10691 / 14700) loss: 0.051390
(Iteration 10701 / 14700) loss: 0.067385
(Iteration 10711 / 14700) loss: 0.054940
(Iteration 10721 / 14700) loss: 0.085931
(Iteration 10731 / 14700) loss: 0.061772
(Iteration 10741 / 14700) loss: 0.035505
(Iteration 10751 / 14700) loss: 0.077648
(Iteration 10761 / 14700) loss: 0.050321
(Iteration 10771 / 14700) loss: 0.097533
(Epoch 22 / 30) train acc: 0.983000; val_acc: 0.574000
(Iteration 10781 / 14700) loss: 0.052289
(Iteration 10791 / 14700) loss: 0.064008
(Iteration 10801 / 14700) loss: 0.044005
(Iteration 10811 / 14700) loss: 0.102645
```

```
(Iteration 10821 / 14700) loss: 0.085465
(Iteration 10831 / 14700) loss: 0.058709
(Iteration 10841 / 14700) loss: 0.071611
(Iteration 10851 / 14700) loss: 0.090858
(Iteration 10861 / 14700) loss: 0.059944
(Iteration 10871 / 14700) loss: 0.073792
(Iteration 10881 / 14700) loss: 0.120688
(Iteration 10891 / 14700) loss: 0.073059
(Iteration 10901 / 14700) loss: 0.080427
(Iteration 10911 / 14700) loss: 0.085340
(Iteration 10921 / 14700) loss: 0.122319
(Iteration 10931 / 14700) loss: 0.066012
(Iteration 10941 / 14700) loss: 0.067663
(Iteration 10951 / 14700) loss: 0.065866
(Iteration 10961 / 14700) loss: 0.065741
(Iteration 10971 / 14700) loss: 0.035313
(Iteration 10981 / 14700) loss: 0.069639
(Iteration 10991 / 14700) loss: 0.103409
(Iteration 11001 / 14700) loss: 0.061181
(Iteration 11011 / 14700) loss: 0.067867
(Iteration 11021 / 14700) loss: 0.052100
(Iteration 11031 / 14700) loss: 0.046280
(Iteration 11041 / 14700) loss: 0.036028
(Iteration 11051 / 14700) loss: 0.067650
(Iteration 11061 / 14700) loss: 0.086091
(Iteration 11071 / 14700) loss: 0.043805
(Iteration 11081 / 14700) loss: 0.111977
(Iteration 11091 / 14700) loss: 0.092501
(Iteration 11101 / 14700) loss: 0.043007
(Iteration 11111 / 14700) loss: 0.061387
(Iteration 11121 / 14700) loss: 0.054322
(Iteration 11131 / 14700) loss: 0.038866
(Iteration 11141 / 14700) loss: 0.047253
(Iteration 11151 / 14700) loss: 0.087707
(Iteration 11161 / 14700) loss: 0.044531
(Iteration 11171 / 14700) loss: 0.059196
(Iteration 11181 / 14700) loss: 0.059948
(Iteration 11191 / 14700) loss: 0.039378
(Iteration 11201 / 14700) loss: 0.054057
(Iteration 11211 / 14700) loss: 0.048818
(Iteration 11221 / 14700) loss: 0.078217
(Iteration 11231 / 14700) loss: 0.068503
(Iteration 11241 / 14700) loss: 0.038239
(Iteration 11251 / 14700) loss: 0.064104
(Iteration 11261 / 14700) loss: 0.059234
(Epoch 23 / 30) train acc: 0.994000; val_acc: 0.567000
(Iteration 11271 / 14700) loss: 0.036783
(Iteration 11281 / 14700) loss: 0.057361
```

```
(Iteration 11291 / 14700) loss: 0.044871
(Iteration 11301 / 14700) loss: 0.049071
(Iteration 11311 / 14700) loss: 0.074004
(Iteration 11321 / 14700) loss: 0.050470
(Iteration 11331 / 14700) loss: 0.050481
(Iteration 11341 / 14700) loss: 0.042398
(Iteration 11351 / 14700) loss: 0.068631
(Iteration 11361 / 14700) loss: 0.044911
(Iteration 11371 / 14700) loss: 0.074506
(Iteration 11381 / 14700) loss: 0.060638
(Iteration 11391 / 14700) loss: 0.054455
(Iteration 11401 / 14700) loss: 0.102683
(Iteration 11411 / 14700) loss: 0.072338
(Iteration 11421 / 14700) loss: 0.107116
(Iteration 11431 / 14700) loss: 0.031939
(Iteration 11441 / 14700) loss: 0.058454
(Iteration 11451 / 14700) loss: 0.035077
(Iteration 11461 / 14700) loss: 0.059008
(Iteration 11471 / 14700) loss: 0.049151
(Iteration 11481 / 14700) loss: 0.041216
(Iteration 11491 / 14700) loss: 0.041719
(Iteration 11501 / 14700) loss: 0.036721
(Iteration 11511 / 14700) loss: 0.030053
(Iteration 11521 / 14700) loss: 0.039530
(Iteration 11531 / 14700) loss: 0.033663
(Iteration 11541 / 14700) loss: 0.038789
(Iteration 11551 / 14700) loss: 0.043826
(Iteration 11561 / 14700) loss: 0.045609
(Iteration 11571 / 14700) loss: 0.045862
(Iteration 11581 / 14700) loss: 0.044418
(Iteration 11591 / 14700) loss: 0.031513
(Iteration 11601 / 14700) loss: 0.035694
(Iteration 11611 / 14700) loss: 0.067386
(Iteration 11621 / 14700) loss: 0.056518
(Iteration 11631 / 14700) loss: 0.059277
(Iteration 11641 / 14700) loss: 0.032373
(Iteration 11651 / 14700) loss: 0.041901
(Iteration 11661 / 14700) loss: 0.060118
(Iteration 11671 / 14700) loss: 0.057356
(Iteration 11681 / 14700) loss: 0.047590
(Iteration 11691 / 14700) loss: 0.057110
(Iteration 11701 / 14700) loss: 0.047574
(Iteration 11711 / 14700) loss: 0.051609
(Iteration 11721 / 14700) loss: 0.046128
(Iteration 11731 / 14700) loss: 0.086109
(Iteration 11741 / 14700) loss: 0.035567
(Iteration 11751 / 14700) loss: 0.052676
(Epoch 24 / 30) train acc: 0.995000; val_acc: 0.573000
```

```
(Iteration 11761 / 14700) loss: 0.043258
(Iteration 11771 / 14700) loss: 0.046072
(Iteration 11781 / 14700) loss: 0.029649
(Iteration 11791 / 14700) loss: 0.056917
(Iteration 11801 / 14700) loss: 0.031256
(Iteration 11811 / 14700) loss: 0.065655
(Iteration 11821 / 14700) loss: 0.035945
(Iteration 11831 / 14700) loss: 0.035850
(Iteration 11841 / 14700) loss: 0.037891
(Iteration 11851 / 14700) loss: 0.037102
(Iteration 11861 / 14700) loss: 0.060078
(Iteration 11871 / 14700) loss: 0.060340
(Iteration 11881 / 14700) loss: 0.025570
(Iteration 11891 / 14700) loss: 0.060289
(Iteration 11901 / 14700) loss: 0.042204
(Iteration 11911 / 14700) loss: 0.044707
(Iteration 11921 / 14700) loss: 0.033027
(Iteration 11931 / 14700) loss: 0.032785
(Iteration 11941 / 14700) loss: 0.037806
(Iteration 11951 / 14700) loss: 0.053192
(Iteration 11961 / 14700) loss: 0.073533
(Iteration 11971 / 14700) loss: 0.040485
(Iteration 11981 / 14700) loss: 0.036872
(Iteration 11991 / 14700) loss: 0.035227
(Iteration 12001 / 14700) loss: 0.075866
(Iteration 12011 / 14700) loss: 0.030535
(Iteration 12021 / 14700) loss: 0.055801
(Iteration 12031 / 14700) loss: 0.050453
(Iteration 12041 / 14700) loss: 0.054446
(Iteration 12051 / 14700) loss: 0.031712
(Iteration 12061 / 14700) loss: 0.038095
(Iteration 12071 / 14700) loss: 0.033749
(Iteration 12081 / 14700) loss: 0.052863
(Iteration 12091 / 14700) loss: 0.055380
(Iteration 12101 / 14700) loss: 0.039775
(Iteration 12111 / 14700) loss: 0.038047
(Iteration 12121 / 14700) loss: 0.039270
(Iteration 12131 / 14700) loss: 0.035820
(Iteration 12141 / 14700) loss: 0.067899
(Iteration 12151 / 14700) loss: 0.035740
(Iteration 12161 / 14700) loss: 0.036015
(Iteration 12171 / 14700) loss: 0.042100
(Iteration 12181 / 14700) loss: 0.053337
(Iteration 12191 / 14700) loss: 0.033182
(Iteration 12201 / 14700) loss: 0.068412
(Iteration 12211 / 14700) loss: 0.038530
(Iteration 12221 / 14700) loss: 0.048236
(Iteration 12231 / 14700) loss: 0.028462
```

```
(Iteration 12241 / 14700) loss: 0.039985
(Epoch 25 / 30) train acc: 0.997000; val_acc: 0.567000
(Iteration 12251 / 14700) loss: 0.050476
(Iteration 12261 / 14700) loss: 0.043209
(Iteration 12271 / 14700) loss: 0.038833
(Iteration 12281 / 14700) loss: 0.038027
(Iteration 12291 / 14700) loss: 0.058794
(Iteration 12301 / 14700) loss: 0.039748
(Iteration 12311 / 14700) loss: 0.030152
(Iteration 12321 / 14700) loss: 0.047469
(Iteration 12331 / 14700) loss: 0.045911
(Iteration 12341 / 14700) loss: 0.049026
(Iteration 12351 / 14700) loss: 0.034609
(Iteration 12361 / 14700) loss: 0.035444
(Iteration 12371 / 14700) loss: 0.036757
(Iteration 12381 / 14700) loss: 0.046268
(Iteration 12391 / 14700) loss: 0.038959
(Iteration 12401 / 14700) loss: 0.030192
(Iteration 12411 / 14700) loss: 0.041324
(Iteration 12421 / 14700) loss: 0.036815
(Iteration 12431 / 14700) loss: 0.044602
(Iteration 12441 / 14700) loss: 0.049462
(Iteration 12451 / 14700) loss: 0.035815
(Iteration 12461 / 14700) loss: 0.030821
(Iteration 12471 / 14700) loss: 0.036761
(Iteration 12481 / 14700) loss: 0.031821
(Iteration 12491 / 14700) loss: 0.052289
(Iteration 12501 / 14700) loss: 0.038832
(Iteration 12511 / 14700) loss: 0.032571
(Iteration 12521 / 14700) loss: 0.028416
(Iteration 12531 / 14700) loss: 0.038458
(Iteration 12541 / 14700) loss: 0.037904
(Iteration 12551 / 14700) loss: 0.033354
(Iteration 12561 / 14700) loss: 0.042390
(Iteration 12571 / 14700) loss: 0.030651
(Iteration 12581 / 14700) loss: 0.038395
(Iteration 12591 / 14700) loss: 0.036492
(Iteration 12601 / 14700) loss: 0.033473
(Iteration 12611 / 14700) loss: 0.039115
(Iteration 12621 / 14700) loss: 0.031334
(Iteration 12631 / 14700) loss: 0.048129
(Iteration 12641 / 14700) loss: 0.027303
(Iteration 12651 / 14700) loss: 0.034245
(Iteration 12661 / 14700) loss: 0.027686
(Iteration 12671 / 14700) loss: 0.041127
(Iteration 12681 / 14700) loss: 0.039519
(Iteration 12691 / 14700) loss: 0.045087
(Iteration 12701 / 14700) loss: 0.048364
```

```
(Iteration 12711 / 14700) loss: 0.034358
(Iteration 12721 / 14700) loss: 0.041012
(Iteration 12731 / 14700) loss: 0.039448
(Epoch 26 / 30) train acc: 0.998000; val_acc: 0.569000
(Iteration 12741 / 14700) loss: 0.029424
(Iteration 12751 / 14700) loss: 0.035610
(Iteration 12761 / 14700) loss: 0.040756
(Iteration 12771 / 14700) loss: 0.030196
(Iteration 12781 / 14700) loss: 0.035823
(Iteration 12791 / 14700) loss: 0.058469
(Iteration 12801 / 14700) loss: 0.028152
(Iteration 12811 / 14700) loss: 0.023411
(Iteration 12821 / 14700) loss: 0.036179
(Iteration 12831 / 14700) loss: 0.042580
(Iteration 12841 / 14700) loss: 0.028034
(Iteration 12851 / 14700) loss: 0.035668
(Iteration 12861 / 14700) loss: 0.046589
(Iteration 12871 / 14700) loss: 0.030979
(Iteration 12881 / 14700) loss: 0.027482
(Iteration 12891 / 14700) loss: 0.028739
(Iteration 12901 / 14700) loss: 0.089146
(Iteration 12911 / 14700) loss: 0.029632
(Iteration 12921 / 14700) loss: 0.030934
(Iteration 12931 / 14700) loss: 0.034637
(Iteration 12941 / 14700) loss: 0.031287
(Iteration 12951 / 14700) loss: 0.032780
(Iteration 12961 / 14700) loss: 0.037702
(Iteration 12971 / 14700) loss: 0.033089
(Iteration 12981 / 14700) loss: 0.030920
(Iteration 12991 / 14700) loss: 0.031784
(Iteration 13001 / 14700) loss: 0.037465
(Iteration 13011 / 14700) loss: 0.026341
(Iteration 13021 / 14700) loss: 0.040874
(Iteration 13031 / 14700) loss: 0.033596
(Iteration 13041 / 14700) loss: 0.033162
(Iteration 13051 / 14700) loss: 0.036452
(Iteration 13061 / 14700) loss: 0.029390
(Iteration 13071 / 14700) loss: 0.034277
(Iteration 13081 / 14700) loss: 0.032080
(Iteration 13091 / 14700) loss: 0.037435
(Iteration 13101 / 14700) loss: 0.028583
(Iteration 13111 / 14700) loss: 0.030486
(Iteration 13121 / 14700) loss: 0.033821
(Iteration 13131 / 14700) loss: 0.034857
(Iteration 13141 / 14700) loss: 0.025023
(Iteration 13151 / 14700) loss: 0.034523
(Iteration 13161 / 14700) loss: 0.032947
(Iteration 13171 / 14700) loss: 0.030193
```

```
(Iteration 13181 / 14700) loss: 0.033617
(Iteration 13191 / 14700) loss: 0.028561
(Iteration 13201 / 14700) loss: 0.033465
(Iteration 13211 / 14700) loss: 0.029950
(Iteration 13221 / 14700) loss: 0.028702
(Epoch 27 / 30) train acc: 0.999000; val_acc: 0.572000
(Iteration 13231 / 14700) loss: 0.029432
(Iteration 13241 / 14700) loss: 0.031331
(Iteration 13251 / 14700) loss: 0.030719
(Iteration 13261 / 14700) loss: 0.037231
(Iteration 13271 / 14700) loss: 0.031949
(Iteration 13281 / 14700) loss: 0.032925
(Iteration 13291 / 14700) loss: 0.034682
(Iteration 13301 / 14700) loss: 0.036942
(Iteration 13311 / 14700) loss: 0.030250
(Iteration 13321 / 14700) loss: 0.032956
(Iteration 13331 / 14700) loss: 0.031054
(Iteration 13341 / 14700) loss: 0.031944
(Iteration 13351 / 14700) loss: 0.037681
(Iteration 13361 / 14700) loss: 0.032234
(Iteration 13371 / 14700) loss: 0.036187
(Iteration 13381 / 14700) loss: 0.026650
(Iteration 13391 / 14700) loss: 0.032561
(Iteration 13401 / 14700) loss: 0.030834
(Iteration 13411 / 14700) loss: 0.028419
(Iteration 13421 / 14700) loss: 0.037301
(Iteration 13431 / 14700) loss: 0.031767
(Iteration 13441 / 14700) loss: 0.030033
(Iteration 13451 / 14700) loss: 0.026954
(Iteration 13461 / 14700) loss: 0.027717
(Iteration 13471 / 14700) loss: 0.040400
(Iteration 13481 / 14700) loss: 0.038082
(Iteration 13491 / 14700) loss: 0.030208
(Iteration 13501 / 14700) loss: 0.036045
(Iteration 13511 / 14700) loss: 0.030650
(Iteration 13521 / 14700) loss: 0.035466
(Iteration 13531 / 14700) loss: 0.042463
(Iteration 13541 / 14700) loss: 0.030166
(Iteration 13551 / 14700) loss: 0.026678
(Iteration 13561 / 14700) loss: 0.030208
(Iteration 13571 / 14700) loss: 0.026148
(Iteration 13581 / 14700) loss: 0.032751
(Iteration 13591 / 14700) loss: 0.029363
(Iteration 13601 / 14700) loss: 0.030957
(Iteration 13611 / 14700) loss: 0.030671
(Iteration 13621 / 14700) loss: 0.029981
(Iteration 13631 / 14700) loss: 0.030094
(Iteration 13641 / 14700) loss: 0.033071
```

```
(Iteration 13651 / 14700) loss: 0.029925
(Iteration 13661 / 14700) loss: 0.026823
(Iteration 13671 / 14700) loss: 0.029401
(Iteration 13681 / 14700) loss: 0.025286
(Iteration 13691 / 14700) loss: 0.033794
(Iteration 13701 / 14700) loss: 0.027280
(Iteration 13711 / 14700) loss: 0.025935
(Epoch 28 / 30) train acc: 0.999000; val_acc: 0.568000
(Iteration 13721 / 14700) loss: 0.040243
(Iteration 13731 / 14700) loss: 0.031346
(Iteration 13741 / 14700) loss: 0.024409
(Iteration 13751 / 14700) loss: 0.029264
(Iteration 13761 / 14700) loss: 0.024941
(Iteration 13771 / 14700) loss: 0.034714
(Iteration 13781 / 14700) loss: 0.033893
(Iteration 13791 / 14700) loss: 0.037020
(Iteration 13801 / 14700) loss: 0.020995
(Iteration 13811 / 14700) loss: 0.028769
(Iteration 13821 / 14700) loss: 0.030902
(Iteration 13831 / 14700) loss: 0.029942
(Iteration 13841 / 14700) loss: 0.030047
(Iteration 13851 / 14700) loss: 0.028658
(Iteration 13861 / 14700) loss: 0.026254
(Iteration 13871 / 14700) loss: 0.026831
(Iteration 13881 / 14700) loss: 0.027521
(Iteration 13891 / 14700) loss: 0.041942
(Iteration 13901 / 14700) loss: 0.041135
(Iteration 13911 / 14700) loss: 0.046833
(Iteration 13921 / 14700) loss: 0.036440
(Iteration 13931 / 14700) loss: 0.026533
(Iteration 13941 / 14700) loss: 0.027895
(Iteration 13951 / 14700) loss: 0.027267
(Iteration 13961 / 14700) loss: 0.025641
(Iteration 13971 / 14700) loss: 0.029916
(Iteration 13981 / 14700) loss: 0.030734
(Iteration 13991 / 14700) loss: 0.068828
(Iteration 14001 / 14700) loss: 0.030548
(Iteration 14011 / 14700) loss: 0.026982
(Iteration 14021 / 14700) loss: 0.027071
(Iteration 14031 / 14700) loss: 0.025394
(Iteration 14041 / 14700) loss: 0.026798
(Iteration 14051 / 14700) loss: 0.034308
(Iteration 14061 / 14700) loss: 0.027476
(Iteration 14071 / 14700) loss: 0.027979
(Iteration 14081 / 14700) loss: 0.030945
(Iteration 14091 / 14700) loss: 0.025445
(Iteration 14101 / 14700) loss: 0.025636
(Iteration 14111 / 14700) loss: 0.027173
```

```
(Iteration 14121 / 14700) loss: 0.028275
(Iteration 14131 / 14700) loss: 0.061298
(Iteration 14141 / 14700) loss: 0.033111
(Iteration 14151 / 14700) loss: 0.025360
(Iteration 14161 / 14700) loss: 0.039330
(Iteration 14171 / 14700) loss: 0.030470
(Iteration 14181 / 14700) loss: 0.023995
(Iteration 14191 / 14700) loss: 0.033196
(Iteration 14201 / 14700) loss: 0.036160
(Epoch 29 / 30) train acc: 1.000000; val_acc: 0.568000
(Iteration 14211 / 14700) loss: 0.030008
(Iteration 14221 / 14700) loss: 0.025589
(Iteration 14231 / 14700) loss: 0.028703
(Iteration 14241 / 14700) loss: 0.027944
(Iteration 14251 / 14700) loss: 0.027969
(Iteration 14261 / 14700) loss: 0.025699
(Iteration 14271 / 14700) loss: 0.032430
(Iteration 14281 / 14700) loss: 0.066146
(Iteration 14291 / 14700) loss: 0.118731
(Iteration 14301 / 14700) loss: 0.033694
(Iteration 14311 / 14700) loss: 0.025304
(Iteration 14321 / 14700) loss: 0.021951
(Iteration 14331 / 14700) loss: 0.031073
(Iteration 14341 / 14700) loss: 0.023542
(Iteration 14351 / 14700) loss: 0.028239
(Iteration 14361 / 14700) loss: 0.033701
(Iteration 14371 / 14700) loss: 0.025671
(Iteration 14381 / 14700) loss: 0.025299
(Iteration 14391 / 14700) loss: 0.024367
(Iteration 14401 / 14700) loss: 0.032577
(Iteration 14411 / 14700) loss: 0.029056
(Iteration 14421 / 14700) loss: 0.025038
(Iteration 14431 / 14700) loss: 0.038726
(Iteration 14441 / 14700) loss: 0.028767
(Iteration 14451 / 14700) loss: 0.037018
(Iteration 14461 / 14700) loss: 0.024326
(Iteration 14471 / 14700) loss: 0.023264
(Iteration 14481 / 14700) loss: 0.025842
(Iteration 14491 / 14700) loss: 0.026919
(Iteration 14501 / 14700) loss: 0.024438
(Iteration 14511 / 14700) loss: 0.028465
(Iteration 14521 / 14700) loss: 0.023882
(Iteration 14531 / 14700) loss: 0.032176
(Iteration 14541 / 14700) loss: 0.022675
(Iteration 14551 / 14700) loss: 0.037933
(Iteration 14561 / 14700) loss: 0.025950
(Iteration 14571 / 14700) loss: 0.026846
(Iteration 14581 / 14700) loss: 0.031291
```

```
(Iteration 14591 / 14700) loss: 0.027455
(Iteration 14601 / 14700) loss: 0.031893
(Iteration 14611 / 14700) loss: 0.030967
(Iteration 14621 / 14700) loss: 0.032746
(Iteration 14631 / 14700) loss: 0.032620
(Iteration 14641 / 14700) loss: 0.031056
(Iteration 14651 / 14700) loss: 0.033060
(Iteration 14661 / 14700) loss: 0.027222
(Iteration 14671 / 14700) loss: 0.024923
(Iteration 14681 / 14700) loss: 0.031481
(Iteration 14691 / 14700) loss: 0.034708
(Epoch 30 / 30) train acc: 1.000000; val_acc: 0.572000
with learning rate 1, regularization strength 5.8e-06, accuracy: 0.577
(Iteration 1 / 14700) loss: 2.302204
(Epoch 0 / 30) train acc: 0.192000; val_acc: 0.196000
(Iteration 11 / 14700) loss: 2.099607
(Iteration 21 / 14700) loss: 1.774866
(Iteration 31 / 14700) loss: 1.710821
(Iteration 41 / 14700) loss: 1.490759
(Iteration 51 / 14700) loss: 1.577958
(Iteration 61 / 14700) loss: 1.382328
(Iteration 71 / 14700) loss: 1.296943
(Iteration 81 / 14700) loss: 1.504289
(Iteration 91 / 14700) loss: 1.447906
(Iteration 101 / 14700) loss: 1.583161
(Iteration 111 / 14700) loss: 1.292073
(Iteration 121 / 14700) loss: 1.420055
(Iteration 131 / 14700) loss: 1.470035
(Iteration 141 / 14700) loss: 1.665184
(Iteration 151 / 14700) loss: 1.369322
(Iteration 161 / 14700) loss: 1.280955
(Iteration 171 / 14700) loss: 1.340837
(Iteration 181 / 14700) loss: 1.286649
(Iteration 191 / 14700) loss: 1.592619
(Iteration 201 / 14700) loss: 1.368844
(Iteration 211 / 14700) loss: 1.445981
(Iteration 221 / 14700) loss: 1.213713
(Iteration 231 / 14700) loss: 1.416555
(Iteration 241 / 14700) loss: 1.238413
(Iteration 251 / 14700) loss: 1.189275
(Iteration 261 / 14700) loss: 1.286773
(Iteration 271 / 14700) loss: 1.300462
(Iteration 281 / 14700) loss: 1.178415
(Iteration 291 / 14700) loss: 1.286332
(Iteration 301 / 14700) loss: 1.309169
(Iteration 311 / 14700) loss: 1.112382
(Iteration 321 / 14700) loss: 1.120641
(Iteration 331 / 14700) loss: 1.407903
```

```
(Iteration 341 / 14700) loss: 1.229953
(Iteration 351 / 14700) loss: 1.302154
(Iteration 361 / 14700) loss: 1.427044
(Iteration 371 / 14700) loss: 1.017994
(Iteration 381 / 14700) loss: 1.279038
(Iteration 391 / 14700) loss: 1.154272
(Iteration 401 / 14700) loss: 1.213147
(Iteration 411 / 14700) loss: 1.494627
(Iteration 421 / 14700) loss: 1.309955
(Iteration 431 / 14700) loss: 1.369081
(Iteration 441 / 14700) loss: 1.222007
(Iteration 451 / 14700) loss: 1.389137
(Iteration 461 / 14700) loss: 1.195080
(Iteration 471 / 14700) loss: 1.211777
(Iteration 481 / 14700) loss: 1.455292
(Epoch 1 / 30) train acc: 0.542000; val_acc: 0.518000
(Iteration 491 / 14700) loss: 1.200131
(Iteration 501 / 14700) loss: 1.144705
(Iteration 511 / 14700) loss: 1.150602
(Iteration 521 / 14700) loss: 1.078030
(Iteration 531 / 14700) loss: 1.143275
(Iteration 541 / 14700) loss: 1.253051
(Iteration 551 / 14700) loss: 1.160697
(Iteration 561 / 14700) loss: 1.249874
(Iteration 571 / 14700) loss: 1.320538
(Iteration 581 / 14700) loss: 0.974423
(Iteration 591 / 14700) loss: 1.235044
(Iteration 601 / 14700) loss: 1.292625
(Iteration 611 / 14700) loss: 1.330429
(Iteration 621 / 14700) loss: 1.135763
(Iteration 631 / 14700) loss: 1.183236
(Iteration 641 / 14700) loss: 1.277727
(Iteration 651 / 14700) loss: 1.361208
(Iteration 661 / 14700) loss: 1.073109
(Iteration 671 / 14700) loss: 1.238403
(Iteration 681 / 14700) loss: 1.016913
(Iteration 691 / 14700) loss: 1.086255
(Iteration 701 / 14700) loss: 1.189026
(Iteration 711 / 14700) loss: 1.104124
(Iteration 721 / 14700) loss: 1.220643
(Iteration 731 / 14700) loss: 1.112233
(Iteration 741 / 14700) loss: 1.301154
(Iteration 751 / 14700) loss: 1.028253
(Iteration 761 / 14700) loss: 1.064540
(Iteration 771 / 14700) loss: 1.105081
(Iteration 781 / 14700) loss: 1.073842
(Iteration 791 / 14700) loss: 1.265700
(Iteration 801 / 14700) loss: 1.064720
```

```
(Iteration 811 / 14700) loss: 1.163597
(Iteration 821 / 14700) loss: 1.052751
(Iteration 831 / 14700) loss: 1.212993
(Iteration 841 / 14700) loss: 1.061854
(Iteration 851 / 14700) loss: 1.197181
(Iteration 861 / 14700) loss: 1.098419
(Iteration 871 / 14700) loss: 1.024327
(Iteration 881 / 14700) loss: 1.099218
(Iteration 891 / 14700) loss: 1.222066
(Iteration 901 / 14700) loss: 1.126525
(Iteration 911 / 14700) loss: 1.018616
(Iteration 921 / 14700) loss: 1.115267
(Iteration 931 / 14700) loss: 1.150178
(Iteration 941 / 14700) loss: 1.071480
(Iteration 951 / 14700) loss: 0.981010
(Iteration 961 / 14700) loss: 1.044239
(Iteration 971 / 14700) loss: 1.088527
(Epoch 2 / 30) train acc: 0.634000; val_acc: 0.560000
(Iteration 981 / 14700) loss: 1.044309
(Iteration 991 / 14700) loss: 0.866090
(Iteration 1001 / 14700) loss: 1.052511
(Iteration 1011 / 14700) loss: 1.019524
(Iteration 1021 / 14700) loss: 0.990776
(Iteration 1031 / 14700) loss: 1.243600
(Iteration 1041 / 14700) loss: 1.264122
(Iteration 1051 / 14700) loss: 1.050032
(Iteration 1061 / 14700) loss: 1.078964
(Iteration 1071 / 14700) loss: 1.259365
(Iteration 1081 / 14700) loss: 1.033094
(Iteration 1091 / 14700) loss: 0.935707
(Iteration 1101 / 14700) loss: 0.879787
(Iteration 1111 / 14700) loss: 0.955017
(Iteration 1121 / 14700) loss: 0.869534
(Iteration 1131 / 14700) loss: 0.807751
(Iteration 1141 / 14700) loss: 0.928403
(Iteration 1151 / 14700) loss: 1.240003
(Iteration 1161 / 14700) loss: 1.021894
(Iteration 1171 / 14700) loss: 1.159446
(Iteration 1181 / 14700) loss: 0.955365
(Iteration 1191 / 14700) loss: 1.112440
(Iteration 1201 / 14700) loss: 1.340008
(Iteration 1211 / 14700) loss: 1.030503
(Iteration 1221 / 14700) loss: 0.891235
(Iteration 1231 / 14700) loss: 1.048044
(Iteration 1241 / 14700) loss: 1.132922
(Iteration 1251 / 14700) loss: 0.920699
(Iteration 1261 / 14700) loss: 1.123380
(Iteration 1271 / 14700) loss: 1.028087
```

```
(Iteration 1281 / 14700) loss: 0.922722
(Iteration 1291 / 14700) loss: 1.127767
(Iteration 1301 / 14700) loss: 1.175614
(Iteration 1311 / 14700) loss: 1.020638
(Iteration 1321 / 14700) loss: 1.011196
(Iteration 1331 / 14700) loss: 0.930423
(Iteration 1341 / 14700) loss: 1.179627
(Iteration 1351 / 14700) loss: 1.146411
(Iteration 1361 / 14700) loss: 1.044452
(Iteration 1371 / 14700) loss: 1.192759
(Iteration 1381 / 14700) loss: 0.825465
(Iteration 1391 / 14700) loss: 0.740207
(Iteration 1401 / 14700) loss: 0.996803
(Iteration 1411 / 14700) loss: 1.039048
(Iteration 1421 / 14700) loss: 0.998852
(Iteration 1431 / 14700) loss: 0.924142
(Iteration 1441 / 14700) loss: 1.138010
(Iteration 1451 / 14700) loss: 0.995202
(Iteration 1461 / 14700) loss: 0.935409
(Epoch 3 / 30) train acc: 0.632000; val_acc: 0.569000
(Iteration 1471 / 14700) loss: 1.228708
(Iteration 1481 / 14700) loss: 0.907591
(Iteration 1491 / 14700) loss: 0.991641
(Iteration 1501 / 14700) loss: 0.878326
(Iteration 1511 / 14700) loss: 0.958795
(Iteration 1521 / 14700) loss: 0.867095
(Iteration 1531 / 14700) loss: 0.854430
(Iteration 1541 / 14700) loss: 0.945609
(Iteration 1551 / 14700) loss: 1.032662
(Iteration 1561 / 14700) loss: 1.081751
(Iteration 1571 / 14700) loss: 1.182500
(Iteration 1581 / 14700) loss: 0.919004
(Iteration 1591 / 14700) loss: 0.825579
(Iteration 1601 / 14700) loss: 0.792915
(Iteration 1611 / 14700) loss: 0.993436
(Iteration 1621 / 14700) loss: 0.916324
(Iteration 1631 / 14700) loss: 0.891837
(Iteration 1641 / 14700) loss: 1.099660
(Iteration 1651 / 14700) loss: 1.050519
(Iteration 1661 / 14700) loss: 0.721810
(Iteration 1671 / 14700) loss: 0.811322
(Iteration 1681 / 14700) loss: 0.966033
(Iteration 1691 / 14700) loss: 0.885604
(Iteration 1701 / 14700) loss: 0.922731
(Iteration 1711 / 14700) loss: 1.014699
(Iteration 1721 / 14700) loss: 0.993488
(Iteration 1731 / 14700) loss: 0.864177
(Iteration 1741 / 14700) loss: 1.028081
```

```
(Iteration 1751 / 14700) loss: 0.958970
(Iteration 1761 / 14700) loss: 1.065927
(Iteration 1771 / 14700) loss: 1.025582
(Iteration 1781 / 14700) loss: 0.918480
(Iteration 1791 / 14700) loss: 0.884324
(Iteration 1801 / 14700) loss: 0.868812
(Iteration 1811 / 14700) loss: 0.946247
(Iteration 1821 / 14700) loss: 1.027061
(Iteration 1831 / 14700) loss: 1.111575
(Iteration 1841 / 14700) loss: 1.090500
(Iteration 1851 / 14700) loss: 0.751027
(Iteration 1861 / 14700) loss: 0.835693
(Iteration 1871 / 14700) loss: 0.896169
(Iteration 1881 / 14700) loss: 0.797931
(Iteration 1891 / 14700) loss: 1.108381
(Iteration 1901 / 14700) loss: 0.864618
(Iteration 1911 / 14700) loss: 0.837283
(Iteration 1921 / 14700) loss: 0.733746
(Iteration 1931 / 14700) loss: 0.903828
(Iteration 1941 / 14700) loss: 0.898650
(Iteration 1951 / 14700) loss: 0.785887
(Epoch 4 / 30) train acc: 0.713000; val_acc: 0.568000
(Iteration 1961 / 14700) loss: 0.842122
(Iteration 1971 / 14700) loss: 0.920778
(Iteration 1981 / 14700) loss: 0.786084
(Iteration 1991 / 14700) loss: 0.932056
(Iteration 2001 / 14700) loss: 0.927545
(Iteration 2011 / 14700) loss: 0.747697
(Iteration 2021 / 14700) loss: 0.928773
(Iteration 2031 / 14700) loss: 0.885268
(Iteration 2041 / 14700) loss: 1.009300
(Iteration 2051 / 14700) loss: 0.869954
(Iteration 2061 / 14700) loss: 0.935127
(Iteration 2071 / 14700) loss: 0.882846
(Iteration 2081 / 14700) loss: 0.836442
(Iteration 2091 / 14700) loss: 0.732732
(Iteration 2101 / 14700) loss: 0.934706
(Iteration 2111 / 14700) loss: 0.842356
(Iteration 2121 / 14700) loss: 0.736042
(Iteration 2131 / 14700) loss: 1.058050
(Iteration 2141 / 14700) loss: 0.908375
(Iteration 2151 / 14700) loss: 0.908540
(Iteration 2161 / 14700) loss: 0.966034
(Iteration 2171 / 14700) loss: 0.893986
(Iteration 2181 / 14700) loss: 0.910904
(Iteration 2191 / 14700) loss: 0.906180
(Iteration 2201 / 14700) loss: 0.903906
(Iteration 2211 / 14700) loss: 0.711740
```

```
(Iteration 2221 / 14700) loss: 0.809019
(Iteration 2231 / 14700) loss: 0.892202
(Iteration 2241 / 14700) loss: 0.878144
(Iteration 2251 / 14700) loss: 0.935628
(Iteration 2261 / 14700) loss: 0.753872
(Iteration 2271 / 14700) loss: 0.875416
(Iteration 2281 / 14700) loss: 0.833886
(Iteration 2291 / 14700) loss: 0.908807
(Iteration 2301 / 14700) loss: 0.669771
(Iteration 2311 / 14700) loss: 0.789455
(Iteration 2321 / 14700) loss: 1.004224
(Iteration 2331 / 14700) loss: 0.593373
(Iteration 2341 / 14700) loss: 0.912265
(Iteration 2351 / 14700) loss: 0.670444
(Iteration 2361 / 14700) loss: 0.697591
(Iteration 2371 / 14700) loss: 0.992905
(Iteration 2381 / 14700) loss: 0.813406
(Iteration 2391 / 14700) loss: 0.768390
(Iteration 2401 / 14700) loss: 0.834531
(Iteration 2411 / 14700) loss: 0.815986
(Iteration 2421 / 14700) loss: 0.732491
(Iteration 2431 / 14700) loss: 0.864999
(Iteration 2441 / 14700) loss: 0.915980
(Epoch 5 / 30) train acc: 0.697000; val_acc: 0.569000
(Iteration 2451 / 14700) loss: 0.756669
(Iteration 2461 / 14700) loss: 0.663386
(Iteration 2471 / 14700) loss: 0.932346
(Iteration 2481 / 14700) loss: 0.780230
(Iteration 2491 / 14700) loss: 0.865984
(Iteration 2501 / 14700) loss: 0.767009
(Iteration 2511 / 14700) loss: 0.811380
(Iteration 2521 / 14700) loss: 0.714657
(Iteration 2531 / 14700) loss: 0.744918
(Iteration 2541 / 14700) loss: 0.780507
(Iteration 2551 / 14700) loss: 0.659259
(Iteration 2561 / 14700) loss: 0.686216
(Iteration 2571 / 14700) loss: 0.806400
(Iteration 2581 / 14700) loss: 0.941127
(Iteration 2591 / 14700) loss: 0.676613
(Iteration 2601 / 14700) loss: 0.724707
(Iteration 2611 / 14700) loss: 0.783594
(Iteration 2621 / 14700) loss: 0.919849
(Iteration 2631 / 14700) loss: 0.827950
(Iteration 2641 / 14700) loss: 0.771104
(Iteration 2651 / 14700) loss: 0.724104
(Iteration 2661 / 14700) loss: 0.793585
(Iteration 2671 / 14700) loss: 0.723632
(Iteration 2681 / 14700) loss: 0.636014
```

```
(Iteration 2691 / 14700) loss: 0.665022
(Iteration 2701 / 14700) loss: 0.578952
(Iteration 2711 / 14700) loss: 1.044743
(Iteration 2721 / 14700) loss: 0.778946
(Iteration 2731 / 14700) loss: 0.947209
(Iteration 2741 / 14700) loss: 0.724314
(Iteration 2751 / 14700) loss: 0.680786
(Iteration 2761 / 14700) loss: 0.805694
(Iteration 2771 / 14700) loss: 0.840929
(Iteration 2781 / 14700) loss: 0.846572
(Iteration 2791 / 14700) loss: 0.829852
(Iteration 2801 / 14700) loss: 0.688643
(Iteration 2811 / 14700) loss: 0.887856
(Iteration 2821 / 14700) loss: 0.810349
(Iteration 2831 / 14700) loss: 0.538490
(Iteration 2841 / 14700) loss: 0.840515
(Iteration 2851 / 14700) loss: 0.760973
(Iteration 2861 / 14700) loss: 0.814534
(Iteration 2871 / 14700) loss: 0.720797
(Iteration 2881 / 14700) loss: 0.616588
(Iteration 2891 / 14700) loss: 0.832496
(Iteration 2901 / 14700) loss: 0.710195
(Iteration 2911 / 14700) loss: 1.113251
(Iteration 2921 / 14700) loss: 0.555265
(Iteration 2931 / 14700) loss: 0.860620
(Epoch 6 / 30) train acc: 0.758000; val_acc: 0.571000
(Iteration 2941 / 14700) loss: 0.651802
(Iteration 2951 / 14700) loss: 0.840869
(Iteration 2961 / 14700) loss: 0.705484
(Iteration 2971 / 14700) loss: 0.817351
(Iteration 2981 / 14700) loss: 0.784204
(Iteration 2991 / 14700) loss: 0.772034
(Iteration 3001 / 14700) loss: 0.780940
(Iteration 3011 / 14700) loss: 0.668441
(Iteration 3021 / 14700) loss: 0.535280
(Iteration 3031 / 14700) loss: 0.919603
(Iteration 3041 / 14700) loss: 0.600021
(Iteration 3051 / 14700) loss: 0.721451
(Iteration 3061 / 14700) loss: 0.867767
(Iteration 3071 / 14700) loss: 0.594860
(Iteration 3081 / 14700) loss: 0.596987
(Iteration 3091 / 14700) loss: 0.771056
(Iteration 3101 / 14700) loss: 0.736746
(Iteration 3111 / 14700) loss: 0.622035
(Iteration 3121 / 14700) loss: 0.652068
(Iteration 3131 / 14700) loss: 0.672431
(Iteration 3141 / 14700) loss: 0.911523
(Iteration 3151 / 14700) loss: 0.740509
```

```
(Iteration 3161 / 14700) loss: 0.701742
(Iteration 3171 / 14700) loss: 0.708727
(Iteration 3181 / 14700) loss: 0.784524
(Iteration 3191 / 14700) loss: 0.880379
(Iteration 3201 / 14700) loss: 0.812175
(Iteration 3211 / 14700) loss: 0.675641
(Iteration 3221 / 14700) loss: 0.531307
(Iteration 3231 / 14700) loss: 0.555963
(Iteration 3241 / 14700) loss: 0.616882
(Iteration 3251 / 14700) loss: 0.666266
(Iteration 3261 / 14700) loss: 0.882753
(Iteration 3271 / 14700) loss: 0.694347
(Iteration 3281 / 14700) loss: 0.702248
(Iteration 3291 / 14700) loss: 0.736705
(Iteration 3301 / 14700) loss: 0.678078
(Iteration 3311 / 14700) loss: 0.542903
(Iteration 3321 / 14700) loss: 0.761360
(Iteration 3331 / 14700) loss: 0.638098
(Iteration 3341 / 14700) loss: 0.629894
(Iteration 3351 / 14700) loss: 1.004912
(Iteration 3361 / 14700) loss: 0.633386
(Iteration 3371 / 14700) loss: 0.745661
(Iteration 3381 / 14700) loss: 0.641577
(Iteration 3391 / 14700) loss: 0.739541
(Iteration 3401 / 14700) loss: 0.722489
(Iteration 3411 / 14700) loss: 0.686502
(Iteration 3421 / 14700) loss: 0.690413
(Epoch 7 / 30) train acc: 0.745000; val_acc: 0.585000
(Iteration 3431 / 14700) loss: 0.524662
(Iteration 3441 / 14700) loss: 0.785863
(Iteration 3451 / 14700) loss: 0.631869
(Iteration 3461 / 14700) loss: 0.740930
(Iteration 3471 / 14700) loss: 0.782876
(Iteration 3481 / 14700) loss: 0.784919
(Iteration 3491 / 14700) loss: 0.890325
(Iteration 3501 / 14700) loss: 0.544522
(Iteration 3511 / 14700) loss: 0.620239
(Iteration 3521 / 14700) loss: 0.604652
(Iteration 3531 / 14700) loss: 0.578599
(Iteration 3541 / 14700) loss: 0.488193
(Iteration 3551 / 14700) loss: 0.671636
(Iteration 3561 / 14700) loss: 0.675797
(Iteration 3571 / 14700) loss: 0.740427
(Iteration 3581 / 14700) loss: 0.760254
(Iteration 3591 / 14700) loss: 0.669803
(Iteration 3601 / 14700) loss: 0.676332
(Iteration 3611 / 14700) loss: 0.589661
(Iteration 3621 / 14700) loss: 0.643803
```

```
(Iteration 3631 / 14700) loss: 0.940037
(Iteration 3641 / 14700) loss: 0.838767
(Iteration 3651 / 14700) loss: 0.960029
(Iteration 3661 / 14700) loss: 0.610572
(Iteration 3671 / 14700) loss: 0.599519
(Iteration 3681 / 14700) loss: 0.677478
(Iteration 3691 / 14700) loss: 0.666549
(Iteration 3701 / 14700) loss: 0.714423
(Iteration 3711 / 14700) loss: 0.698869
(Iteration 3721 / 14700) loss: 0.559950
(Iteration 3731 / 14700) loss: 0.561912
(Iteration 3741 / 14700) loss: 0.743490
(Iteration 3751 / 14700) loss: 0.643209
(Iteration 3761 / 14700) loss: 0.823705
(Iteration 3771 / 14700) loss: 0.491682
(Iteration 3781 / 14700) loss: 0.852080
(Iteration 3791 / 14700) loss: 0.706215
(Iteration 3801 / 14700) loss: 0.499424
(Iteration 3811 / 14700) loss: 0.613208
(Iteration 3821 / 14700) loss: 0.690716
(Iteration 3831 / 14700) loss: 0.538413
(Iteration 3841 / 14700) loss: 0.630590
(Iteration 3851 / 14700) loss: 0.665546
(Iteration 3861 / 14700) loss: 0.649251
(Iteration 3871 / 14700) loss: 0.651695
(Iteration 3881 / 14700) loss: 0.465013
(Iteration 3891 / 14700) loss: 0.644373
(Iteration 3901 / 14700) loss: 0.585005
(Iteration 3911 / 14700) loss: 0.693296
(Epoch 8 / 30) train acc: 0.782000; val_acc: 0.573000
(Iteration 3921 / 14700) loss: 0.477912
(Iteration 3931 / 14700) loss: 0.601632
(Iteration 3941 / 14700) loss: 0.523986
(Iteration 3951 / 14700) loss: 0.525518
(Iteration 3961 / 14700) loss: 0.707443
(Iteration 3971 / 14700) loss: 0.741883
(Iteration 3981 / 14700) loss: 0.736203
(Iteration 3991 / 14700) loss: 0.585331
(Iteration 4001 / 14700) loss: 0.592553
(Iteration 4011 / 14700) loss: 0.539580
(Iteration 4021 / 14700) loss: 0.495174
(Iteration 4031 / 14700) loss: 0.520957
(Iteration 4041 / 14700) loss: 0.604081
(Iteration 4051 / 14700) loss: 0.576261
(Iteration 4061 / 14700) loss: 0.602585
(Iteration 4071 / 14700) loss: 0.646109
(Iteration 4081 / 14700) loss: 0.546971
(Iteration 4091 / 14700) loss: 0.526898
```

```
(Iteration 4101 / 14700) loss: 0.540176
(Iteration 4111 / 14700) loss: 0.686306
(Iteration 4121 / 14700) loss: 0.666463
(Iteration 4131 / 14700) loss: 0.545865
(Iteration 4141 / 14700) loss: 0.882505
(Iteration 4151 / 14700) loss: 0.685644
(Iteration 4161 / 14700) loss: 0.749625
(Iteration 4171 / 14700) loss: 0.523290
(Iteration 4181 / 14700) loss: 0.679132
(Iteration 4191 / 14700) loss: 0.914168
(Iteration 4201 / 14700) loss: 0.497734
(Iteration 4211 / 14700) loss: 0.741810
(Iteration 4221 / 14700) loss: 0.859573
(Iteration 4231 / 14700) loss: 0.623680
(Iteration 4241 / 14700) loss: 0.574669
(Iteration 4251 / 14700) loss: 0.587238
(Iteration 4261 / 14700) loss: 0.655830
(Iteration 4271 / 14700) loss: 0.732040
(Iteration 4281 / 14700) loss: 0.641181
(Iteration 4291 / 14700) loss: 0.549843
(Iteration 4301 / 14700) loss: 0.494172
(Iteration 4311 / 14700) loss: 0.511233
(Iteration 4321 / 14700) loss: 0.557088
(Iteration 4331 / 14700) loss: 0.556531
(Iteration 4341 / 14700) loss: 0.642215
(Iteration 4351 / 14700) loss: 0.824666
(Iteration 4361 / 14700) loss: 0.867074
(Iteration 4371 / 14700) loss: 0.684668
(Iteration 4381 / 14700) loss: 0.500641
(Iteration 4391 / 14700) loss: 0.604659
(Iteration 4401 / 14700) loss: 0.683570
(Epoch 9 / 30) train acc: 0.822000; val_acc: 0.575000
(Iteration 4411 / 14700) loss: 0.521227
(Iteration 4421 / 14700) loss: 0.628972
(Iteration 4431 / 14700) loss: 0.527689
(Iteration 4441 / 14700) loss: 0.561042
(Iteration 4451 / 14700) loss: 0.524170
(Iteration 4461 / 14700) loss: 0.674333
(Iteration 4471 / 14700) loss: 0.485824
(Iteration 4481 / 14700) loss: 0.608499
(Iteration 4491 / 14700) loss: 0.689011
(Iteration 4501 / 14700) loss: 0.584950
(Iteration 4511 / 14700) loss: 0.662925
(Iteration 4521 / 14700) loss: 0.463002
(Iteration 4531 / 14700) loss: 0.615765
(Iteration 4541 / 14700) loss: 0.423147
(Iteration 4551 / 14700) loss: 0.520064
(Iteration 4561 / 14700) loss: 0.672765
```

```
(Iteration 4571 / 14700) loss: 0.559918
(Iteration 4581 / 14700) loss: 0.333931
(Iteration 4591 / 14700) loss: 0.505881
(Iteration 4601 / 14700) loss: 0.648110
(Iteration 4611 / 14700) loss: 0.514576
(Iteration 4621 / 14700) loss: 0.469643
(Iteration 4631 / 14700) loss: 0.436492
(Iteration 4641 / 14700) loss: 0.627079
(Iteration 4651 / 14700) loss: 0.720902
(Iteration 4661 / 14700) loss: 0.527482
(Iteration 4671 / 14700) loss: 0.369131
(Iteration 4681 / 14700) loss: 0.542751
(Iteration 4691 / 14700) loss: 0.513379
(Iteration 4701 / 14700) loss: 0.517835
(Iteration 4711 / 14700) loss: 0.564043
(Iteration 4721 / 14700) loss: 0.367064
(Iteration 4731 / 14700) loss: 0.446410
(Iteration 4741 / 14700) loss: 0.660872
(Iteration 4751 / 14700) loss: 0.656575
(Iteration 4761 / 14700) loss: 0.663519
(Iteration 4771 / 14700) loss: 0.424886
(Iteration 4781 / 14700) loss: 0.574145
(Iteration 4791 / 14700) loss: 0.464794
(Iteration 4801 / 14700) loss: 0.642533
(Iteration 4811 / 14700) loss: 0.528637
(Iteration 4821 / 14700) loss: 0.735449
(Iteration 4831 / 14700) loss: 0.418148
(Iteration 4841 / 14700) loss: 0.577079
(Iteration 4851 / 14700) loss: 0.567979
(Iteration 4861 / 14700) loss: 0.524597
(Iteration 4871 / 14700) loss: 0.614843
(Iteration 4881 / 14700) loss: 0.400274
(Iteration 4891 / 14700) loss: 0.374551
(Epoch 10 / 30) train acc: 0.809000; val_acc: 0.560000
(Iteration 4901 / 14700) loss: 0.574588
(Iteration 4911 / 14700) loss: 0.335369
(Iteration 4921 / 14700) loss: 0.867034
(Iteration 4931 / 14700) loss: 0.517482
(Iteration 4941 / 14700) loss: 0.472803
(Iteration 4951 / 14700) loss: 0.413159
(Iteration 4961 / 14700) loss: 0.677150
(Iteration 4971 / 14700) loss: 0.517607
(Iteration 4981 / 14700) loss: 0.433494
(Iteration 4991 / 14700) loss: 0.431131
(Iteration 5001 / 14700) loss: 0.454309
(Iteration 5011 / 14700) loss: 0.461485
(Iteration 5021 / 14700) loss: 0.475803
(Iteration 5031 / 14700) loss: 0.508394
```

```
(Iteration 5041 / 14700) loss: 0.432755
(Iteration 5051 / 14700) loss: 0.694796
(Iteration 5061 / 14700) loss: 0.608006
(Iteration 5071 / 14700) loss: 0.349118
(Iteration 5081 / 14700) loss: 0.436616
(Iteration 5091 / 14700) loss: 0.384545
(Iteration 5101 / 14700) loss: 0.585312
(Iteration 5111 / 14700) loss: 0.411239
(Iteration 5121 / 14700) loss: 0.605151
(Iteration 5131 / 14700) loss: 0.426866
(Iteration 5141 / 14700) loss: 0.386987
(Iteration 5151 / 14700) loss: 0.489114
(Iteration 5161 / 14700) loss: 0.580559
(Iteration 5171 / 14700) loss: 0.570586
(Iteration 5181 / 14700) loss: 0.448969
(Iteration 5191 / 14700) loss: 0.336872
(Iteration 5201 / 14700) loss: 0.414330
(Iteration 5211 / 14700) loss: 0.480411
(Iteration 5221 / 14700) loss: 0.493546
(Iteration 5231 / 14700) loss: 0.694967
(Iteration 5241 / 14700) loss: 0.291731
(Iteration 5251 / 14700) loss: 0.497431
(Iteration 5261 / 14700) loss: 0.511891
(Iteration 5271 / 14700) loss: 0.496696
(Iteration 5281 / 14700) loss: 0.422241
(Iteration 5291 / 14700) loss: 0.465761
(Iteration 5301 / 14700) loss: 0.484390
(Iteration 5311 / 14700) loss: 0.551721
(Iteration 5321 / 14700) loss: 0.578451
(Iteration 5331 / 14700) loss: 0.527864
(Iteration 5341 / 14700) loss: 0.418109
(Iteration 5351 / 14700) loss: 0.529991
(Iteration 5361 / 14700) loss: 0.319754
(Iteration 5371 / 14700) loss: 0.608649
(Iteration 5381 / 14700) loss: 0.546101
(Epoch 11 / 30) train acc: 0.815000; val_acc: 0.572000
(Iteration 5391 / 14700) loss: 0.623359
(Iteration 5401 / 14700) loss: 0.473433
(Iteration 5411 / 14700) loss: 0.447920
(Iteration 5421 / 14700) loss: 0.466170
(Iteration 5431 / 14700) loss: 0.408723
(Iteration 5441 / 14700) loss: 0.500138
(Iteration 5451 / 14700) loss: 0.353956
(Iteration 5461 / 14700) loss: 0.473960
(Iteration 5471 / 14700) loss: 0.514352
(Iteration 5481 / 14700) loss: 0.414962
(Iteration 5491 / 14700) loss: 0.319244
(Iteration 5501 / 14700) loss: 0.435129
```

```
(Iteration 5511 / 14700) loss: 0.448779
(Iteration 5521 / 14700) loss: 0.320396
(Iteration 5531 / 14700) loss: 0.579040
(Iteration 5541 / 14700) loss: 0.350169
(Iteration 5551 / 14700) loss: 0.408272
(Iteration 5561 / 14700) loss: 0.396385
(Iteration 5571 / 14700) loss: 0.531421
(Iteration 5581 / 14700) loss: 0.375085
(Iteration 5591 / 14700) loss: 0.415381
(Iteration 5601 / 14700) loss: 0.528701
(Iteration 5611 / 14700) loss: 0.303390
(Iteration 5621 / 14700) loss: 0.469099
(Iteration 5631 / 14700) loss: 0.461344
(Iteration 5641 / 14700) loss: 0.431233
(Iteration 5651 / 14700) loss: 0.312060
(Iteration 5661 / 14700) loss: 0.552324
(Iteration 5671 / 14700) loss: 0.512439
(Iteration 5681 / 14700) loss: 0.502057
(Iteration 5691 / 14700) loss: 0.551623
(Iteration 5701 / 14700) loss: 0.300941
(Iteration 5711 / 14700) loss: 0.479539
(Iteration 5721 / 14700) loss: 0.725256
(Iteration 5731 / 14700) loss: 0.531199
(Iteration 5741 / 14700) loss: 0.578896
(Iteration 5751 / 14700) loss: 0.430728
(Iteration 5761 / 14700) loss: 0.487212
(Iteration 5771 / 14700) loss: 0.440534
(Iteration 5781 / 14700) loss: 0.379172
(Iteration 5791 / 14700) loss: 0.512342
(Iteration 5801 / 14700) loss: 0.548980
(Iteration 5811 / 14700) loss: 0.439806
(Iteration 5821 / 14700) loss: 0.360862
(Iteration 5831 / 14700) loss: 0.369463
(Iteration 5841 / 14700) loss: 0.472875
(Iteration 5851 / 14700) loss: 0.299951
(Iteration 5861 / 14700) loss: 0.483827
(Iteration 5871 / 14700) loss: 0.621435
(Epoch 12 / 30) train acc: 0.856000; val_acc: 0.566000
(Iteration 5881 / 14700) loss: 0.341391
(Iteration 5891 / 14700) loss: 0.420787
(Iteration 5901 / 14700) loss: 0.264878
(Iteration 5911 / 14700) loss: 0.487713
(Iteration 5921 / 14700) loss: 0.376205
(Iteration 5931 / 14700) loss: 0.301772
(Iteration 5941 / 14700) loss: 0.346270
(Iteration 5951 / 14700) loss: 0.438831
(Iteration 5961 / 14700) loss: 0.349146
(Iteration 5971 / 14700) loss: 0.329130
```

```
(Iteration 5981 / 14700) loss: 0.411963
(Iteration 5991 / 14700) loss: 0.334802
(Iteration 6001 / 14700) loss: 0.294469
(Iteration 6011 / 14700) loss: 0.489939
(Iteration 6021 / 14700) loss: 0.380426
(Iteration 6031 / 14700) loss: 0.326587
(Iteration 6041 / 14700) loss: 0.524139
(Iteration 6051 / 14700) loss: 0.613982
(Iteration 6061 / 14700) loss: 0.578678
(Iteration 6071 / 14700) loss: 0.407683
(Iteration 6081 / 14700) loss: 0.365537
(Iteration 6091 / 14700) loss: 0.339830
(Iteration 6101 / 14700) loss: 0.297380
(Iteration 6111 / 14700) loss: 0.395456
(Iteration 6121 / 14700) loss: 0.401882
(Iteration 6131 / 14700) loss: 0.457854
(Iteration 6141 / 14700) loss: 0.341395
(Iteration 6151 / 14700) loss: 0.339291
(Iteration 6161 / 14700) loss: 0.394320
(Iteration 6171 / 14700) loss: 0.425911
(Iteration 6181 / 14700) loss: 0.383186
(Iteration 6191 / 14700) loss: 0.333937
(Iteration 6201 / 14700) loss: 0.307128
(Iteration 6211 / 14700) loss: 0.369094
(Iteration 6221 / 14700) loss: 0.319158
(Iteration 6231 / 14700) loss: 0.470901
(Iteration 6241 / 14700) loss: 0.411234
(Iteration 6251 / 14700) loss: 0.355634
(Iteration 6261 / 14700) loss: 0.357994
(Iteration 6271 / 14700) loss: 0.281456
(Iteration 6281 / 14700) loss: 0.371639
(Iteration 6291 / 14700) loss: 0.278819
(Iteration 6301 / 14700) loss: 0.345630
(Iteration 6311 / 14700) loss: 0.353486
(Iteration 6321 / 14700) loss: 0.324498
(Iteration 6331 / 14700) loss: 0.543055
(Iteration 6341 / 14700) loss: 0.354368
(Iteration 6351 / 14700) loss: 0.320831
(Iteration 6361 / 14700) loss: 0.406450
(Epoch 13 / 30) train acc: 0.895000; val_acc: 0.576000
(Iteration 6371 / 14700) loss: 0.667738
(Iteration 6381 / 14700) loss: 0.258638
(Iteration 6391 / 14700) loss: 0.321078
(Iteration 6401 / 14700) loss: 0.356141
(Iteration 6411 / 14700) loss: 0.258385
(Iteration 6421 / 14700) loss: 0.249937
(Iteration 6431 / 14700) loss: 0.344331
(Iteration 6441 / 14700) loss: 0.323983
```

```
(Iteration 6451 / 14700) loss: 0.484758
(Iteration 6461 / 14700) loss: 0.385727
(Iteration 6471 / 14700) loss: 0.338714
(Iteration 6481 / 14700) loss: 0.285618
(Iteration 6491 / 14700) loss: 0.344827
(Iteration 6501 / 14700) loss: 0.239338
(Iteration 6511 / 14700) loss: 0.367320
(Iteration 6521 / 14700) loss: 0.241176
(Iteration 6531 / 14700) loss: 0.286634
(Iteration 6541 / 14700) loss: 0.461014
(Iteration 6551 / 14700) loss: 0.369594
(Iteration 6561 / 14700) loss: 0.279840
(Iteration 6571 / 14700) loss: 0.475825
(Iteration 6581 / 14700) loss: 0.268567
(Iteration 6591 / 14700) loss: 0.351698
(Iteration 6601 / 14700) loss: 0.424293
(Iteration 6611 / 14700) loss: 0.557787
(Iteration 6621 / 14700) loss: 0.277109
(Iteration 6631 / 14700) loss: 0.370157
(Iteration 6641 / 14700) loss: 0.363205
(Iteration 6651 / 14700) loss: 0.298150
(Iteration 6661 / 14700) loss: 0.358282
(Iteration 6671 / 14700) loss: 0.315515
(Iteration 6681 / 14700) loss: 0.328901
(Iteration 6691 / 14700) loss: 0.361480
(Iteration 6701 / 14700) loss: 0.522958
(Iteration 6711 / 14700) loss: 0.257171
(Iteration 6721 / 14700) loss: 0.249497
(Iteration 6731 / 14700) loss: 0.171362
(Iteration 6741 / 14700) loss: 0.375555
(Iteration 6751 / 14700) loss: 0.378229
(Iteration 6761 / 14700) loss: 0.398290
(Iteration 6771 / 14700) loss: 0.261547
(Iteration 6781 / 14700) loss: 0.323671
(Iteration 6791 / 14700) loss: 0.267433
(Iteration 6801 / 14700) loss: 0.347214
(Iteration 6811 / 14700) loss: 0.242550
(Iteration 6821 / 14700) loss: 0.346367
(Iteration 6831 / 14700) loss: 0.363337
(Iteration 6841 / 14700) loss: 0.180530
(Iteration 6851 / 14700) loss: 0.324555
(Epoch 14 / 30) train acc: 0.904000; val_acc: 0.562000
(Iteration 6861 / 14700) loss: 0.332569
(Iteration 6871 / 14700) loss: 0.176450
(Iteration 6881 / 14700) loss: 0.293175
(Iteration 6891 / 14700) loss: 0.359168
(Iteration 6901 / 14700) loss: 0.290541
(Iteration 6911 / 14700) loss: 0.309970
```

```
(Iteration 6921 / 14700) loss: 0.214931
(Iteration 6931 / 14700) loss: 0.370114
(Iteration 6941 / 14700) loss: 0.297776
(Iteration 6951 / 14700) loss: 0.253330
(Iteration 6961 / 14700) loss: 0.212731
(Iteration 6971 / 14700) loss: 0.474160
(Iteration 6981 / 14700) loss: 0.263170
(Iteration 6991 / 14700) loss: 0.246795
(Iteration 7001 / 14700) loss: 0.340874
(Iteration 7011 / 14700) loss: 0.397571
(Iteration 7021 / 14700) loss: 0.381559
(Iteration 7031 / 14700) loss: 0.263977
(Iteration 7041 / 14700) loss: 0.489863
(Iteration 7051 / 14700) loss: 0.240156
(Iteration 7061 / 14700) loss: 0.318450
(Iteration 7071 / 14700) loss: 0.273545
(Iteration 7081 / 14700) loss: 0.426734
(Iteration 7091 / 14700) loss: 0.245226
(Iteration 7101 / 14700) loss: 0.324919
(Iteration 7111 / 14700) loss: 0.406265
(Iteration 7121 / 14700) loss: 0.320312
(Iteration 7131 / 14700) loss: 0.243625
(Iteration 7141 / 14700) loss: 0.343824
(Iteration 7151 / 14700) loss: 0.266563
(Iteration 7161 / 14700) loss: 0.321306
(Iteration 7171 / 14700) loss: 0.294242
(Iteration 7181 / 14700) loss: 0.360289
(Iteration 7191 / 14700) loss: 0.255152
(Iteration 7201 / 14700) loss: 0.153115
(Iteration 7211 / 14700) loss: 0.276593
(Iteration 7221 / 14700) loss: 0.219323
(Iteration 7231 / 14700) loss: 0.288001
(Iteration 7241 / 14700) loss: 0.214850
(Iteration 7251 / 14700) loss: 0.317215
(Iteration 7261 / 14700) loss: 0.404959
(Iteration 7271 / 14700) loss: 0.299459
(Iteration 7281 / 14700) loss: 0.347761
(Iteration 7291 / 14700) loss: 0.194621
(Iteration 7301 / 14700) loss: 0.223186
(Iteration 7311 / 14700) loss: 0.250812
(Iteration 7321 / 14700) loss: 0.283557
(Iteration 7331 / 14700) loss: 0.169933
(Iteration 7341 / 14700) loss: 0.292815
(Epoch 15 / 30) train acc: 0.908000; val_acc: 0.550000
(Iteration 7351 / 14700) loss: 0.139001
(Iteration 7361 / 14700) loss: 0.224838
(Iteration 7371 / 14700) loss: 0.175181
(Iteration 7381 / 14700) loss: 0.356305
```

```
(Iteration 7391 / 14700) loss: 0.313040
(Iteration 7401 / 14700) loss: 0.376706
(Iteration 7411 / 14700) loss: 0.230402
(Iteration 7421 / 14700) loss: 0.167006
(Iteration 7431 / 14700) loss: 0.306297
(Iteration 7441 / 14700) loss: 0.236330
(Iteration 7451 / 14700) loss: 0.230277
(Iteration 7461 / 14700) loss: 0.376552
(Iteration 7471 / 14700) loss: 0.238703
(Iteration 7481 / 14700) loss: 0.156043
(Iteration 7491 / 14700) loss: 0.260075
(Iteration 7501 / 14700) loss: 0.176002
(Iteration 7511 / 14700) loss: 0.352072
(Iteration 7521 / 14700) loss: 0.244323
(Iteration 7531 / 14700) loss: 0.201079
(Iteration 7541 / 14700) loss: 0.192795
(Iteration 7551 / 14700) loss: 0.227069
(Iteration 7561 / 14700) loss: 0.240486
(Iteration 7571 / 14700) loss: 0.277298
(Iteration 7581 / 14700) loss: 0.264238
(Iteration 7591 / 14700) loss: 0.189572
(Iteration 7601 / 14700) loss: 0.321411
(Iteration 7611 / 14700) loss: 0.190209
(Iteration 7621 / 14700) loss: 0.245850
(Iteration 7631 / 14700) loss: 0.283312
(Iteration 7641 / 14700) loss: 0.199125
(Iteration 7651 / 14700) loss: 0.348194
(Iteration 7661 / 14700) loss: 0.336412
(Iteration 7671 / 14700) loss: 0.272760
(Iteration 7681 / 14700) loss: 0.310144
(Iteration 7691 / 14700) loss: 0.230336
(Iteration 7701 / 14700) loss: 0.221970
(Iteration 7711 / 14700) loss: 0.201706
(Iteration 7721 / 14700) loss: 0.283474
(Iteration 7731 / 14700) loss: 0.215739
(Iteration 7741 / 14700) loss: 0.361887
(Iteration 7751 / 14700) loss: 0.282674
(Iteration 7761 / 14700) loss: 0.208696
(Iteration 7771 / 14700) loss: 0.371515
(Iteration 7781 / 14700) loss: 0.235493
(Iteration 7791 / 14700) loss: 0.323167
(Iteration 7801 / 14700) loss: 0.319495
(Iteration 7811 / 14700) loss: 0.164694
(Iteration 7821 / 14700) loss: 0.342282
(Iteration 7831 / 14700) loss: 0.197196
(Epoch 16 / 30) train acc: 0.921000; val_acc: 0.566000
(Iteration 7841 / 14700) loss: 0.235720
(Iteration 7851 / 14700) loss: 0.244949
```

```
(Iteration 7861 / 14700) loss: 0.268042
(Iteration 7871 / 14700) loss: 0.251065
(Iteration 7881 / 14700) loss: 0.195380
(Iteration 7891 / 14700) loss: 0.245164
(Iteration 7901 / 14700) loss: 0.184139
(Iteration 7911 / 14700) loss: 0.210270
(Iteration 7921 / 14700) loss: 0.160959
(Iteration 7931 / 14700) loss: 0.156804
(Iteration 7941 / 14700) loss: 0.180789
(Iteration 7951 / 14700) loss: 0.360593
(Iteration 7961 / 14700) loss: 0.203624
(Iteration 7971 / 14700) loss: 0.356208
(Iteration 7981 / 14700) loss: 0.223601
(Iteration 7991 / 14700) loss: 0.207081
(Iteration 8001 / 14700) loss: 0.179879
(Iteration 8011 / 14700) loss: 0.353422
(Iteration 8021 / 14700) loss: 0.283780
(Iteration 8031 / 14700) loss: 0.250447
(Iteration 8041 / 14700) loss: 0.197983
(Iteration 8051 / 14700) loss: 0.234677
(Iteration 8061 / 14700) loss: 0.175192
(Iteration 8071 / 14700) loss: 0.230485
(Iteration 8081 / 14700) loss: 0.318113
(Iteration 8091 / 14700) loss: 0.387325
(Iteration 8101 / 14700) loss: 0.222812
(Iteration 8111 / 14700) loss: 0.232210
(Iteration 8121 / 14700) loss: 0.200321
(Iteration 8131 / 14700) loss: 0.263046
(Iteration 8141 / 14700) loss: 0.245393
(Iteration 8151 / 14700) loss: 0.157314
(Iteration 8161 / 14700) loss: 0.292346
(Iteration 8171 / 14700) loss: 0.224462
(Iteration 8181 / 14700) loss: 0.124178
(Iteration 8191 / 14700) loss: 0.191569
(Iteration 8201 / 14700) loss: 0.172047
(Iteration 8211 / 14700) loss: 0.217418
(Iteration 8221 / 14700) loss: 0.278168
(Iteration 8231 / 14700) loss: 0.302707
(Iteration 8241 / 14700) loss: 0.288737
(Iteration 8251 / 14700) loss: 0.183238
(Iteration 8261 / 14700) loss: 0.422976
(Iteration 8271 / 14700) loss: 0.253920
(Iteration 8281 / 14700) loss: 0.165394
(Iteration 8291 / 14700) loss: 0.259634
(Iteration 8301 / 14700) loss: 0.196775
(Iteration 8311 / 14700) loss: 0.315207
(Iteration 8321 / 14700) loss: 0.184278
(Epoch 17 / 30) train acc: 0.928000; val_acc: 0.566000
```

```
(Iteration 8331 / 14700) loss: 0.145015
(Iteration 8341 / 14700) loss: 0.182130
(Iteration 8351 / 14700) loss: 0.230732
(Iteration 8361 / 14700) loss: 0.256672
(Iteration 8371 / 14700) loss: 0.133300
(Iteration 8381 / 14700) loss: 0.134219
(Iteration 8391 / 14700) loss: 0.274478
(Iteration 8401 / 14700) loss: 0.274027
(Iteration 8411 / 14700) loss: 0.145673
(Iteration 8421 / 14700) loss: 0.150122
(Iteration 8431 / 14700) loss: 0.257181
(Iteration 8441 / 14700) loss: 0.220058
(Iteration 8451 / 14700) loss: 0.186479
(Iteration 8461 / 14700) loss: 0.155959
(Iteration 8471 / 14700) loss: 0.099536
(Iteration 8481 / 14700) loss: 0.103574
(Iteration 8491 / 14700) loss: 0.189109
(Iteration 8501 / 14700) loss: 0.156333
(Iteration 8511 / 14700) loss: 0.216532
(Iteration 8521 / 14700) loss: 0.143206
(Iteration 8531 / 14700) loss: 0.261746
(Iteration 8541 / 14700) loss: 0.166118
(Iteration 8551 / 14700) loss: 0.339323
(Iteration 8561 / 14700) loss: 0.144890
(Iteration 8571 / 14700) loss: 0.193450
(Iteration 8581 / 14700) loss: 0.213496
(Iteration 8591 / 14700) loss: 0.155733
(Iteration 8601 / 14700) loss: 0.159869
(Iteration 8611 / 14700) loss: 0.167555
(Iteration 8621 / 14700) loss: 0.093280
(Iteration 8631 / 14700) loss: 0.193536
(Iteration 8641 / 14700) loss: 0.146508
(Iteration 8651 / 14700) loss: 0.131705
(Iteration 8661 / 14700) loss: 0.237365
(Iteration 8671 / 14700) loss: 0.197766
(Iteration 8681 / 14700) loss: 0.132788
(Iteration 8691 / 14700) loss: 0.183855
(Iteration 8701 / 14700) loss: 0.195308
(Iteration 8711 / 14700) loss: 0.182043
(Iteration 8721 / 14700) loss: 0.155624
(Iteration 8731 / 14700) loss: 0.200522
(Iteration 8741 / 14700) loss: 0.158542
(Iteration 8751 / 14700) loss: 0.111006
(Iteration 8761 / 14700) loss: 0.194211
(Iteration 8771 / 14700) loss: 0.187680
(Iteration 8781 / 14700) loss: 0.185243
(Iteration 8791 / 14700) loss: 0.220907
(Iteration 8801 / 14700) loss: 0.154338
```

```
(Iteration 8811 / 14700) loss: 0.163521
(Epoch 18 / 30) train acc: 0.944000; val_acc: 0.563000
(Iteration 8821 / 14700) loss: 0.147011
(Iteration 8831 / 14700) loss: 0.128343
(Iteration 8841 / 14700) loss: 0.149300
(Iteration 8851 / 14700) loss: 0.179777
(Iteration 8861 / 14700) loss: 0.224138
(Iteration 8871 / 14700) loss: 0.177557
(Iteration 8881 / 14700) loss: 0.162741
(Iteration 8891 / 14700) loss: 0.216167
(Iteration 8901 / 14700) loss: 0.208823
(Iteration 8911 / 14700) loss: 0.132408
(Iteration 8921 / 14700) loss: 0.194125
(Iteration 8931 / 14700) loss: 0.113338
(Iteration 8941 / 14700) loss: 0.162449
(Iteration 8951 / 14700) loss: 0.113612
(Iteration 8961 / 14700) loss: 0.212921
(Iteration 8971 / 14700) loss: 0.251395
(Iteration 8981 / 14700) loss: 0.160957
(Iteration 8991 / 14700) loss: 0.195451
(Iteration 9001 / 14700) loss: 0.189125
(Iteration 9011 / 14700) loss: 0.134609
(Iteration 9021 / 14700) loss: 0.169360
(Iteration 9031 / 14700) loss: 0.210460
(Iteration 9041 / 14700) loss: 0.220882
(Iteration 9051 / 14700) loss: 0.145337
(Iteration 9061 / 14700) loss: 0.143299
(Iteration 9071 / 14700) loss: 0.172224
(Iteration 9081 / 14700) loss: 0.140059
(Iteration 9091 / 14700) loss: 0.087726
(Iteration 9101 / 14700) loss: 0.141797
(Iteration 9111 / 14700) loss: 0.213578
(Iteration 9121 / 14700) loss: 0.220991
(Iteration 9131 / 14700) loss: 0.157921
(Iteration 9141 / 14700) loss: 0.168954
(Iteration 9151 / 14700) loss: 0.147687
(Iteration 9161 / 14700) loss: 0.090097
(Iteration 9171 / 14700) loss: 0.142495
(Iteration 9181 / 14700) loss: 0.119287
(Iteration 9191 / 14700) loss: 0.226189
(Iteration 9201 / 14700) loss: 0.193379
(Iteration 9211 / 14700) loss: 0.185095
(Iteration 9221 / 14700) loss: 0.127775
(Iteration 9231 / 14700) loss: 0.157817
(Iteration 9241 / 14700) loss: 0.130329
(Iteration 9251 / 14700) loss: 0.162174
(Iteration 9261 / 14700) loss: 0.158053
(Iteration 9271 / 14700) loss: 0.148418
```

```
(Iteration 9281 / 14700) loss: 0.081086
(Iteration 9291 / 14700) loss: 0.284763
(Iteration 9301 / 14700) loss: 0.159754
(Epoch 19 / 30) train acc: 0.963000; val_acc: 0.564000
(Iteration 9311 / 14700) loss: 0.145170
(Iteration 9321 / 14700) loss: 0.122668
(Iteration 9331 / 14700) loss: 0.137121
(Iteration 9341 / 14700) loss: 0.118073
(Iteration 9351 / 14700) loss: 0.197655
(Iteration 9361 / 14700) loss: 0.128495
(Iteration 9371 / 14700) loss: 0.151566
(Iteration 9381 / 14700) loss: 0.136599
(Iteration 9391 / 14700) loss: 0.192362
(Iteration 9401 / 14700) loss: 0.158613
(Iteration 9411 / 14700) loss: 0.172838
(Iteration 9421 / 14700) loss: 0.073498
(Iteration 9431 / 14700) loss: 0.100646
(Iteration 9441 / 14700) loss: 0.183859
(Iteration 9451 / 14700) loss: 0.104088
(Iteration 9461 / 14700) loss: 0.105533
(Iteration 9471 / 14700) loss: 0.182027
(Iteration 9481 / 14700) loss: 0.074825
(Iteration 9491 / 14700) loss: 0.104527
(Iteration 9501 / 14700) loss: 0.151222
(Iteration 9511 / 14700) loss: 0.179910
(Iteration 9521 / 14700) loss: 0.168425
(Iteration 9531 / 14700) loss: 0.076117
(Iteration 9541 / 14700) loss: 0.214200
(Iteration 9551 / 14700) loss: 0.059231
(Iteration 9561 / 14700) loss: 0.180812
(Iteration 9571 / 14700) loss: 0.174980
(Iteration 9581 / 14700) loss: 0.111922
(Iteration 9591 / 14700) loss: 0.166160
(Iteration 9601 / 14700) loss: 0.178607
(Iteration 9611 / 14700) loss: 0.178460
(Iteration 9621 / 14700) loss: 0.096972
(Iteration 9631 / 14700) loss: 0.175329
(Iteration 9641 / 14700) loss: 0.157153
(Iteration 9651 / 14700) loss: 0.101140
(Iteration 9661 / 14700) loss: 0.081852
(Iteration 9671 / 14700) loss: 0.046910
(Iteration 9681 / 14700) loss: 0.160243
(Iteration 9691 / 14700) loss: 0.083028
(Iteration 9701 / 14700) loss: 0.120027
(Iteration 9711 / 14700) loss: 0.117425
(Iteration 9721 / 14700) loss: 0.093212
(Iteration 9731 / 14700) loss: 0.199712
(Iteration 9741 / 14700) loss: 0.118880
```

```
(Iteration 9751 / 14700) loss: 0.176777
(Iteration 9761 / 14700) loss: 0.127130
(Iteration 9771 / 14700) loss: 0.147741
(Iteration 9781 / 14700) loss: 0.088599
(Iteration 9791 / 14700) loss: 0.194121
(Epoch 20 / 30) train acc: 0.965000; val_acc: 0.572000
(Iteration 9801 / 14700) loss: 0.153114
(Iteration 9811 / 14700) loss: 0.090793
(Iteration 9821 / 14700) loss: 0.123244
(Iteration 9831 / 14700) loss: 0.138850
(Iteration 9841 / 14700) loss: 0.151612
(Iteration 9851 / 14700) loss: 0.176347
(Iteration 9861 / 14700) loss: 0.145616
(Iteration 9871 / 14700) loss: 0.051699
(Iteration 9881 / 14700) loss: 0.143635
(Iteration 9891 / 14700) loss: 0.172104
(Iteration 9901 / 14700) loss: 0.060983
(Iteration 9911 / 14700) loss: 0.146540
(Iteration 9921 / 14700) loss: 0.101796
(Iteration 9931 / 14700) loss: 0.077939
(Iteration 9941 / 14700) loss: 0.118588
(Iteration 9951 / 14700) loss: 0.140442
(Iteration 9961 / 14700) loss: 0.162246
(Iteration 9971 / 14700) loss: 0.115547
(Iteration 9981 / 14700) loss: 0.085182
(Iteration 9991 / 14700) loss: 0.113506
(Iteration 10001 / 14700) loss: 0.109744
(Iteration 10011 / 14700) loss: 0.091018
(Iteration 10021 / 14700) loss: 0.138156
(Iteration 10031 / 14700) loss: 0.102824
(Iteration 10041 / 14700) loss: 0.138496
(Iteration 10051 / 14700) loss: 0.128971
(Iteration 10061 / 14700) loss: 0.127249
(Iteration 10071 / 14700) loss: 0.077982
(Iteration 10081 / 14700) loss: 0.104152
(Iteration 10091 / 14700) loss: 0.104327
(Iteration 10101 / 14700) loss: 0.101754
(Iteration 10111 / 14700) loss: 0.129170
(Iteration 10121 / 14700) loss: 0.139564
(Iteration 10131 / 14700) loss: 0.123846
(Iteration 10141 / 14700) loss: 0.120953
(Iteration 10151 / 14700) loss: 0.075979
(Iteration 10161 / 14700) loss: 0.119281
(Iteration 10171 / 14700) loss: 0.090205
(Iteration 10181 / 14700) loss: 0.101599
(Iteration 10191 / 14700) loss: 0.125227
(Iteration 10201 / 14700) loss: 0.156445
(Iteration 10211 / 14700) loss: 0.079485
```

```
(Iteration 10221 / 14700) loss: 0.134862
(Iteration 10231 / 14700) loss: 0.109786
(Iteration 10241 / 14700) loss: 0.079648
(Iteration 10251 / 14700) loss: 0.136330
(Iteration 10261 / 14700) loss: 0.116372
(Iteration 10271 / 14700) loss: 0.098039
(Iteration 10281 / 14700) loss: 0.097801
(Epoch 21 / 30) train acc: 0.976000; val_acc: 0.562000
(Iteration 10291 / 14700) loss: 0.070705
(Iteration 10301 / 14700) loss: 0.133252
(Iteration 10311 / 14700) loss: 0.100733
(Iteration 10321 / 14700) loss: 0.144695
(Iteration 10331 / 14700) loss: 0.141205
(Iteration 10341 / 14700) loss: 0.164333
(Iteration 10351 / 14700) loss: 0.097198
(Iteration 10361 / 14700) loss: 0.079208
(Iteration 10371 / 14700) loss: 0.079870
(Iteration 10381 / 14700) loss: 0.136419
(Iteration 10391 / 14700) loss: 0.076105
(Iteration 10401 / 14700) loss: 0.114929
(Iteration 10411 / 14700) loss: 0.181129
(Iteration 10421 / 14700) loss: 0.103771
(Iteration 10431 / 14700) loss: 0.073739
(Iteration 10441 / 14700) loss: 0.091309
(Iteration 10451 / 14700) loss: 0.153469
(Iteration 10461 / 14700) loss: 0.085623
(Iteration 10471 / 14700) loss: 0.127199
(Iteration 10481 / 14700) loss: 0.070588
(Iteration 10491 / 14700) loss: 0.068149
(Iteration 10501 / 14700) loss: 0.107520
(Iteration 10511 / 14700) loss: 0.069068
(Iteration 10521 / 14700) loss: 0.116045
(Iteration 10531 / 14700) loss: 0.075541
(Iteration 10541 / 14700) loss: 0.113673
(Iteration 10551 / 14700) loss: 0.083890
(Iteration 10561 / 14700) loss: 0.059396
(Iteration 10571 / 14700) loss: 0.122241
(Iteration 10581 / 14700) loss: 0.126481
(Iteration 10591 / 14700) loss: 0.167931
(Iteration 10601 / 14700) loss: 0.109166
(Iteration 10611 / 14700) loss: 0.080530
(Iteration 10621 / 14700) loss: 0.086561
(Iteration 10631 / 14700) loss: 0.073522
(Iteration 10641 / 14700) loss: 0.086300
(Iteration 10651 / 14700) loss: 0.095091
(Iteration 10661 / 14700) loss: 0.093093
(Iteration 10671 / 14700) loss: 0.151062
(Iteration 10681 / 14700) loss: 0.083440
```

```
(Iteration 10691 / 14700) loss: 0.064612
(Iteration 10701 / 14700) loss: 0.133894
(Iteration 10711 / 14700) loss: 0.119583
(Iteration 10721 / 14700) loss: 0.098601
(Iteration 10731 / 14700) loss: 0.059746
(Iteration 10741 / 14700) loss: 0.114376
(Iteration 10751 / 14700) loss: 0.080039
(Iteration 10761 / 14700) loss: 0.048101
(Iteration 10771 / 14700) loss: 0.058194
(Epoch 22 / 30) train acc: 0.980000; val_acc: 0.573000
(Iteration 10781 / 14700) loss: 0.154342
(Iteration 10791 / 14700) loss: 0.116256
(Iteration 10801 / 14700) loss: 0.074032
(Iteration 10811 / 14700) loss: 0.065098
(Iteration 10821 / 14700) loss: 0.093092
(Iteration 10831 / 14700) loss: 0.060582
(Iteration 10841 / 14700) loss: 0.087670
(Iteration 10851 / 14700) loss: 0.078726
(Iteration 10861 / 14700) loss: 0.106988
(Iteration 10871 / 14700) loss: 0.076217
(Iteration 10881 / 14700) loss: 0.089683
(Iteration 10891 / 14700) loss: 0.086029
(Iteration 10901 / 14700) loss: 0.077604
(Iteration 10911 / 14700) loss: 0.074470
(Iteration 10921 / 14700) loss: 0.069209
(Iteration 10931 / 14700) loss: 0.117279
(Iteration 10941 / 14700) loss: 0.109849
(Iteration 10951 / 14700) loss: 0.095527
(Iteration 10961 / 14700) loss: 0.088294
(Iteration 10971 / 14700) loss: 0.048235
(Iteration 10981 / 14700) loss: 0.093841
(Iteration 10991 / 14700) loss: 0.097759
(Iteration 11001 / 14700) loss: 0.055092
(Iteration 11011 / 14700) loss: 0.113081
(Iteration 11021 / 14700) loss: 0.059964
(Iteration 11031 / 14700) loss: 0.090937
(Iteration 11041 / 14700) loss: 0.109800
(Iteration 11051 / 14700) loss: 0.054401
(Iteration 11061 / 14700) loss: 0.059904
(Iteration 11071 / 14700) loss: 0.120614
(Iteration 11081 / 14700) loss: 0.074402
(Iteration 11091 / 14700) loss: 0.038275
(Iteration 11101 / 14700) loss: 0.069816
(Iteration 11111 / 14700) loss: 0.128341
(Iteration 11121 / 14700) loss: 0.090577
(Iteration 11131 / 14700) loss: 0.067269
(Iteration 11141 / 14700) loss: 0.149983
(Iteration 11151 / 14700) loss: 0.092257
```

```
(Iteration 11161 / 14700) loss: 0.076515
(Iteration 11171 / 14700) loss: 0.093098
(Iteration 11181 / 14700) loss: 0.087159
(Iteration 11191 / 14700) loss: 0.069733
(Iteration 11201 / 14700) loss: 0.076761
(Iteration 11211 / 14700) loss: 0.057826
(Iteration 11221 / 14700) loss: 0.092618
(Iteration 11231 / 14700) loss: 0.078714
(Iteration 11241 / 14700) loss: 0.052325
(Iteration 11251 / 14700) loss: 0.087329
(Iteration 11261 / 14700) loss: 0.081535
(Epoch 23 / 30) train acc: 0.986000; val_acc: 0.570000
(Iteration 11271 / 14700) loss: 0.096641
(Iteration 11281 / 14700) loss: 0.052963
(Iteration 11291 / 14700) loss: 0.062941
(Iteration 11301 / 14700) loss: 0.073119
(Iteration 11311 / 14700) loss: 0.063006
(Iteration 11321 / 14700) loss: 0.071111
(Iteration 11331 / 14700) loss: 0.037847
(Iteration 11341 / 14700) loss: 0.068474
(Iteration 11351 / 14700) loss: 0.061507
(Iteration 11361 / 14700) loss: 0.047611
(Iteration 11371 / 14700) loss: 0.103798
(Iteration 11381 / 14700) loss: 0.129829
(Iteration 11391 / 14700) loss: 0.058283
(Iteration 11401 / 14700) loss: 0.075705
(Iteration 11411 / 14700) loss: 0.096106
(Iteration 11421 / 14700) loss: 0.057494
(Iteration 11431 / 14700) loss: 0.056554
(Iteration 11441 / 14700) loss: 0.049965
(Iteration 11451 / 14700) loss: 0.055191
(Iteration 11461 / 14700) loss: 0.098399
(Iteration 11471 / 14700) loss: 0.062553
(Iteration 11481 / 14700) loss: 0.066226
(Iteration 11491 / 14700) loss: 0.062380
(Iteration 11501 / 14700) loss: 0.053591
(Iteration 11511 / 14700) loss: 0.066605
(Iteration 11521 / 14700) loss: 0.038603
(Iteration 11531 / 14700) loss: 0.046622
(Iteration 11541 / 14700) loss: 0.055829
(Iteration 11551 / 14700) loss: 0.040943
(Iteration 11561 / 14700) loss: 0.083281
(Iteration 11571 / 14700) loss: 0.108463
(Iteration 11581 / 14700) loss: 0.079830
(Iteration 11591 / 14700) loss: 0.056880
(Iteration 11601 / 14700) loss: 0.057468
(Iteration 11611 / 14700) loss: 0.057838
(Iteration 11621 / 14700) loss: 0.062606
```

```
(Iteration 11631 / 14700) loss: 0.072098
(Iteration 11641 / 14700) loss: 0.065554
(Iteration 11651 / 14700) loss: 0.105387
(Iteration 11661 / 14700) loss: 0.029606
(Iteration 11671 / 14700) loss: 0.061882
(Iteration 11681 / 14700) loss: 0.054374
(Iteration 11691 / 14700) loss: 0.048824
(Iteration 11701 / 14700) loss: 0.068839
(Iteration 11711 / 14700) loss: 0.102351
(Iteration 11721 / 14700) loss: 0.055993
(Iteration 11731 / 14700) loss: 0.084630
(Iteration 11741 / 14700) loss: 0.070613
(Iteration 11751 / 14700) loss: 0.050940
(Epoch 24 / 30) train acc: 0.994000; val_acc: 0.579000
(Iteration 11761 / 14700) loss: 0.080930
(Iteration 11771 / 14700) loss: 0.056862
(Iteration 11781 / 14700) loss: 0.108709
(Iteration 11791 / 14700) loss: 0.070279
(Iteration 11801 / 14700) loss: 0.077217
(Iteration 11811 / 14700) loss: 0.095820
(Iteration 11821 / 14700) loss: 0.041093
(Iteration 11831 / 14700) loss: 0.058298
(Iteration 11841 / 14700) loss: 0.044536
(Iteration 11851 / 14700) loss: 0.067738
(Iteration 11861 / 14700) loss: 0.044238
(Iteration 11871 / 14700) loss: 0.037576
(Iteration 11881 / 14700) loss: 0.040793
(Iteration 11891 / 14700) loss: 0.076931
(Iteration 11901 / 14700) loss: 0.071714
(Iteration 11911 / 14700) loss: 0.037627
(Iteration 11921 / 14700) loss: 0.053568
(Iteration 11931 / 14700) loss: 0.054001
(Iteration 11941 / 14700) loss: 0.042559
(Iteration 11951 / 14700) loss: 0.054646
(Iteration 11961 / 14700) loss: 0.038759
(Iteration 11971 / 14700) loss: 0.076352
(Iteration 11981 / 14700) loss: 0.062457
(Iteration 11991 / 14700) loss: 0.095561
(Iteration 12001 / 14700) loss: 0.037032
(Iteration 12011 / 14700) loss: 0.073465
(Iteration 12021 / 14700) loss: 0.054042
(Iteration 12031 / 14700) loss: 0.082317
(Iteration 12041 / 14700) loss: 0.033051
(Iteration 12051 / 14700) loss: 0.049602
(Iteration 12061 / 14700) loss: 0.070497
(Iteration 12071 / 14700) loss: 0.062358
(Iteration 12081 / 14700) loss: 0.066180
(Iteration 12091 / 14700) loss: 0.072227
```

```
(Iteration 12101 / 14700) loss: 0.062837
(Iteration 12111 / 14700) loss: 0.055964
(Iteration 12121 / 14700) loss: 0.034732
(Iteration 12131 / 14700) loss: 0.067510
(Iteration 12141 / 14700) loss: 0.069504
(Iteration 12151 / 14700) loss: 0.043166
(Iteration 12161 / 14700) loss: 0.052624
(Iteration 12171 / 14700) loss: 0.087873
(Iteration 12181 / 14700) loss: 0.059187
(Iteration 12191 / 14700) loss: 0.059513
(Iteration 12201 / 14700) loss: 0.029574
(Iteration 12211 / 14700) loss: 0.062189
(Iteration 12221 / 14700) loss: 0.043302
(Iteration 12231 / 14700) loss: 0.052147
(Iteration 12241 / 14700) loss: 0.065672
(Epoch 25 / 30) train acc: 0.993000; val_acc: 0.575000
(Iteration 12251 / 14700) loss: 0.063308
(Iteration 12261 / 14700) loss: 0.054086
(Iteration 12271 / 14700) loss: 0.055032
(Iteration 12281 / 14700) loss: 0.055393
(Iteration 12291 / 14700) loss: 0.064752
(Iteration 12301 / 14700) loss: 0.077577
(Iteration 12311 / 14700) loss: 0.039503
(Iteration 12321 / 14700) loss: 0.026410
(Iteration 12331 / 14700) loss: 0.039554
(Iteration 12341 / 14700) loss: 0.052262
(Iteration 12351 / 14700) loss: 0.097878
(Iteration 12361 / 14700) loss: 0.044708
(Iteration 12371 / 14700) loss: 0.042001
(Iteration 12381 / 14700) loss: 0.092527
(Iteration 12391 / 14700) loss: 0.057859
(Iteration 12401 / 14700) loss: 0.069018
(Iteration 12411 / 14700) loss: 0.070649
(Iteration 12421 / 14700) loss: 0.053269
(Iteration 12431 / 14700) loss: 0.047604
(Iteration 12441 / 14700) loss: 0.084650
(Iteration 12451 / 14700) loss: 0.048775
(Iteration 12461 / 14700) loss: 0.050185
(Iteration 12471 / 14700) loss: 0.038722
(Iteration 12481 / 14700) loss: 0.034894
(Iteration 12491 / 14700) loss: 0.066522
(Iteration 12501 / 14700) loss: 0.044490
(Iteration 12511 / 14700) loss: 0.048293
(Iteration 12521 / 14700) loss: 0.052110
(Iteration 12531 / 14700) loss: 0.040825
(Iteration 12541 / 14700) loss: 0.052611
(Iteration 12551 / 14700) loss: 0.063695
(Iteration 12561 / 14700) loss: 0.064793
```

```
(Iteration 12571 / 14700) loss: 0.064999
(Iteration 12581 / 14700) loss: 0.053820
(Iteration 12591 / 14700) loss: 0.062178
(Iteration 12601 / 14700) loss: 0.055039
(Iteration 12611 / 14700) loss: 0.064689
(Iteration 12621 / 14700) loss: 0.072330
(Iteration 12631 / 14700) loss: 0.045189
(Iteration 12641 / 14700) loss: 0.034931
(Iteration 12651 / 14700) loss: 0.092535
(Iteration 12661 / 14700) loss: 0.030115
(Iteration 12671 / 14700) loss: 0.042196
(Iteration 12681 / 14700) loss: 0.055115
(Iteration 12691 / 14700) loss: 0.050311
(Iteration 12701 / 14700) loss: 0.075265
(Iteration 12711 / 14700) loss: 0.043945
(Iteration 12721 / 14700) loss: 0.055274
(Iteration 12731 / 14700) loss: 0.028164
(Epoch 26 / 30) train acc: 0.995000; val_acc: 0.574000
(Iteration 12741 / 14700) loss: 0.044317
(Iteration 12751 / 14700) loss: 0.049795
(Iteration 12761 / 14700) loss: 0.033039
(Iteration 12771 / 14700) loss: 0.050533
(Iteration 12781 / 14700) loss: 0.050030
(Iteration 12791 / 14700) loss: 0.065001
(Iteration 12801 / 14700) loss: 0.054980
(Iteration 12811 / 14700) loss: 0.033550
(Iteration 12821 / 14700) loss: 0.044715
(Iteration 12831 / 14700) loss: 0.037589
(Iteration 12841 / 14700) loss: 0.059679
(Iteration 12851 / 14700) loss: 0.046318
(Iteration 12861 / 14700) loss: 0.057017
(Iteration 12871 / 14700) loss: 0.040278
(Iteration 12881 / 14700) loss: 0.062162
(Iteration 12891 / 14700) loss: 0.062339
(Iteration 12901 / 14700) loss: 0.033289
(Iteration 12911 / 14700) loss: 0.044241
(Iteration 12921 / 14700) loss: 0.052101
(Iteration 12931 / 14700) loss: 0.055667
(Iteration 12941 / 14700) loss: 0.049129
(Iteration 12951 / 14700) loss: 0.032180
(Iteration 12961 / 14700) loss: 0.034017
(Iteration 12971 / 14700) loss: 0.036971
(Iteration 12981 / 14700) loss: 0.038052
(Iteration 12991 / 14700) loss: 0.051480
(Iteration 13001 / 14700) loss: 0.027325
(Iteration 13011 / 14700) loss: 0.049099
(Iteration 13021 / 14700) loss: 0.049523
(Iteration 13031 / 14700) loss: 0.046981
```

```
(Iteration 13041 / 14700) loss: 0.066144
(Iteration 13051 / 14700) loss: 0.063071
(Iteration 13061 / 14700) loss: 0.063495
(Iteration 13071 / 14700) loss: 0.038044
(Iteration 13081 / 14700) loss: 0.062165
(Iteration 13091 / 14700) loss: 0.043524
(Iteration 13101 / 14700) loss: 0.043027
(Iteration 13111 / 14700) loss: 0.058742
(Iteration 13121 / 14700) loss: 0.038145
(Iteration 13131 / 14700) loss: 0.029130
(Iteration 13141 / 14700) loss: 0.040440
(Iteration 13151 / 14700) loss: 0.055960
(Iteration 13161 / 14700) loss: 0.026340
(Iteration 13171 / 14700) loss: 0.041714
(Iteration 13181 / 14700) loss: 0.041007
(Iteration 13191 / 14700) loss: 0.057045
(Iteration 13201 / 14700) loss: 0.065623
(Iteration 13211 / 14700) loss: 0.037928
(Iteration 13221 / 14700) loss: 0.039549
(Epoch 27 / 30) train acc: 0.998000; val_acc: 0.573000
(Iteration 13231 / 14700) loss: 0.043758
(Iteration 13241 / 14700) loss: 0.035705
(Iteration 13251 / 14700) loss: 0.048094
(Iteration 13261 / 14700) loss: 0.072210
(Iteration 13271 / 14700) loss: 0.042799
(Iteration 13281 / 14700) loss: 0.037322
(Iteration 13291 / 14700) loss: 0.067488
(Iteration 13301 / 14700) loss: 0.038900
(Iteration 13311 / 14700) loss: 0.035964
(Iteration 13321 / 14700) loss: 0.036770
(Iteration 13331 / 14700) loss: 0.074964
(Iteration 13341 / 14700) loss: 0.050388
(Iteration 13351 / 14700) loss: 0.080571
(Iteration 13361 / 14700) loss: 0.057906
(Iteration 13371 / 14700) loss: 0.033400
(Iteration 13381 / 14700) loss: 0.037429
(Iteration 13391 / 14700) loss: 0.051538
(Iteration 13401 / 14700) loss: 0.041083
(Iteration 13411 / 14700) loss: 0.033450
(Iteration 13421 / 14700) loss: 0.043506
(Iteration 13431 / 14700) loss: 0.054263
(Iteration 13441 / 14700) loss: 0.032457
(Iteration 13451 / 14700) loss: 0.032476
(Iteration 13461 / 14700) loss: 0.039087
(Iteration 13471 / 14700) loss: 0.038973
(Iteration 13481 / 14700) loss: 0.060369
(Iteration 13491 / 14700) loss: 0.035770
(Iteration 13501 / 14700) loss: 0.040282
```

```
(Iteration 13511 / 14700) loss: 0.039029
(Iteration 13521 / 14700) loss: 0.077027
(Iteration 13531 / 14700) loss: 0.042527
(Iteration 13541 / 14700) loss: 0.036678
(Iteration 13551 / 14700) loss: 0.039800
(Iteration 13561 / 14700) loss: 0.048598
(Iteration 13571 / 14700) loss: 0.033853
(Iteration 13581 / 14700) loss: 0.047445
(Iteration 13591 / 14700) loss: 0.035970
(Iteration 13601 / 14700) loss: 0.038177
(Iteration 13611 / 14700) loss: 0.035271
(Iteration 13621 / 14700) loss: 0.037100
(Iteration 13631 / 14700) loss: 0.040965
(Iteration 13641 / 14700) loss: 0.033095
(Iteration 13651 / 14700) loss: 0.027195
(Iteration 13661 / 14700) loss: 0.054256
(Iteration 13671 / 14700) loss: 0.027063
(Iteration 13681 / 14700) loss: 0.048034
(Iteration 13691 / 14700) loss: 0.027749
(Iteration 13701 / 14700) loss: 0.029752
(Iteration 13711 / 14700) loss: 0.038449
(Epoch 28 / 30) train acc: 0.999000; val_acc: 0.570000
(Iteration 13721 / 14700) loss: 0.052732
(Iteration 13731 / 14700) loss: 0.036530
(Iteration 13741 / 14700) loss: 0.039406
(Iteration 13751 / 14700) loss: 0.032919
(Iteration 13761 / 14700) loss: 0.033799
(Iteration 13771 / 14700) loss: 0.052733
(Iteration 13781 / 14700) loss: 0.055529
(Iteration 13791 / 14700) loss: 0.056109
(Iteration 13801 / 14700) loss: 0.035903
(Iteration 13811 / 14700) loss: 0.026938
(Iteration 13821 / 14700) loss: 0.028620
(Iteration 13831 / 14700) loss: 0.036008
(Iteration 13841 / 14700) loss: 0.037712
(Iteration 13851 / 14700) loss: 0.048293
(Iteration 13861 / 14700) loss: 0.034743
(Iteration 13871 / 14700) loss: 0.041136
(Iteration 13881 / 14700) loss: 0.036662
(Iteration 13891 / 14700) loss: 0.037360
(Iteration 13901 / 14700) loss: 0.041277
(Iteration 13911 / 14700) loss: 0.027686
(Iteration 13921 / 14700) loss: 0.038770
(Iteration 13931 / 14700) loss: 0.036930
(Iteration 13941 / 14700) loss: 0.028149
(Iteration 13951 / 14700) loss: 0.035034
(Iteration 13961 / 14700) loss: 0.048904
(Iteration 13971 / 14700) loss: 0.029333
```

```
(Iteration 13981 / 14700) loss: 0.032926
(Iteration 13991 / 14700) loss: 0.029911
(Iteration 14001 / 14700) loss: 0.040413
(Iteration 14011 / 14700) loss: 0.076769
(Iteration 14021 / 14700) loss: 0.032415
(Iteration 14031 / 14700) loss: 0.047617
(Iteration 14041 / 14700) loss: 0.025361
(Iteration 14051 / 14700) loss: 0.034040
(Iteration 14061 / 14700) loss: 0.041611
(Iteration 14071 / 14700) loss: 0.035394
(Iteration 14081 / 14700) loss: 0.040971
(Iteration 14091 / 14700) loss: 0.035447
(Iteration 14101 / 14700) loss: 0.036929
(Iteration 14111 / 14700) loss: 0.064026
(Iteration 14121 / 14700) loss: 0.033736
(Iteration 14131 / 14700) loss: 0.035196
(Iteration 14141 / 14700) loss: 0.042491
(Iteration 14151 / 14700) loss: 0.038754
(Iteration 14161 / 14700) loss: 0.033182
(Iteration 14171 / 14700) loss: 0.037049
(Iteration 14181 / 14700) loss: 0.033843
(Iteration 14191 / 14700) loss: 0.044932
(Iteration 14201 / 14700) loss: 0.030124
(Epoch 29 / 30) train acc: 0.996000; val_acc: 0.568000
(Iteration 14211 / 14700) loss: 0.048117
(Iteration 14221 / 14700) loss: 0.069502
(Iteration 14231 / 14700) loss: 0.039416
(Iteration 14241 / 14700) loss: 0.025065
(Iteration 14251 / 14700) loss: 0.036936
(Iteration 14261 / 14700) loss: 0.037185
(Iteration 14271 / 14700) loss: 0.025028
(Iteration 14281 / 14700) loss: 0.048892
(Iteration 14291 / 14700) loss: 0.037219
(Iteration 14301 / 14700) loss: 0.045735
(Iteration 14311 / 14700) loss: 0.036276
(Iteration 14321 / 14700) loss: 0.034696
(Iteration 14331 / 14700) loss: 0.053380
(Iteration 14341 / 14700) loss: 0.036706
(Iteration 14351 / 14700) loss: 0.036586
(Iteration 14361 / 14700) loss: 0.040118
(Iteration 14371 / 14700) loss: 0.035204
(Iteration 14381 / 14700) loss: 0.028366
(Iteration 14391 / 14700) loss: 0.034975
(Iteration 14401 / 14700) loss: 0.028975
(Iteration 14411 / 14700) loss: 0.034867
(Iteration 14421 / 14700) loss: 0.028861
(Iteration 14431 / 14700) loss: 0.044793
(Iteration 14441 / 14700) loss: 0.027299
```

```
(Iteration 14451 / 14700) loss: 0.041879
(Iteration 14461 / 14700) loss: 0.050376
(Iteration 14471 / 14700) loss: 0.036902
(Iteration 14481 / 14700) loss: 0.035996
(Iteration 14491 / 14700) loss: 0.033731
(Iteration 14501 / 14700) loss: 0.032572
(Iteration 14511 / 14700) loss: 0.017749
(Iteration 14521 / 14700) loss: 0.026652
(Iteration 14531 / 14700) loss: 0.038587
(Iteration 14541 / 14700) loss: 0.027461
(Iteration 14551 / 14700) loss: 0.029548
(Iteration 14561 / 14700) loss: 0.040552
(Iteration 14571 / 14700) loss: 0.033639
(Iteration 14581 / 14700) loss: 0.021494
(Iteration 14591 / 14700) loss: 0.028204
(Iteration 14601 / 14700) loss: 0.039360
(Iteration 14611 / 14700) loss: 0.034280
(Iteration 14621 / 14700) loss: 0.032302
(Iteration 14631 / 14700) loss: 0.047675
(Iteration 14641 / 14700) loss: 0.034774
(Iteration 14651 / 14700) loss: 0.036384
(Iteration 14661 / 14700) loss: 0.038323
(Iteration 14671 / 14700) loss: 0.030499
(Iteration 14681 / 14700) loss: 0.059594
(Iteration 14691 / 14700) loss: 0.036844
(Epoch 30 / 30) train acc: 1.000000; val_acc: 0.577000
with learning rate 0.5, regularization strength 0, accuracy: 0.585
(Iteration 1 / 14700) loss: 2.304408
(Epoch 0 / 30) train acc: 0.129000; val_acc: 0.160000
(Iteration 11 / 14700) loss: 2.156691
(Iteration 21 / 14700) loss: 1.758211
(Iteration 31 / 14700) loss: 1.526383
(Iteration 41 / 14700) loss: 1.543705
(Iteration 51 / 14700) loss: 1.447301
(Iteration 61 / 14700) loss: 1.426566
(Iteration 71 / 14700) loss: 1.608326
(Iteration 81 / 14700) loss: 1.398568
(Iteration 91 / 14700) loss: 1.410492
(Iteration 101 / 14700) loss: 1.218117
(Iteration 111 / 14700) loss: 1.305100
(Iteration 121 / 14700) loss: 1.358560
(Iteration 131 / 14700) loss: 1.466717
(Iteration 141 / 14700) loss: 1.523420
(Iteration 151 / 14700) loss: 1.373073
(Iteration 161 / 14700) loss: 1.516295
(Iteration 171 / 14700) loss: 1.471210
(Iteration 181 / 14700) loss: 1.243424
(Iteration 191 / 14700) loss: 1.306246
```

```
(Iteration 201 / 14700) loss: 1.336478
(Iteration 211 / 14700) loss: 1.388476
(Iteration 221 / 14700) loss: 1.419168
(Iteration 231 / 14700) loss: 1.264229
(Iteration 241 / 14700) loss: 1.679825
(Iteration 251 / 14700) loss: 1.114893
(Iteration 261 / 14700) loss: 1.242674
(Iteration 271 / 14700) loss: 1.232197
(Iteration 281 / 14700) loss: 1.375042
(Iteration 291 / 14700) loss: 1.499545
(Iteration 301 / 14700) loss: 1.161060
(Iteration 311 / 14700) loss: 1.098848
(Iteration 321 / 14700) loss: 1.581536
(Iteration 331 / 14700) loss: 1.333971
(Iteration 341 / 14700) loss: 1.212062
(Iteration 351 / 14700) loss: 1.212241
(Iteration 361 / 14700) loss: 1.315343
(Iteration 371 / 14700) loss: 1.366043
(Iteration 381 / 14700) loss: 1.125355
(Iteration 391 / 14700) loss: 1.288677
(Iteration 401 / 14700) loss: 1.227475
(Iteration 411 / 14700) loss: 1.127985
(Iteration 421 / 14700) loss: 1.249639
(Iteration 431 / 14700) loss: 1.183983
(Iteration 441 / 14700) loss: 1.120194
(Iteration 451 / 14700) loss: 1.228357
(Iteration 461 / 14700) loss: 1.135718
(Iteration 471 / 14700) loss: 1.015827
(Iteration 481 / 14700) loss: 1.384986
(Epoch 1 / 30) train acc: 0.568000; val_acc: 0.527000
(Iteration 491 / 14700) loss: 1.369245
(Iteration 501 / 14700) loss: 1.325018
(Iteration 511 / 14700) loss: 1.321769
(Iteration 521 / 14700) loss: 1.240895
(Iteration 531 / 14700) loss: 1.075589
(Iteration 541 / 14700) loss: 1.123913
(Iteration 551 / 14700) loss: 1.000422
(Iteration 561 / 14700) loss: 1.002832
(Iteration 571 / 14700) loss: 1.127375
(Iteration 581 / 14700) loss: 1.065503
(Iteration 591 / 14700) loss: 1.066513
(Iteration 601 / 14700) loss: 1.056879
(Iteration 611 / 14700) loss: 1.195605
(Iteration 621 / 14700) loss: 1.137496
(Iteration 631 / 14700) loss: 1.362342
(Iteration 641 / 14700) loss: 1.075863
(Iteration 651 / 14700) loss: 1.040593
(Iteration 661 / 14700) loss: 1.353524
```

```
(Iteration 671 / 14700) loss: 1.278997
(Iteration 681 / 14700) loss: 1.197345
(Iteration 691 / 14700) loss: 0.943019
(Iteration 701 / 14700) loss: 1.155673
(Iteration 711 / 14700) loss: 1.239212
(Iteration 721 / 14700) loss: 1.208029
(Iteration 731 / 14700) loss: 0.983506
(Iteration 741 / 14700) loss: 1.150622
(Iteration 751 / 14700) loss: 1.113852
(Iteration 761 / 14700) loss: 1.049767
(Iteration 771 / 14700) loss: 1.076641
(Iteration 781 / 14700) loss: 1.200896
(Iteration 791 / 14700) loss: 1.180077
(Iteration 801 / 14700) loss: 1.291421
(Iteration 811 / 14700) loss: 1.011137
(Iteration 821 / 14700) loss: 0.915392
(Iteration 831 / 14700) loss: 1.218979
(Iteration 841 / 14700) loss: 1.262996
(Iteration 851 / 14700) loss: 1.125752
(Iteration 861 / 14700) loss: 0.956365
(Iteration 871 / 14700) loss: 1.127433
(Iteration 881 / 14700) loss: 1.198568
(Iteration 891 / 14700) loss: 1.053884
(Iteration 901 / 14700) loss: 1.202903
(Iteration 911 / 14700) loss: 1.285192
(Iteration 921 / 14700) loss: 1.016732
(Iteration 931 / 14700) loss: 0.874548
(Iteration 941 / 14700) loss: 1.110628
(Iteration 951 / 14700) loss: 1.097726
(Iteration 961 / 14700) loss: 1.115709
(Iteration 971 / 14700) loss: 0.964364
(Epoch 2 / 30) train acc: 0.642000; val_acc: 0.593000
(Iteration 981 / 14700) loss: 1.012150
(Iteration 991 / 14700) loss: 0.964245
(Iteration 1001 / 14700) loss: 0.957457
(Iteration 1011 / 14700) loss: 1.144455
(Iteration 1021 / 14700) loss: 0.946257
(Iteration 1031 / 14700) loss: 0.885395
(Iteration 1041 / 14700) loss: 1.231295
(Iteration 1051 / 14700) loss: 1.123246
(Iteration 1061 / 14700) loss: 1.035087
(Iteration 1071 / 14700) loss: 1.258753
(Iteration 1081 / 14700) loss: 1.215321
(Iteration 1091 / 14700) loss: 0.874151
(Iteration 1101 / 14700) loss: 0.978655
(Iteration 1111 / 14700) loss: 1.312952
(Iteration 1121 / 14700) loss: 1.156362
(Iteration 1131 / 14700) loss: 1.422581
```

```
(Iteration 1141 / 14700) loss: 1.209508
(Iteration 1151 / 14700) loss: 1.099390
(Iteration 1161 / 14700) loss: 1.112164
(Iteration 1171 / 14700) loss: 1.096098
(Iteration 1181 / 14700) loss: 1.084270
(Iteration 1191 / 14700) loss: 0.957657
(Iteration 1201 / 14700) loss: 1.078083
(Iteration 1211 / 14700) loss: 1.092707
(Iteration 1221 / 14700) loss: 1.105565
(Iteration 1231 / 14700) loss: 0.885372
(Iteration 1241 / 14700) loss: 1.040456
(Iteration 1251 / 14700) loss: 0.983579
(Iteration 1261 / 14700) loss: 1.235114
(Iteration 1271 / 14700) loss: 1.207570
(Iteration 1281 / 14700) loss: 0.991184
(Iteration 1291 / 14700) loss: 0.972825
(Iteration 1301 / 14700) loss: 1.061349
(Iteration 1311 / 14700) loss: 0.991179
(Iteration 1321 / 14700) loss: 1.068153
(Iteration 1331 / 14700) loss: 0.869547
(Iteration 1341 / 14700) loss: 0.977054
(Iteration 1351 / 14700) loss: 0.779417
(Iteration 1361 / 14700) loss: 0.928431
(Iteration 1371 / 14700) loss: 1.142564
(Iteration 1381 / 14700) loss: 0.913242
(Iteration 1391 / 14700) loss: 1.369749
(Iteration 1401 / 14700) loss: 1.020644
(Iteration 1411 / 14700) loss: 1.058778
(Iteration 1421 / 14700) loss: 1.161595
(Iteration 1431 / 14700) loss: 1.175409
(Iteration 1441 / 14700) loss: 0.948359
(Iteration 1451 / 14700) loss: 1.156477
(Iteration 1461 / 14700) loss: 0.770186
(Epoch 3 / 30) train acc: 0.660000; val_acc: 0.564000
(Iteration 1471 / 14700) loss: 1.059039
(Iteration 1481 / 14700) loss: 1.233559
(Iteration 1491 / 14700) loss: 0.919579
(Iteration 1501 / 14700) loss: 0.869117
(Iteration 1511 / 14700) loss: 0.930294
(Iteration 1521 / 14700) loss: 1.109277
(Iteration 1531 / 14700) loss: 0.974019
(Iteration 1541 / 14700) loss: 1.074194
(Iteration 1551 / 14700) loss: 1.029687
(Iteration 1561 / 14700) loss: 1.079446
(Iteration 1571 / 14700) loss: 0.924911
(Iteration 1581 / 14700) loss: 0.911895
(Iteration 1591 / 14700) loss: 0.896969
(Iteration 1601 / 14700) loss: 0.977959
```

```
(Iteration 1611 / 14700) loss: 1.194314
(Iteration 1621 / 14700) loss: 1.064805
(Iteration 1631 / 14700) loss: 0.869352
(Iteration 1641 / 14700) loss: 0.937661
(Iteration 1651 / 14700) loss: 0.936551
(Iteration 1661 / 14700) loss: 1.028171
(Iteration 1671 / 14700) loss: 0.988769
(Iteration 1681 / 14700) loss: 1.244630
(Iteration 1691 / 14700) loss: 0.812769
(Iteration 1701 / 14700) loss: 1.105878
(Iteration 1711 / 14700) loss: 1.093939
(Iteration 1721 / 14700) loss: 1.206833
(Iteration 1731 / 14700) loss: 0.790657
(Iteration 1741 / 14700) loss: 0.917361
(Iteration 1751 / 14700) loss: 0.934819
(Iteration 1761 / 14700) loss: 0.933934
(Iteration 1771 / 14700) loss: 0.890424
(Iteration 1781 / 14700) loss: 1.005030
(Iteration 1791 / 14700) loss: 0.839463
(Iteration 1801 / 14700) loss: 0.887174
(Iteration 1811 / 14700) loss: 1.144694
(Iteration 1821 / 14700) loss: 0.946603
(Iteration 1831 / 14700) loss: 0.949058
(Iteration 1841 / 14700) loss: 0.969846
(Iteration 1851 / 14700) loss: 1.043772
(Iteration 1861 / 14700) loss: 0.932728
(Iteration 1871 / 14700) loss: 1.092439
(Iteration 1881 / 14700) loss: 0.987020
(Iteration 1891 / 14700) loss: 0.771601
(Iteration 1901 / 14700) loss: 0.790702
(Iteration 1911 / 14700) loss: 0.846306
(Iteration 1921 / 14700) loss: 1.095056
(Iteration 1931 / 14700) loss: 0.952680
(Iteration 1941 / 14700) loss: 1.117721
(Iteration 1951 / 14700) loss: 0.902993
(Epoch 4 / 30) train acc: 0.683000; val_acc: 0.562000
(Iteration 1961 / 14700) loss: 0.961406
(Iteration 1971 / 14700) loss: 0.972816
(Iteration 1981 / 14700) loss: 1.067542
(Iteration 1991 / 14700) loss: 0.909765
(Iteration 2001 / 14700) loss: 0.731348
(Iteration 2011 / 14700) loss: 0.950845
(Iteration 2021 / 14700) loss: 0.721814
(Iteration 2031 / 14700) loss: 0.908338
(Iteration 2041 / 14700) loss: 0.911009
(Iteration 2051 / 14700) loss: 0.822576
(Iteration 2061 / 14700) loss: 0.882093
(Iteration 2071 / 14700) loss: 0.906295
```

```
(Iteration 2081 / 14700) loss: 0.770350
(Iteration 2091 / 14700) loss: 0.994919
(Iteration 2101 / 14700) loss: 0.820161
(Iteration 2111 / 14700) loss: 0.813981
(Iteration 2121 / 14700) loss: 0.989644
(Iteration 2131 / 14700) loss: 0.832895
(Iteration 2141 / 14700) loss: 0.805293
(Iteration 2151 / 14700) loss: 0.629366
(Iteration 2161 / 14700) loss: 0.885108
(Iteration 2171 / 14700) loss: 0.945565
(Iteration 2181 / 14700) loss: 0.898437
(Iteration 2191 / 14700) loss: 1.033395
(Iteration 2201 / 14700) loss: 0.935618
(Iteration 2211 / 14700) loss: 0.978549
(Iteration 2221 / 14700) loss: 0.798424
(Iteration 2231 / 14700) loss: 0.922700
(Iteration 2241 / 14700) loss: 1.113430
(Iteration 2251 / 14700) loss: 0.937140
(Iteration 2261 / 14700) loss: 0.825994
(Iteration 2271 / 14700) loss: 0.901195
(Iteration 2281 / 14700) loss: 0.794439
(Iteration 2291 / 14700) loss: 0.967652
(Iteration 2301 / 14700) loss: 0.884878
(Iteration 2311 / 14700) loss: 0.810892
(Iteration 2321 / 14700) loss: 0.738938
(Iteration 2331 / 14700) loss: 0.592415
(Iteration 2341 / 14700) loss: 0.731184
(Iteration 2351 / 14700) loss: 1.000408
(Iteration 2361 / 14700) loss: 0.897937
(Iteration 2371 / 14700) loss: 0.688930
(Iteration 2381 / 14700) loss: 0.794573
(Iteration 2391 / 14700) loss: 1.000098
(Iteration 2401 / 14700) loss: 0.764694
(Iteration 2411 / 14700) loss: 0.848042
(Iteration 2421 / 14700) loss: 0.924104
(Iteration 2431 / 14700) loss: 0.770786
(Iteration 2441 / 14700) loss: 0.802633
(Epoch 5 / 30) train acc: 0.730000; val_acc: 0.553000
(Iteration 2451 / 14700) loss: 0.860144
(Iteration 2461 / 14700) loss: 0.813304
(Iteration 2471 / 14700) loss: 0.842103
(Iteration 2481 / 14700) loss: 0.867851
(Iteration 2491 / 14700) loss: 0.569121
(Iteration 2501 / 14700) loss: 0.611198
(Iteration 2511 / 14700) loss: 0.918059
(Iteration 2521 / 14700) loss: 0.809859
(Iteration 2531 / 14700) loss: 0.654194
(Iteration 2541 / 14700) loss: 0.824483
```

```
(Iteration 2551 / 14700) loss: 0.925159
(Iteration 2561 / 14700) loss: 0.659599
(Iteration 2571 / 14700) loss: 0.814079
(Iteration 2581 / 14700) loss: 0.970865
(Iteration 2591 / 14700) loss: 0.884250
(Iteration 2601 / 14700) loss: 0.938488
(Iteration 2611 / 14700) loss: 0.560527
(Iteration 2621 / 14700) loss: 0.836687
(Iteration 2631 / 14700) loss: 0.852954
(Iteration 2641 / 14700) loss: 0.780669
(Iteration 2651 / 14700) loss: 0.654081
(Iteration 2661 / 14700) loss: 0.885212
(Iteration 2671 / 14700) loss: 0.644944
(Iteration 2681 / 14700) loss: 0.976038
(Iteration 2691 / 14700) loss: 0.890221
(Iteration 2701 / 14700) loss: 1.056636
(Iteration 2711 / 14700) loss: 1.048323
(Iteration 2721 / 14700) loss: 0.673754
(Iteration 2731 / 14700) loss: 0.703076
(Iteration 2741 / 14700) loss: 0.797133
(Iteration 2751 / 14700) loss: 0.688228
(Iteration 2761 / 14700) loss: 0.690679
(Iteration 2771 / 14700) loss: 0.913895
(Iteration 2781 / 14700) loss: 0.798830
(Iteration 2791 / 14700) loss: 0.893336
(Iteration 2801 / 14700) loss: 0.848235
(Iteration 2811 / 14700) loss: 0.839667
(Iteration 2821 / 14700) loss: 0.680996
(Iteration 2831 / 14700) loss: 0.837324
(Iteration 2841 / 14700) loss: 0.829382
(Iteration 2851 / 14700) loss: 0.880927
(Iteration 2861 / 14700) loss: 0.530305
(Iteration 2871 / 14700) loss: 0.715705
(Iteration 2881 / 14700) loss: 0.772923
(Iteration 2891 / 14700) loss: 0.928716
(Iteration 2901 / 14700) loss: 0.858675
(Iteration 2911 / 14700) loss: 0.618695
(Iteration 2921 / 14700) loss: 0.719571
(Iteration 2931 / 14700) loss: 0.688524
(Epoch 6 / 30) train acc: 0.712000; val_acc: 0.552000
(Iteration 2941 / 14700) loss: 0.863459
(Iteration 2951 / 14700) loss: 0.848197
(Iteration 2961 / 14700) loss: 0.766017
(Iteration 2971 / 14700) loss: 0.663128
(Iteration 2981 / 14700) loss: 0.587468
(Iteration 2991 / 14700) loss: 0.958116
(Iteration 3001 / 14700) loss: 0.755454
(Iteration 3011 / 14700) loss: 0.667700
```

```
(Iteration 3021 / 14700) loss: 0.645203
(Iteration 3031 / 14700) loss: 0.649787
(Iteration 3041 / 14700) loss: 0.711688
(Iteration 3051 / 14700) loss: 1.113542
(Iteration 3061 / 14700) loss: 0.686174
(Iteration 3071 / 14700) loss: 0.848756
(Iteration 3081 / 14700) loss: 0.784536
(Iteration 3091 / 14700) loss: 0.908444
(Iteration 3101 / 14700) loss: 0.926884
(Iteration 3111 / 14700) loss: 0.848105
(Iteration 3121 / 14700) loss: 0.766726
(Iteration 3131 / 14700) loss: 0.938225
(Iteration 3141 / 14700) loss: 0.960868
(Iteration 3151 / 14700) loss: 0.698146
(Iteration 3161 / 14700) loss: 0.627758
(Iteration 3171 / 14700) loss: 0.761247
(Iteration 3181 / 14700) loss: 0.843764
(Iteration 3191 / 14700) loss: 0.803579
(Iteration 3201 / 14700) loss: 0.673709
(Iteration 3211 / 14700) loss: 0.854353
(Iteration 3221 / 14700) loss: 0.783499
(Iteration 3231 / 14700) loss: 0.837083
(Iteration 3241 / 14700) loss: 0.685656
(Iteration 3251 / 14700) loss: 0.579369
(Iteration 3261 / 14700) loss: 0.639560
(Iteration 3271 / 14700) loss: 0.837665
(Iteration 3281 / 14700) loss: 0.547702
(Iteration 3291 / 14700) loss: 0.720516
(Iteration 3301 / 14700) loss: 0.727450
(Iteration 3311 / 14700) loss: 0.733139
(Iteration 3321 / 14700) loss: 0.670664
(Iteration 3331 / 14700) loss: 0.574030
(Iteration 3341 / 14700) loss: 0.632417
(Iteration 3351 / 14700) loss: 0.683435
(Iteration 3361 / 14700) loss: 0.615871
(Iteration 3371 / 14700) loss: 0.673430
(Iteration 3381 / 14700) loss: 0.616447
(Iteration 3391 / 14700) loss: 0.675352
(Iteration 3401 / 14700) loss: 0.423061
(Iteration 3411 / 14700) loss: 0.869708
(Iteration 3421 / 14700) loss: 0.573077
(Epoch 7 / 30) train acc: 0.763000; val_acc: 0.549000
(Iteration 3431 / 14700) loss: 0.766268
(Iteration 3441 / 14700) loss: 0.714897
(Iteration 3451 / 14700) loss: 0.537580
(Iteration 3461 / 14700) loss: 0.638227
(Iteration 3471 / 14700) loss: 0.653473
(Iteration 3481 / 14700) loss: 0.412966
```

```
(Iteration 3491 / 14700) loss: 0.719958
(Iteration 3501 / 14700) loss: 0.819641
(Iteration 3511 / 14700) loss: 0.480273
(Iteration 3521 / 14700) loss: 0.513405
(Iteration 3531 / 14700) loss: 0.659609
(Iteration 3541 / 14700) loss: 0.407212
(Iteration 3551 / 14700) loss: 0.588903
(Iteration 3561 / 14700) loss: 0.639855
(Iteration 3571 / 14700) loss: 0.808317
(Iteration 3581 / 14700) loss: 0.781010
(Iteration 3591 / 14700) loss: 0.574204
(Iteration 3601 / 14700) loss: 0.691697
(Iteration 3611 / 14700) loss: 0.620526
(Iteration 3621 / 14700) loss: 0.498386
(Iteration 3631 / 14700) loss: 0.692843
(Iteration 3641 / 14700) loss: 0.484280
(Iteration 3651 / 14700) loss: 0.681534
(Iteration 3661 / 14700) loss: 0.681614
(Iteration 3671 / 14700) loss: 0.740158
(Iteration 3681 / 14700) loss: 0.513105
(Iteration 3691 / 14700) loss: 0.722128
(Iteration 3701 / 14700) loss: 1.080183
(Iteration 3711 / 14700) loss: 0.789316
(Iteration 3721 / 14700) loss: 0.591594
(Iteration 3731 / 14700) loss: 0.529214
(Iteration 3741 / 14700) loss: 0.701310
(Iteration 3751 / 14700) loss: 0.687831
(Iteration 3761 / 14700) loss: 0.616483
(Iteration 3771 / 14700) loss: 0.664538
(Iteration 3781 / 14700) loss: 0.671090
(Iteration 3791 / 14700) loss: 0.599637
(Iteration 3801 / 14700) loss: 0.455567
(Iteration 3811 / 14700) loss: 0.667995
(Iteration 3821 / 14700) loss: 0.569288
(Iteration 3831 / 14700) loss: 0.792797
(Iteration 3841 / 14700) loss: 0.819361
(Iteration 3851 / 14700) loss: 0.563770
(Iteration 3861 / 14700) loss: 0.685910
(Iteration 3871 / 14700) loss: 0.587014
(Iteration 3881 / 14700) loss: 0.542920
(Iteration 3891 / 14700) loss: 0.641820
(Iteration 3901 / 14700) loss: 0.771299
(Iteration 3911 / 14700) loss: 0.723506
(Epoch 8 / 30) train acc: 0.774000; val_acc: 0.548000
(Iteration 3921 / 14700) loss: 0.634350
(Iteration 3931 / 14700) loss: 0.570790
(Iteration 3941 / 14700) loss: 0.768280
(Iteration 3951 / 14700) loss: 0.559950
```

```
(Iteration 3961 / 14700) loss: 0.585238
(Iteration 3971 / 14700) loss: 0.641285
(Iteration 3981 / 14700) loss: 0.476780
(Iteration 3991 / 14700) loss: 0.454875
(Iteration 4001 / 14700) loss: 0.408556
(Iteration 4011 / 14700) loss: 0.612082
(Iteration 4021 / 14700) loss: 0.563604
(Iteration 4031 / 14700) loss: 0.462967
(Iteration 4041 / 14700) loss: 0.533522
(Iteration 4051 / 14700) loss: 0.608106
(Iteration 4061 / 14700) loss: 0.737994
(Iteration 4071 / 14700) loss: 0.656118
(Iteration 4081 / 14700) loss: 0.682703
(Iteration 4091 / 14700) loss: 0.429084
(Iteration 4101 / 14700) loss: 0.527527
(Iteration 4111 / 14700) loss: 0.578663
(Iteration 4121 / 14700) loss: 0.710833
(Iteration 4131 / 14700) loss: 0.578593
(Iteration 4141 / 14700) loss: 0.622752
(Iteration 4151 / 14700) loss: 0.400801
(Iteration 4161 / 14700) loss: 0.605382
(Iteration 4171 / 14700) loss: 0.599575
(Iteration 4181 / 14700) loss: 0.654549
(Iteration 4191 / 14700) loss: 0.405737
(Iteration 4201 / 14700) loss: 0.432776
(Iteration 4211 / 14700) loss: 0.351534
(Iteration 4221 / 14700) loss: 0.460420
(Iteration 4231 / 14700) loss: 0.465813
(Iteration 4241 / 14700) loss: 0.549720
(Iteration 4251 / 14700) loss: 0.691662
(Iteration 4261 / 14700) loss: 0.556296
(Iteration 4271 / 14700) loss: 0.443680
(Iteration 4281 / 14700) loss: 0.655561
(Iteration 4291 / 14700) loss: 0.488612
(Iteration 4301 / 14700) loss: 0.406904
(Iteration 4311 / 14700) loss: 0.511207
(Iteration 4321 / 14700) loss: 0.685816
(Iteration 4331 / 14700) loss: 0.603626
(Iteration 4341 / 14700) loss: 0.599710
(Iteration 4351 / 14700) loss: 0.563707
(Iteration 4361 / 14700) loss: 0.637689
(Iteration 4371 / 14700) loss: 0.725801
(Iteration 4381 / 14700) loss: 0.389501
(Iteration 4391 / 14700) loss: 0.476318
(Iteration 4401 / 14700) loss: 0.753751
(Epoch 9 / 30) train acc: 0.783000; val_acc: 0.550000
(Iteration 4411 / 14700) loss: 0.773013
(Iteration 4421 / 14700) loss: 0.604231
```

```
(Iteration 4431 / 14700) loss: 0.574024
(Iteration 4441 / 14700) loss: 0.673848
(Iteration 4451 / 14700) loss: 0.587146
(Iteration 4461 / 14700) loss: 0.480675
(Iteration 4471 / 14700) loss: 0.654288
(Iteration 4481 / 14700) loss: 0.652792
(Iteration 4491 / 14700) loss: 0.736200
(Iteration 4501 / 14700) loss: 0.569540
(Iteration 4511 / 14700) loss: 0.617380
(Iteration 4521 / 14700) loss: 0.789019
(Iteration 4531 / 14700) loss: 0.627985
(Iteration 4541 / 14700) loss: 0.634579
(Iteration 4551 / 14700) loss: 0.337911
(Iteration 4561 / 14700) loss: 0.578895
(Iteration 4571 / 14700) loss: 0.506240
(Iteration 4581 / 14700) loss: 0.584657
(Iteration 4591 / 14700) loss: 0.587616
(Iteration 4601 / 14700) loss: 0.670205
(Iteration 4611 / 14700) loss: 0.444403
(Iteration 4621 / 14700) loss: 0.525781
(Iteration 4631 / 14700) loss: 0.564381
(Iteration 4641 / 14700) loss: 0.542388
(Iteration 4651 / 14700) loss: 0.682063
(Iteration 4661 / 14700) loss: 0.605624
(Iteration 4671 / 14700) loss: 0.482727
(Iteration 4681 / 14700) loss: 0.608587
(Iteration 4691 / 14700) loss: 0.458788
(Iteration 4701 / 14700) loss: 0.446940
(Iteration 4711 / 14700) loss: 0.498862
(Iteration 4721 / 14700) loss: 0.528055
(Iteration 4731 / 14700) loss: 0.619381
(Iteration 4741 / 14700) loss: 0.574395
(Iteration 4751 / 14700) loss: 0.438931
(Iteration 4761 / 14700) loss: 0.542232
(Iteration 4771 / 14700) loss: 0.471736
(Iteration 4781 / 14700) loss: 0.545462
(Iteration 4791 / 14700) loss: 0.515855
(Iteration 4801 / 14700) loss: 0.490943
(Iteration 4811 / 14700) loss: 0.278097
(Iteration 4821 / 14700) loss: 0.652353
(Iteration 4831 / 14700) loss: 0.542666
(Iteration 4841 / 14700) loss: 0.374348
(Iteration 4851 / 14700) loss: 0.461924
(Iteration 4861 / 14700) loss: 0.607466
(Iteration 4871 / 14700) loss: 0.523421
(Iteration 4881 / 14700) loss: 0.667227
(Iteration 4891 / 14700) loss: 0.394382
(Epoch 10 / 30) train acc: 0.806000; val_acc: 0.565000
```

```
(Iteration 4901 / 14700) loss: 0.572372
(Iteration 4911 / 14700) loss: 0.580517
(Iteration 4921 / 14700) loss: 0.599079
(Iteration 4931 / 14700) loss: 0.421467
(Iteration 4941 / 14700) loss: 0.487017
(Iteration 4951 / 14700) loss: 0.386579
(Iteration 4961 / 14700) loss: 0.433641
(Iteration 4971 / 14700) loss: 0.462687
(Iteration 4981 / 14700) loss: 0.550213
(Iteration 4991 / 14700) loss: 0.407968
(Iteration 5001 / 14700) loss: 0.453839
(Iteration 5011 / 14700) loss: 0.529458
(Iteration 5021 / 14700) loss: 0.442328
(Iteration 5031 / 14700) loss: 0.486948
(Iteration 5041 / 14700) loss: 0.274532
(Iteration 5051 / 14700) loss: 0.580698
(Iteration 5061 / 14700) loss: 0.501753
(Iteration 5071 / 14700) loss: 0.328978
(Iteration 5081 / 14700) loss: 0.386132
(Iteration 5091 / 14700) loss: 0.684955
(Iteration 5101 / 14700) loss: 0.458861
(Iteration 5111 / 14700) loss: 0.498640
(Iteration 5121 / 14700) loss: 0.439132
(Iteration 5131 / 14700) loss: 0.649662
(Iteration 5141 / 14700) loss: 0.488499
(Iteration 5151 / 14700) loss: 0.520910
(Iteration 5161 / 14700) loss: 0.618106
(Iteration 5171 / 14700) loss: 0.498274
(Iteration 5181 / 14700) loss: 0.344360
(Iteration 5191 / 14700) loss: 0.441455
(Iteration 5201 / 14700) loss: 0.454091
(Iteration 5211 / 14700) loss: 0.381821
(Iteration 5221 / 14700) loss: 0.499350
(Iteration 5231 / 14700) loss: 0.578908
(Iteration 5241 / 14700) loss: 0.399364
(Iteration 5251 / 14700) loss: 0.375145
(Iteration 5261 / 14700) loss: 0.458088
(Iteration 5271 / 14700) loss: 0.436152
(Iteration 5281 / 14700) loss: 0.667608
(Iteration 5291 / 14700) loss: 0.575011
(Iteration 5301 / 14700) loss: 0.658676
(Iteration 5311 / 14700) loss: 0.557283
(Iteration 5321 / 14700) loss: 0.585148
(Iteration 5331 / 14700) loss: 0.635371
(Iteration 5341 / 14700) loss: 0.414455
(Iteration 5351 / 14700) loss: 0.603547
(Iteration 5361 / 14700) loss: 0.589667
(Iteration 5371 / 14700) loss: 0.238303
```

```
(Iteration 5381 / 14700) loss: 0.502390
(Epoch 11 / 30) train acc: 0.849000; val_acc: 0.570000
(Iteration 5391 / 14700) loss: 0.467520
(Iteration 5401 / 14700) loss: 0.341591
(Iteration 5411 / 14700) loss: 0.512755
(Iteration 5421 / 14700) loss: 0.403406
(Iteration 5431 / 14700) loss: 0.467834
(Iteration 5441 / 14700) loss: 0.419724
(Iteration 5451 / 14700) loss: 0.535330
(Iteration 5461 / 14700) loss: 0.472417
(Iteration 5471 / 14700) loss: 0.690882
(Iteration 5481 / 14700) loss: 0.474409
(Iteration 5491 / 14700) loss: 0.348059
(Iteration 5501 / 14700) loss: 0.416140
(Iteration 5511 / 14700) loss: 0.656542
(Iteration 5521 / 14700) loss: 0.696966
(Iteration 5531 / 14700) loss: 0.651135
(Iteration 5541 / 14700) loss: 0.417234
(Iteration 5551 / 14700) loss: 0.353315
(Iteration 5561 / 14700) loss: 0.352365
(Iteration 5571 / 14700) loss: 0.335280
(Iteration 5581 / 14700) loss: 0.441325
(Iteration 5591 / 14700) loss: 0.678709
(Iteration 5601 / 14700) loss: 0.609193
(Iteration 5611 / 14700) loss: 0.318384
(Iteration 5621 / 14700) loss: 0.358615
(Iteration 5631 / 14700) loss: 0.471204
(Iteration 5641 / 14700) loss: 0.379520
(Iteration 5651 / 14700) loss: 0.558919
(Iteration 5661 / 14700) loss: 0.575910
(Iteration 5671 / 14700) loss: 0.487533
(Iteration 5681 / 14700) loss: 0.489168
(Iteration 5691 / 14700) loss: 0.417857
(Iteration 5701 / 14700) loss: 0.423265
(Iteration 5711 / 14700) loss: 0.457152
(Iteration 5721 / 14700) loss: 0.378931
(Iteration 5731 / 14700) loss: 0.560348
(Iteration 5741 / 14700) loss: 0.406249
(Iteration 5751 / 14700) loss: 0.402769
(Iteration 5761 / 14700) loss: 0.299406
(Iteration 5771 / 14700) loss: 0.453307
(Iteration 5781 / 14700) loss: 0.345092
(Iteration 5791 / 14700) loss: 0.474036
(Iteration 5801 / 14700) loss: 0.404850
(Iteration 5811 / 14700) loss: 0.339228
(Iteration 5821 / 14700) loss: 0.461714
(Iteration 5831 / 14700) loss: 0.382265
(Iteration 5841 / 14700) loss: 0.259283
```

```
(Iteration 5851 / 14700) loss: 0.414007
(Iteration 5861 / 14700) loss: 0.560884
(Iteration 5871 / 14700) loss: 0.374970
(Epoch 12 / 30) train acc: 0.859000; val_acc: 0.547000
(Iteration 5881 / 14700) loss: 0.475448
(Iteration 5891 / 14700) loss: 0.555287
(Iteration 5901 / 14700) loss: 0.448282
(Iteration 5911 / 14700) loss: 0.309759
(Iteration 5921 / 14700) loss: 0.361121
(Iteration 5931 / 14700) loss: 0.419864
(Iteration 5941 / 14700) loss: 0.489509
(Iteration 5951 / 14700) loss: 0.407311
(Iteration 5961 / 14700) loss: 0.365499
(Iteration 5971 / 14700) loss: 0.321253
(Iteration 5981 / 14700) loss: 0.335892
(Iteration 5991 / 14700) loss: 0.458112
(Iteration 6001 / 14700) loss: 0.354184
(Iteration 6011 / 14700) loss: 0.529276
(Iteration 6021 / 14700) loss: 0.321392
(Iteration 6031 / 14700) loss: 0.358732
(Iteration 6041 / 14700) loss: 0.443345
(Iteration 6051 / 14700) loss: 0.392158
(Iteration 6061 / 14700) loss: 0.458607
(Iteration 6071 / 14700) loss: 0.381712
(Iteration 6081 / 14700) loss: 0.551129
(Iteration 6091 / 14700) loss: 0.353133
(Iteration 6101 / 14700) loss: 0.504072
(Iteration 6111 / 14700) loss: 0.364503
(Iteration 6121 / 14700) loss: 0.274242
(Iteration 6131 / 14700) loss: 0.324135
(Iteration 6141 / 14700) loss: 0.457030
(Iteration 6151 / 14700) loss: 0.325839
(Iteration 6161 / 14700) loss: 0.413293
(Iteration 6171 / 14700) loss: 0.362914
(Iteration 6181 / 14700) loss: 0.321970
(Iteration 6191 / 14700) loss: 0.361741
(Iteration 6201 / 14700) loss: 0.312048
(Iteration 6211 / 14700) loss: 0.349054
(Iteration 6221 / 14700) loss: 0.471696
(Iteration 6231 / 14700) loss: 0.313311
(Iteration 6241 / 14700) loss: 0.387056
(Iteration 6251 / 14700) loss: 0.413611
(Iteration 6261 / 14700) loss: 0.426549
(Iteration 6271 / 14700) loss: 0.560673
(Iteration 6281 / 14700) loss: 0.293210
(Iteration 6291 / 14700) loss: 0.421322
(Iteration 6301 / 14700) loss: 0.564523
(Iteration 6311 / 14700) loss: 0.422006
```

```
(Iteration 6321 / 14700) loss: 0.321101
(Iteration 6331 / 14700) loss: 0.399410
(Iteration 6341 / 14700) loss: 0.295902
(Iteration 6351 / 14700) loss: 0.588820
(Iteration 6361 / 14700) loss: 0.355881
(Epoch 13 / 30) train acc: 0.884000; val_acc: 0.549000
(Iteration 6371 / 14700) loss: 0.444993
(Iteration 6381 / 14700) loss: 0.335059
(Iteration 6391 / 14700) loss: 0.468235
(Iteration 6401 / 14700) loss: 0.304061
(Iteration 6411 / 14700) loss: 0.379137
(Iteration 6421 / 14700) loss: 0.234277
(Iteration 6431 / 14700) loss: 0.448215
(Iteration 6441 / 14700) loss: 0.276298
(Iteration 6451 / 14700) loss: 0.369976
(Iteration 6461 / 14700) loss: 0.339629
(Iteration 6471 / 14700) loss: 0.291199
(Iteration 6481 / 14700) loss: 0.286293
(Iteration 6491 / 14700) loss: 0.339535
(Iteration 6501 / 14700) loss: 0.356150
(Iteration 6511 / 14700) loss: 0.167829
(Iteration 6521 / 14700) loss: 0.454496
(Iteration 6531 / 14700) loss: 0.428914
(Iteration 6541 / 14700) loss: 0.191280
(Iteration 6551 / 14700) loss: 0.229526
(Iteration 6561 / 14700) loss: 0.353517
(Iteration 6571 / 14700) loss: 0.355740
(Iteration 6581 / 14700) loss: 0.278324
(Iteration 6591 / 14700) loss: 0.226181
(Iteration 6601 / 14700) loss: 0.419012
(Iteration 6611 / 14700) loss: 0.277225
(Iteration 6621 / 14700) loss: 0.456158
(Iteration 6631 / 14700) loss: 0.292337
(Iteration 6641 / 14700) loss: 0.235934
(Iteration 6651 / 14700) loss: 0.186729
(Iteration 6661 / 14700) loss: 0.410550
(Iteration 6671 / 14700) loss: 0.323040
(Iteration 6681 / 14700) loss: 0.244881
(Iteration 6691 / 14700) loss: 0.350442
(Iteration 6701 / 14700) loss: 0.349557
(Iteration 6711 / 14700) loss: 0.418328
(Iteration 6721 / 14700) loss: 0.426217
(Iteration 6731 / 14700) loss: 0.286974
(Iteration 6741 / 14700) loss: 0.229846
(Iteration 6751 / 14700) loss: 0.345847
(Iteration 6761 / 14700) loss: 0.279390
(Iteration 6771 / 14700) loss: 0.203773
(Iteration 6781 / 14700) loss: 0.284958
```

```
(Iteration 6791 / 14700) loss: 0.387124
(Iteration 6801 / 14700) loss: 0.333121
(Iteration 6811 / 14700) loss: 0.296184
(Iteration 6821 / 14700) loss: 0.253128
(Iteration 6831 / 14700) loss: 0.420531
(Iteration 6841 / 14700) loss: 0.327555
(Iteration 6851 / 14700) loss: 0.375427
(Epoch 14 / 30) train acc: 0.906000; val_acc: 0.540000
(Iteration 6861 / 14700) loss: 0.233952
(Iteration 6871 / 14700) loss: 0.300990
(Iteration 6881 / 14700) loss: 0.402246
(Iteration 6891 / 14700) loss: 0.371327
(Iteration 6901 / 14700) loss: 0.283248
(Iteration 6911 / 14700) loss: 0.351217
(Iteration 6921 / 14700) loss: 0.321178
(Iteration 6931 / 14700) loss: 0.242882
(Iteration 6941 / 14700) loss: 0.385391
(Iteration 6951 / 14700) loss: 0.352278
(Iteration 6961 / 14700) loss: 0.438838
(Iteration 6971 / 14700) loss: 0.448874
(Iteration 6981 / 14700) loss: 0.359694
(Iteration 6991 / 14700) loss: 0.248795
(Iteration 7001 / 14700) loss: 0.242357
(Iteration 7011 / 14700) loss: 0.407860
(Iteration 7021 / 14700) loss: 0.348701
(Iteration 7031 / 14700) loss: 0.374207
(Iteration 7041 / 14700) loss: 0.366390
(Iteration 7051 / 14700) loss: 0.253238
(Iteration 7061 / 14700) loss: 0.331216
(Iteration 7071 / 14700) loss: 0.302333
(Iteration 7081 / 14700) loss: 0.311352
(Iteration 7091 / 14700) loss: 0.368415
(Iteration 7101 / 14700) loss: 0.299833
(Iteration 7111 / 14700) loss: 0.269194
(Iteration 7121 / 14700) loss: 0.227159
(Iteration 7131 / 14700) loss: 0.244212
(Iteration 7141 / 14700) loss: 0.361874
(Iteration 7151 / 14700) loss: 0.281917
(Iteration 7161 / 14700) loss: 0.456234
(Iteration 7171 / 14700) loss: 0.284049
(Iteration 7181 / 14700) loss: 0.207512
(Iteration 7191 / 14700) loss: 0.255377
(Iteration 7201 / 14700) loss: 0.262525
(Iteration 7211 / 14700) loss: 0.212675
(Iteration 7221 / 14700) loss: 0.410065
(Iteration 7231 / 14700) loss: 0.301582
(Iteration 7241 / 14700) loss: 0.230341
(Iteration 7251 / 14700) loss: 0.339407
```

```
(Iteration 7261 / 14700) loss: 0.246707
(Iteration 7271 / 14700) loss: 0.267335
(Iteration 7281 / 14700) loss: 0.231729
(Iteration 7291 / 14700) loss: 0.335906
(Iteration 7301 / 14700) loss: 0.356769
(Iteration 7311 / 14700) loss: 0.361252
(Iteration 7321 / 14700) loss: 0.288878
(Iteration 7331 / 14700) loss: 0.326662
(Iteration 7341 / 14700) loss: 0.296328
(Epoch 15 / 30) train acc: 0.904000; val_acc: 0.565000
(Iteration 7351 / 14700) loss: 0.252138
(Iteration 7361 / 14700) loss: 0.391859
(Iteration 7371 / 14700) loss: 0.337332
(Iteration 7381 / 14700) loss: 0.240467
(Iteration 7391 / 14700) loss: 0.208806
(Iteration 7401 / 14700) loss: 0.269475
(Iteration 7411 / 14700) loss: 0.278794
(Iteration 7421 / 14700) loss: 0.286576
(Iteration 7431 / 14700) loss: 0.334330
(Iteration 7441 / 14700) loss: 0.228379
(Iteration 7451 / 14700) loss: 0.196277
(Iteration 7461 / 14700) loss: 0.306289
(Iteration 7471 / 14700) loss: 0.218057
(Iteration 7481 / 14700) loss: 0.287167
(Iteration 7491 / 14700) loss: 0.236078
(Iteration 7501 / 14700) loss: 0.222360
(Iteration 7511 / 14700) loss: 0.223794
(Iteration 7521 / 14700) loss: 0.215574
(Iteration 7531 / 14700) loss: 0.366705
(Iteration 7541 / 14700) loss: 0.176686
(Iteration 7551 / 14700) loss: 0.282089
(Iteration 7561 / 14700) loss: 0.225225
(Iteration 7571 / 14700) loss: 0.240663
(Iteration 7581 / 14700) loss: 0.425031
(Iteration 7591 / 14700) loss: 0.225615
(Iteration 7601 / 14700) loss: 0.137202
(Iteration 7611 / 14700) loss: 0.211898
(Iteration 7621 / 14700) loss: 0.240181
(Iteration 7631 / 14700) loss: 0.218359
(Iteration 7641 / 14700) loss: 0.257510
(Iteration 7651 / 14700) loss: 0.270057
(Iteration 7661 / 14700) loss: 0.339156
(Iteration 7671 / 14700) loss: 0.283264
(Iteration 7681 / 14700) loss: 0.215737
(Iteration 7691 / 14700) loss: 0.255598
(Iteration 7701 / 14700) loss: 0.254520
(Iteration 7711 / 14700) loss: 0.309936
(Iteration 7721 / 14700) loss: 0.275352
```

```
(Iteration 7731 / 14700) loss: 0.204468
(Iteration 7741 / 14700) loss: 0.302038
(Iteration 7751 / 14700) loss: 0.253593
(Iteration 7761 / 14700) loss: 0.299237
(Iteration 7771 / 14700) loss: 0.244536
(Iteration 7781 / 14700) loss: 0.206269
(Iteration 7791 / 14700) loss: 0.149425
(Iteration 7801 / 14700) loss: 0.173034
(Iteration 7811 / 14700) loss: 0.220595
(Iteration 7821 / 14700) loss: 0.196392
(Iteration 7831 / 14700) loss: 0.264491
(Epoch 16 / 30) train acc: 0.913000; val_acc: 0.553000
(Iteration 7841 / 14700) loss: 0.267011
(Iteration 7851 / 14700) loss: 0.207695
(Iteration 7861 / 14700) loss: 0.231391
(Iteration 7871 / 14700) loss: 0.214946
(Iteration 7881 / 14700) loss: 0.233676
(Iteration 7891 / 14700) loss: 0.216287
(Iteration 7901 / 14700) loss: 0.217336
(Iteration 7911 / 14700) loss: 0.164822
(Iteration 7921 / 14700) loss: 0.299372
(Iteration 7931 / 14700) loss: 0.192575
(Iteration 7941 / 14700) loss: 0.205350
(Iteration 7951 / 14700) loss: 0.254877
(Iteration 7961 / 14700) loss: 0.406666
(Iteration 7971 / 14700) loss: 0.178650
(Iteration 7981 / 14700) loss: 0.228094
(Iteration 7991 / 14700) loss: 0.239261
(Iteration 8001 / 14700) loss: 0.237143
(Iteration 8011 / 14700) loss: 0.213815
(Iteration 8021 / 14700) loss: 0.164741
(Iteration 8031 / 14700) loss: 0.198628
(Iteration 8041 / 14700) loss: 0.211586
(Iteration 8051 / 14700) loss: 0.316514
(Iteration 8061 / 14700) loss: 0.179591
(Iteration 8071 / 14700) loss: 0.178961
(Iteration 8081 / 14700) loss: 0.221266
(Iteration 8091 / 14700) loss: 0.189570
(Iteration 8101 / 14700) loss: 0.161579
(Iteration 8111 / 14700) loss: 0.238387
(Iteration 8121 / 14700) loss: 0.221199
(Iteration 8131 / 14700) loss: 0.330326
(Iteration 8141 / 14700) loss: 0.351507
(Iteration 8151 / 14700) loss: 0.233221
(Iteration 8161 / 14700) loss: 0.344716
(Iteration 8171 / 14700) loss: 0.223323
(Iteration 8181 / 14700) loss: 0.154068
(Iteration 8191 / 14700) loss: 0.254564
```

```
(Iteration 8201 / 14700) loss: 0.206759
(Iteration 8211 / 14700) loss: 0.243756
(Iteration 8221 / 14700) loss: 0.273190
(Iteration 8231 / 14700) loss: 0.202240
(Iteration 8241 / 14700) loss: 0.237300
(Iteration 8251 / 14700) loss: 0.304372
(Iteration 8261 / 14700) loss: 0.145406
(Iteration 8271 / 14700) loss: 0.264820
(Iteration 8281 / 14700) loss: 0.227145
(Iteration 8291 / 14700) loss: 0.312433
(Iteration 8301 / 14700) loss: 0.215035
(Iteration 8311 / 14700) loss: 0.139994
(Iteration 8321 / 14700) loss: 0.359779
(Epoch 17 / 30) train acc: 0.930000; val_acc: 0.557000
(Iteration 8331 / 14700) loss: 0.312476
(Iteration 8341 / 14700) loss: 0.193272
(Iteration 8351 / 14700) loss: 0.137242
(Iteration 8361 / 14700) loss: 0.258160
(Iteration 8371 / 14700) loss: 0.251156
(Iteration 8381 / 14700) loss: 0.203605
(Iteration 8391 / 14700) loss: 0.184507
(Iteration 8401 / 14700) loss: 0.192063
(Iteration 8411 / 14700) loss: 0.153387
(Iteration 8421 / 14700) loss: 0.131823
(Iteration 8431 / 14700) loss: 0.185612
(Iteration 8441 / 14700) loss: 0.213753
(Iteration 8451 / 14700) loss: 0.173506
(Iteration 8461 / 14700) loss: 0.284571
(Iteration 8471 / 14700) loss: 0.161161
(Iteration 8481 / 14700) loss: 0.207550
(Iteration 8491 / 14700) loss: 0.132492
(Iteration 8501 / 14700) loss: 0.193896
(Iteration 8511 / 14700) loss: 0.268878
(Iteration 8521 / 14700) loss: 0.144269
(Iteration 8531 / 14700) loss: 0.223106
(Iteration 8541 / 14700) loss: 0.212250
(Iteration 8551 / 14700) loss: 0.241540
(Iteration 8561 / 14700) loss: 0.210578
(Iteration 8571 / 14700) loss: 0.318379
(Iteration 8581 / 14700) loss: 0.220370
(Iteration 8591 / 14700) loss: 0.286105
(Iteration 8601 / 14700) loss: 0.144122
(Iteration 8611 / 14700) loss: 0.167496
(Iteration 8621 / 14700) loss: 0.237656
(Iteration 8631 / 14700) loss: 0.153083
(Iteration 8641 / 14700) loss: 0.188801
(Iteration 8651 / 14700) loss: 0.249776
(Iteration 8661 / 14700) loss: 0.173646
```

```
(Iteration 8671 / 14700) loss: 0.282409
(Iteration 8681 / 14700) loss: 0.239236
(Iteration 8691 / 14700) loss: 0.192204
(Iteration 8701 / 14700) loss: 0.164413
(Iteration 8711 / 14700) loss: 0.135315
(Iteration 8721 / 14700) loss: 0.243679
(Iteration 8731 / 14700) loss: 0.232534
(Iteration 8741 / 14700) loss: 0.138976
(Iteration 8751 / 14700) loss: 0.254647
(Iteration 8761 / 14700) loss: 0.229573
(Iteration 8771 / 14700) loss: 0.206266
(Iteration 8781 / 14700) loss: 0.231026
(Iteration 8791 / 14700) loss: 0.185872
(Iteration 8801 / 14700) loss: 0.117659
(Iteration 8811 / 14700) loss: 0.101883
(Epoch 18 / 30) train acc: 0.946000; val_acc: 0.567000
(Iteration 8821 / 14700) loss: 0.168176
(Iteration 8831 / 14700) loss: 0.143770
(Iteration 8841 / 14700) loss: 0.140375
(Iteration 8851 / 14700) loss: 0.156672
(Iteration 8861 / 14700) loss: 0.162907
(Iteration 8871 / 14700) loss: 0.232484
(Iteration 8881 / 14700) loss: 0.250274
(Iteration 8891 / 14700) loss: 0.199388
(Iteration 8901 / 14700) loss: 0.145267
(Iteration 8911 / 14700) loss: 0.158100
(Iteration 8921 / 14700) loss: 0.139312
(Iteration 8931 / 14700) loss: 0.222763
(Iteration 8941 / 14700) loss: 0.152379
(Iteration 8951 / 14700) loss: 0.173436
(Iteration 8961 / 14700) loss: 0.206863
(Iteration 8971 / 14700) loss: 0.256087
(Iteration 8981 / 14700) loss: 0.116118
(Iteration 8991 / 14700) loss: 0.107200
(Iteration 9001 / 14700) loss: 0.224940
(Iteration 9011 / 14700) loss: 0.130102
(Iteration 9021 / 14700) loss: 0.187272
(Iteration 9031 / 14700) loss: 0.198325
(Iteration 9041 / 14700) loss: 0.214714
(Iteration 9051 / 14700) loss: 0.172537
(Iteration 9061 / 14700) loss: 0.108270
(Iteration 9071 / 14700) loss: 0.126859
(Iteration 9081 / 14700) loss: 0.113640
(Iteration 9091 / 14700) loss: 0.190583
(Iteration 9101 / 14700) loss: 0.163980
(Iteration 9111 / 14700) loss: 0.179329
(Iteration 9121 / 14700) loss: 0.144998
(Iteration 9131 / 14700) loss: 0.187265
```

```
(Iteration 9141 / 14700) loss: 0.185487
(Iteration 9151 / 14700) loss: 0.231221
(Iteration 9161 / 14700) loss: 0.269018
(Iteration 9171 / 14700) loss: 0.110555
(Iteration 9181 / 14700) loss: 0.194439
(Iteration 9191 / 14700) loss: 0.227996
(Iteration 9201 / 14700) loss: 0.148287
(Iteration 9211 / 14700) loss: 0.142049
(Iteration 9221 / 14700) loss: 0.149997
(Iteration 9231 / 14700) loss: 0.222574
(Iteration 9241 / 14700) loss: 0.116229
(Iteration 9251 / 14700) loss: 0.094604
(Iteration 9261 / 14700) loss: 0.176251
(Iteration 9271 / 14700) loss: 0.112521
(Iteration 9281 / 14700) loss: 0.220063
(Iteration 9291 / 14700) loss: 0.107826
(Iteration 9301 / 14700) loss: 0.160585
(Epoch 19 / 30) train acc: 0.957000; val_acc: 0.562000
(Iteration 9311 / 14700) loss: 0.179004
(Iteration 9321 / 14700) loss: 0.125898
(Iteration 9331 / 14700) loss: 0.235186
(Iteration 9341 / 14700) loss: 0.131761
(Iteration 9351 / 14700) loss: 0.132336
(Iteration 9361 / 14700) loss: 0.108975
(Iteration 9371 / 14700) loss: 0.153171
(Iteration 9381 / 14700) loss: 0.102208
(Iteration 9391 / 14700) loss: 0.095349
(Iteration 9401 / 14700) loss: 0.095641
(Iteration 9411 / 14700) loss: 0.145690
(Iteration 9421 / 14700) loss: 0.143587
(Iteration 9431 / 14700) loss: 0.212545
(Iteration 9441 / 14700) loss: 0.155967
(Iteration 9451 / 14700) loss: 0.129919
(Iteration 9461 / 14700) loss: 0.149740
(Iteration 9471 / 14700) loss: 0.264285
(Iteration 9481 / 14700) loss: 0.151542
(Iteration 9491 / 14700) loss: 0.154861
(Iteration 9501 / 14700) loss: 0.084344
(Iteration 9511 / 14700) loss: 0.118190
(Iteration 9521 / 14700) loss: 0.174123
(Iteration 9531 / 14700) loss: 0.182172
(Iteration 9541 / 14700) loss: 0.139364
(Iteration 9551 / 14700) loss: 0.109396
(Iteration 9561 / 14700) loss: 0.166017
(Iteration 9571 / 14700) loss: 0.098302
(Iteration 9581 / 14700) loss: 0.159893
(Iteration 9591 / 14700) loss: 0.237760
(Iteration 9601 / 14700) loss: 0.224687
```

```
(Iteration 9611 / 14700) loss: 0.092784
(Iteration 9621 / 14700) loss: 0.182935
(Iteration 9631 / 14700) loss: 0.075593
(Iteration 9641 / 14700) loss: 0.187930
(Iteration 9651 / 14700) loss: 0.114153
(Iteration 9661 / 14700) loss: 0.156581
(Iteration 9671 / 14700) loss: 0.118464
(Iteration 9681 / 14700) loss: 0.148954
(Iteration 9691 / 14700) loss: 0.088198
(Iteration 9701 / 14700) loss: 0.094222
(Iteration 9711 / 14700) loss: 0.214330
(Iteration 9721 / 14700) loss: 0.153185
(Iteration 9731 / 14700) loss: 0.080062
(Iteration 9741 / 14700) loss: 0.119733
(Iteration 9751 / 14700) loss: 0.065197
(Iteration 9761 / 14700) loss: 0.097018
(Iteration 9771 / 14700) loss: 0.099711
(Iteration 9781 / 14700) loss: 0.077275
(Iteration 9791 / 14700) loss: 0.129530
(Epoch 20 / 30) train acc: 0.966000; val_acc: 0.547000
(Iteration 9801 / 14700) loss: 0.187943
(Iteration 9811 / 14700) loss: 0.110592
(Iteration 9821 / 14700) loss: 0.097813
(Iteration 9831 / 14700) loss: 0.103753
(Iteration 9841 / 14700) loss: 0.115300
(Iteration 9851 / 14700) loss: 0.120857
(Iteration 9861 / 14700) loss: 0.116092
(Iteration 9871 / 14700) loss: 0.130267
(Iteration 9881 / 14700) loss: 0.133439
(Iteration 9891 / 14700) loss: 0.092510
(Iteration 9901 / 14700) loss: 0.126110
(Iteration 9911 / 14700) loss: 0.104468
(Iteration 9921 / 14700) loss: 0.178639
(Iteration 9931 / 14700) loss: 0.093781
(Iteration 9941 / 14700) loss: 0.126495
(Iteration 9951 / 14700) loss: 0.083005
(Iteration 9961 / 14700) loss: 0.122650
(Iteration 9971 / 14700) loss: 0.097663
(Iteration 9981 / 14700) loss: 0.183674
(Iteration 9991 / 14700) loss: 0.107989
(Iteration 10001 / 14700) loss: 0.078563
(Iteration 10011 / 14700) loss: 0.073353
(Iteration 10021 / 14700) loss: 0.092847
(Iteration 10031 / 14700) loss: 0.075756
(Iteration 10041 / 14700) loss: 0.083563
(Iteration 10051 / 14700) loss: 0.090916
(Iteration 10061 / 14700) loss: 0.156762
(Iteration 10071 / 14700) loss: 0.067799
```

```
(Iteration 10081 / 14700) loss: 0.171569
(Iteration 10091 / 14700) loss: 0.171892
(Iteration 10101 / 14700) loss: 0.215698
(Iteration 10111 / 14700) loss: 0.155851
(Iteration 10121 / 14700) loss: 0.101673
(Iteration 10131 / 14700) loss: 0.107728
(Iteration 10141 / 14700) loss: 0.094439
(Iteration 10151 / 14700) loss: 0.103193
(Iteration 10161 / 14700) loss: 0.171783
(Iteration 10171 / 14700) loss: 0.074175
(Iteration 10181 / 14700) loss: 0.133162
(Iteration 10191 / 14700) loss: 0.221035
(Iteration 10201 / 14700) loss: 0.108023
(Iteration 10211 / 14700) loss: 0.102142
(Iteration 10221 / 14700) loss: 0.122069
(Iteration 10231 / 14700) loss: 0.112709
(Iteration 10241 / 14700) loss: 0.114263
(Iteration 10251 / 14700) loss: 0.100894
(Iteration 10261 / 14700) loss: 0.063376
(Iteration 10271 / 14700) loss: 0.115792
(Iteration 10281 / 14700) loss: 0.083919
(Epoch 21 / 30) train acc: 0.973000; val_acc: 0.557000
(Iteration 10291 / 14700) loss: 0.065822
(Iteration 10301 / 14700) loss: 0.150779
(Iteration 10311 / 14700) loss: 0.096986
(Iteration 10321 / 14700) loss: 0.071572
(Iteration 10331 / 14700) loss: 0.073261
(Iteration 10341 / 14700) loss: 0.102860
(Iteration 10351 / 14700) loss: 0.178361
(Iteration 10361 / 14700) loss: 0.059307
(Iteration 10371 / 14700) loss: 0.115111
(Iteration 10381 / 14700) loss: 0.123237
(Iteration 10391 / 14700) loss: 0.106073
(Iteration 10401 / 14700) loss: 0.124684
(Iteration 10411 / 14700) loss: 0.124334
(Iteration 10421 / 14700) loss: 0.106166
(Iteration 10431 / 14700) loss: 0.119194
(Iteration 10441 / 14700) loss: 0.128136
(Iteration 10451 / 14700) loss: 0.165025
(Iteration 10461 / 14700) loss: 0.177950
(Iteration 10471 / 14700) loss: 0.069740
(Iteration 10481 / 14700) loss: 0.110135
(Iteration 10491 / 14700) loss: 0.142528
(Iteration 10501 / 14700) loss: 0.094175
(Iteration 10511 / 14700) loss: 0.103728
(Iteration 10521 / 14700) loss: 0.190680
(Iteration 10531 / 14700) loss: 0.069614
(Iteration 10541 / 14700) loss: 0.085307
```

```
(Iteration 10551 / 14700) loss: 0.108010
(Iteration 10561 / 14700) loss: 0.097385
(Iteration 10571 / 14700) loss: 0.134437
(Iteration 10581 / 14700) loss: 0.076988
(Iteration 10591 / 14700) loss: 0.066816
(Iteration 10601 / 14700) loss: 0.147075
(Iteration 10611 / 14700) loss: 0.084771
(Iteration 10621 / 14700) loss: 0.099875
(Iteration 10631 / 14700) loss: 0.075203
(Iteration 10641 / 14700) loss: 0.134545
(Iteration 10651 / 14700) loss: 0.119772
(Iteration 10661 / 14700) loss: 0.091882
(Iteration 10671 / 14700) loss: 0.071431
(Iteration 10681 / 14700) loss: 0.115079
(Iteration 10691 / 14700) loss: 0.074817
(Iteration 10701 / 14700) loss: 0.129156
(Iteration 10711 / 14700) loss: 0.092375
(Iteration 10721 / 14700) loss: 0.115133
(Iteration 10731 / 14700) loss: 0.086813
(Iteration 10741 / 14700) loss: 0.122578
(Iteration 10751 / 14700) loss: 0.091346
(Iteration 10761 / 14700) loss: 0.081660
(Iteration 10771 / 14700) loss: 0.083078
(Epoch 22 / 30) train acc: 0.988000; val_acc: 0.549000
(Iteration 10781 / 14700) loss: 0.060423
(Iteration 10791 / 14700) loss: 0.143914
(Iteration 10801 / 14700) loss: 0.104996
(Iteration 10811 / 14700) loss: 0.085890
(Iteration 10821 / 14700) loss: 0.085936
(Iteration 10831 / 14700) loss: 0.127105
(Iteration 10841 / 14700) loss: 0.084788
(Iteration 10851 / 14700) loss: 0.089527
(Iteration 10861 / 14700) loss: 0.089691
(Iteration 10871 / 14700) loss: 0.081778
(Iteration 10881 / 14700) loss: 0.079866
(Iteration 10891 / 14700) loss: 0.094317
(Iteration 10901 / 14700) loss: 0.108646
(Iteration 10911 / 14700) loss: 0.077489
(Iteration 10921 / 14700) loss: 0.068407
(Iteration 10931 / 14700) loss: 0.090327
(Iteration 10941 / 14700) loss: 0.081380
(Iteration 10951 / 14700) loss: 0.110459
(Iteration 10961 / 14700) loss: 0.094833
(Iteration 10971 / 14700) loss: 0.049696
(Iteration 10981 / 14700) loss: 0.086795
(Iteration 10991 / 14700) loss: 0.055674
(Iteration 11001 / 14700) loss: 0.058129
(Iteration 11011 / 14700) loss: 0.083036
```

```
(Iteration 11021 / 14700) loss: 0.130277
(Iteration 11031 / 14700) loss: 0.051780
(Iteration 11041 / 14700) loss: 0.077377
(Iteration 11051 / 14700) loss: 0.088490
(Iteration 11061 / 14700) loss: 0.080058
(Iteration 11071 / 14700) loss: 0.079303
(Iteration 11081 / 14700) loss: 0.097444
(Iteration 11091 / 14700) loss: 0.107679
(Iteration 11101 / 14700) loss: 0.100954
(Iteration 11111 / 14700) loss: 0.130606
(Iteration 11121 / 14700) loss: 0.076296
(Iteration 11131 / 14700) loss: 0.174957
(Iteration 11141 / 14700) loss: 0.113528
(Iteration 11151 / 14700) loss: 0.096128
(Iteration 11161 / 14700) loss: 0.105317
(Iteration 11171 / 14700) loss: 0.059555
(Iteration 11181 / 14700) loss: 0.187430
(Iteration 11191 / 14700) loss: 0.102759
(Iteration 11201 / 14700) loss: 0.091679
(Iteration 11211 / 14700) loss: 0.107086
(Iteration 11221 / 14700) loss: 0.072042
(Iteration 11231 / 14700) loss: 0.102979
(Iteration 11241 / 14700) loss: 0.052496
(Iteration 11251 / 14700) loss: 0.095113
(Iteration 11261 / 14700) loss: 0.099046
(Epoch 23 / 30) train acc: 0.996000; val_acc: 0.546000
(Iteration 11271 / 14700) loss: 0.059205
(Iteration 11281 / 14700) loss: 0.111991
(Iteration 11291 / 14700) loss: 0.079686
(Iteration 11301 / 14700) loss: 0.101980
(Iteration 11311 / 14700) loss: 0.086871
(Iteration 11321 / 14700) loss: 0.069383
(Iteration 11331 / 14700) loss: 0.073721
(Iteration 11341 / 14700) loss: 0.106648
(Iteration 11351 / 14700) loss: 0.064956
(Iteration 11361 / 14700) loss: 0.093539
(Iteration 11371 / 14700) loss: 0.113459
(Iteration 11381 / 14700) loss: 0.126472
(Iteration 11391 / 14700) loss: 0.076512
(Iteration 11401 / 14700) loss: 0.102613
(Iteration 11411 / 14700) loss: 0.089650
(Iteration 11421 / 14700) loss: 0.079164
(Iteration 11431 / 14700) loss: 0.079982
(Iteration 11441 / 14700) loss: 0.087457
(Iteration 11451 / 14700) loss: 0.070490
(Iteration 11461 / 14700) loss: 0.046530
(Iteration 11471 / 14700) loss: 0.105492
(Iteration 11481 / 14700) loss: 0.073336
```

```
(Iteration 11491 / 14700) loss: 0.053289
(Iteration 11501 / 14700) loss: 0.065305
(Iteration 11511 / 14700) loss: 0.067064
(Iteration 11521 / 14700) loss: 0.091450
(Iteration 11531 / 14700) loss: 0.082219
(Iteration 11541 / 14700) loss: 0.047719
(Iteration 11551 / 14700) loss: 0.085089
(Iteration 11561 / 14700) loss: 0.046953
(Iteration 11571 / 14700) loss: 0.059023
(Iteration 11581 / 14700) loss: 0.066926
(Iteration 11591 / 14700) loss: 0.053524
(Iteration 11601 / 14700) loss: 0.071352
(Iteration 11611 / 14700) loss: 0.077195
(Iteration 11621 / 14700) loss: 0.059446
(Iteration 11631 / 14700) loss: 0.069989
(Iteration 11641 / 14700) loss: 0.081527
(Iteration 11651 / 14700) loss: 0.072046
(Iteration 11661 / 14700) loss: 0.096840
(Iteration 11671 / 14700) loss: 0.044224
(Iteration 11681 / 14700) loss: 0.052915
(Iteration 11691 / 14700) loss: 0.053107
(Iteration 11701 / 14700) loss: 0.069867
(Iteration 11711 / 14700) loss: 0.095252
(Iteration 11721 / 14700) loss: 0.085006
(Iteration 11731 / 14700) loss: 0.078052
(Iteration 11741 / 14700) loss: 0.076291
(Iteration 11751 / 14700) loss: 0.071627
(Epoch 24 / 30) train acc: 0.990000; val_acc: 0.550000
(Iteration 11761 / 14700) loss: 0.050008
(Iteration 11771 / 14700) loss: 0.064557
(Iteration 11781 / 14700) loss: 0.063605
(Iteration 11791 / 14700) loss: 0.092653
(Iteration 11801 / 14700) loss: 0.094219
(Iteration 11811 / 14700) loss: 0.079714
(Iteration 11821 / 14700) loss: 0.068266
(Iteration 11831 / 14700) loss: 0.058741
(Iteration 11841 / 14700) loss: 0.043455
(Iteration 11851 / 14700) loss: 0.092973
(Iteration 11861 / 14700) loss: 0.056082
(Iteration 11871 / 14700) loss: 0.072184
(Iteration 11881 / 14700) loss: 0.058734
(Iteration 11891 / 14700) loss: 0.070688
(Iteration 11901 / 14700) loss: 0.053768
(Iteration 11911 / 14700) loss: 0.046776
(Iteration 11921 / 14700) loss: 0.051748
(Iteration 11931 / 14700) loss: 0.040189
(Iteration 11941 / 14700) loss: 0.055481
(Iteration 11951 / 14700) loss: 0.051506
```

```
(Iteration 11961 / 14700) loss: 0.075726
(Iteration 11971 / 14700) loss: 0.070674
(Iteration 11981 / 14700) loss: 0.082646
(Iteration 11991 / 14700) loss: 0.065074
(Iteration 12001 / 14700) loss: 0.057218
(Iteration 12011 / 14700) loss: 0.090600
(Iteration 12021 / 14700) loss: 0.073619
(Iteration 12031 / 14700) loss: 0.049251
(Iteration 12041 / 14700) loss: 0.047001
(Iteration 12051 / 14700) loss: 0.052875
(Iteration 12061 / 14700) loss: 0.062779
(Iteration 12071 / 14700) loss: 0.057679
(Iteration 12081 / 14700) loss: 0.069093
(Iteration 12091 / 14700) loss: 0.088402
(Iteration 12101 / 14700) loss: 0.080243
(Iteration 12111 / 14700) loss: 0.054738
(Iteration 12121 / 14700) loss: 0.054917
(Iteration 12131 / 14700) loss: 0.055421
(Iteration 12141 / 14700) loss: 0.081655
(Iteration 12151 / 14700) loss: 0.071511
(Iteration 12161 / 14700) loss: 0.049585
(Iteration 12171 / 14700) loss: 0.057872
(Iteration 12181 / 14700) loss: 0.055913
(Iteration 12191 / 14700) loss: 0.070004
(Iteration 12201 / 14700) loss: 0.037874
(Iteration 12211 / 14700) loss: 0.095728
(Iteration 12221 / 14700) loss: 0.057471
(Iteration 12231 / 14700) loss: 0.094188
(Iteration 12241 / 14700) loss: 0.069828
(Epoch 25 / 30) train acc: 0.997000; val_acc: 0.557000
(Iteration 12251 / 14700) loss: 0.080449
(Iteration 12261 / 14700) loss: 0.068113
(Iteration 12271 / 14700) loss: 0.058356
(Iteration 12281 / 14700) loss: 0.055561
(Iteration 12291 / 14700) loss: 0.039376
(Iteration 12301 / 14700) loss: 0.044312
(Iteration 12311 / 14700) loss: 0.071915
(Iteration 12321 / 14700) loss: 0.046105
(Iteration 12331 / 14700) loss: 0.051336
(Iteration 12341 / 14700) loss: 0.051103
(Iteration 12351 / 14700) loss: 0.064249
(Iteration 12361 / 14700) loss: 0.063752
(Iteration 12371 / 14700) loss: 0.055149
(Iteration 12381 / 14700) loss: 0.059389
(Iteration 12391 / 14700) loss: 0.058083
(Iteration 12401 / 14700) loss: 0.050652
(Iteration 12411 / 14700) loss: 0.054121
(Iteration 12421 / 14700) loss: 0.073629
```

```
(Iteration 12431 / 14700) loss: 0.051844
(Iteration 12441 / 14700) loss: 0.068705
(Iteration 12451 / 14700) loss: 0.068930
(Iteration 12461 / 14700) loss: 0.047248
(Iteration 12471 / 14700) loss: 0.081417
(Iteration 12481 / 14700) loss: 0.067909
(Iteration 12491 / 14700) loss: 0.054803
(Iteration 12501 / 14700) loss: 0.049819
(Iteration 12511 / 14700) loss: 0.064965
(Iteration 12521 / 14700) loss: 0.047290
(Iteration 12531 / 14700) loss: 0.055605
(Iteration 12541 / 14700) loss: 0.060455
(Iteration 12551 / 14700) loss: 0.045044
(Iteration 12561 / 14700) loss: 0.067280
(Iteration 12571 / 14700) loss: 0.065636
(Iteration 12581 / 14700) loss: 0.061541
(Iteration 12591 / 14700) loss: 0.058543
(Iteration 12601 / 14700) loss: 0.036315
(Iteration 12611 / 14700) loss: 0.073159
(Iteration 12621 / 14700) loss: 0.058816
(Iteration 12631 / 14700) loss: 0.073419
(Iteration 12641 / 14700) loss: 0.052597
(Iteration 12651 / 14700) loss: 0.055427
(Iteration 12661 / 14700) loss: 0.051834
(Iteration 12671 / 14700) loss: 0.051489
(Iteration 12681 / 14700) loss: 0.054426
(Iteration 12691 / 14700) loss: 0.080079
(Iteration 12701 / 14700) loss: 0.046000
(Iteration 12711 / 14700) loss: 0.062622
(Iteration 12721 / 14700) loss: 0.041456
(Iteration 12731 / 14700) loss: 0.073003
(Epoch 26 / 30) train acc: 0.998000; val_acc: 0.559000
(Iteration 12741 / 14700) loss: 0.067086
(Iteration 12751 / 14700) loss: 0.055002
(Iteration 12761 / 14700) loss: 0.039747
(Iteration 12771 / 14700) loss: 0.046223
(Iteration 12781 / 14700) loss: 0.041160
(Iteration 12791 / 14700) loss: 0.044526
(Iteration 12801 / 14700) loss: 0.084649
(Iteration 12811 / 14700) loss: 0.041016
(Iteration 12821 / 14700) loss: 0.051117
(Iteration 12831 / 14700) loss: 0.042670
(Iteration 12841 / 14700) loss: 0.059903
(Iteration 12851 / 14700) loss: 0.047698
(Iteration 12861 / 14700) loss: 0.064515
(Iteration 12871 / 14700) loss: 0.073593
(Iteration 12881 / 14700) loss: 0.053577
(Iteration 12891 / 14700) loss: 0.046117
```

```
(Iteration 12901 / 14700) loss: 0.042923
(Iteration 12911 / 14700) loss: 0.044435
(Iteration 12921 / 14700) loss: 0.056042
(Iteration 12931 / 14700) loss: 0.067021
(Iteration 12941 / 14700) loss: 0.062706
(Iteration 12951 / 14700) loss: 0.038016
(Iteration 12961 / 14700) loss: 0.050632
(Iteration 12971 / 14700) loss: 0.077104
(Iteration 12981 / 14700) loss: 0.088714
(Iteration 12991 / 14700) loss: 0.054252
(Iteration 13001 / 14700) loss: 0.037858
(Iteration 13011 / 14700) loss: 0.039378
(Iteration 13021 / 14700) loss: 0.045737
(Iteration 13031 / 14700) loss: 0.073481
(Iteration 13041 / 14700) loss: 0.040518
(Iteration 13051 / 14700) loss: 0.044264
(Iteration 13061 / 14700) loss: 0.070508
(Iteration 13071 / 14700) loss: 0.058111
(Iteration 13081 / 14700) loss: 0.042236
(Iteration 13091 / 14700) loss: 0.050844
(Iteration 13101 / 14700) loss: 0.056285
(Iteration 13111 / 14700) loss: 0.040364
(Iteration 13121 / 14700) loss: 0.039216
(Iteration 13131 / 14700) loss: 0.066449
(Iteration 13141 / 14700) loss: 0.046789
(Iteration 13151 / 14700) loss: 0.057881
(Iteration 13161 / 14700) loss: 0.054714
(Iteration 13171 / 14700) loss: 0.042112
(Iteration 13181 / 14700) loss: 0.055987
(Iteration 13191 / 14700) loss: 0.061507
(Iteration 13201 / 14700) loss: 0.045215
(Iteration 13211 / 14700) loss: 0.058766
(Iteration 13221 / 14700) loss: 0.055965
(Epoch 27 / 30) train acc: 0.995000; val_acc: 0.554000
(Iteration 13231 / 14700) loss: 0.041585
(Iteration 13241 / 14700) loss: 0.041206
(Iteration 13251 / 14700) loss: 0.041365
(Iteration 13261 / 14700) loss: 0.067152
(Iteration 13271 / 14700) loss: 0.053071
(Iteration 13281 / 14700) loss: 0.054462
(Iteration 13291 / 14700) loss: 0.044300
(Iteration 13301 / 14700) loss: 0.053637
(Iteration 13311 / 14700) loss: 0.036287
(Iteration 13321 / 14700) loss: 0.071916
(Iteration 13331 / 14700) loss: 0.039092
(Iteration 13341 / 14700) loss: 0.053869
(Iteration 13351 / 14700) loss: 0.066925
(Iteration 13361 / 14700) loss: 0.075299
```

```
(Iteration 13371 / 14700) loss: 0.042989
(Iteration 13381 / 14700) loss: 0.064511
(Iteration 13391 / 14700) loss: 0.050871
(Iteration 13401 / 14700) loss: 0.058855
(Iteration 13411 / 14700) loss: 0.055687
(Iteration 13421 / 14700) loss: 0.043779
(Iteration 13431 / 14700) loss: 0.052853
(Iteration 13441 / 14700) loss: 0.047474
(Iteration 13451 / 14700) loss: 0.053870
(Iteration 13461 / 14700) loss: 0.046335
(Iteration 13471 / 14700) loss: 0.050067
(Iteration 13481 / 14700) loss: 0.062425
(Iteration 13491 / 14700) loss: 0.041815
(Iteration 13501 / 14700) loss: 0.048736
(Iteration 13511 / 14700) loss: 0.047784
(Iteration 13521 / 14700) loss: 0.055352
(Iteration 13531 / 14700) loss: 0.049506
(Iteration 13541 / 14700) loss: 0.038193
(Iteration 13551 / 14700) loss: 0.050574
(Iteration 13561 / 14700) loss: 0.036397
(Iteration 13571 / 14700) loss: 0.094228
(Iteration 13581 / 14700) loss: 0.041985
(Iteration 13591 / 14700) loss: 0.050011
(Iteration 13601 / 14700) loss: 0.042047
(Iteration 13611 / 14700) loss: 0.044009
(Iteration 13621 / 14700) loss: 0.058665
(Iteration 13631 / 14700) loss: 0.052617
(Iteration 13641 / 14700) loss: 0.055592
(Iteration 13651 / 14700) loss: 0.075131
(Iteration 13661 / 14700) loss: 0.054063
(Iteration 13671 / 14700) loss: 0.076245
(Iteration 13681 / 14700) loss: 0.048854
(Iteration 13691 / 14700) loss: 0.039287
(Iteration 13701 / 14700) loss: 0.062611
(Iteration 13711 / 14700) loss: 0.045920
(Epoch 28 / 30) train acc: 0.999000; val_acc: 0.560000
(Iteration 13721 / 14700) loss: 0.086397
(Iteration 13731 / 14700) loss: 0.055709
(Iteration 13741 / 14700) loss: 0.051794
(Iteration 13751 / 14700) loss: 0.040264
(Iteration 13761 / 14700) loss: 0.055599
(Iteration 13771 / 14700) loss: 0.054813
(Iteration 13781 / 14700) loss: 0.040241
(Iteration 13791 / 14700) loss: 0.038258
(Iteration 13801 / 14700) loss: 0.049790
(Iteration 13811 / 14700) loss: 0.056390
(Iteration 13821 / 14700) loss: 0.048418
(Iteration 13831 / 14700) loss: 0.026835
```

```
(Iteration 13841 / 14700) loss: 0.045251
(Iteration 13851 / 14700) loss: 0.040692
(Iteration 13861 / 14700) loss: 0.036674
(Iteration 13871 / 14700) loss: 0.045462
(Iteration 13881 / 14700) loss: 0.057695
(Iteration 13891 / 14700) loss: 0.042114
(Iteration 13901 / 14700) loss: 0.048746
(Iteration 13911 / 14700) loss: 0.045668
(Iteration 13921 / 14700) loss: 0.053120
(Iteration 13931 / 14700) loss: 0.067543
(Iteration 13941 / 14700) loss: 0.050453
(Iteration 13951 / 14700) loss: 0.050762
(Iteration 13961 / 14700) loss: 0.046761
(Iteration 13971 / 14700) loss: 0.040366
(Iteration 13981 / 14700) loss: 0.046466
(Iteration 13991 / 14700) loss: 0.062342
(Iteration 14001 / 14700) loss: 0.035830
(Iteration 14011 / 14700) loss: 0.076833
(Iteration 14021 / 14700) loss: 0.038565
(Iteration 14031 / 14700) loss: 0.046036
(Iteration 14041 / 14700) loss: 0.065157
(Iteration 14051 / 14700) loss: 0.041084
(Iteration 14061 / 14700) loss: 0.098520
(Iteration 14071 / 14700) loss: 0.047903
(Iteration 14081 / 14700) loss: 0.046056
(Iteration 14091 / 14700) loss: 0.066205
(Iteration 14101 / 14700) loss: 0.054936
(Iteration 14111 / 14700) loss: 0.038475
(Iteration 14121 / 14700) loss: 0.036662
(Iteration 14131 / 14700) loss: 0.067983
(Iteration 14141 / 14700) loss: 0.081217
(Iteration 14151 / 14700) loss: 0.046502
(Iteration 14161 / 14700) loss: 0.034089
(Iteration 14171 / 14700) loss: 0.042900
(Iteration 14181 / 14700) loss: 0.050018
(Iteration 14191 / 14700) loss: 0.042629
(Iteration 14201 / 14700) loss: 0.058964
(Epoch 29 / 30) train acc: 0.999000; val_acc: 0.562000
(Iteration 14211 / 14700) loss: 0.040797
(Iteration 14221 / 14700) loss: 0.057438
(Iteration 14231 / 14700) loss: 0.042807
(Iteration 14241 / 14700) loss: 0.042347
(Iteration 14251 / 14700) loss: 0.039718
(Iteration 14261 / 14700) loss: 0.038270
(Iteration 14271 / 14700) loss: 0.043605
(Iteration 14281 / 14700) loss: 0.039331
(Iteration 14291 / 14700) loss: 0.053961
(Iteration 14301 / 14700) loss: 0.033333
```

```
(Iteration 14311 / 14700) loss: 0.044969
(Iteration 14321 / 14700) loss: 0.050008
(Iteration 14331 / 14700) loss: 0.039181
(Iteration 14341 / 14700) loss: 0.055717
(Iteration 14351 / 14700) loss: 0.050281
(Iteration 14361 / 14700) loss: 0.043820
(Iteration 14371 / 14700) loss: 0.059076
(Iteration 14381 / 14700) loss: 0.045647
(Iteration 14391 / 14700) loss: 0.033017
(Iteration 14401 / 14700) loss: 0.058718
(Iteration 14411 / 14700) loss: 0.039425
(Iteration 14421 / 14700) loss: 0.056612
(Iteration 14431 / 14700) loss: 0.047754
(Iteration 14441 / 14700) loss: 0.035925
(Iteration 14451 / 14700) loss: 0.045329
(Iteration 14461 / 14700) loss: 0.041786
(Iteration 14471 / 14700) loss: 0.044980
(Iteration 14481 / 14700) loss: 0.036170
(Iteration 14491 / 14700) loss: 0.042318
(Iteration 14501 / 14700) loss: 0.039642
(Iteration 14511 / 14700) loss: 0.033885
(Iteration 14521 / 14700) loss: 0.040303
(Iteration 14531 / 14700) loss: 0.076744
(Iteration 14541 / 14700) loss: 0.035779
(Iteration 14551 / 14700) loss: 0.041673
(Iteration 14561 / 14700) loss: 0.043889
(Iteration 14571 / 14700) loss: 0.036820
(Iteration 14581 / 14700) loss: 0.044126
(Iteration 14591 / 14700) loss: 0.054785
(Iteration 14601 / 14700) loss: 0.035552
(Iteration 14611 / 14700) loss: 0.043830
(Iteration 14621 / 14700) loss: 0.041122
(Iteration 14631 / 14700) loss: 0.041860
(Iteration 14641 / 14700) loss: 0.034784
(Iteration 14651 / 14700) loss: 0.041459
(Iteration 14661 / 14700) loss: 0.043642
(Iteration 14671 / 14700) loss: 0.037949
(Iteration 14681 / 14700) loss: 0.038452
(Iteration 14691 / 14700) loss: 0.043854
(Epoch 30 / 30) train acc: 0.999000; val_acc: 0.557000
with learning rate 0.5, regularization strength 6.2e-06, accuracy: 0.593
(Iteration 1 / 14700) loss: 2.300888
(Epoch 0 / 30) train acc: 0.156000; val_acc: 0.166000
(Iteration 11 / 14700) loss: 2.158406
(Iteration 21 / 14700) loss: 1.819201
(Iteration 31 / 14700) loss: 1.684112
(Iteration 41 / 14700) loss: 1.531912
(Iteration 51 / 14700) loss: 1.504155
```

```
(Iteration 61 / 14700) loss: 1.462383
(Iteration 71 / 14700) loss: 1.572073
(Iteration 81 / 14700) loss: 1.340952
(Iteration 91 / 14700) loss: 1.329483
(Iteration 101 / 14700) loss: 1.296486
(Iteration 111 / 14700) loss: 1.365941
(Iteration 121 / 14700) loss: 1.487231
(Iteration 131 / 14700) loss: 1.096437
(Iteration 141 / 14700) loss: 1.320652
(Iteration 151 / 14700) loss: 1.312360
(Iteration 161 / 14700) loss: 1.338933
(Iteration 171 / 14700) loss: 1.400536
(Iteration 181 / 14700) loss: 1.551477
(Iteration 191 / 14700) loss: 1.386011
(Iteration 201 / 14700) loss: 1.324180
(Iteration 211 / 14700) loss: 1.308132
(Iteration 221 / 14700) loss: 1.268968
(Iteration 231 / 14700) loss: 1.266383
(Iteration 241 / 14700) loss: 1.351776
(Iteration 251 / 14700) loss: 1.378960
(Iteration 261 / 14700) loss: 1.550880
(Iteration 271 / 14700) loss: 1.260050
(Iteration 281 / 14700) loss: 1.419694
(Iteration 291 / 14700) loss: 1.312544
(Iteration 301 / 14700) loss: 1.436241
(Iteration 311 / 14700) loss: 1.397458
(Iteration 321 / 14700) loss: 1.139253
(Iteration 331 / 14700) loss: 1.446064
(Iteration 341 / 14700) loss: 1.129623
(Iteration 351 / 14700) loss: 1.230631
(Iteration 361 / 14700) loss: 1.185862
(Iteration 371 / 14700) loss: 1.205232
(Iteration 381 / 14700) loss: 1.539372
(Iteration 391 / 14700) loss: 1.172192
(Iteration 401 / 14700) loss: 1.146597
(Iteration 411 / 14700) loss: 1.449281
(Iteration 421 / 14700) loss: 1.272507
(Iteration 431 / 14700) loss: 1.169723
(Iteration 441 / 14700) loss: 1.275453
(Iteration 451 / 14700) loss: 1.076080
(Iteration 461 / 14700) loss: 1.293935
(Iteration 471 / 14700) loss: 1.191611
(Iteration 481 / 14700) loss: 1.205589
(Epoch 1 / 30) train acc: 0.572000; val_acc: 0.530000
(Iteration 491 / 14700) loss: 1.292536
(Iteration 501 / 14700) loss: 1.119613
(Iteration 511 / 14700) loss: 1.274725
(Iteration 521 / 14700) loss: 1.137421
```

```
(Iteration 531 / 14700) loss: 1.079745
(Iteration 541 / 14700) loss: 1.252619
(Iteration 551 / 14700) loss: 1.292826
(Iteration 561 / 14700) loss: 1.077680
(Iteration 571 / 14700) loss: 1.190709
(Iteration 581 / 14700) loss: 1.263656
(Iteration 591 / 14700) loss: 1.167329
(Iteration 601 / 14700) loss: 1.490570
(Iteration 611 / 14700) loss: 1.429174
(Iteration 621 / 14700) loss: 1.105969
(Iteration 631 / 14700) loss: 1.207329
(Iteration 641 / 14700) loss: 1.251344
(Iteration 651 / 14700) loss: 1.046990
(Iteration 661 / 14700) loss: 1.229227
(Iteration 671 / 14700) loss: 1.298852
(Iteration 681 / 14700) loss: 1.241444
(Iteration 691 / 14700) loss: 1.277462
(Iteration 701 / 14700) loss: 1.246918
(Iteration 711 / 14700) loss: 1.241451
(Iteration 721 / 14700) loss: 1.117036
(Iteration 731 / 14700) loss: 1.052805
(Iteration 741 / 14700) loss: 1.179468
(Iteration 751 / 14700) loss: 0.970798
(Iteration 761 / 14700) loss: 1.353897
(Iteration 771 / 14700) loss: 1.167723
(Iteration 781 / 14700) loss: 1.279823
(Iteration 791 / 14700) loss: 1.147661
(Iteration 801 / 14700) loss: 1.344329
(Iteration 811 / 14700) loss: 1.102319
(Iteration 821 / 14700) loss: 1.160082
(Iteration 831 / 14700) loss: 1.068506
(Iteration 841 / 14700) loss: 1.013594
(Iteration 851 / 14700) loss: 1.104200
(Iteration 861 / 14700) loss: 1.194533
(Iteration 871 / 14700) loss: 1.143268
(Iteration 881 / 14700) loss: 1.188938
(Iteration 891 / 14700) loss: 1.063402
(Iteration 901 / 14700) loss: 1.083512
(Iteration 911 / 14700) loss: 0.954559
(Iteration 921 / 14700) loss: 1.200206
(Iteration 931 / 14700) loss: 1.195776
(Iteration 941 / 14700) loss: 1.202117
(Iteration 951 / 14700) loss: 1.078212
(Iteration 961 / 14700) loss: 1.184515
(Iteration 971 / 14700) loss: 0.947640
(Epoch 2 / 30) train acc: 0.652000; val_acc: 0.542000
(Iteration 981 / 14700) loss: 1.144053
(Iteration 991 / 14700) loss: 1.009763
```

```
(Iteration 1001 / 14700) loss: 0.931698
(Iteration 1011 / 14700) loss: 0.952374
(Iteration 1021 / 14700) loss: 1.080817
(Iteration 1031 / 14700) loss: 1.177191
(Iteration 1041 / 14700) loss: 1.076417
(Iteration 1051 / 14700) loss: 0.883864
(Iteration 1061 / 14700) loss: 1.027893
(Iteration 1071 / 14700) loss: 0.972992
(Iteration 1081 / 14700) loss: 1.062967
(Iteration 1091 / 14700) loss: 0.892608
(Iteration 1101 / 14700) loss: 1.031514
(Iteration 1111 / 14700) loss: 0.980416
(Iteration 1121 / 14700) loss: 1.028006
(Iteration 1131 / 14700) loss: 0.937637
(Iteration 1141 / 14700) loss: 1.064411
(Iteration 1151 / 14700) loss: 1.227470
(Iteration 1161 / 14700) loss: 0.828699
(Iteration 1171 / 14700) loss: 0.877221
(Iteration 1181 / 14700) loss: 1.180084
(Iteration 1191 / 14700) loss: 1.066674
(Iteration 1201 / 14700) loss: 1.107008
(Iteration 1211 / 14700) loss: 0.957920
(Iteration 1221 / 14700) loss: 1.044211
(Iteration 1231 / 14700) loss: 0.944185
(Iteration 1241 / 14700) loss: 1.033694
(Iteration 1251 / 14700) loss: 1.635586
(Iteration 1261 / 14700) loss: 1.099931
(Iteration 1271 / 14700) loss: 0.936473
(Iteration 1281 / 14700) loss: 0.996460
(Iteration 1291 / 14700) loss: 1.088124
(Iteration 1301 / 14700) loss: 0.927021
(Iteration 1311 / 14700) loss: 0.810255
(Iteration 1321 / 14700) loss: 0.958290
(Iteration 1331 / 14700) loss: 0.895256
(Iteration 1341 / 14700) loss: 0.963707
(Iteration 1351 / 14700) loss: 1.103492
(Iteration 1361 / 14700) loss: 1.200225
(Iteration 1371 / 14700) loss: 1.052968
(Iteration 1381 / 14700) loss: 1.167928
(Iteration 1391 / 14700) loss: 1.132935
(Iteration 1401 / 14700) loss: 1.007373
(Iteration 1411 / 14700) loss: 1.068637
(Iteration 1421 / 14700) loss: 0.940678
(Iteration 1431 / 14700) loss: 0.993764
(Iteration 1441 / 14700) loss: 0.923104
(Iteration 1451 / 14700) loss: 1.095159
(Iteration 1461 / 14700) loss: 1.077746
(Epoch 3 / 30) train acc: 0.642000; val_acc: 0.545000
```

```
(Iteration 1471 / 14700) loss: 1.002782
(Iteration 1481 / 14700) loss: 1.195153
(Iteration 1491 / 14700) loss: 0.838893
(Iteration 1501 / 14700) loss: 0.993332
(Iteration 1511 / 14700) loss: 0.893376
(Iteration 1521 / 14700) loss: 1.015029
(Iteration 1531 / 14700) loss: 1.001997
(Iteration 1541 / 14700) loss: 0.913341
(Iteration 1551 / 14700) loss: 0.989680
(Iteration 1561 / 14700) loss: 0.860243
(Iteration 1571 / 14700) loss: 1.013036
(Iteration 1581 / 14700) loss: 0.912072
(Iteration 1591 / 14700) loss: 1.159622
(Iteration 1601 / 14700) loss: 0.868350
(Iteration 1611 / 14700) loss: 0.800046
(Iteration 1621 / 14700) loss: 1.006891
(Iteration 1631 / 14700) loss: 1.054194
(Iteration 1641 / 14700) loss: 0.952098
(Iteration 1651 / 14700) loss: 1.030936
(Iteration 1661 / 14700) loss: 1.122110
(Iteration 1671 / 14700) loss: 0.853036
(Iteration 1681 / 14700) loss: 1.012675
(Iteration 1691 / 14700) loss: 1.154437
(Iteration 1701 / 14700) loss: 0.850344
(Iteration 1711 / 14700) loss: 0.940399
(Iteration 1721 / 14700) loss: 0.928619
(Iteration 1731 / 14700) loss: 1.082515
(Iteration 1741 / 14700) loss: 1.031938
(Iteration 1751 / 14700) loss: 0.955820
(Iteration 1761 / 14700) loss: 0.865781
(Iteration 1771 / 14700) loss: 0.961414
(Iteration 1781 / 14700) loss: 1.005692
(Iteration 1791 / 14700) loss: 0.916742
(Iteration 1801 / 14700) loss: 0.922141
(Iteration 1811 / 14700) loss: 0.996001
(Iteration 1821 / 14700) loss: 0.948562
(Iteration 1831 / 14700) loss: 0.982566
(Iteration 1841 / 14700) loss: 0.796154
(Iteration 1851 / 14700) loss: 0.903122
(Iteration 1861 / 14700) loss: 1.005362
(Iteration 1871 / 14700) loss: 1.079142
(Iteration 1881 / 14700) loss: 0.825609
(Iteration 1891 / 14700) loss: 0.991053
(Iteration 1901 / 14700) loss: 0.991841
(Iteration 1911 / 14700) loss: 0.865710
(Iteration 1921 / 14700) loss: 0.884400
(Iteration 1931 / 14700) loss: 0.701092
(Iteration 1941 / 14700) loss: 0.755027
```

```
(Iteration 1951 / 14700) loss: 0.966084
(Epoch 4 / 30) train acc: 0.714000; val_acc: 0.557000
(Iteration 1961 / 14700) loss: 0.732358
(Iteration 1971 / 14700) loss: 0.807123
(Iteration 1981 / 14700) loss: 1.041314
(Iteration 1991 / 14700) loss: 0.932859
(Iteration 2001 / 14700) loss: 0.776513
(Iteration 2011 / 14700) loss: 0.804475
(Iteration 2021 / 14700) loss: 0.990310
(Iteration 2031 / 14700) loss: 0.909985
(Iteration 2041 / 14700) loss: 0.755067
(Iteration 2051 / 14700) loss: 0.794123
(Iteration 2061 / 14700) loss: 0.690933
(Iteration 2071 / 14700) loss: 0.880952
(Iteration 2081 / 14700) loss: 0.771672
(Iteration 2091 / 14700) loss: 1.153829
(Iteration 2101 / 14700) loss: 0.816304
(Iteration 2111 / 14700) loss: 0.826457
(Iteration 2121 / 14700) loss: 0.804819
(Iteration 2131 / 14700) loss: 0.731973
(Iteration 2141 / 14700) loss: 0.720743
(Iteration 2151 / 14700) loss: 0.888930
(Iteration 2161 / 14700) loss: 0.745757
(Iteration 2171 / 14700) loss: 0.930097
(Iteration 2181 / 14700) loss: 0.737997
(Iteration 2191 / 14700) loss: 0.802129
(Iteration 2201 / 14700) loss: 0.947527
(Iteration 2211 / 14700) loss: 0.932753
(Iteration 2221 / 14700) loss: 0.628434
(Iteration 2231 / 14700) loss: 0.986268
(Iteration 2241 / 14700) loss: 0.866060
(Iteration 2251 / 14700) loss: 0.913483
(Iteration 2261 / 14700) loss: 1.009303
(Iteration 2271 / 14700) loss: 0.990304
(Iteration 2281 / 14700) loss: 0.849824
(Iteration 2291 / 14700) loss: 0.909996
(Iteration 2301 / 14700) loss: 0.893646
(Iteration 2311 / 14700) loss: 0.979287
(Iteration 2321 / 14700) loss: 0.873136
(Iteration 2331 / 14700) loss: 0.702655
(Iteration 2341 / 14700) loss: 0.970175
(Iteration 2351 / 14700) loss: 0.939937
(Iteration 2361 / 14700) loss: 0.945837
(Iteration 2371 / 14700) loss: 0.819309
(Iteration 2381 / 14700) loss: 0.652479
(Iteration 2391 / 14700) loss: 0.807291
(Iteration 2401 / 14700) loss: 0.888200
(Iteration 2411 / 14700) loss: 0.763632
```

```
(Iteration 2421 / 14700) loss: 0.640889
(Iteration 2431 / 14700) loss: 0.683713
(Iteration 2441 / 14700) loss: 0.963352
(Epoch 5 / 30) train acc: 0.724000; val_acc: 0.574000
(Iteration 2451 / 14700) loss: 0.915755
(Iteration 2461 / 14700) loss: 0.856215
(Iteration 2471 / 14700) loss: 0.685422
(Iteration 2481 / 14700) loss: 0.835611
(Iteration 2491 / 14700) loss: 0.765370
(Iteration 2501 / 14700) loss: 0.695270
(Iteration 2511 / 14700) loss: 0.700611
(Iteration 2521 / 14700) loss: 0.813401
(Iteration 2531 / 14700) loss: 0.756023
(Iteration 2541 / 14700) loss: 0.698275
(Iteration 2551 / 14700) loss: 0.961485
(Iteration 2561 / 14700) loss: 0.912774
(Iteration 2571 / 14700) loss: 0.713229
(Iteration 2581 / 14700) loss: 0.956940
(Iteration 2591 / 14700) loss: 0.859946
(Iteration 2601 / 14700) loss: 0.732602
(Iteration 2611 / 14700) loss: 1.006495
(Iteration 2621 / 14700) loss: 0.805479
(Iteration 2631 / 14700) loss: 0.843112
(Iteration 2641 / 14700) loss: 0.797554
(Iteration 2651 / 14700) loss: 0.885800
(Iteration 2661 / 14700) loss: 0.976095
(Iteration 2671 / 14700) loss: 0.982270
(Iteration 2681 / 14700) loss: 0.965979
(Iteration 2691 / 14700) loss: 0.860188
(Iteration 2701 / 14700) loss: 0.731674
(Iteration 2711 / 14700) loss: 0.773154
(Iteration 2721 / 14700) loss: 0.802965
(Iteration 2731 / 14700) loss: 0.787425
(Iteration 2741 / 14700) loss: 0.761315
(Iteration 2751 / 14700) loss: 0.505253
(Iteration 2761 / 14700) loss: 0.793467
(Iteration 2771 / 14700) loss: 0.898976
(Iteration 2781 / 14700) loss: 0.772905
(Iteration 2791 / 14700) loss: 0.830576
(Iteration 2801 / 14700) loss: 0.874091
(Iteration 2811 / 14700) loss: 0.746819
(Iteration 2821 / 14700) loss: 0.820039
(Iteration 2831 / 14700) loss: 0.603924
(Iteration 2841 / 14700) loss: 0.787039
(Iteration 2851 / 14700) loss: 0.621295
(Iteration 2861 / 14700) loss: 0.752049
(Iteration 2871 / 14700) loss: 0.712648
(Iteration 2881 / 14700) loss: 0.670907
```

```
(Iteration 2891 / 14700) loss: 0.741970
(Iteration 2901 / 14700) loss: 0.742583
(Iteration 2911 / 14700) loss: 0.793316
(Iteration 2921 / 14700) loss: 1.116905
(Iteration 2931 / 14700) loss: 0.515659
(Epoch 6 / 30) train acc: 0.749000; val_acc: 0.564000
(Iteration 2941 / 14700) loss: 0.785649
(Iteration 2951 / 14700) loss: 0.718730
(Iteration 2961 / 14700) loss: 0.787412
(Iteration 2971 / 14700) loss: 0.747069
(Iteration 2981 / 14700) loss: 0.714631
(Iteration 2991 / 14700) loss: 0.742176
(Iteration 3001 / 14700) loss: 0.883045
(Iteration 3011 / 14700) loss: 0.777359
(Iteration 3021 / 14700) loss: 0.725911
(Iteration 3031 / 14700) loss: 0.593949
(Iteration 3041 / 14700) loss: 0.994329
(Iteration 3051 / 14700) loss: 0.953240
(Iteration 3061 / 14700) loss: 0.763245
(Iteration 3071 / 14700) loss: 0.676624
(Iteration 3081 / 14700) loss: 0.913318
(Iteration 3091 / 14700) loss: 0.761455
(Iteration 3101 / 14700) loss: 0.763744
(Iteration 3111 / 14700) loss: 0.659636
(Iteration 3121 / 14700) loss: 0.821143
(Iteration 3131 / 14700) loss: 0.769073
(Iteration 3141 / 14700) loss: 0.779655
(Iteration 3151 / 14700) loss: 0.744371
(Iteration 3161 / 14700) loss: 0.647592
(Iteration 3171 / 14700) loss: 0.657510
(Iteration 3181 / 14700) loss: 0.724712
(Iteration 3191 / 14700) loss: 0.699111
(Iteration 3201 / 14700) loss: 0.719789
(Iteration 3211 / 14700) loss: 0.561711
(Iteration 3221 / 14700) loss: 0.653286
(Iteration 3231 / 14700) loss: 0.476106
(Iteration 3241 / 14700) loss: 0.754561
(Iteration 3251 / 14700) loss: 0.570274
(Iteration 3261 / 14700) loss: 0.719724
(Iteration 3271 / 14700) loss: 0.479743
(Iteration 3281 / 14700) loss: 0.552817
(Iteration 3291 / 14700) loss: 0.715313
(Iteration 3301 / 14700) loss: 0.674306
(Iteration 3311 / 14700) loss: 0.913764
(Iteration 3321 / 14700) loss: 0.725958
(Iteration 3331 / 14700) loss: 0.644331
(Iteration 3341 / 14700) loss: 0.719749
(Iteration 3351 / 14700) loss: 0.692812
```

```
(Iteration 3361 / 14700) loss: 0.438679
(Iteration 3371 / 14700) loss: 0.808576
(Iteration 3381 / 14700) loss: 0.677693
(Iteration 3391 / 14700) loss: 0.639765
(Iteration 3401 / 14700) loss: 0.734137
(Iteration 3411 / 14700) loss: 0.698168
(Iteration 3421 / 14700) loss: 0.682589
(Epoch 7 / 30) train acc: 0.748000; val_acc: 0.562000
(Iteration 3431 / 14700) loss: 0.628463
(Iteration 3441 / 14700) loss: 0.782742
(Iteration 3451 / 14700) loss: 0.591513
(Iteration 3461 / 14700) loss: 0.947801
(Iteration 3471 / 14700) loss: 0.889588
(Iteration 3481 / 14700) loss: 0.560961
(Iteration 3491 / 14700) loss: 0.717646
(Iteration 3501 / 14700) loss: 0.741513
(Iteration 3511 / 14700) loss: 0.540549
(Iteration 3521 / 14700) loss: 0.596026
(Iteration 3531 / 14700) loss: 0.766757
(Iteration 3541 / 14700) loss: 0.691986
(Iteration 3551 / 14700) loss: 0.756443
(Iteration 3561 / 14700) loss: 0.497367
(Iteration 3571 / 14700) loss: 0.697603
(Iteration 3581 / 14700) loss: 0.588670
(Iteration 3591 / 14700) loss: 0.712355
(Iteration 3601 / 14700) loss: 0.565101
(Iteration 3611 / 14700) loss: 0.675213
(Iteration 3621 / 14700) loss: 0.585275
(Iteration 3631 / 14700) loss: 0.794713
(Iteration 3641 / 14700) loss: 0.513386
(Iteration 3651 / 14700) loss: 0.714426
(Iteration 3661 / 14700) loss: 0.714916
(Iteration 3671 / 14700) loss: 0.623348
(Iteration 3681 / 14700) loss: 0.605299
(Iteration 3691 / 14700) loss: 0.574736
(Iteration 3701 / 14700) loss: 0.797336
(Iteration 3711 / 14700) loss: 0.724522
(Iteration 3721 / 14700) loss: 0.960612
(Iteration 3731 / 14700) loss: 0.601742
(Iteration 3741 / 14700) loss: 0.581778
(Iteration 3751 / 14700) loss: 0.791955
(Iteration 3761 / 14700) loss: 0.707887
(Iteration 3771 / 14700) loss: 0.882737
(Iteration 3781 / 14700) loss: 0.636560
(Iteration 3791 / 14700) loss: 0.483871
(Iteration 3801 / 14700) loss: 0.643210
(Iteration 3811 / 14700) loss: 0.625989
(Iteration 3821 / 14700) loss: 0.582301
```

```
(Iteration 3831 / 14700) loss: 0.652624
(Iteration 3841 / 14700) loss: 0.675460
(Iteration 3851 / 14700) loss: 0.663330
(Iteration 3861 / 14700) loss: 0.872796
(Iteration 3871 / 14700) loss: 0.813265
(Iteration 3881 / 14700) loss: 0.494340
(Iteration 3891 / 14700) loss: 0.681197
(Iteration 3901 / 14700) loss: 0.727026
(Iteration 3911 / 14700) loss: 0.501388
(Epoch 8 / 30) train acc: 0.801000; val_acc: 0.549000
(Iteration 3921 / 14700) loss: 0.807707
(Iteration 3931 / 14700) loss: 0.624472
(Iteration 3941 / 14700) loss: 0.745619
(Iteration 3951 / 14700) loss: 0.532516
(Iteration 3961 / 14700) loss: 0.639119
(Iteration 3971 / 14700) loss: 0.702623
(Iteration 3981 / 14700) loss: 0.655577
(Iteration 3991 / 14700) loss: 0.475215
(Iteration 4001 / 14700) loss: 0.633002
(Iteration 4011 / 14700) loss: 0.509751
(Iteration 4021 / 14700) loss: 0.674119
(Iteration 4031 / 14700) loss: 0.659575
(Iteration 4041 / 14700) loss: 0.848054
(Iteration 4051 / 14700) loss: 0.569748
(Iteration 4061 / 14700) loss: 0.804046
(Iteration 4071 / 14700) loss: 0.625971
(Iteration 4081 / 14700) loss: 0.614478
(Iteration 4091 / 14700) loss: 0.557526
(Iteration 4101 / 14700) loss: 0.644221
(Iteration 4111 / 14700) loss: 0.614053
(Iteration 4121 / 14700) loss: 0.556462
(Iteration 4131 / 14700) loss: 0.506867
(Iteration 4141 / 14700) loss: 0.745735
(Iteration 4151 / 14700) loss: 0.531965
(Iteration 4161 / 14700) loss: 0.520850
(Iteration 4171 / 14700) loss: 0.660134
(Iteration 4181 / 14700) loss: 0.618179
(Iteration 4191 / 14700) loss: 0.836759
(Iteration 4201 / 14700) loss: 0.441547
(Iteration 4211 / 14700) loss: 0.532818
(Iteration 4221 / 14700) loss: 0.529294
(Iteration 4231 / 14700) loss: 0.416555
(Iteration 4241 / 14700) loss: 0.531913
(Iteration 4251 / 14700) loss: 0.548514
(Iteration 4261 / 14700) loss: 0.740952
(Iteration 4271 / 14700) loss: 0.665672
(Iteration 4281 / 14700) loss: 0.665224
(Iteration 4291 / 14700) loss: 0.594676
```

```
(Iteration 4301 / 14700) loss: 0.580499
(Iteration 4311 / 14700) loss: 0.539648
(Iteration 4321 / 14700) loss: 0.709128
(Iteration 4331 / 14700) loss: 0.490063
(Iteration 4341 / 14700) loss: 0.663404
(Iteration 4351 / 14700) loss: 0.637485
(Iteration 4361 / 14700) loss: 0.410941
(Iteration 4371 / 14700) loss: 0.484307
(Iteration 4381 / 14700) loss: 0.559420
(Iteration 4391 / 14700) loss: 0.640524
(Iteration 4401 / 14700) loss: 0.582320
(Epoch 9 / 30) train acc: 0.818000; val_acc: 0.561000
(Iteration 4411 / 14700) loss: 0.596998
(Iteration 4421 / 14700) loss: 0.623779
(Iteration 4431 / 14700) loss: 0.577893
(Iteration 4441 / 14700) loss: 0.627114
(Iteration 4451 / 14700) loss: 0.513646
(Iteration 4461 / 14700) loss: 0.601668
(Iteration 4471 / 14700) loss: 0.518458
(Iteration 4481 / 14700) loss: 0.603079
(Iteration 4491 / 14700) loss: 0.511944
(Iteration 4501 / 14700) loss: 0.726273
(Iteration 4511 / 14700) loss: 0.527665
(Iteration 4521 / 14700) loss: 0.650534
(Iteration 4531 / 14700) loss: 0.592546
(Iteration 4541 / 14700) loss: 0.543540
(Iteration 4551 / 14700) loss: 0.432103
(Iteration 4561 / 14700) loss: 0.528931
(Iteration 4571 / 14700) loss: 0.623091
(Iteration 4581 / 14700) loss: 0.703610
(Iteration 4591 / 14700) loss: 0.552237
(Iteration 4601 / 14700) loss: 0.612918
(Iteration 4611 / 14700) loss: 0.619305
(Iteration 4621 / 14700) loss: 0.596577
(Iteration 4631 / 14700) loss: 0.598418
(Iteration 4641 / 14700) loss: 0.591589
(Iteration 4651 / 14700) loss: 0.467593
(Iteration 4661 / 14700) loss: 0.541158
(Iteration 4671 / 14700) loss: 0.437765
(Iteration 4681 / 14700) loss: 0.466996
(Iteration 4691 / 14700) loss: 0.570543
(Iteration 4701 / 14700) loss: 0.551830
(Iteration 4711 / 14700) loss: 0.393305
(Iteration 4721 / 14700) loss: 0.561571
(Iteration 4731 / 14700) loss: 0.437342
(Iteration 4741 / 14700) loss: 0.543964
(Iteration 4751 / 14700) loss: 0.550252
(Iteration 4761 / 14700) loss: 0.404999
```

```
(Iteration 4771 / 14700) loss: 0.550118
(Iteration 4781 / 14700) loss: 0.490059
(Iteration 4791 / 14700) loss: 0.296776
(Iteration 4801 / 14700) loss: 0.658879
(Iteration 4811 / 14700) loss: 0.420037
(Iteration 4821 / 14700) loss: 0.651993
(Iteration 4831 / 14700) loss: 0.595812
(Iteration 4841 / 14700) loss: 0.735170
(Iteration 4851 / 14700) loss: 0.778722
(Iteration 4861 / 14700) loss: 0.743117
(Iteration 4871 / 14700) loss: 0.357108
(Iteration 4881 / 14700) loss: 0.698981
(Iteration 4891 / 14700) loss: 0.453618
(Epoch 10 / 30) train acc: 0.824000; val_acc: 0.556000
(Iteration 4901 / 14700) loss: 0.504673
(Iteration 4911 / 14700) loss: 0.567287
(Iteration 4921 / 14700) loss: 0.525596
(Iteration 4931 / 14700) loss: 0.446892
(Iteration 4941 / 14700) loss: 0.367290
(Iteration 4951 / 14700) loss: 0.451891
(Iteration 4961 / 14700) loss: 0.586648
(Iteration 4971 / 14700) loss: 0.592256
(Iteration 4981 / 14700) loss: 0.494431
(Iteration 4991 / 14700) loss: 0.523833
(Iteration 5001 / 14700) loss: 0.476296
(Iteration 5011 / 14700) loss: 0.389777
(Iteration 5021 / 14700) loss: 0.517971
(Iteration 5031 / 14700) loss: 0.662883
(Iteration 5041 / 14700) loss: 0.557078
(Iteration 5051 / 14700) loss: 0.560619
(Iteration 5061 / 14700) loss: 0.542092
(Iteration 5071 / 14700) loss: 0.534304
(Iteration 5081 / 14700) loss: 0.286259
(Iteration 5091 / 14700) loss: 0.483202
(Iteration 5101 / 14700) loss: 0.567637
(Iteration 5111 / 14700) loss: 0.639750
(Iteration 5121 / 14700) loss: 0.483397
(Iteration 5131 / 14700) loss: 0.569706
(Iteration 5141 / 14700) loss: 0.304039
(Iteration 5151 / 14700) loss: 0.720507
(Iteration 5161 / 14700) loss: 0.449341
(Iteration 5171 / 14700) loss: 0.481994
(Iteration 5181 / 14700) loss: 0.319591
(Iteration 5191 / 14700) loss: 0.604344
(Iteration 5201 / 14700) loss: 0.432160
(Iteration 5211 / 14700) loss: 0.481476
(Iteration 5221 / 14700) loss: 0.654216
(Iteration 5231 / 14700) loss: 0.444355
```

```
(Iteration 5241 / 14700) loss: 0.455959
(Iteration 5251 / 14700) loss: 0.695821
(Iteration 5261 / 14700) loss: 0.473073
(Iteration 5271 / 14700) loss: 0.366480
(Iteration 5281 / 14700) loss: 0.470446
(Iteration 5291 / 14700) loss: 0.378296
(Iteration 5301 / 14700) loss: 0.458932
(Iteration 5311 / 14700) loss: 0.442562
(Iteration 5321 / 14700) loss: 0.770811
(Iteration 5331 / 14700) loss: 0.614680
(Iteration 5341 / 14700) loss: 0.430850
(Iteration 5351 / 14700) loss: 0.540028
(Iteration 5361 / 14700) loss: 0.336759
(Iteration 5371 / 14700) loss: 0.568277
(Iteration 5381 / 14700) loss: 0.398273
(Epoch 11 / 30) train acc: 0.840000; val_acc: 0.548000
(Iteration 5391 / 14700) loss: 0.613621
(Iteration 5401 / 14700) loss: 0.480086
(Iteration 5411 / 14700) loss: 0.428785
(Iteration 5421 / 14700) loss: 0.540754
(Iteration 5431 / 14700) loss: 0.429893
(Iteration 5441 / 14700) loss: 0.455745
(Iteration 5451 / 14700) loss: 0.430877
(Iteration 5461 / 14700) loss: 0.483511
(Iteration 5471 / 14700) loss: 0.522204
(Iteration 5481 / 14700) loss: 0.400260
(Iteration 5491 / 14700) loss: 0.455009
(Iteration 5501 / 14700) loss: 0.381830
(Iteration 5511 / 14700) loss: 0.500013
(Iteration 5521 / 14700) loss: 0.358684
(Iteration 5531 / 14700) loss: 0.319996
(Iteration 5541 / 14700) loss: 0.362799
(Iteration 5551 / 14700) loss: 0.465039
(Iteration 5561 / 14700) loss: 0.396156
(Iteration 5571 / 14700) loss: 0.521330
(Iteration 5581 / 14700) loss: 0.366416
(Iteration 5591 / 14700) loss: 0.313830
(Iteration 5601 / 14700) loss: 0.432104
(Iteration 5611 / 14700) loss: 0.336705
(Iteration 5621 / 14700) loss: 0.473379
(Iteration 5631 / 14700) loss: 0.365767
(Iteration 5641 / 14700) loss: 0.381022
(Iteration 5651 / 14700) loss: 0.560694
(Iteration 5661 / 14700) loss: 0.574459
(Iteration 5671 / 14700) loss: 0.424041
(Iteration 5681 / 14700) loss: 0.408681
(Iteration 5691 / 14700) loss: 0.321498
(Iteration 5701 / 14700) loss: 0.441663
```

```
(Iteration 5711 / 14700) loss: 0.535204
(Iteration 5721 / 14700) loss: 0.438198
(Iteration 5731 / 14700) loss: 0.470156
(Iteration 5741 / 14700) loss: 0.695423
(Iteration 5751 / 14700) loss: 0.357883
(Iteration 5761 / 14700) loss: 0.443496
(Iteration 5771 / 14700) loss: 0.396046
(Iteration 5781 / 14700) loss: 0.414058
(Iteration 5791 / 14700) loss: 0.480836
(Iteration 5801 / 14700) loss: 0.444366
(Iteration 5811 / 14700) loss: 0.447109
(Iteration 5821 / 14700) loss: 0.438255
(Iteration 5831 / 14700) loss: 0.578570
(Iteration 5841 / 14700) loss: 0.359555
(Iteration 5851 / 14700) loss: 0.412265
(Iteration 5861 / 14700) loss: 0.378869
(Iteration 5871 / 14700) loss: 0.498077
(Epoch 12 / 30) train acc: 0.849000; val_acc: 0.575000
(Iteration 5881 / 14700) loss: 0.560274
(Iteration 5891 / 14700) loss: 0.473252
(Iteration 5901 / 14700) loss: 0.347272
(Iteration 5911 / 14700) loss: 0.377527
(Iteration 5921 / 14700) loss: 0.384179
(Iteration 5931 / 14700) loss: 0.285289
(Iteration 5941 / 14700) loss: 0.416437
(Iteration 5951 / 14700) loss: 0.314693
(Iteration 5961 / 14700) loss: 0.556806
(Iteration 5971 / 14700) loss: 0.366301
(Iteration 5981 / 14700) loss: 0.278899
(Iteration 5991 / 14700) loss: 0.464708
(Iteration 6001 / 14700) loss: 0.398595
(Iteration 6011 / 14700) loss: 0.351595
(Iteration 6021 / 14700) loss: 0.402970
(Iteration 6031 / 14700) loss: 0.417706
(Iteration 6041 / 14700) loss: 0.447641
(Iteration 6051 / 14700) loss: 0.393827
(Iteration 6061 / 14700) loss: 0.482325
(Iteration 6071 / 14700) loss: 0.319215
(Iteration 6081 / 14700) loss: 0.543738
(Iteration 6091 / 14700) loss: 0.407687
(Iteration 6101 / 14700) loss: 0.501381
(Iteration 6111 / 14700) loss: 0.366459
(Iteration 6121 / 14700) loss: 0.493157
(Iteration 6131 / 14700) loss: 0.382606
(Iteration 6141 / 14700) loss: 0.360374
(Iteration 6151 / 14700) loss: 0.685123
(Iteration 6161 / 14700) loss: 0.360613
(Iteration 6171 / 14700) loss: 0.465284
```

```
(Iteration 6181 / 14700) loss: 0.353016
(Iteration 6191 / 14700) loss: 0.399746
(Iteration 6201 / 14700) loss: 0.407218
(Iteration 6211 / 14700) loss: 0.450419
(Iteration 6221 / 14700) loss: 0.455571
(Iteration 6231 / 14700) loss: 0.323988
(Iteration 6241 / 14700) loss: 0.392371
(Iteration 6251 / 14700) loss: 0.445448
(Iteration 6261 / 14700) loss: 0.356274
(Iteration 6271 / 14700) loss: 0.413521
(Iteration 6281 / 14700) loss: 0.290789
(Iteration 6291 / 14700) loss: 0.410344
(Iteration 6301 / 14700) loss: 0.302052
(Iteration 6311 / 14700) loss: 0.477634
(Iteration 6321 / 14700) loss: 0.487168
(Iteration 6331 / 14700) loss: 0.288631
(Iteration 6341 / 14700) loss: 0.337953
(Iteration 6351 / 14700) loss: 0.487046
(Iteration 6361 / 14700) loss: 0.293670
(Epoch 13 / 30) train acc: 0.873000; val_acc: 0.547000
(Iteration 6371 / 14700) loss: 0.411663
(Iteration 6381 / 14700) loss: 0.256767
(Iteration 6391 / 14700) loss: 0.278428
(Iteration 6401 / 14700) loss: 0.228513
(Iteration 6411 / 14700) loss: 0.329103
(Iteration 6421 / 14700) loss: 0.301205
(Iteration 6431 / 14700) loss: 0.452781
(Iteration 6441 / 14700) loss: 0.250687
(Iteration 6451 / 14700) loss: 0.272838
(Iteration 6461 / 14700) loss: 0.340663
(Iteration 6471 / 14700) loss: 0.351145
(Iteration 6481 / 14700) loss: 0.353961
(Iteration 6491 / 14700) loss: 0.260802
(Iteration 6501 / 14700) loss: 0.331951
(Iteration 6511 / 14700) loss: 0.501341
(Iteration 6521 / 14700) loss: 0.374276
(Iteration 6531 / 14700) loss: 0.430872
(Iteration 6541 / 14700) loss: 0.281936
(Iteration 6551 / 14700) loss: 0.380830
(Iteration 6561 / 14700) loss: 0.391502
(Iteration 6571 / 14700) loss: 0.191483
(Iteration 6581 / 14700) loss: 0.221411
(Iteration 6591 / 14700) loss: 0.154199
(Iteration 6601 / 14700) loss: 0.648435
(Iteration 6611 / 14700) loss: 0.312397
(Iteration 6621 / 14700) loss: 0.266891
(Iteration 6631 / 14700) loss: 0.322083
(Iteration 6641 / 14700) loss: 0.450705
```

```
(Iteration 6651 / 14700) loss: 0.228445
(Iteration 6661 / 14700) loss: 0.306568
(Iteration 6671 / 14700) loss: 0.294841
(Iteration 6681 / 14700) loss: 0.215851
(Iteration 6691 / 14700) loss: 0.246611
(Iteration 6701 / 14700) loss: 0.283887
(Iteration 6711 / 14700) loss: 0.309973
(Iteration 6721 / 14700) loss: 0.282221
(Iteration 6731 / 14700) loss: 0.361736
(Iteration 6741 / 14700) loss: 0.328057
(Iteration 6751 / 14700) loss: 0.398865
(Iteration 6761 / 14700) loss: 0.301377
(Iteration 6771 / 14700) loss: 0.193469
(Iteration 6781 / 14700) loss: 0.325982
(Iteration 6791 / 14700) loss: 0.271662
(Iteration 6801 / 14700) loss: 0.387653
(Iteration 6811 / 14700) loss: 0.420686
(Iteration 6821 / 14700) loss: 0.240376
(Iteration 6831 / 14700) loss: 0.394039
(Iteration 6841 / 14700) loss: 0.332049
(Iteration 6851 / 14700) loss: 0.284304
(Epoch 14 / 30) train acc: 0.862000; val_acc: 0.548000
(Iteration 6861 / 14700) loss: 0.340398
(Iteration 6871 / 14700) loss: 0.303607
(Iteration 6881 / 14700) loss: 0.196485
(Iteration 6891 / 14700) loss: 0.423165
(Iteration 6901 / 14700) loss: 0.288676
(Iteration 6911 / 14700) loss: 0.242436
(Iteration 6921 / 14700) loss: 0.357404
(Iteration 6931 / 14700) loss: 0.337269
(Iteration 6941 / 14700) loss: 0.236335
(Iteration 6951 / 14700) loss: 0.357879
(Iteration 6961 / 14700) loss: 0.319650
(Iteration 6971 / 14700) loss: 0.470655
(Iteration 6981 / 14700) loss: 0.483224
(Iteration 6991 / 14700) loss: 0.281433
(Iteration 7001 / 14700) loss: 0.403590
(Iteration 7011 / 14700) loss: 0.277045
(Iteration 7021 / 14700) loss: 0.223605
(Iteration 7031 / 14700) loss: 0.320826
(Iteration 7041 / 14700) loss: 0.273829
(Iteration 7051 / 14700) loss: 0.247561
(Iteration 7061 / 14700) loss: 0.297195
(Iteration 7071 / 14700) loss: 0.198936
(Iteration 7081 / 14700) loss: 0.362001
(Iteration 7091 / 14700) loss: 0.319592
(Iteration 7101 / 14700) loss: 0.456553
(Iteration 7111 / 14700) loss: 0.304730
```

```
(Iteration 7121 / 14700) loss: 0.325460
(Iteration 7131 / 14700) loss: 0.390736
(Iteration 7141 / 14700) loss: 0.320963
(Iteration 7151 / 14700) loss: 0.335262
(Iteration 7161 / 14700) loss: 0.294228
(Iteration 7171 / 14700) loss: 0.330308
(Iteration 7181 / 14700) loss: 0.200790
(Iteration 7191 / 14700) loss: 0.233544
(Iteration 7201 / 14700) loss: 0.364393
(Iteration 7211 / 14700) loss: 0.219215
(Iteration 7221 / 14700) loss: 0.168196
(Iteration 7231 / 14700) loss: 0.301681
(Iteration 7241 / 14700) loss: 0.209664
(Iteration 7251 / 14700) loss: 0.249409
(Iteration 7261 / 14700) loss: 0.239900
(Iteration 7271 / 14700) loss: 0.245798
(Iteration 7281 / 14700) loss: 0.303082
(Iteration 7291 / 14700) loss: 0.296565
(Iteration 7301 / 14700) loss: 0.387008
(Iteration 7311 / 14700) loss: 0.309918
(Iteration 7321 / 14700) loss: 0.295968
(Iteration 7331 / 14700) loss: 0.248326
(Iteration 7341 / 14700) loss: 0.247270
(Epoch 15 / 30) train acc: 0.900000; val_acc: 0.556000
(Iteration 7351 / 14700) loss: 0.310164
(Iteration 7361 / 14700) loss: 0.256839
(Iteration 7371 / 14700) loss: 0.370458
(Iteration 7381 / 14700) loss: 0.241547
(Iteration 7391 / 14700) loss: 0.348935
(Iteration 7401 / 14700) loss: 0.323078
(Iteration 7411 / 14700) loss: 0.340487
(Iteration 7421 / 14700) loss: 0.197517
(Iteration 7431 / 14700) loss: 0.293367
(Iteration 7441 / 14700) loss: 0.190009
(Iteration 7451 / 14700) loss: 0.293425
(Iteration 7461 / 14700) loss: 0.263498
(Iteration 7471 / 14700) loss: 0.346243
(Iteration 7481 / 14700) loss: 0.220811
(Iteration 7491 / 14700) loss: 0.394292
(Iteration 7501 / 14700) loss: 0.177909
(Iteration 7511 / 14700) loss: 0.288086
(Iteration 7521 / 14700) loss: 0.229486
(Iteration 7531 / 14700) loss: 0.260194
(Iteration 7541 / 14700) loss: 0.310529
(Iteration 7551 / 14700) loss: 0.378162
(Iteration 7561 / 14700) loss: 0.255289
(Iteration 7571 / 14700) loss: 0.324793
(Iteration 7581 / 14700) loss: 0.210566
```

```
(Iteration 7591 / 14700) loss: 0.296803
(Iteration 7601 / 14700) loss: 0.232934
(Iteration 7611 / 14700) loss: 0.291024
(Iteration 7621 / 14700) loss: 0.372545
(Iteration 7631 / 14700) loss: 0.220014
(Iteration 7641 / 14700) loss: 0.203488
(Iteration 7651 / 14700) loss: 0.227328
(Iteration 7661 / 14700) loss: 0.315232
(Iteration 7671 / 14700) loss: 0.263378
(Iteration 7681 / 14700) loss: 0.235773
(Iteration 7691 / 14700) loss: 0.215494
(Iteration 7701 / 14700) loss: 0.216138
(Iteration 7711 / 14700) loss: 0.333154
(Iteration 7721 / 14700) loss: 0.199564
(Iteration 7731 / 14700) loss: 0.172993
(Iteration 7741 / 14700) loss: 0.284544
(Iteration 7751 / 14700) loss: 0.234449
(Iteration 7761 / 14700) loss: 0.380779
(Iteration 7771 / 14700) loss: 0.196160
(Iteration 7781 / 14700) loss: 0.315252
(Iteration 7791 / 14700) loss: 0.376524
(Iteration 7801 / 14700) loss: 0.249917
(Iteration 7811 / 14700) loss: 0.282267
(Iteration 7821 / 14700) loss: 0.236647
(Iteration 7831 / 14700) loss: 0.185457
(Epoch 16 / 30) train acc: 0.909000; val_acc: 0.558000
(Iteration 7841 / 14700) loss: 0.253055
(Iteration 7851 / 14700) loss: 0.236693
(Iteration 7861 / 14700) loss: 0.235599
(Iteration 7871 / 14700) loss: 0.160377
(Iteration 7881 / 14700) loss: 0.339223
(Iteration 7891 / 14700) loss: 0.169246
(Iteration 7901 / 14700) loss: 0.329026
(Iteration 7911 / 14700) loss: 0.209619
(Iteration 7921 / 14700) loss: 0.143213
(Iteration 7931 / 14700) loss: 0.294576
(Iteration 7941 / 14700) loss: 0.163864
(Iteration 7951 / 14700) loss: 0.336454
(Iteration 7961 / 14700) loss: 0.185611
(Iteration 7971 / 14700) loss: 0.320825
(Iteration 7981 / 14700) loss: 0.230887
(Iteration 7991 / 14700) loss: 0.179531
(Iteration 8001 / 14700) loss: 0.247742
(Iteration 8011 / 14700) loss: 0.234873
(Iteration 8021 / 14700) loss: 0.193744
(Iteration 8031 / 14700) loss: 0.193592
(Iteration 8041 / 14700) loss: 0.237274
(Iteration 8051 / 14700) loss: 0.170936
```

```
(Iteration 8061 / 14700) loss: 0.203045
(Iteration 8071 / 14700) loss: 0.197741
(Iteration 8081 / 14700) loss: 0.240930
(Iteration 8091 / 14700) loss: 0.201680
(Iteration 8101 / 14700) loss: 0.212729
(Iteration 8111 / 14700) loss: 0.216576
(Iteration 8121 / 14700) loss: 0.299944
(Iteration 8131 / 14700) loss: 0.285292
(Iteration 8141 / 14700) loss: 0.297099
(Iteration 8151 / 14700) loss: 0.233218
(Iteration 8161 / 14700) loss: 0.169139
(Iteration 8171 / 14700) loss: 0.314247
(Iteration 8181 / 14700) loss: 0.284961
(Iteration 8191 / 14700) loss: 0.239676
(Iteration 8201 / 14700) loss: 0.221603
(Iteration 8211 / 14700) loss: 0.281875
(Iteration 8221 / 14700) loss: 0.264767
(Iteration 8231 / 14700) loss: 0.241955
(Iteration 8241 / 14700) loss: 0.228753
(Iteration 8251 / 14700) loss: 0.328781
(Iteration 8261 / 14700) loss: 0.237120
(Iteration 8271 / 14700) loss: 0.342299
(Iteration 8281 / 14700) loss: 0.195027
(Iteration 8291 / 14700) loss: 0.169797
(Iteration 8301 / 14700) loss: 0.199919
(Iteration 8311 / 14700) loss: 0.234787
(Iteration 8321 / 14700) loss: 0.259026
(Epoch 17 / 30) train acc: 0.941000; val_acc: 0.564000
(Iteration 8331 / 14700) loss: 0.334103
(Iteration 8341 / 14700) loss: 0.220992
(Iteration 8351 / 14700) loss: 0.372794
(Iteration 8361 / 14700) loss: 0.165474
(Iteration 8371 / 14700) loss: 0.190464
(Iteration 8381 / 14700) loss: 0.158635
(Iteration 8391 / 14700) loss: 0.361527
(Iteration 8401 / 14700) loss: 0.186600
(Iteration 8411 / 14700) loss: 0.219491
(Iteration 8421 / 14700) loss: 0.233365
(Iteration 8431 / 14700) loss: 0.184694
(Iteration 8441 / 14700) loss: 0.213214
(Iteration 8451 / 14700) loss: 0.189612
(Iteration 8461 / 14700) loss: 0.186625
(Iteration 8471 / 14700) loss: 0.154102
(Iteration 8481 / 14700) loss: 0.157444
(Iteration 8491 / 14700) loss: 0.227466
(Iteration 8501 / 14700) loss: 0.264304
(Iteration 8511 / 14700) loss: 0.125605
(Iteration 8521 / 14700) loss: 0.129447
```

```
(Iteration 8531 / 14700) loss: 0.216694
(Iteration 8541 / 14700) loss: 0.158742
(Iteration 8551 / 14700) loss: 0.099436
(Iteration 8561 / 14700) loss: 0.148014
(Iteration 8571 / 14700) loss: 0.233926
(Iteration 8581 / 14700) loss: 0.121175
(Iteration 8591 / 14700) loss: 0.196680
(Iteration 8601 / 14700) loss: 0.229336
(Iteration 8611 / 14700) loss: 0.250363
(Iteration 8621 / 14700) loss: 0.177901
(Iteration 8631 / 14700) loss: 0.251387
(Iteration 8641 / 14700) loss: 0.236678
(Iteration 8651 / 14700) loss: 0.143028
(Iteration 8661 / 14700) loss: 0.193074
(Iteration 8671 / 14700) loss: 0.228386
(Iteration 8681 / 14700) loss: 0.191518
(Iteration 8691 / 14700) loss: 0.189408
(Iteration 8701 / 14700) loss: 0.188448
(Iteration 8711 / 14700) loss: 0.166999
(Iteration 8721 / 14700) loss: 0.170974
(Iteration 8731 / 14700) loss: 0.221486
(Iteration 8741 / 14700) loss: 0.323593
(Iteration 8751 / 14700) loss: 0.262682
(Iteration 8761 / 14700) loss: 0.182399
(Iteration 8771 / 14700) loss: 0.210355
(Iteration 8781 / 14700) loss: 0.226726
(Iteration 8791 / 14700) loss: 0.143075
(Iteration 8801 / 14700) loss: 0.177514
(Iteration 8811 / 14700) loss: 0.209944
(Epoch 18 / 30) train acc: 0.963000; val_acc: 0.561000
(Iteration 8821 / 14700) loss: 0.266243
(Iteration 8831 / 14700) loss: 0.133250
(Iteration 8841 / 14700) loss: 0.118966
(Iteration 8851 / 14700) loss: 0.197411
(Iteration 8861 / 14700) loss: 0.136227
(Iteration 8871 / 14700) loss: 0.172540
(Iteration 8881 / 14700) loss: 0.206865
(Iteration 8891 / 14700) loss: 0.153315
(Iteration 8901 / 14700) loss: 0.134428
(Iteration 8911 / 14700) loss: 0.305375
(Iteration 8921 / 14700) loss: 0.189910
(Iteration 8931 / 14700) loss: 0.153269
(Iteration 8941 / 14700) loss: 0.206850
(Iteration 8951 / 14700) loss: 0.150394
(Iteration 8961 / 14700) loss: 0.226259
(Iteration 8971 / 14700) loss: 0.159738
(Iteration 8981 / 14700) loss: 0.159121
(Iteration 8991 / 14700) loss: 0.165633
```

```
(Iteration 9001 / 14700) loss: 0.217150
(Iteration 9011 / 14700) loss: 0.176943
(Iteration 9021 / 14700) loss: 0.236065
(Iteration 9031 / 14700) loss: 0.165269
(Iteration 9041 / 14700) loss: 0.222034
(Iteration 9051 / 14700) loss: 0.161721
(Iteration 9061 / 14700) loss: 0.087816
(Iteration 9071 / 14700) loss: 0.165339
(Iteration 9081 / 14700) loss: 0.098896
(Iteration 9091 / 14700) loss: 0.199804
(Iteration 9101 / 14700) loss: 0.137330
(Iteration 9111 / 14700) loss: 0.206511
(Iteration 9121 / 14700) loss: 0.163232
(Iteration 9131 / 14700) loss: 0.145900
(Iteration 9141 / 14700) loss: 0.163356
(Iteration 9151 / 14700) loss: 0.124225
(Iteration 9161 / 14700) loss: 0.243571
(Iteration 9171 / 14700) loss: 0.119147
(Iteration 9181 / 14700) loss: 0.236986
(Iteration 9191 / 14700) loss: 0.105837
(Iteration 9201 / 14700) loss: 0.149793
(Iteration 9211 / 14700) loss: 0.162781
(Iteration 9221 / 14700) loss: 0.136946
(Iteration 9231 / 14700) loss: 0.231327
(Iteration 9241 / 14700) loss: 0.199871
(Iteration 9251 / 14700) loss: 0.207379
(Iteration 9261 / 14700) loss: 0.139690
(Iteration 9271 / 14700) loss: 0.247410
(Iteration 9281 / 14700) loss: 0.142092
(Iteration 9291 / 14700) loss: 0.170890
(Iteration 9301 / 14700) loss: 0.196580
(Epoch 19 / 30) train acc: 0.973000; val_acc: 0.554000
(Iteration 9311 / 14700) loss: 0.137987
(Iteration 9321 / 14700) loss: 0.159836
(Iteration 9331 / 14700) loss: 0.189886
(Iteration 9341 / 14700) loss: 0.151554
(Iteration 9351 / 14700) loss: 0.129346
(Iteration 9361 / 14700) loss: 0.111331
(Iteration 9371 / 14700) loss: 0.152724
(Iteration 9381 / 14700) loss: 0.127119
(Iteration 9391 / 14700) loss: 0.099259
(Iteration 9401 / 14700) loss: 0.195055
(Iteration 9411 / 14700) loss: 0.206717
(Iteration 9421 / 14700) loss: 0.168114
(Iteration 9431 / 14700) loss: 0.103380
(Iteration 9441 / 14700) loss: 0.183077
(Iteration 9451 / 14700) loss: 0.126275
(Iteration 9461 / 14700) loss: 0.207318
```

```
(Iteration 9471 / 14700) loss: 0.091371
(Iteration 9481 / 14700) loss: 0.198061
(Iteration 9491 / 14700) loss: 0.172739
(Iteration 9501 / 14700) loss: 0.115694
(Iteration 9511 / 14700) loss: 0.112781
(Iteration 9521 / 14700) loss: 0.160276
(Iteration 9531 / 14700) loss: 0.099380
(Iteration 9541 / 14700) loss: 0.114943
(Iteration 9551 / 14700) loss: 0.079435
(Iteration 9561 / 14700) loss: 0.088595
(Iteration 9571 / 14700) loss: 0.170538
(Iteration 9581 / 14700) loss: 0.155761
(Iteration 9591 / 14700) loss: 0.124060
(Iteration 9601 / 14700) loss: 0.098514
(Iteration 9611 / 14700) loss: 0.170237
(Iteration 9621 / 14700) loss: 0.096950
(Iteration 9631 / 14700) loss: 0.136661
(Iteration 9641 / 14700) loss: 0.124147
(Iteration 9651 / 14700) loss: 0.136003
(Iteration 9661 / 14700) loss: 0.104293
(Iteration 9671 / 14700) loss: 0.087261
(Iteration 9681 / 14700) loss: 0.118471
(Iteration 9691 / 14700) loss: 0.083205
(Iteration 9701 / 14700) loss: 0.253737
(Iteration 9711 / 14700) loss: 0.123677
(Iteration 9721 / 14700) loss: 0.164666
(Iteration 9731 / 14700) loss: 0.129309
(Iteration 9741 / 14700) loss: 0.095512
(Iteration 9751 / 14700) loss: 0.167916
(Iteration 9761 / 14700) loss: 0.116687
(Iteration 9771 / 14700) loss: 0.183708
(Iteration 9781 / 14700) loss: 0.130424
(Iteration 9791 / 14700) loss: 0.115171
(Epoch 20 / 30) train acc: 0.976000; val_acc: 0.541000
(Iteration 9801 / 14700) loss: 0.103990
(Iteration 9811 / 14700) loss: 0.114815
(Iteration 9821 / 14700) loss: 0.111507
(Iteration 9831 / 14700) loss: 0.106727
(Iteration 9841 / 14700) loss: 0.145102
(Iteration 9851 / 14700) loss: 0.144446
(Iteration 9861 / 14700) loss: 0.119628
(Iteration 9871 / 14700) loss: 0.192367
(Iteration 9881 / 14700) loss: 0.134941
(Iteration 9891 / 14700) loss: 0.110089
(Iteration 9901 / 14700) loss: 0.064576
(Iteration 9911 / 14700) loss: 0.077741
(Iteration 9921 / 14700) loss: 0.138123
(Iteration 9931 / 14700) loss: 0.109047
```

```
(Iteration 9941 / 14700) loss: 0.076774
(Iteration 9951 / 14700) loss: 0.096004
(Iteration 9961 / 14700) loss: 0.136563
(Iteration 9971 / 14700) loss: 0.125600
(Iteration 9981 / 14700) loss: 0.113016
(Iteration 9991 / 14700) loss: 0.154173
(Iteration 10001 / 14700) loss: 0.096002
(Iteration 10011 / 14700) loss: 0.145274
(Iteration 10021 / 14700) loss: 0.109519
(Iteration 10031 / 14700) loss: 0.142366
(Iteration 10041 / 14700) loss: 0.113232
(Iteration 10051 / 14700) loss: 0.131366
(Iteration 10061 / 14700) loss: 0.129849
(Iteration 10071 / 14700) loss: 0.134849
(Iteration 10081 / 14700) loss: 0.122079
(Iteration 10091 / 14700) loss: 0.112800
(Iteration 10101 / 14700) loss: 0.129718
(Iteration 10111 / 14700) loss: 0.135470
(Iteration 10121 / 14700) loss: 0.169324
(Iteration 10131 / 14700) loss: 0.118378
(Iteration 10141 / 14700) loss: 0.120644
(Iteration 10151 / 14700) loss: 0.062352
(Iteration 10161 / 14700) loss: 0.060646
(Iteration 10171 / 14700) loss: 0.085130
(Iteration 10181 / 14700) loss: 0.210669
(Iteration 10191 / 14700) loss: 0.089211
(Iteration 10201 / 14700) loss: 0.144939
(Iteration 10211 / 14700) loss: 0.106890
(Iteration 10221 / 14700) loss: 0.132774
(Iteration 10231 / 14700) loss: 0.130534
(Iteration 10241 / 14700) loss: 0.090504
(Iteration 10251 / 14700) loss: 0.143568
(Iteration 10261 / 14700) loss: 0.092675
(Iteration 10271 / 14700) loss: 0.102431
(Iteration 10281 / 14700) loss: 0.086671
(Epoch 21 / 30) train acc: 0.979000; val_acc: 0.570000
(Iteration 10291 / 14700) loss: 0.082345
(Iteration 10301 / 14700) loss: 0.106823
(Iteration 10311 / 14700) loss: 0.171637
(Iteration 10321 / 14700) loss: 0.093702
(Iteration 10331 / 14700) loss: 0.148613
(Iteration 10341 / 14700) loss: 0.100959
(Iteration 10351 / 14700) loss: 0.072740
(Iteration 10361 / 14700) loss: 0.118766
(Iteration 10371 / 14700) loss: 0.186505
(Iteration 10381 / 14700) loss: 0.118461
(Iteration 10391 / 14700) loss: 0.137158
(Iteration 10401 / 14700) loss: 0.115378
```

```
(Iteration 10411 / 14700) loss: 0.078974
(Iteration 10421 / 14700) loss: 0.060160
(Iteration 10431 / 14700) loss: 0.104396
(Iteration 10441 / 14700) loss: 0.081549
(Iteration 10451 / 14700) loss: 0.172349
(Iteration 10461 / 14700) loss: 0.079637
(Iteration 10471 / 14700) loss: 0.080326
(Iteration 10481 / 14700) loss: 0.085062
(Iteration 10491 / 14700) loss: 0.085903
(Iteration 10501 / 14700) loss: 0.069877
(Iteration 10511 / 14700) loss: 0.079597
(Iteration 10521 / 14700) loss: 0.097009
(Iteration 10531 / 14700) loss: 0.112761
(Iteration 10541 / 14700) loss: 0.055819
(Iteration 10551 / 14700) loss: 0.092746
(Iteration 10561 / 14700) loss: 0.073679
(Iteration 10571 / 14700) loss: 0.108899
(Iteration 10581 / 14700) loss: 0.070423
(Iteration 10591 / 14700) loss: 0.082750
(Iteration 10601 / 14700) loss: 0.074733
(Iteration 10611 / 14700) loss: 0.108397
(Iteration 10621 / 14700) loss: 0.084349
(Iteration 10631 / 14700) loss: 0.083456
(Iteration 10641 / 14700) loss: 0.097788
(Iteration 10651 / 14700) loss: 0.076224
(Iteration 10661 / 14700) loss: 0.062324
(Iteration 10671 / 14700) loss: 0.082755
(Iteration 10681 / 14700) loss: 0.130106
(Iteration 10691 / 14700) loss: 0.104903
(Iteration 10701 / 14700) loss: 0.120500
(Iteration 10711 / 14700) loss: 0.081364
(Iteration 10721 / 14700) loss: 0.088243
(Iteration 10731 / 14700) loss: 0.081806
(Iteration 10741 / 14700) loss: 0.119454
(Iteration 10751 / 14700) loss: 0.067616
(Iteration 10761 / 14700) loss: 0.098242
(Iteration 10771 / 14700) loss: 0.087666
(Epoch 22 / 30) train acc: 0.986000; val_acc: 0.556000
(Iteration 10781 / 14700) loss: 0.091800
(Iteration 10791 / 14700) loss: 0.111014
(Iteration 10801 / 14700) loss: 0.068957
(Iteration 10811 / 14700) loss: 0.068181
(Iteration 10821 / 14700) loss: 0.083388
(Iteration 10831 / 14700) loss: 0.072920
(Iteration 10841 / 14700) loss: 0.089410
(Iteration 10851 / 14700) loss: 0.094868
(Iteration 10861 / 14700) loss: 0.082885
(Iteration 10871 / 14700) loss: 0.062293
```

```
(Iteration 10881 / 14700) loss: 0.095037
(Iteration 10891 / 14700) loss: 0.067172
(Iteration 10901 / 14700) loss: 0.103601
(Iteration 10911 / 14700) loss: 0.093406
(Iteration 10921 / 14700) loss: 0.119672
(Iteration 10931 / 14700) loss: 0.070820
(Iteration 10941 / 14700) loss: 0.093241
(Iteration 10951 / 14700) loss: 0.080048
(Iteration 10961 / 14700) loss: 0.083498
(Iteration 10971 / 14700) loss: 0.100122
(Iteration 10981 / 14700) loss: 0.112001
(Iteration 10991 / 14700) loss: 0.081677
(Iteration 11001 / 14700) loss: 0.114248
(Iteration 11011 / 14700) loss: 0.071436
(Iteration 11021 / 14700) loss: 0.085337
(Iteration 11031 / 14700) loss: 0.079287
(Iteration 11041 / 14700) loss: 0.075610
(Iteration 11051 / 14700) loss: 0.091488
(Iteration 11061 / 14700) loss: 0.146511
(Iteration 11071 / 14700) loss: 0.072947
(Iteration 11081 / 14700) loss: 0.046849
(Iteration 11091 / 14700) loss: 0.070629
(Iteration 11101 / 14700) loss: 0.098325
(Iteration 11111 / 14700) loss: 0.075713
(Iteration 11121 / 14700) loss: 0.084251
(Iteration 11131 / 14700) loss: 0.073571
(Iteration 11141 / 14700) loss: 0.079315
(Iteration 11151 / 14700) loss: 0.057498
(Iteration 11161 / 14700) loss: 0.139781
(Iteration 11171 / 14700) loss: 0.062182
(Iteration 11181 / 14700) loss: 0.096673
(Iteration 11191 / 14700) loss: 0.077980
(Iteration 11201 / 14700) loss: 0.063958
(Iteration 11211 / 14700) loss: 0.068615
(Iteration 11221 / 14700) loss: 0.129708
(Iteration 11231 / 14700) loss: 0.078245
(Iteration 11241 / 14700) loss: 0.070184
(Iteration 11251 / 14700) loss: 0.039702
(Iteration 11261 / 14700) loss: 0.098071
(Epoch 23 / 30) train acc: 0.986000; val_acc: 0.558000
(Iteration 11271 / 14700) loss: 0.099016
(Iteration 11281 / 14700) loss: 0.086505
(Iteration 11291 / 14700) loss: 0.070035
(Iteration 11301 / 14700) loss: 0.100116
(Iteration 11311 / 14700) loss: 0.075355
(Iteration 11321 / 14700) loss: 0.063924
(Iteration 11331 / 14700) loss: 0.068438
(Iteration 11341 / 14700) loss: 0.079183
```

```
(Iteration 11351 / 14700) loss: 0.076910
(Iteration 11361 / 14700) loss: 0.088234
(Iteration 11371 / 14700) loss: 0.064878
(Iteration 11381 / 14700) loss: 0.094540
(Iteration 11391 / 14700) loss: 0.067247
(Iteration 11401 / 14700) loss: 0.048786
(Iteration 11411 / 14700) loss: 0.072718
(Iteration 11421 / 14700) loss: 0.055705
(Iteration 11431 / 14700) loss: 0.086802
(Iteration 11441 / 14700) loss: 0.089312
(Iteration 11451 / 14700) loss: 0.082952
(Iteration 11461 / 14700) loss: 0.111906
(Iteration 11471 / 14700) loss: 0.090087
(Iteration 11481 / 14700) loss: 0.094926
(Iteration 11491 / 14700) loss: 0.059992
(Iteration 11501 / 14700) loss: 0.073421
(Iteration 11511 / 14700) loss: 0.099105
(Iteration 11521 / 14700) loss: 0.083842
(Iteration 11531 / 14700) loss: 0.058223
(Iteration 11541 / 14700) loss: 0.076302
(Iteration 11551 / 14700) loss: 0.083994
(Iteration 11561 / 14700) loss: 0.069134
(Iteration 11571 / 14700) loss: 0.064759
(Iteration 11581 / 14700) loss: 0.063111
(Iteration 11591 / 14700) loss: 0.106127
(Iteration 11601 / 14700) loss: 0.068241
(Iteration 11611 / 14700) loss: 0.070833
(Iteration 11621 / 14700) loss: 0.067115
(Iteration 11631 / 14700) loss: 0.087783
(Iteration 11641 / 14700) loss: 0.076597
(Iteration 11651 / 14700) loss: 0.074500
(Iteration 11661 / 14700) loss: 0.059102
(Iteration 11671 / 14700) loss: 0.037075
(Iteration 11681 / 14700) loss: 0.065485
(Iteration 11691 / 14700) loss: 0.055688
(Iteration 11701 / 14700) loss: 0.056216
(Iteration 11711 / 14700) loss: 0.051101
(Iteration 11721 / 14700) loss: 0.064548
(Iteration 11731 / 14700) loss: 0.069048
(Iteration 11741 / 14700) loss: 0.070476
(Iteration 11751 / 14700) loss: 0.081873
(Epoch 24 / 30) train acc: 0.993000; val_acc: 0.548000
(Iteration 11761 / 14700) loss: 0.071887
(Iteration 11771 / 14700) loss: 0.064218
(Iteration 11781 / 14700) loss: 0.134390
(Iteration 11791 / 14700) loss: 0.064272
(Iteration 11801 / 14700) loss: 0.057047
(Iteration 11811 / 14700) loss: 0.089645
```

```
(Iteration 11821 / 14700) loss: 0.051072
(Iteration 11831 / 14700) loss: 0.092336
(Iteration 11841 / 14700) loss: 0.097341
(Iteration 11851 / 14700) loss: 0.055608
(Iteration 11861 / 14700) loss: 0.067777
(Iteration 11871 / 14700) loss: 0.053726
(Iteration 11881 / 14700) loss: 0.054708
(Iteration 11891 / 14700) loss: 0.088606
(Iteration 11901 / 14700) loss: 0.085508
(Iteration 11911 / 14700) loss: 0.065794
(Iteration 11921 / 14700) loss: 0.046861
(Iteration 11931 / 14700) loss: 0.064658
(Iteration 11941 / 14700) loss: 0.104260
(Iteration 11951 / 14700) loss: 0.048554
(Iteration 11961 / 14700) loss: 0.104330
(Iteration 11971 / 14700) loss: 0.048521
(Iteration 11981 / 14700) loss: 0.067030
(Iteration 11991 / 14700) loss: 0.064251
(Iteration 12001 / 14700) loss: 0.052303
(Iteration 12011 / 14700) loss: 0.070665
(Iteration 12021 / 14700) loss: 0.052998
(Iteration 12031 / 14700) loss: 0.044787
(Iteration 12041 / 14700) loss: 0.053659
(Iteration 12051 / 14700) loss: 0.045439
(Iteration 12061 / 14700) loss: 0.069720
(Iteration 12071 / 14700) loss: 0.073890
(Iteration 12081 / 14700) loss: 0.045767
(Iteration 12091 / 14700) loss: 0.043885
(Iteration 12101 / 14700) loss: 0.073937
(Iteration 12111 / 14700) loss: 0.054074
(Iteration 12121 / 14700) loss: 0.053587
(Iteration 12131 / 14700) loss: 0.061419
(Iteration 12141 / 14700) loss: 0.059816
(Iteration 12151 / 14700) loss: 0.053441
(Iteration 12161 / 14700) loss: 0.074507
(Iteration 12171 / 14700) loss: 0.069185
(Iteration 12181 / 14700) loss: 0.051687
(Iteration 12191 / 14700) loss: 0.060792
(Iteration 12201 / 14700) loss: 0.057475
(Iteration 12211 / 14700) loss: 0.063383
(Iteration 12221 / 14700) loss: 0.053163
(Iteration 12231 / 14700) loss: 0.074449
(Iteration 12241 / 14700) loss: 0.049980
(Epoch 25 / 30) train acc: 0.996000; val_acc: 0.557000
(Iteration 12251 / 14700) loss: 0.099449
(Iteration 12261 / 14700) loss: 0.076191
(Iteration 12271 / 14700) loss: 0.065077
(Iteration 12281 / 14700) loss: 0.057504
```

```
(Iteration 12291 / 14700) loss: 0.067943
(Iteration 12301 / 14700) loss: 0.108649
(Iteration 12311 / 14700) loss: 0.067067
(Iteration 12321 / 14700) loss: 0.107344
(Iteration 12331 / 14700) loss: 0.060060
(Iteration 12341 / 14700) loss: 0.058062
(Iteration 12351 / 14700) loss: 0.053686
(Iteration 12361 / 14700) loss: 0.040501
(Iteration 12371 / 14700) loss: 0.042195
(Iteration 12381 / 14700) loss: 0.064071
(Iteration 12391 / 14700) loss: 0.045152
(Iteration 12401 / 14700) loss: 0.046536
(Iteration 12411 / 14700) loss: 0.050182
(Iteration 12421 / 14700) loss: 0.077322
(Iteration 12431 / 14700) loss: 0.083374
(Iteration 12441 / 14700) loss: 0.053385
(Iteration 12451 / 14700) loss: 0.038104
(Iteration 12461 / 14700) loss: 0.057275
(Iteration 12471 / 14700) loss: 0.077520
(Iteration 12481 / 14700) loss: 0.048657
(Iteration 12491 / 14700) loss: 0.063072
(Iteration 12501 / 14700) loss: 0.091226
(Iteration 12511 / 14700) loss: 0.037472
(Iteration 12521 / 14700) loss: 0.049600
(Iteration 12531 / 14700) loss: 0.050633
(Iteration 12541 / 14700) loss: 0.054125
(Iteration 12551 / 14700) loss: 0.067310
(Iteration 12561 / 14700) loss: 0.067853
(Iteration 12571 / 14700) loss: 0.055027
(Iteration 12581 / 14700) loss: 0.054928
(Iteration 12591 / 14700) loss: 0.071568
(Iteration 12601 / 14700) loss: 0.065226
(Iteration 12611 / 14700) loss: 0.057389
(Iteration 12621 / 14700) loss: 0.050941
(Iteration 12631 / 14700) loss: 0.065009
(Iteration 12641 / 14700) loss: 0.055030
(Iteration 12651 / 14700) loss: 0.046864
(Iteration 12661 / 14700) loss: 0.061184
(Iteration 12671 / 14700) loss: 0.037702
(Iteration 12681 / 14700) loss: 0.050534
(Iteration 12691 / 14700) loss: 0.050097
(Iteration 12701 / 14700) loss: 0.078235
(Iteration 12711 / 14700) loss: 0.059759
(Iteration 12721 / 14700) loss: 0.047964
(Iteration 12731 / 14700) loss: 0.121793
(Epoch 26 / 30) train acc: 1.000000; val_acc: 0.560000
(Iteration 12741 / 14700) loss: 0.061341
(Iteration 12751 / 14700) loss: 0.059871
```

```
(Iteration 12761 / 14700) loss: 0.076689
(Iteration 12771 / 14700) loss: 0.052215
(Iteration 12781 / 14700) loss: 0.043003
(Iteration 12791 / 14700) loss: 0.043620
(Iteration 12801 / 14700) loss: 0.081378
(Iteration 12811 / 14700) loss: 0.072714
(Iteration 12821 / 14700) loss: 0.045063
(Iteration 12831 / 14700) loss: 0.049219
(Iteration 12841 / 14700) loss: 0.067493
(Iteration 12851 / 14700) loss: 0.054093
(Iteration 12861 / 14700) loss: 0.039489
(Iteration 12871 / 14700) loss: 0.045336
(Iteration 12881 / 14700) loss: 0.068852
(Iteration 12891 / 14700) loss: 0.077575
(Iteration 12901 / 14700) loss: 0.067425
(Iteration 12911 / 14700) loss: 0.089539
(Iteration 12921 / 14700) loss: 0.058727
(Iteration 12931 / 14700) loss: 0.047732
(Iteration 12941 / 14700) loss: 0.059226
(Iteration 12951 / 14700) loss: 0.053786
(Iteration 12961 / 14700) loss: 0.044453
(Iteration 12971 / 14700) loss: 0.059622
(Iteration 12981 / 14700) loss: 0.061297
(Iteration 12991 / 14700) loss: 0.056298
(Iteration 13001 / 14700) loss: 0.039197
(Iteration 13011 / 14700) loss: 0.053404
(Iteration 13021 / 14700) loss: 0.050914
(Iteration 13031 / 14700) loss: 0.043471
(Iteration 13041 / 14700) loss: 0.049154
(Iteration 13051 / 14700) loss: 0.073724
(Iteration 13061 / 14700) loss: 0.046676
(Iteration 13071 / 14700) loss: 0.052779
(Iteration 13081 / 14700) loss: 0.041066
(Iteration 13091 / 14700) loss: 0.047688
(Iteration 13101 / 14700) loss: 0.042617
(Iteration 13111 / 14700) loss: 0.042628
(Iteration 13121 / 14700) loss: 0.038153
(Iteration 13131 / 14700) loss: 0.053930
(Iteration 13141 / 14700) loss: 0.054978
(Iteration 13151 / 14700) loss: 0.046585
(Iteration 13161 / 14700) loss: 0.049944
(Iteration 13171 / 14700) loss: 0.053142
(Iteration 13181 / 14700) loss: 0.046488
(Iteration 13191 / 14700) loss: 0.053609
(Iteration 13201 / 14700) loss: 0.046101
(Iteration 13211 / 14700) loss: 0.053752
(Iteration 13221 / 14700) loss: 0.062968
(Epoch 27 / 30) train acc: 0.994000; val_acc: 0.562000
```

```
(Iteration 13231 / 14700) loss: 0.041458
(Iteration 13241 / 14700) loss: 0.063024
(Iteration 13251 / 14700) loss: 0.036512
(Iteration 13261 / 14700) loss: 0.068305
(Iteration 13271 / 14700) loss: 0.063995
(Iteration 13281 / 14700) loss: 0.048268
(Iteration 13291 / 14700) loss: 0.048687
(Iteration 13301 / 14700) loss: 0.034155
(Iteration 13311 / 14700) loss: 0.035907
(Iteration 13321 / 14700) loss: 0.051379
(Iteration 13331 / 14700) loss: 0.034991
(Iteration 13341 / 14700) loss: 0.045981
(Iteration 13351 / 14700) loss: 0.042258
(Iteration 13361 / 14700) loss: 0.059533
(Iteration 13371 / 14700) loss: 0.049370
(Iteration 13381 / 14700) loss: 0.037101
(Iteration 13391 / 14700) loss: 0.054181
(Iteration 13401 / 14700) loss: 0.063347
(Iteration 13411 / 14700) loss: 0.055115
(Iteration 13421 / 14700) loss: 0.048975
(Iteration 13431 / 14700) loss: 0.050544
(Iteration 13441 / 14700) loss: 0.054658
(Iteration 13451 / 14700) loss: 0.043724
(Iteration 13461 / 14700) loss: 0.041367
(Iteration 13471 / 14700) loss: 0.044909
(Iteration 13481 / 14700) loss: 0.044669
(Iteration 13491 / 14700) loss: 0.040066
(Iteration 13501 / 14700) loss: 0.052091
(Iteration 13511 / 14700) loss: 0.046992
(Iteration 13521 / 14700) loss: 0.034461
(Iteration 13531 / 14700) loss: 0.048773
(Iteration 13541 / 14700) loss: 0.044824
(Iteration 13551 / 14700) loss: 0.035998
(Iteration 13561 / 14700) loss: 0.052288
(Iteration 13571 / 14700) loss: 0.057604
(Iteration 13581 / 14700) loss: 0.047813
(Iteration 13591 / 14700) loss: 0.041713
(Iteration 13601 / 14700) loss: 0.043003
(Iteration 13611 / 14700) loss: 0.093000
(Iteration 13621 / 14700) loss: 0.043402
(Iteration 13631 / 14700) loss: 0.040965
(Iteration 13641 / 14700) loss: 0.047998
(Iteration 13651 / 14700) loss: 0.054668
(Iteration 13661 / 14700) loss: 0.053222
(Iteration 13671 / 14700) loss: 0.041971
(Iteration 13681 / 14700) loss: 0.041215
(Iteration 13691 / 14700) loss: 0.053286
(Iteration 13701 / 14700) loss: 0.056068
```

```
(Iteration 13711 / 14700) loss: 0.054647
(Epoch 28 / 30) train acc: 0.998000; val_acc: 0.563000
(Iteration 13721 / 14700) loss: 0.036011
(Iteration 13731 / 14700) loss: 0.040450
(Iteration 13741 / 14700) loss: 0.054417
(Iteration 13751 / 14700) loss: 0.058851
(Iteration 13761 / 14700) loss: 0.034344
(Iteration 13771 / 14700) loss: 0.039031
(Iteration 13781 / 14700) loss: 0.036944
(Iteration 13791 / 14700) loss: 0.058907
(Iteration 13801 / 14700) loss: 0.041209
(Iteration 13811 / 14700) loss: 0.042165
(Iteration 13821 / 14700) loss: 0.038399
(Iteration 13831 / 14700) loss: 0.055164
(Iteration 13841 / 14700) loss: 0.042675
(Iteration 13851 / 14700) loss: 0.056102
(Iteration 13861 / 14700) loss: 0.047449
(Iteration 13871 / 14700) loss: 0.042565
(Iteration 13881 / 14700) loss: 0.039674
(Iteration 13891 / 14700) loss: 0.040639
(Iteration 13901 / 14700) loss: 0.047363
(Iteration 13911 / 14700) loss: 0.045798
(Iteration 13921 / 14700) loss: 0.043995
(Iteration 13931 / 14700) loss: 0.045074
(Iteration 13941 / 14700) loss: 0.063748
(Iteration 13951 / 14700) loss: 0.068137
(Iteration 13961 / 14700) loss: 0.042471
(Iteration 13971 / 14700) loss: 0.035341
(Iteration 13981 / 14700) loss: 0.079754
(Iteration 13991 / 14700) loss: 0.033203
(Iteration 14001 / 14700) loss: 0.037058
(Iteration 14011 / 14700) loss: 0.051595
(Iteration 14021 / 14700) loss: 0.062294
(Iteration 14031 / 14700) loss: 0.051160
(Iteration 14041 / 14700) loss: 0.039046
(Iteration 14051 / 14700) loss: 0.034603
(Iteration 14061 / 14700) loss: 0.056321
(Iteration 14071 / 14700) loss: 0.045448
(Iteration 14081 / 14700) loss: 0.064632
(Iteration 14091 / 14700) loss: 0.053176
(Iteration 14101 / 14700) loss: 0.071798
(Iteration 14111 / 14700) loss: 0.068915
(Iteration 14121 / 14700) loss: 0.047112
(Iteration 14131 / 14700) loss: 0.053195
(Iteration 14141 / 14700) loss: 0.039948
(Iteration 14151 / 14700) loss: 0.043135
(Iteration 14161 / 14700) loss: 0.033794
(Iteration 14171 / 14700) loss: 0.057115
```

```
(Iteration 14181 / 14700) loss: 0.033049
(Iteration 14191 / 14700) loss: 0.042748
(Iteration 14201 / 14700) loss: 0.042137
(Epoch 29 / 30) train acc: 0.999000; val_acc: 0.569000
(Iteration 14211 / 14700) loss: 0.040395
(Iteration 14221 / 14700) loss: 0.043953
(Iteration 14231 / 14700) loss: 0.044564
(Iteration 14241 / 14700) loss: 0.032107
(Iteration 14251 / 14700) loss: 0.046350
(Iteration 14261 / 14700) loss: 0.093176
(Iteration 14271 / 14700) loss: 0.081585
(Iteration 14281 / 14700) loss: 0.039355
(Iteration 14291 / 14700) loss: 0.035696
(Iteration 14301 / 14700) loss: 0.041662
(Iteration 14311 / 14700) loss: 0.076771
(Iteration 14321 / 14700) loss: 0.048074
(Iteration 14331 / 14700) loss: 0.050403
(Iteration 14341 / 14700) loss: 0.038633
(Iteration 14351 / 14700) loss: 0.038055
(Iteration 14361 / 14700) loss: 0.043745
(Iteration 14371 / 14700) loss: 0.045481
(Iteration 14381 / 14700) loss: 0.037625
(Iteration 14391 / 14700) loss: 0.042732
(Iteration 14401 / 14700) loss: 0.033372
(Iteration 14411 / 14700) loss: 0.026351
(Iteration 14421 / 14700) loss: 0.032079
(Iteration 14431 / 14700) loss: 0.039083
(Iteration 14441 / 14700) loss: 0.034201
(Iteration 14451 / 14700) loss: 0.034301
(Iteration 14461 / 14700) loss: 0.054567
(Iteration 14471 / 14700) loss: 0.039883
(Iteration 14481 / 14700) loss: 0.063367
(Iteration 14491 / 14700) loss: 0.048763
(Iteration 14501 / 14700) loss: 0.033395
(Iteration 14511 / 14700) loss: 0.061949
(Iteration 14521 / 14700) loss: 0.042457
(Iteration 14531 / 14700) loss: 0.042171
(Iteration 14541 / 14700) loss: 0.046332
(Iteration 14551 / 14700) loss: 0.054879
(Iteration 14561 / 14700) loss: 0.040846
(Iteration 14571 / 14700) loss: 0.051178
(Iteration 14581 / 14700) loss: 0.050783
(Iteration 14591 / 14700) loss: 0.041428
(Iteration 14601 / 14700) loss: 0.039652
(Iteration 14611 / 14700) loss: 0.033563
(Iteration 14621 / 14700) loss: 0.039340
(Iteration 14631 / 14700) loss: 0.032866
(Iteration 14641 / 14700) loss: 0.044186
```

```
(Iteration 14651 / 14700) loss: 0.036423
(Iteration 14661 / 14700) loss: 0.109786
(Iteration 14671 / 14700) loss: 0.038298
(Iteration 14681 / 14700) loss: 0.038168
(Iteration 14691 / 14700) loss: 0.027018
(Epoch 30 / 30) train acc: 0.999000; val_acc: 0.567000
with learning rate 0.5, regularization strength 5.8e-06, accuracy: 0.575
the best accuracy on validation set is 0.61
```

[70]:
```
# Run your best neural net classifier on the test set. You should be able
# to get more than 58% accuracy. It is also possible to get >60% accuracy
# with careful tuning.

y_test_pred = np.argmax(best_net.loss(data['X_test']), axis=1)
test_acc = (y_test_pred == data['y_test']).mean()
print(test_acc)
```

```
0.594
```

[71]:
```
# Save best model
best_net.save("best_two_layer_net_features.npy")
```

```
best_two_layer_net_features.npy saved.
```

FullyConnectedNets

April 5, 2026

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment1/'
FOLDERNAME = 'CS231n/assignment1/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/MyDrive/{}'.format(FOLDERNAME))

# This downloads the CIFAR-10 dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/MyDrive/$FOLDERNAME/cs231n/datasets/
# %cd /content/drive/MyDrive/$FOLDERNAME
!bash get_datasets.sh
%cd /content/drive/MyDrive/$FOLDERNAME
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
/content/drive/MyDrive/CS231n/assignment1/cs231n/datasets
/content/drive/MyDrive/CS231n/assignment1
```

1 Multi-Layer Fully Connected Network

In this exercise, you will implement a fully connected network with an arbitrary number of hidden layers.

[2]:
```
# from google.colab import drive
# drive.mount('/content/drive')
```

Read through the `FullyConnectedNet` class in the file `cs231n/classifiers/fc_net.py`.

Implement the network initialization, forward pass, and backward pass. Throughout this assign-

ment, you will be implementing layers in `cs231n/layers.py`. You can re-use your implementations for `affine_forward`, `affine_backward`, `relu_forward`, `relu_backward`, and `softmax_loss` from before. For right now, don't worry about implementing dropout or batch/layer normalization yet, as you will add those features later.

[3]:
```
# Setup cell.
import time
import numpy as np
import matplotlib.pyplot as plt
from cs231n.classifiers.fc_net import *
from cs231n.data_utils import get_CIFAR10_data
from cs231n.gradient_check import eval_numerical_gradient,␣
↪eval_numerical_gradient_array
from cs231n.solver import Solver
import sys
import importlib
sys.modules['imp'] = importlib
%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0)  # Set default size of plots.
plt.rcParams["image.interpolation"] = "nearest"
plt.rcParams["image.cmap"] = "gray"

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

[4]:
```
# Load the (preprocessed) CIFAR-10 data.
data = get_CIFAR10_data()
for k, v in list(data.items()):
    print(f"{k}: {v.shape}")
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

1.1 Initial Loss and Gradient Check

As a sanity check, run the following to check the initial loss and to gradient check the network both with and without regularization. This is a good way to see if the initial losses seem reasonable.

For gradient checking, you should expect to see errors around 1e-7 or less.

[5]:
```
np.random.seed(231)
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for reg in [0, 3.14]:
    print("Running check with reg = ", reg)
    model = FullyConnectedNet(
        [H1, H2],
        input_dim=D,
        num_classes=C,
        reg=reg,
        weight_scale=5e-2,
        dtype=np.float64
    )

    loss, grads = model.loss(X, y)
    print("Initial loss: ", loss)

    # Most of the errors should be on the order of e-7 or smaller.
    # NOTE: It is fine however to see an error for W2 on the order of e-5
    # for the check when reg = 0.0
    for name in sorted(grads):
        f = lambda _: model.loss(X, y)[0]
        grad_num = eval_numerical_gradient(f, model.params[name],␣
 ↪verbose=False, h=1e-5)
        print(f"{name} relative error: {rel_error(grad_num, grads[name])}")
```

```
Running check with reg =  0
Initial loss:  2.300479089768492
W1 relative error: 1.0252674471656573e-07
W2 relative error: 2.2120479295080622e-05
W3 relative error: 4.5623278736665505e-07
b1 relative error: 4.6600944653202505e-09
b2 relative error: 2.085654276112763e-09
b3 relative error: 1.689724888469736e-10
Running check with reg =  3.14
Initial loss:  7.052114776533016
W1 relative error: 3.904541941902138e-09
W2 relative error: 6.86942277940646e-08
W3 relative error: 3.483989247437803e-08
b1 relative error: 1.4752427965311745e-08
b2 relative error: 1.4615869332918208e-09
b3 relative error: 1.3200479211447775e-10
```

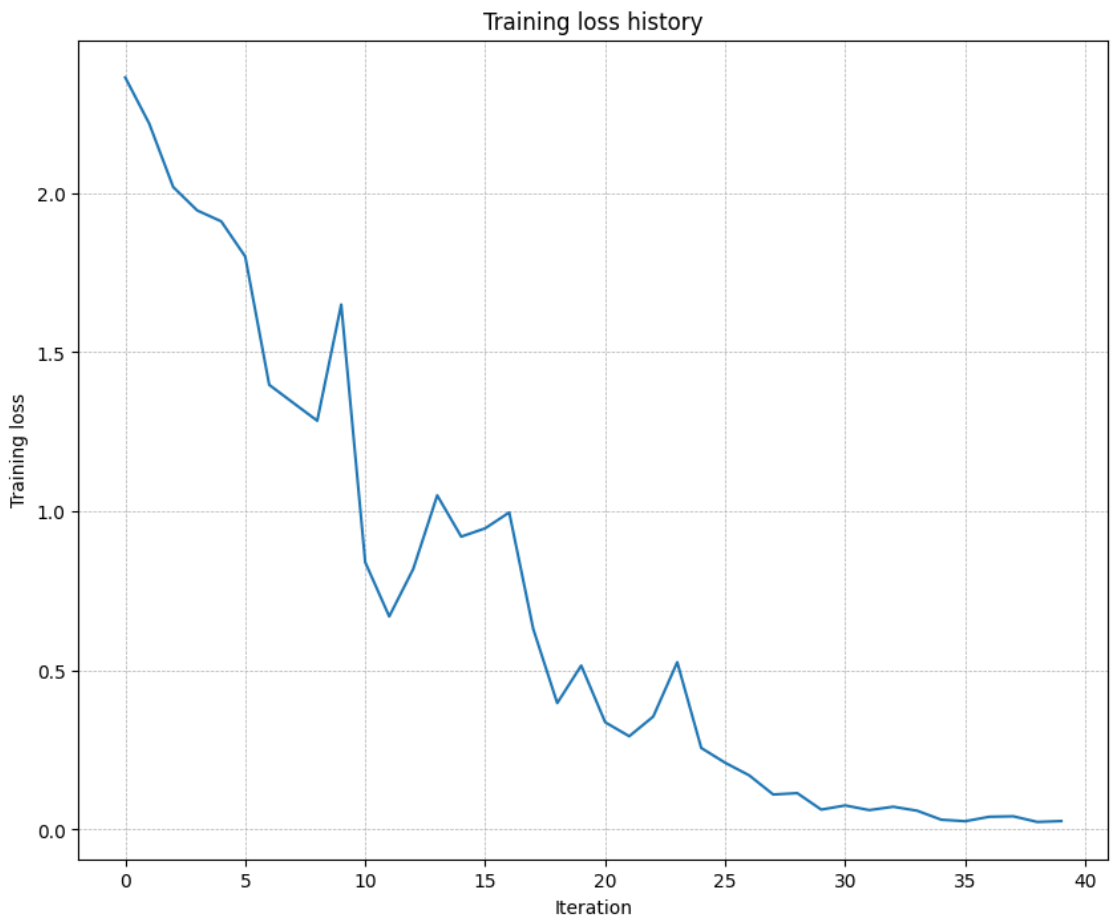
As another sanity check, make sure your network can overfit on a small dataset of 50 images. First, we will try a three-layer network with 100 units in each hidden layer. In the following cell, tweak the **learning rate** and **weight initialization scale** to overfit and achieve 100% training accuracy

within 20 epochs.

[6]:
```
# TODO: Use a three-layer Net to overfit 50 training examples by
# tweaking just the learning rate and initialization scale.

num_train = 50
small_data = {
  "X_train": data["X_train"][:num_train],
  "y_train": data["y_train"][:num_train],
  "X_val": data["X_val"],
  "y_val": data["y_val"],
}

weight_scale = 1e-2   # Experiment with this!
learning_rate = 1e-2  # Experiment with this!


model = FullyConnectedNet(
    [100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule="sgd",
    optim_config={"learning_rate": learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title("Training loss history")
plt.xlabel("Iteration")
plt.ylabel("Training loss")
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 2.363364
(Epoch 0 / 20) train acc: 0.180000; val_acc: 0.108000
(Epoch 1 / 20) train acc: 0.320000; val_acc: 0.127000
(Epoch 2 / 20) train acc: 0.440000; val_acc: 0.172000
(Epoch 3 / 20) train acc: 0.500000; val_acc: 0.184000
(Epoch 4 / 20) train acc: 0.540000; val_acc: 0.181000
(Epoch 5 / 20) train acc: 0.740000; val_acc: 0.190000
(Iteration 11 / 40) loss: 0.839976
```

```
(Epoch 6 / 20) train acc: 0.740000; val_acc: 0.187000
(Epoch 7 / 20) train acc: 0.740000; val_acc: 0.183000
(Epoch 8 / 20) train acc: 0.820000; val_acc: 0.177000
(Epoch 9 / 20) train acc: 0.860000; val_acc: 0.200000
(Epoch 10 / 20) train acc: 0.920000; val_acc: 0.191000
(Iteration 21 / 40) loss: 0.337174
(Epoch 11 / 20) train acc: 0.960000; val_acc: 0.189000
(Epoch 12 / 20) train acc: 0.940000; val_acc: 0.180000
(Epoch 13 / 20) train acc: 1.000000; val_acc: 0.199000
(Epoch 14 / 20) train acc: 1.000000; val_acc: 0.199000
(Epoch 15 / 20) train acc: 1.000000; val_acc: 0.195000
(Iteration 31 / 40) loss: 0.075911
(Epoch 16 / 20) train acc: 1.000000; val_acc: 0.182000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.201000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.207000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.185000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.192000
```



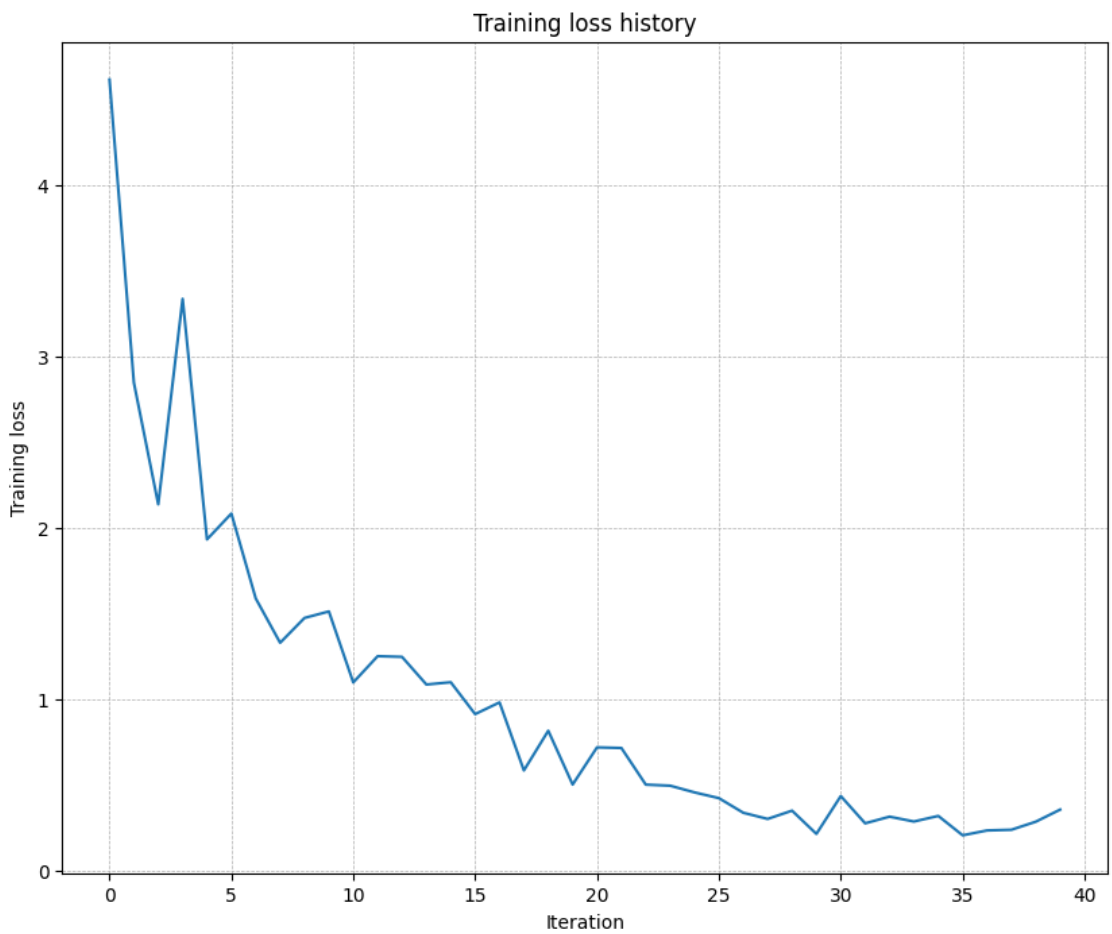
Now, try to use a five-layer network with 100 units on each layer to overfit on 50 training examples.

Again, you will have to adjust the learning rate and weight initialization scale, but you should be able to achieve 100% training accuracy within 20 epochs.

[8]:
```
# TODO: Use a five-layer Net to overfit 50 training examples by
# tweaking just the learning rate and initialization scale.

num_train = 50
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

learning_rate = 2e-3  # Experiment with this!
weight_scale = 5e-2   # Experiment with this!


model = FullyConnectedNet(
    [100, 100, 100, 100],
    weight_scale=weight_scale,
    dtype=np.float64
)
solver = Solver(
    model,
    small_data,
    print_every=10,
    num_epochs=20,
    batch_size=25,
    update_rule='sgd',
    optim_config={'learning_rate': learning_rate},
)
solver.train()

plt.plot(solver.loss_history)
plt.title('Training loss history')
plt.xlabel('Iteration')
plt.ylabel('Training loss')
plt.grid(linestyle='--', linewidth=0.5)
plt.show()
```

```
(Iteration 1 / 40) loss: 4.618178
(Epoch 0 / 20) train acc: 0.160000; val_acc: 0.095000
(Epoch 1 / 20) train acc: 0.220000; val_acc: 0.098000
(Epoch 2 / 20) train acc: 0.300000; val_acc: 0.115000
(Epoch 3 / 20) train acc: 0.500000; val_acc: 0.119000
(Epoch 4 / 20) train acc: 0.540000; val_acc: 0.116000
(Epoch 5 / 20) train acc: 0.620000; val_acc: 0.141000
```

```
(Iteration 11 / 40) loss: 1.101288
(Epoch 6 / 20) train acc: 0.760000; val_acc: 0.127000
(Epoch 7 / 20) train acc: 0.820000; val_acc: 0.146000
(Epoch 8 / 20) train acc: 0.840000; val_acc: 0.136000
(Epoch 9 / 20) train acc: 0.860000; val_acc: 0.154000
(Epoch 10 / 20) train acc: 0.920000; val_acc: 0.142000
(Iteration 21 / 40) loss: 0.722512
(Epoch 11 / 20) train acc: 0.920000; val_acc: 0.148000
(Epoch 12 / 20) train acc: 0.960000; val_acc: 0.145000
(Epoch 13 / 20) train acc: 0.960000; val_acc: 0.148000
(Epoch 14 / 20) train acc: 0.980000; val_acc: 0.150000
(Epoch 15 / 20) train acc: 0.960000; val_acc: 0.150000
(Iteration 31 / 40) loss: 0.438442
(Epoch 16 / 20) train acc: 0.980000; val_acc: 0.135000
(Epoch 17 / 20) train acc: 1.000000; val_acc: 0.147000
(Epoch 18 / 20) train acc: 1.000000; val_acc: 0.145000
(Epoch 19 / 20) train acc: 1.000000; val_acc: 0.153000
(Epoch 20 / 20) train acc: 1.000000; val_acc: 0.143000
```



1.2 Inline Question 1:

Did you notice anything about the comparative difficulty of training the three-layer network vs. training the five-layer network? In particular, based on your experience, which network seemed more sensitive to the initialization scale? Why do you think that is the case?

1.3 Answer:

The learning rate for 5 layers needs to be larger.

5 layers. The variance would accumulate.

2 Update rules

So far we have used vanilla stochastic gradient descent (SGD) as our update rule. More sophisticated update rules can make it easier to train deep networks. We will implement a few of the most commonly used update rules and compare them to vanilla SGD.

2.1 SGD+Momentum

Stochastic gradient descent with momentum is a widely used update rule that tends to make deep networks converge faster than vanilla stochastic gradient descent. See the Momentum Update section at http://cs231n.github.io/neural-networks-3/#sgd for more information.

Open the file `cs231n/optim.py` and read the documentation at the top of the file to make sure you understand the API. Implement the SGD+momentum update rule in the function `sgd_momentum` and run the following to check your implementation. You should see errors less than e-8.

[10]:
```
from cs231n.optim import sgd_momentum

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
v = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {"learning_rate": 1e-3, "velocity": v}
next_w, _ = sgd_momentum(w, dw, config=config)

expected_next_w = np.asarray([
  [ 0.1406,      0.20738947,  0.27417895,  0.34096842,  0.40775789],
  [ 0.47454737,  0.54133684,  0.60812632,  0.67491579,  0.74170526],
  [ 0.80849474,  0.87528421,  0.94207368,  1.00886316,  1.07565263],
  [ 1.14244211,  1.20923158,  1.27602105,  1.34281053,  1.4096    ]])
expected_velocity = np.asarray([
  [ 0.5406,      0.55475789,  0.56891579, 0.58307368,  0.59723158],
  [ 0.61138947,  0.62554737,  0.63970526,  0.65386316,  0.66802105],
  [ 0.68217895,  0.69633684,  0.71049474,  0.72465263,  0.73881053],
  [ 0.75296842,  0.76712632,  0.78128421,  0.79544211,  0.8096    ]])
```

```
# Should see relative errors around e-8 or less
print("next_w error: ", rel_error(next_w, expected_next_w))
print("velocity error: ", rel_error(expected_velocity, config["velocity"]))
```

```
next_w error:  8.882347033505819e-09
velocity error:  4.269287743278663e-09
```

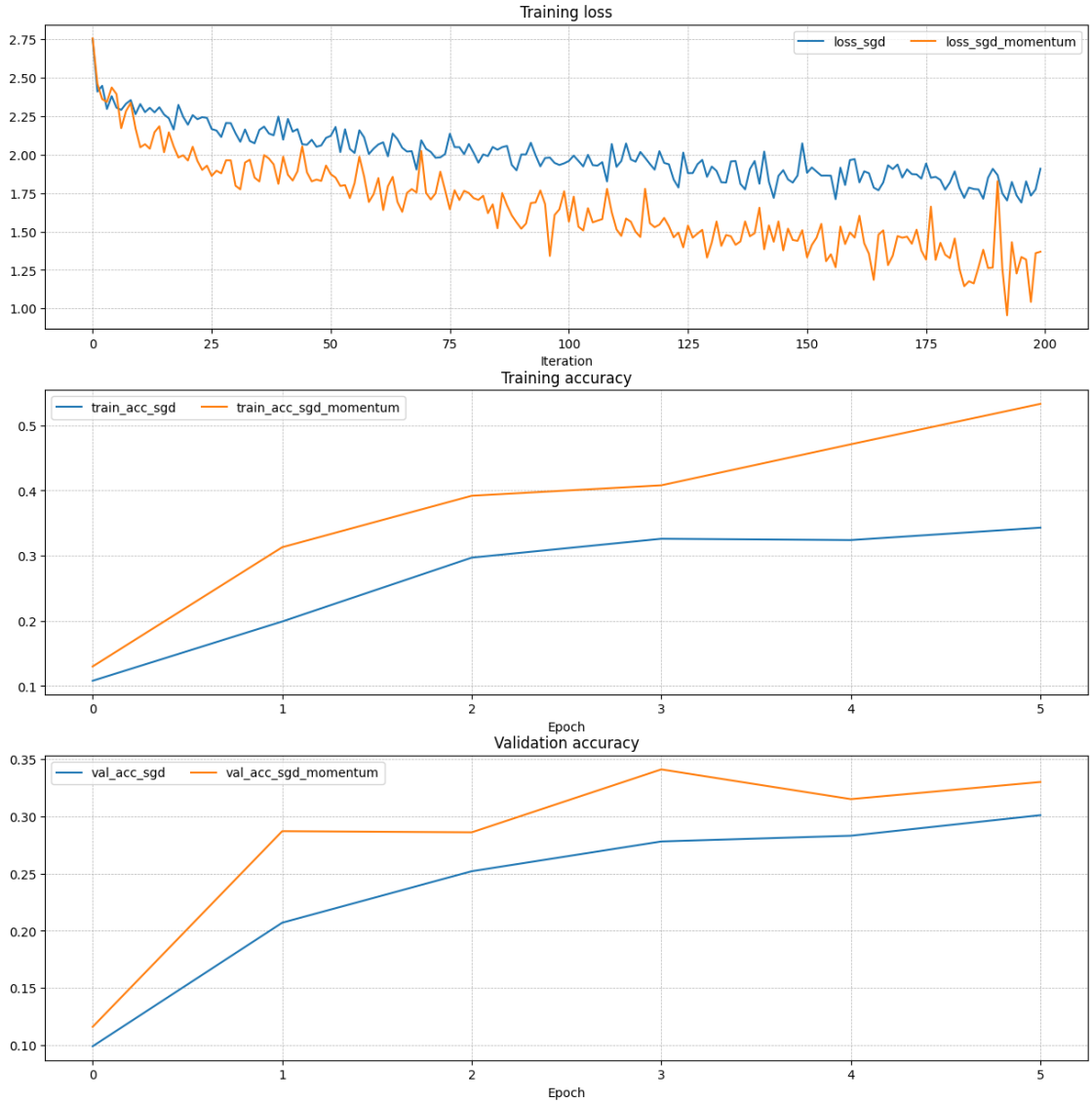
Once you have done so, run the following to train a six-layer network with both SGD and SGD+momentum. You should see the SGD+momentum update rule converge faster.

[11]:
```
num_train = 4000
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

solvers = {}

for update_rule in ['sgd', 'sgd_momentum']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )

    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': 5e-3},
        verbose=True,
    )
    solvers[update_rule] = solver
    solver.train()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')
```

```python
for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"loss_{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"train_acc_{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"val_acc_{update_rule}")

for ax in axes:
    ax.legend(loc="best", ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  sgd
(Iteration 1 / 200) loss: 2.748818
(Epoch 0 / 5) train acc: 0.108000; val_acc: 0.099000
(Iteration 11 / 200) loss: 2.328016
(Iteration 21 / 200) loss: 2.193851
(Iteration 31 / 200) loss: 2.137887
(Epoch 1 / 5) train acc: 0.199000; val_acc: 0.207000
(Iteration 41 / 200) loss: 2.096686
(Iteration 51 / 200) loss: 2.121650
(Iteration 61 / 200) loss: 2.065826
(Iteration 71 / 200) loss: 2.037420
(Epoch 2 / 5) train acc: 0.297000; val_acc: 0.252000
(Iteration 81 / 200) loss: 2.010506
(Iteration 91 / 200) loss: 2.000336
(Iteration 101 / 200) loss: 1.958244
(Iteration 111 / 200) loss: 1.920142
(Epoch 3 / 5) train acc: 0.326000; val_acc: 0.278000
(Iteration 121 / 200) loss: 1.945631
(Iteration 131 / 200) loss: 1.921746
(Iteration 141 / 200) loss: 1.810008
(Iteration 151 / 200) loss: 1.881182
(Epoch 4 / 5) train acc: 0.324000; val_acc: 0.283000
(Iteration 161 / 200) loss: 1.970306
(Iteration 171 / 200) loss: 1.850312
(Iteration 181 / 200) loss: 1.818468
(Iteration 191 / 200) loss: 1.866706
(Epoch 5 / 5) train acc: 0.343000; val_acc: 0.301000
Running with  sgd_momentum
(Iteration 1 / 200) loss: 2.754692
(Epoch 0 / 5) train acc: 0.130000; val_acc: 0.116000
(Iteration 11 / 200) loss: 2.046967
(Iteration 21 / 200) loss: 1.961226
(Iteration 31 / 200) loss: 1.797457
(Epoch 1 / 5) train acc: 0.313000; val_acc: 0.287000
(Iteration 41 / 200) loss: 1.986396
(Iteration 51 / 200) loss: 1.872173
```

```
(Iteration 61 / 200) loss: 1.847111
(Iteration 71 / 200) loss: 1.750988
(Epoch 2 / 5) train acc: 0.392000; val_acc: 0.286000
(Iteration 81 / 200) loss: 1.716712
(Iteration 91 / 200) loss: 1.517957
(Iteration 101 / 200) loss: 1.563843
(Iteration 111 / 200) loss: 1.513238
(Epoch 3 / 5) train acc: 0.408000; val_acc: 0.341000
(Iteration 121 / 200) loss: 1.588529
(Iteration 131 / 200) loss: 1.428497
(Iteration 141 / 200) loss: 1.654260
(Iteration 151 / 200) loss: 1.331691
(Epoch 4 / 5) train acc: 0.471000; val_acc: 0.315000
(Iteration 161 / 200) loss: 1.460159
(Iteration 171 / 200) loss: 1.459527
(Iteration 181 / 200) loss: 1.326956
(Iteration 191 / 200) loss: 1.826592
(Epoch 5 / 5) train acc: 0.533000; val_acc: 0.330000
```



2.2 RMSProp and Adam

RMSProp [1] and Adam [2] are update rules that set per-parameter learning rates by using a running average of the second moments of gradients.

In the file `cs231n/optim.py`, implement the RMSProp update rule in the `rmsprop` function and implement the Adam update rule in the `adam` function, and check your implementations using the tests below.

NOTE: Please implement the *complete* Adam update rule (with the bias correction mechanism), not the first simplified version mentioned in the course notes.

[1] Tijmen Tieleman and Geoffrey Hinton. "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude." COURSERA: Neural Networks for Machine Learning 4 (2012).

[2] Diederik Kingma and Jimmy Ba, "Adam: A Method for Stochastic Optimization", ICLR 2015.

[17]:
```
# Test RMSProp implementation
from cs231n.optim import rmsprop

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
cache = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'cache': cache}
next_w, _ = rmsprop(w, dw, config=config)

expected_next_w = np.asarray([
  [-0.39223849, -0.34037513, -0.28849239, -0.23659121, -0.18467247],
  [-0.132737,   -0.08078555, -0.02881884,  0.02316247,  0.07515774],
  [ 0.12716641,  0.17918792,  0.23122175,  0.28326742,  0.33532447],
  [ 0.38739248,  0.43947102,  0.49155973,  0.54365823,  0.59576619]])
expected_cache = np.asarray([
  [ 0.5976,      0.6126277,   0.6277108,   0.64284931,  0.65804321],
  [ 0.67329252,  0.68859723,  0.70395734,  0.71937285,  0.73484377],
  [ 0.75037008,  0.7659518,   0.78158892,  0.79728144,  0.81302936],
  [ 0.82883269,  0.84469141,  0.86060554,  0.87657507,  0.8926    ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('cache error: ', rel_error(expected_cache, config['cache']))
```

```
next_w error:  9.524687511038133e-08
cache error:  2.6477955807156126e-09
```

[19]:
```
# Test Adam implementation
from cs231n.optim import adam

N, D = 4, 5
w = np.linspace(-0.4, 0.6, num=N*D).reshape(N, D)
dw = np.linspace(-0.6, 0.4, num=N*D).reshape(N, D)
m = np.linspace(0.6, 0.9, num=N*D).reshape(N, D)
v = np.linspace(0.7, 0.5, num=N*D).reshape(N, D)

config = {'learning_rate': 1e-2, 'm': m, 'v': v, 't': 5}
next_w, _ = adam(w, dw, config=config)

expected_next_w = np.asarray([
  [-0.40094747, -0.34836187, -0.29577703, -0.24319299, -0.19060977],
  [-0.1380274,  -0.08544591, -0.03286534,  0.01971428,  0.0722929],
  [ 0.1248705,   0.17744702,  0.23002243,  0.28259667,  0.33516969],
  [ 0.38774145,  0.44031188,  0.49288093,  0.54544852,  0.59801459]])
```

```
expected_v = np.asarray([
  [ 0.69966,     0.68908382,  0.67851319,  0.66794809,  0.65738853,],
  [ 0.64683452,  0.63628604,  0.6257431,   0.61520571,  0.60467385,],
  [ 0.59414753,  0.58362676,  0.57311152,  0.56260183,  0.55209767,],
  [ 0.54159906,  0.53110598,  0.52061845,  0.51013645,  0.49966,   ]])
expected_m = np.asarray([
  [ 0.48,        0.49947368,  0.51894737,  0.53842105,  0.55789474],
  [ 0.57736842,  0.59684211,  0.61631579,  0.63578947,  0.65526316],
  [ 0.67473684,  0.69421053,  0.71368421,  0.73315789,  0.75263158],
  [ 0.77210526,  0.79157895,  0.81105263,  0.83052632,  0.85      ]])

# You should see relative errors around e-7 or less
print('next_w error: ', rel_error(expected_next_w, next_w))
print('v error: ', rel_error(expected_v, config['v']))
print('m error: ', rel_error(expected_m, config['m']))
```

```
next_w error:  1.1395691798535431e-07
v error:  4.208314038113071e-09
m error:  4.214963193114416e-09
```

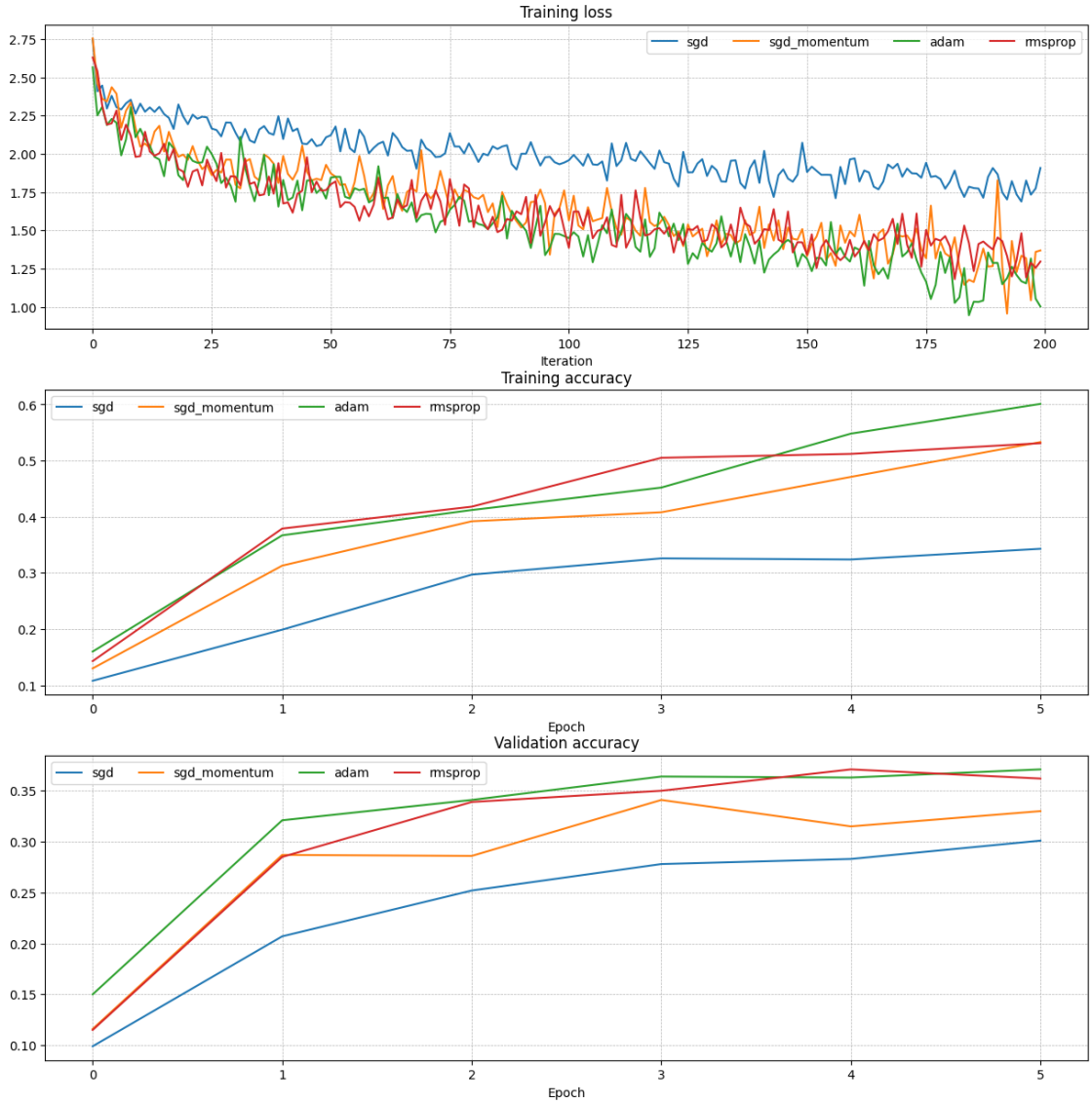
Once you have debugged your RMSProp and Adam implementations, run the following to train a pair of deep networks using these new update rules:

[20]:
```
learning_rates = {'rmsprop': 1e-4, 'adam': 1e-3}
for update_rule in ['adam', 'rmsprop']:
    print('Running with ', update_rule)
    model = FullyConnectedNet(
        [100, 100, 100, 100, 100],
        weight_scale=5e-2
    )
    solver = Solver(
        model,
        small_data,
        num_epochs=5,
        batch_size=100,
        update_rule=update_rule,
        optim_config={'learning_rate': learning_rates[update_rule]},
        verbose=True
    )
    solvers[update_rule] = solver
    solver.train()
    print()

fig, axes = plt.subplots(3, 1, figsize=(15, 15))

axes[0].set_title('Training loss')
axes[0].set_xlabel('Iteration')
axes[1].set_title('Training accuracy')
```

```python
axes[1].set_xlabel('Epoch')
axes[2].set_title('Validation accuracy')
axes[2].set_xlabel('Epoch')

for update_rule, solver in solvers.items():
    axes[0].plot(solver.loss_history, label=f"{update_rule}")
    axes[1].plot(solver.train_acc_history, label=f"{update_rule}")
    axes[2].plot(solver.val_acc_history, label=f"{update_rule}")

for ax in axes:
    ax.legend(loc='best', ncol=4)
    ax.grid(linestyle='--', linewidth=0.5)

plt.show()
```

```
Running with  adam
(Iteration 1 / 200) loss: 2.566155
(Epoch 0 / 5) train acc: 0.160000; val_acc: 0.150000
(Iteration 11 / 200) loss: 2.163810
(Iteration 21 / 200) loss: 1.997342
(Iteration 31 / 200) loss: 1.686885
(Epoch 1 / 5) train acc: 0.367000; val_acc: 0.321000
(Iteration 41 / 200) loss: 1.829010
(Iteration 51 / 200) loss: 1.844484
(Iteration 61 / 200) loss: 1.919071
(Iteration 71 / 200) loss: 1.608266
(Epoch 2 / 5) train acc: 0.412000; val_acc: 0.341000
(Iteration 81 / 200) loss: 1.563809
(Iteration 91 / 200) loss: 1.537605
(Iteration 101 / 200) loss: 1.449089
(Iteration 111 / 200) loss: 1.414424
(Epoch 3 / 5) train acc: 0.452000; val_acc: 0.364000
(Iteration 121 / 200) loss: 1.560739
(Iteration 131 / 200) loss: 1.360069
(Iteration 141 / 200) loss: 1.433769
(Iteration 151 / 200) loss: 1.313879
(Epoch 4 / 5) train acc: 0.548000; val_acc: 0.363000
(Iteration 161 / 200) loss: 1.387982
(Iteration 171 / 200) loss: 1.328627
(Iteration 181 / 200) loss: 1.324767
(Iteration 191 / 200) loss: 1.287378
(Epoch 5 / 5) train acc: 0.601000; val_acc: 0.371000

Running with  rmsprop
(Iteration 1 / 200) loss: 2.630047
(Epoch 0 / 5) train acc: 0.143000; val_acc: 0.115000
(Iteration 11 / 200) loss: 1.984898
```

```
(Iteration 21 / 200) loss: 1.783943
(Iteration 31 / 200) loss: 1.850880
(Epoch 1 / 5) train acc: 0.379000; val_acc: 0.285000
(Iteration 41 / 200) loss: 1.675548
(Iteration 51 / 200) loss: 1.800409
(Iteration 61 / 200) loss: 1.839983
(Iteration 71 / 200) loss: 1.744196
(Epoch 2 / 5) train acc: 0.418000; val_acc: 0.339000
(Iteration 81 / 200) loss: 1.520826
(Iteration 91 / 200) loss: 1.605029
(Iteration 101 / 200) loss: 1.385624
(Iteration 111 / 200) loss: 1.391699
(Epoch 3 / 5) train acc: 0.505000; val_acc: 0.350000
(Iteration 121 / 200) loss: 1.477524
(Iteration 131 / 200) loss: 1.539054
(Iteration 141 / 200) loss: 1.445163
(Iteration 151 / 200) loss: 1.340662
(Epoch 4 / 5) train acc: 0.512000; val_acc: 0.371000
(Iteration 161 / 200) loss: 1.327932
(Iteration 171 / 200) loss: 1.608787
(Iteration 181 / 200) loss: 1.395082
(Iteration 191 / 200) loss: 1.453611
(Epoch 5 / 5) train acc: 0.531000; val_acc: 0.362000
```



2.3 Inline Question 2:

AdaGrad, like Adam, is a per-parameter optimization method that uses the following update rule:

```
cache += dw**2
w += - learning_rate * dw / (np.sqrt(cache) + eps)
```

John notices that when he was training a network with AdaGrad that the updates became very small, and that his network was learning slowly. Using your knowledge of the AdaGrad update rule, why do you think the updates would become very small? Would Adam have the same issue?

2.4 Answer:

The cache gradually increases, which causes the update coefficient become smaller

No. in Adam, v change only change a bit, bacaue $1-\beta_2$ is small. And if dw become greatly larger, then m would become greatly larger

3 Train a Good Model!

Train the best fully connected model that you can on CIFAR-10, storing your best model in the `best_model` variable. We require you to get at least 50% accuracy on the validation set using a fully connected network.

If you are careful it should be possible to get accuracies above 55%, but we don't require it for this part and won't assign extra credit for doing so. Later in the next assignment, we will ask you to train the best convolutional network that you can on CIFAR-10, and we would prefer that you spend your effort working on convolutional networks rather than fully connected networks.

Note: In the next assignment, you will learn techniques like BatchNormalization and Dropout which can help you train powerful models.

[36]:
```
################################################################################
# TODO: Train the best FullyConnectedNet that you can on CIFAR-10. You might   #
# find batch/layer normalization and dropout useful. Store your best model in  #
# the best_model variable.                                                     #
################################################################################
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
num_train = 50
weight_scale = 1e-2
learning_rate = 6e-4
hidden_dim = [100,300,500,200,100,100]
reg = 1e-4

model = FullyConnectedNet(
    hidden_dims = hidden_dim,
    input_dim = 3*32*32,
    num_classes = 10,
    dropout_keep_ratio=1,
    normalization=None,
    reg=reg,
    weight_scale=weight_scale,
    dtype=np.float32,
    seed=None,
)
solver = Solver(
    model,
    data,
    update_rule = 'adam',
    lr_decay = 0.95,
    optim_config = {'learning_rate':learning_rate},
    batch_size = 300,
    num_epochs = 20,
```

```
)
solver.train()
acc = solver.check_accuracy(data['X_val'],data['y_val'])
print(f"lr:{learning_rate},weight_scale:{weight_scale},reg:{reg},layer_num:
 ↪{len(hidden_dim)},acc = {acc}")


best_model = model

# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
################################################################################
#                                END OF YOUR CODE                              #
################################################################################
```

```
(Iteration 1 / 3260) loss: 2.305680
(Epoch 0 / 20) train acc: 0.111000; val_acc: 0.108000
(Iteration 11 / 3260) loss: 2.214580
(Iteration 21 / 3260) loss: 2.224440
(Iteration 31 / 3260) loss: 2.113802
(Iteration 41 / 3260) loss: 2.122817
(Iteration 51 / 3260) loss: 2.085051
(Iteration 61 / 3260) loss: 2.041845
(Iteration 71 / 3260) loss: 2.088666
(Iteration 81 / 3260) loss: 2.022615
(Iteration 91 / 3260) loss: 2.031595
(Iteration 101 / 3260) loss: 2.035232
(Iteration 111 / 3260) loss: 2.087455
(Iteration 121 / 3260) loss: 1.990949
(Iteration 131 / 3260) loss: 2.024419
(Iteration 141 / 3260) loss: 2.002379
(Iteration 151 / 3260) loss: 2.008687
(Iteration 161 / 3260) loss: 1.981039
(Epoch 1 / 20) train acc: 0.161000; val_acc: 0.179000
(Iteration 171 / 3260) loss: 1.981995
(Iteration 181 / 3260) loss: 1.949293
(Iteration 191 / 3260) loss: 2.005523
(Iteration 201 / 3260) loss: 2.000781
(Iteration 211 / 3260) loss: 2.002521
(Iteration 221 / 3260) loss: 2.013667
(Iteration 231 / 3260) loss: 1.980323
(Iteration 241 / 3260) loss: 1.982023
(Iteration 251 / 3260) loss: 1.980105
(Iteration 261 / 3260) loss: 1.958826
(Iteration 271 / 3260) loss: 1.953150
(Iteration 281 / 3260) loss: 2.008771
(Iteration 291 / 3260) loss: 1.962542
(Iteration 301 / 3260) loss: 1.960841
```

```
(Iteration 311 / 3260) loss: 1.940190
(Iteration 321 / 3260) loss: 1.937306
(Epoch 2 / 20) train acc: 0.181000; val_acc: 0.202000
(Iteration 331 / 3260) loss: 1.965046
(Iteration 341 / 3260) loss: 1.903959
(Iteration 351 / 3260) loss: 1.942012
(Iteration 361 / 3260) loss: 1.946629
(Iteration 371 / 3260) loss: 1.956240
(Iteration 381 / 3260) loss: 1.959869
(Iteration 391 / 3260) loss: 1.891558
(Iteration 401 / 3260) loss: 1.953115
(Iteration 411 / 3260) loss: 1.839468
(Iteration 421 / 3260) loss: 1.815935
(Iteration 431 / 3260) loss: 1.937792
(Iteration 441 / 3260) loss: 1.872505
(Iteration 451 / 3260) loss: 1.771752
(Iteration 461 / 3260) loss: 1.834106
(Iteration 471 / 3260) loss: 1.784560
(Iteration 481 / 3260) loss: 1.823589
(Epoch 3 / 20) train acc: 0.293000; val_acc: 0.299000
(Iteration 491 / 3260) loss: 1.756994
(Iteration 501 / 3260) loss: 1.687494
(Iteration 511 / 3260) loss: 1.649584
(Iteration 521 / 3260) loss: 1.723278
(Iteration 531 / 3260) loss: 1.824768
(Iteration 541 / 3260) loss: 1.769396
(Iteration 551 / 3260) loss: 1.713673
(Iteration 561 / 3260) loss: 1.711925
(Iteration 571 / 3260) loss: 1.717050
(Iteration 581 / 3260) loss: 1.685405
(Iteration 591 / 3260) loss: 1.549417
(Iteration 601 / 3260) loss: 1.669405
(Iteration 611 / 3260) loss: 1.755049
(Iteration 621 / 3260) loss: 1.654728
(Iteration 631 / 3260) loss: 1.716295
(Iteration 641 / 3260) loss: 1.614760
(Iteration 651 / 3260) loss: 1.749676
(Epoch 4 / 20) train acc: 0.329000; val_acc: 0.377000
(Iteration 661 / 3260) loss: 1.751048
(Iteration 671 / 3260) loss: 1.664852
(Iteration 681 / 3260) loss: 1.635128
(Iteration 691 / 3260) loss: 1.626746
(Iteration 701 / 3260) loss: 1.643240
(Iteration 711 / 3260) loss: 1.611314
(Iteration 721 / 3260) loss: 1.628619
(Iteration 731 / 3260) loss: 1.690028
(Iteration 741 / 3260) loss: 1.574474
(Iteration 751 / 3260) loss: 1.557484
```

```
(Iteration 761 / 3260) loss: 1.548862
(Iteration 771 / 3260) loss: 1.648286
(Iteration 781 / 3260) loss: 1.586262
(Iteration 791 / 3260) loss: 1.630070
(Iteration 801 / 3260) loss: 1.541965
(Iteration 811 / 3260) loss: 1.653294
(Epoch 5 / 20) train acc: 0.390000; val_acc: 0.409000
(Iteration 821 / 3260) loss: 1.531717
(Iteration 831 / 3260) loss: 1.652539
(Iteration 841 / 3260) loss: 1.513008
(Iteration 851 / 3260) loss: 1.548915
(Iteration 861 / 3260) loss: 1.657524
(Iteration 871 / 3260) loss: 1.578894
(Iteration 881 / 3260) loss: 1.567847
(Iteration 891 / 3260) loss: 1.446998
(Iteration 901 / 3260) loss: 1.513896
(Iteration 911 / 3260) loss: 1.496448
(Iteration 921 / 3260) loss: 1.559751
(Iteration 931 / 3260) loss: 1.420192
(Iteration 941 / 3260) loss: 1.584785
(Iteration 951 / 3260) loss: 1.480402
(Iteration 961 / 3260) loss: 1.445472
(Iteration 971 / 3260) loss: 1.427162
(Epoch 6 / 20) train acc: 0.444000; val_acc: 0.449000
(Iteration 981 / 3260) loss: 1.433580
(Iteration 991 / 3260) loss: 1.400230
(Iteration 1001 / 3260) loss: 1.453525
(Iteration 1011 / 3260) loss: 1.409459
(Iteration 1021 / 3260) loss: 1.521113
(Iteration 1031 / 3260) loss: 1.374431
(Iteration 1041 / 3260) loss: 1.510234
(Iteration 1051 / 3260) loss: 1.434981
(Iteration 1061 / 3260) loss: 1.413785
(Iteration 1071 / 3260) loss: 1.434774
(Iteration 1081 / 3260) loss: 1.398878
(Iteration 1091 / 3260) loss: 1.420719
(Iteration 1101 / 3260) loss: 1.398551
(Iteration 1111 / 3260) loss: 1.483582
(Iteration 1121 / 3260) loss: 1.317006
(Iteration 1131 / 3260) loss: 1.316808
(Iteration 1141 / 3260) loss: 1.317971
(Epoch 7 / 20) train acc: 0.512000; val_acc: 0.499000
(Iteration 1151 / 3260) loss: 1.510683
(Iteration 1161 / 3260) loss: 1.285491
(Iteration 1171 / 3260) loss: 1.369538
(Iteration 1181 / 3260) loss: 1.332315
(Iteration 1191 / 3260) loss: 1.292381
(Iteration 1201 / 3260) loss: 1.269120
```

```
(Iteration 1211 / 3260) loss: 1.377900
(Iteration 1221 / 3260) loss: 1.441852
(Iteration 1231 / 3260) loss: 1.276620
(Iteration 1241 / 3260) loss: 1.266733
(Iteration 1251 / 3260) loss: 1.325410
(Iteration 1261 / 3260) loss: 1.310975
(Iteration 1271 / 3260) loss: 1.285512
(Iteration 1281 / 3260) loss: 1.348066
(Iteration 1291 / 3260) loss: 1.253233
(Iteration 1301 / 3260) loss: 1.240814
(Epoch 8 / 20) train acc: 0.565000; val_acc: 0.505000
(Iteration 1311 / 3260) loss: 1.205703
(Iteration 1321 / 3260) loss: 1.344215
(Iteration 1331 / 3260) loss: 1.308049
(Iteration 1341 / 3260) loss: 1.216938
(Iteration 1351 / 3260) loss: 1.359214
(Iteration 1361 / 3260) loss: 1.298583
(Iteration 1371 / 3260) loss: 1.253377
(Iteration 1381 / 3260) loss: 1.207249
(Iteration 1391 / 3260) loss: 1.261354
(Iteration 1401 / 3260) loss: 1.255880
(Iteration 1411 / 3260) loss: 1.173309
(Iteration 1421 / 3260) loss: 1.250351
(Iteration 1431 / 3260) loss: 1.256306
(Iteration 1441 / 3260) loss: 1.244742
(Iteration 1451 / 3260) loss: 1.145528
(Iteration 1461 / 3260) loss: 1.143808
(Epoch 9 / 20) train acc: 0.580000; val_acc: 0.516000
(Iteration 1471 / 3260) loss: 1.203131
(Iteration 1481 / 3260) loss: 1.170376
(Iteration 1491 / 3260) loss: 1.290633
(Iteration 1501 / 3260) loss: 1.185480
(Iteration 1511 / 3260) loss: 1.262691
(Iteration 1521 / 3260) loss: 1.395087
(Iteration 1531 / 3260) loss: 1.218957
(Iteration 1541 / 3260) loss: 1.292243
(Iteration 1551 / 3260) loss: 1.260102
(Iteration 1561 / 3260) loss: 1.200734
(Iteration 1571 / 3260) loss: 1.105535
(Iteration 1581 / 3260) loss: 1.158342
(Iteration 1591 / 3260) loss: 1.209920
(Iteration 1601 / 3260) loss: 1.201069
(Iteration 1611 / 3260) loss: 1.196152
(Iteration 1621 / 3260) loss: 1.213179
(Epoch 10 / 20) train acc: 0.587000; val_acc: 0.510000
(Iteration 1631 / 3260) loss: 1.124297
(Iteration 1641 / 3260) loss: 1.210357
(Iteration 1651 / 3260) loss: 1.118386
```

```
(Iteration 1661 / 3260) loss: 1.089711
(Iteration 1671 / 3260) loss: 1.156598
(Iteration 1681 / 3260) loss: 1.130097
(Iteration 1691 / 3260) loss: 1.135729
(Iteration 1701 / 3260) loss: 1.185588
(Iteration 1711 / 3260) loss: 1.097887
(Iteration 1721 / 3260) loss: 1.117112
(Iteration 1731 / 3260) loss: 1.183163
(Iteration 1741 / 3260) loss: 1.046266
(Iteration 1751 / 3260) loss: 1.195892
(Iteration 1761 / 3260) loss: 1.181839
(Iteration 1771 / 3260) loss: 1.155146
(Iteration 1781 / 3260) loss: 1.111465
(Iteration 1791 / 3260) loss: 1.172964
(Epoch 11 / 20) train acc: 0.604000; val_acc: 0.530000
(Iteration 1801 / 3260) loss: 1.001929
(Iteration 1811 / 3260) loss: 1.177663
(Iteration 1821 / 3260) loss: 0.903832
(Iteration 1831 / 3260) loss: 1.137024
(Iteration 1841 / 3260) loss: 1.054776
(Iteration 1851 / 3260) loss: 1.103575
(Iteration 1861 / 3260) loss: 1.063887
(Iteration 1871 / 3260) loss: 0.967013
(Iteration 1881 / 3260) loss: 1.120910
(Iteration 1891 / 3260) loss: 1.110611
(Iteration 1901 / 3260) loss: 1.198098
(Iteration 1911 / 3260) loss: 1.113862
(Iteration 1921 / 3260) loss: 1.063695
(Iteration 1931 / 3260) loss: 0.983098
(Iteration 1941 / 3260) loss: 1.019448
(Iteration 1951 / 3260) loss: 1.054036
(Epoch 12 / 20) train acc: 0.612000; val_acc: 0.532000
(Iteration 1961 / 3260) loss: 0.987148
(Iteration 1971 / 3260) loss: 1.062667
(Iteration 1981 / 3260) loss: 1.110595
(Iteration 1991 / 3260) loss: 1.040176
(Iteration 2001 / 3260) loss: 1.002144
(Iteration 2011 / 3260) loss: 1.059010
(Iteration 2021 / 3260) loss: 1.026946
(Iteration 2031 / 3260) loss: 1.113681
(Iteration 2041 / 3260) loss: 1.041009
(Iteration 2051 / 3260) loss: 1.010226
(Iteration 2061 / 3260) loss: 1.048625
(Iteration 2071 / 3260) loss: 1.032212
(Iteration 2081 / 3260) loss: 1.043551
(Iteration 2091 / 3260) loss: 1.048630
(Iteration 2101 / 3260) loss: 1.041220
(Iteration 2111 / 3260) loss: 1.016404
```

```
(Epoch 13 / 20) train acc: 0.604000; val_acc: 0.517000
(Iteration 2121 / 3260) loss: 1.065391
(Iteration 2131 / 3260) loss: 0.967863
(Iteration 2141 / 3260) loss: 1.051589
(Iteration 2151 / 3260) loss: 0.982553
(Iteration 2161 / 3260) loss: 1.092004
(Iteration 2171 / 3260) loss: 1.069718
(Iteration 2181 / 3260) loss: 1.033337
(Iteration 2191 / 3260) loss: 0.949538
(Iteration 2201 / 3260) loss: 0.969910
(Iteration 2211 / 3260) loss: 1.035652
(Iteration 2221 / 3260) loss: 0.930378
(Iteration 2231 / 3260) loss: 0.946149
(Iteration 2241 / 3260) loss: 1.072305
(Iteration 2251 / 3260) loss: 1.041237
(Iteration 2261 / 3260) loss: 0.988954
(Iteration 2271 / 3260) loss: 0.885170
(Iteration 2281 / 3260) loss: 0.991767
(Epoch 14 / 20) train acc: 0.685000; val_acc: 0.526000
(Iteration 2291 / 3260) loss: 0.826763
(Iteration 2301 / 3260) loss: 0.989265
(Iteration 2311 / 3260) loss: 1.041309
(Iteration 2321 / 3260) loss: 0.860309
(Iteration 2331 / 3260) loss: 0.979017
(Iteration 2341 / 3260) loss: 1.033221
(Iteration 2351 / 3260) loss: 0.864722
(Iteration 2361 / 3260) loss: 0.863373
(Iteration 2371 / 3260) loss: 0.857666
(Iteration 2381 / 3260) loss: 0.874276
(Iteration 2391 / 3260) loss: 0.929147
(Iteration 2401 / 3260) loss: 0.897855
(Iteration 2411 / 3260) loss: 0.970166
(Iteration 2421 / 3260) loss: 0.965993
(Iteration 2431 / 3260) loss: 0.965234
(Iteration 2441 / 3260) loss: 0.924094
(Epoch 15 / 20) train acc: 0.677000; val_acc: 0.520000
(Iteration 2451 / 3260) loss: 0.914338
(Iteration 2461 / 3260) loss: 0.992909
(Iteration 2471 / 3260) loss: 0.925571
(Iteration 2481 / 3260) loss: 0.937424
(Iteration 2491 / 3260) loss: 0.893232
(Iteration 2501 / 3260) loss: 0.985878
(Iteration 2511 / 3260) loss: 0.952429
(Iteration 2521 / 3260) loss: 0.915838
(Iteration 2531 / 3260) loss: 0.844347
(Iteration 2541 / 3260) loss: 0.918812
(Iteration 2551 / 3260) loss: 0.881454
(Iteration 2561 / 3260) loss: 0.886538
```

```
(Iteration 2571 / 3260) loss: 0.835053
(Iteration 2581 / 3260) loss: 0.893417
(Iteration 2591 / 3260) loss: 0.878143
(Iteration 2601 / 3260) loss: 0.942319
(Epoch 16 / 20) train acc: 0.705000; val_acc: 0.508000
(Iteration 2611 / 3260) loss: 0.733327
(Iteration 2621 / 3260) loss: 0.932349
(Iteration 2631 / 3260) loss: 0.817309
(Iteration 2641 / 3260) loss: 0.806564
(Iteration 2651 / 3260) loss: 0.856842
(Iteration 2661 / 3260) loss: 0.833663
(Iteration 2671 / 3260) loss: 0.867647
(Iteration 2681 / 3260) loss: 0.848362
(Iteration 2691 / 3260) loss: 0.765775
(Iteration 2701 / 3260) loss: 0.837513
(Iteration 2711 / 3260) loss: 0.885728
(Iteration 2721 / 3260) loss: 0.856807
(Iteration 2731 / 3260) loss: 0.861904
(Iteration 2741 / 3260) loss: 0.799806
(Iteration 2751 / 3260) loss: 0.912406
(Iteration 2761 / 3260) loss: 0.856294
(Iteration 2771 / 3260) loss: 0.873870
(Epoch 17 / 20) train acc: 0.725000; val_acc: 0.523000
(Iteration 2781 / 3260) loss: 0.920828
(Iteration 2791 / 3260) loss: 0.805072
(Iteration 2801 / 3260) loss: 0.825101
(Iteration 2811 / 3260) loss: 0.814987
(Iteration 2821 / 3260) loss: 0.856038
(Iteration 2831 / 3260) loss: 0.807031
(Iteration 2841 / 3260) loss: 0.733055
(Iteration 2851 / 3260) loss: 0.802774
(Iteration 2861 / 3260) loss: 0.773706
(Iteration 2871 / 3260) loss: 0.797108
(Iteration 2881 / 3260) loss: 0.833339
(Iteration 2891 / 3260) loss: 0.835485
(Iteration 2901 / 3260) loss: 0.788998
(Iteration 2911 / 3260) loss: 0.706761
(Iteration 2921 / 3260) loss: 0.817007
(Iteration 2931 / 3260) loss: 0.780563
(Epoch 18 / 20) train acc: 0.718000; val_acc: 0.528000
(Iteration 2941 / 3260) loss: 0.726138
(Iteration 2951 / 3260) loss: 0.694232
(Iteration 2961 / 3260) loss: 0.765319
(Iteration 2971 / 3260) loss: 0.665498
(Iteration 2981 / 3260) loss: 0.754957
(Iteration 2991 / 3260) loss: 0.755919
(Iteration 3001 / 3260) loss: 0.709123
(Iteration 3011 / 3260) loss: 0.747290
```

```
(Iteration 3021 / 3260) loss: 0.736516
(Iteration 3031 / 3260) loss: 0.783233
(Iteration 3041 / 3260) loss: 0.744931
(Iteration 3051 / 3260) loss: 0.716002
(Iteration 3061 / 3260) loss: 0.676588
(Iteration 3071 / 3260) loss: 0.680908
(Iteration 3081 / 3260) loss: 0.682637
(Iteration 3091 / 3260) loss: 0.708400
(Epoch 19 / 20) train acc: 0.766000; val_acc: 0.511000
(Iteration 3101 / 3260) loss: 0.807701
(Iteration 3111 / 3260) loss: 0.830073
(Iteration 3121 / 3260) loss: 0.754342
(Iteration 3131 / 3260) loss: 0.646469
(Iteration 3141 / 3260) loss: 0.755273
(Iteration 3151 / 3260) loss: 0.623805
(Iteration 3161 / 3260) loss: 0.724624
(Iteration 3171 / 3260) loss: 0.684737
(Iteration 3181 / 3260) loss: 0.728850
(Iteration 3191 / 3260) loss: 0.765591
(Iteration 3201 / 3260) loss: 0.684455
(Iteration 3211 / 3260) loss: 0.721286
(Iteration 3221 / 3260) loss: 0.650779
(Iteration 3231 / 3260) loss: 0.554282
(Iteration 3241 / 3260) loss: 0.638809
(Iteration 3251 / 3260) loss: 0.709897
(Epoch 20 / 20) train acc: 0.780000; val_acc: 0.501000
lr:0.0006,weight_scale:0.01,reg:0.0001,layer_num:6,acc = 0.532
```

4 Test Your Model!

Run your best model on the validation and test sets. You should achieve at least 50% accuracy on the validation set and the test set.

[37]:
```
y_test_pred = np.argmax(best_model.loss(data['X_test']), axis=1)
y_val_pred = np.argmax(best_model.loss(data['X_val']), axis=1)
print('Validation set accuracy: ', (y_val_pred == data['y_val']).mean())
print('Test set accuracy: ', (y_test_pred == data['y_test']).mean())
```

```
Validation set accuracy:  0.532
Test set accuracy:  0.524
```